SoK: Zero-Knowledge Friendly Hash Functions over Prime Fields

Clémence Bouvier[1], Lorenzo Grassi[2], Katharina Koschatko[3], Christian Rechberger[3,4], Fabian Schmid[3], Matthias Johann Steiner[5], Zhuo Wu[6] and Hailun Yan[7]

[1] Université de Lorraine, CNRS, Inria, LORIA, Nancy, France
[2] Eindhoven University of Technology, Eindhoven, Netherlands
[3] Graz University of Technology, Graz, Austria
[4] TACEO, Graz, Austria
[5] Freischaffender Forscher, Klagenfurt am Wörthersee, Austria
[6] Digital Technologies, Ant Group, China
[7] University of Chinese Academy of Sciences, Beijing, China

Abstract

Zero-Knowledge (ZK) proof systems have become a cornerstone of privacy-preserving technologies and blockchain scalability. However, traditional hash functions are often inefficient within ZK protocols due to their high constraint complexity in arithmetic circuits. While various ZK-friendly hash functions have been proposed to address this bottleneck, their diverse algebraic structures and varying security margins make it difficult for practitioners to select the optimal primitive.

We conduct a systematic survey of ZK-friendly hash functions, covering both algorithmic design paradigms and modes of operation. We consolidate the relevant algebraic cryptanalysis techniques and review the third-party cryptanalysis of each design. For parameter sets broken by the best known attacks, we estimate the minimum number of rounds needed to withstand them – not as new recommendations, but to bring all constructions to a comparable level. On this basis, we analyze each construction across different arithmetization styles (R1CS, AIR, PLONK), both theoretically, by deriving the underlying constraint or gate counts, and empirically, through extensive benchmarks.

This paper comes with a fully open-source implementation suite comprising a SageMath/Python reference framework, native Rust implementations, in-circuit implementations, and a reusable TikZ figure library.

Keywords: Zero-Knowledge Proofs, Hash Functions, Arithmetization, Algebraic Cryptanalysis

1 Introduction

Zero-Knowledge (ZK) proof systems have emerged as a foundational technology for blockchain and privacy-preserving computations. Nowadays, ZK proofs are increasingly used in real-world applications, including authentication, access control, blockchain and

Emails: clemence.bouvier@inria.fr (Clémence Bouvier), l.grassi@tue.nl (Lorenzo Grassi), katharina.koschatko@tugraz.at (Katharina Koschatko), christian.rechberger@tugraz.at (Christian Rechberger), fabian.schmid@tugraz.at (Fabian Schmid), steiner.matthias@gmx.at (Matthias Johann Steiner), pucun.wz@antgroup.com (Zhuo Wu), hailun.yan@ucas.ac.cn (Hailun Yan)

This work is licensed under a "CC BY 4.0" license.
Date of this document: 2026-09-08.

smart contract systems, and cryptocurrencies. At the core of these proof systems lies the underlying hash function, which plays a critical role in determining their overall efficiency. Although conventional standard hash functions, such as members of the SHA-3 and SHA-2 families, have a high level of assurance against cryptanalysis and are highly efficient on CPU architectures, they are often relatively inefficient when implemented within ZK proof systems. Consequently, several ZK-friendly hash functions have been proposed in the literature in the last decade, aiming to achieve greater efficiency within proof systems while preserving robust security guarantees. When working with ZK-friendly hash functions, one might ask:

> *After a decade of proposals, which designs are relevant when judged by cryptanalysis and performance?*

This SoK paper aims to answer this non-trivial question for the case of ZK protocols instantiated over prime fields.

1.1 Importance of ZK-Friendly Hash Functions

In this context, the efficiency of cryptographic primitives – most notably hash functions – is a decisive factor for the overall performance of the proof system, affecting both proof generation time and verification costs.

Key Purposes of Hash Functions in ZK Proof Systems. To make things clear, let us start by pointing out the following:

- *hash functions are not inherently required by the mathematical definition of zero-knowledge proofs.* An abstract interactive ZK proof can exist without any hash function;
- at the same time, *virtually all practical ZK proofs systems* (e.g., Groth16, PLONK, Halo2, STARKs, Spartan) *rely on hash functions* for commitments, Fiat–Shamir, Merkle trees, transcript generation, or polynomial commitment schemes.

Hash functions are crucial for ZK proofs as they allow a verifier to reason about large amounts of data through a small, fixed-size digest while preserving security. In other words, hash functions are crucial as they reduce the amount of data that must be committed to, authenticated, or referenced during the protocol, while ensuring the integrity and consistency of the information used throughout the proof protocol. Although the succinctness of modern ZK proofs is primarily achieved by the proof system itself, hash functions are indispensable for making proofs over large datasets computationally and communication efficient.

Limitation of Traditional Hash Functions. Traditional cryptographic hash functions such as SHA-2 [NIS15a] and SHA-3 [NIS15b] (Keccak [BDPVA11b, BDPV13]) are designed to optimize security and performance in conventional computing environments rather than the computational model of ZK proof systems. In more detail, their internal structures rely on operations that are expensive to represent as arithmetic constraints, including extensive bitwise logic (AND and XOR gates) and bit-rotations. When translated into a ZK proof circuit, these operations require a large number of constraints, significantly increasing proving time, memory consumption, and proof generation costs. Since the efficiency of modern ZK proof systems is largely determined by the size and complexity of the underlying constraint system, the direct use of SHA-2 or SHA-3 often becomes a major performance bottleneck.

Emergence of ZK-Friendly Hash Functions. To address this challenge, a new generation of *ZK-friendly*, or *Arithmetization-Oriented* (AO), hash functions has been developed, aiming to achieve an efficient arithmetization of ZK proof verification. These constructions achieve substantially lower proving costs by relying on algebraic operations that map efficiently to the finite-field arithmetic used in modern proof systems, making them significantly more practical for privacy-preserving protocols and succinct proof generation.

Motivation and Goals of this Work. The rapid development of ZK proof systems has led to the proposal of a wide range of ZK-friendly hash functions. The goal of minimizing the cost of proving hash computations inside ZK circuits has resulted in a diverse landscape of constructions employing fundamentally different design strategies, including distinct algebraic structures, nonlinear layers, linear diffusion mechanisms, and security assumptions.

At the same time, despite this progress, the field remains relatively young. Many proposed designs have subsequently been shown to admit attacks that reduce their claimed security or break important security properties altogether. Concrete examples include algebraic Gröbner basis attacks [ACG+19] that break the MARVELLOUS family (JARVIS and FRIDAY) [AD18], the preimage attack [GKRS22] on full *Grendel* [Sze21], or the practical rebound distinguisher on the original 10-round Skyscraper [BGK+25a] permutation, which leads to practical collisions on the 9-round compression function [Bak25]. All these attacks were proposed soon after publication of the primitive.

These developments highlight both the difficulty of designing secure ZK-friendly hash functions and the evolving understanding of the associated design principles. As a consequence, there is currently no clear consensus on which ZK-friendly hash function – or, more fundamentally, which design strategy – offers the best balance of security, efficiency, and practicality. Existing constructions make different trade-offs, are evaluated under different cost models, and often target different proof systems or implementation environments, making direct comparison challenging. Furthermore, the rapid pace of new proposals and cryptanalytic results makes meaningful comparison increasingly difficult. Many hash functions have different security margins, while others have been partially or completely broken, rendering direct comparisons of efficiency misleading.

This motivates a systematic study of ZK-friendly hash functions. Similar to what Bak, Boeuf, Bouvier, Manterola Ayala, Øygarden, Perrin, Phanse and Raddum did with the STAP (Symmetric Techniques for Advanced Protocols) Zoo website [MAPB23], this paper aims to provide a unified perspective on the field, by organizing existing constructions according to their underlying design principles, examining the rationale behind their components, and reviewing the current state of cryptanalysis.

Still, we do not limit ourselves to provide a simple survey of the existing literature. For primitives whose security has been compromised due to insufficient security margins, we reassess their security in light of the latest cryptanalytic results. This enables a more meaningful comparison across designs by evaluating them at comparable security levels rather than as originally proposed. Building on this reassessment, we present a more realistic performance comparison that accounts for both proving efficiency and the security margin, thereby providing practitioners with a sounder basis for selecting ZK-friendly hash functions and helping researchers better understand the trade-offs between competing design strategies.

1.2 Our Contribution

In this paper, we provide a SoK (systematization of knowledge) of ZK-friendly hash functions specifically tailored for prime fields. While ZK-friendly hash functions have also been proposed over binary fields, and new proof systems allowing for more efficient proofs of binary circuits have recently been proposed [DP25, DP26, Set26, BRW26] we deliberately

exclude them from our scope.[8] The main contributions of this work are summarized as follows.

1.2.1 Classification of Existing ZK-friendly Hash Functions

We provide a formal classification of ZK-friendly hash functions into *four types/categories*, extending the three-type classification initially proposed by Rechberger [Rec23], which is now widely adopted in the literature. In details:

Type-I corresponds to the first wave of designs, which aim to minimize the number of nonlinear operations by relying on low-degree inner functions. Representative primitives include GMiMC [AGP+19a] (and Feistel-MiMC [AGR+16]), Poseidon [GKR+21], Neptune [GOPS22], Poseidon2 [GKS23a], Moirai [BP25b], and pSquare-hash [GMBM+26]. See Section 5.

Type-II includes designs that can be verified efficiently in a given arithmetic circuit via an equivalent representation. Representative primitives include *Rescue* [AAB+20], *Rescue-Prime* [SAD20], *Rescue-Prime Optimized* (RPO) [AKM+22], `Anemoi` [BBC+23], Griffin [GHR+23], Arion [CRST26], and XHash [ABK+26]. See Section 6.

Type-III captures constructions whose defining strategy is to decompose field elements into smaller chunks and apply non-linear operations through look-up tables on these chunks. Representative constructions include Reinforced Concrete [GKL+22], `Monolith` [GKL+24], Tip5 [SLST23], Tip4/Tip4' [Sal23], and Skyscraper [BGK+25a]. See Section 7.

Type-IV is a new type aiming at capturing constructions that use, as an essential component, an m-th power residue character. Such constructions were previously grouped with lookup-based designs, although conceptually they are defined by the residue character itself, not by its possible lookup realization. The representative constructions are *Grendel* [Sze21] and Polocolo [HHL+25]. See Section 8.

An overview of the timeline of ZK-friendly primitives is proposed in Figure 1.[9] In there, we list the various primitives studied in this paper, classified by type. Previous works (like [Bou23] or [MAPB23]) often assign a single date to each primitive, without distinguishing between its pre-publication (ePrint or arXiv) and its publication in a journal or conference proceedings. Due to significant time delay - sometimes several years - between these two events, we have chosen to include both dates systematically.

1.2.2 Extensive Open-Source Implementation Suite

Beyond surveying the literature, we provide a unified, fully open-source implementation suite covering the primitives considered in this work. It supports reproducibility, consistent comparison, and community extensions.

Reference Framework (SageMath/Python). A documented executable specification with a uniform structure for parameter validation, design-specified derivations (round numbers, constants, matrices), and shared sponge and compression modes, written to support both cryptanalysis and the instantiation of valid parameter sets.

https://github.com/stap-zoo/ref-sage

[8] Binary fields exhibit characteristic-two-specific properties that give rise to additional cryptanalytic techniques and attack vectors absent in the prime-field setting. Given the already broad scope of this work, treating these designs adequately would therefore require a separate analysis.

[9] We deliberately omit Feistel-MiMC published in 2016 for reasons explained in Section 5.1.

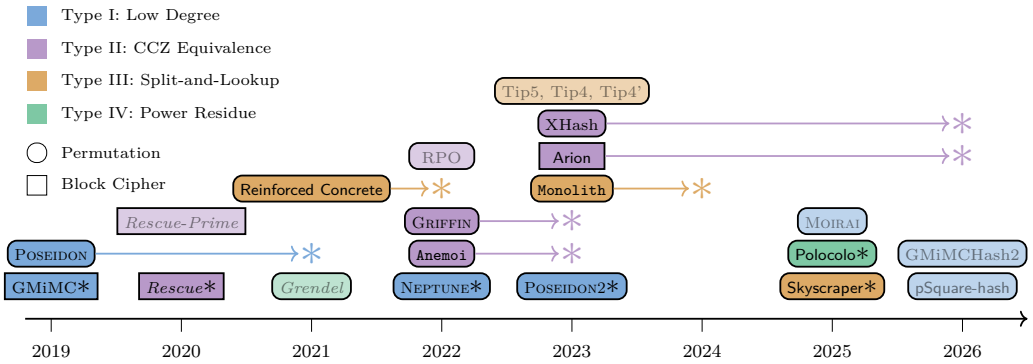


Figure 1: Timeline of ZK-friendly primitives. Low-opacity entries indicate preprint-only works; opaque entries show published works (∗ indicates their publication year).

Plain Implementation (Rust). The baseline native implementation used for the benchmarks in Section 10.1.

https://github.com/stap-zoo/plain-rust

Proof-System Implementations. In-circuit implementations across several backends and arithmetizations, used for the benchmarks in Section 10.2.

https://github.com/stap-zoo/proof-plonky3
https://github.com/stap-zoo/proof-gnark

Figure Library (LaTeX). A TikZ library and reusable `tikzpictures` for all primitives.

https://github.com/stap-zoo/latex-figures

1.2.3 Restoring Security

ZK-friendly primitives have attracted a substantial amount of cryptanalytic attention, with particularly significant advances in algebraic cryptanalysis over the past years, as shown in detail in Figure 4. We therefore revisit the most relevant algebraic attacks on these constructions in Section 4, providing a consolidated view of the modeling and solving techniques employed and their evolution. The primitive-specific sections complement this overview with a detailed account of the cryptanalysis of each construction, with the goal of making the progression of attacks, their assumptions, and their concrete impact on the proposed parameter sets explicit.[10]

For common parameter sets broken by the best known attacks, we estimate the minimum number of rounds required to recover the claimed security level. We emphasize that these values are *not* proposed as updated round numbers or new security recommendations, but as lower bounds on what would be necessary to withstand the respective attacks, without any additional security margin. A formal security analysis is beyond the scope of this work, and we do not aim to take the place of the respective designers. Our goal is to have a rough estimation of the number of rounds in order to provide a fair performance comparison of the different ZK-friendly schemes.

[10] Only attacks that appeared in the literature or online *before September 01, 2026* are considered in this work.

1.2.4 GMiMCHash2: An Updated Variant of GMiMCHash

While restoring the security was possible for all considered hash functions, analyzing the cryptanalysis literature of GMiMCHash [AGP+19a] turned out to be a challenging task due to its diversity. GMiMCHash is an expanding Feistel Network with $x \mapsto (x+c)^\alpha$ as source of non-linearity, where the exponent α induces a power permutation. As one of the first primitives in the literature, see Figure 1, GMiMCHash is also one of the most studied primitives with regard to cryptanalysis. At the time of writing this survey it is fair to say that the initial cryptanalysis did not uphold its claims as several works could significantly improve upon the complexity of attacks [BCD+20, LWL23, Ste26]. Moreover, dominant attack vectors vary with the underlying prime size. For example, for fields $p \approx 2^{32}$ the most competitive attacks are differential [BCD+20] and interpolation attacks [BV25a], also see Appendix D.1.3, but for fields $p \approx 2^{64}$ the best attack is a hybrid algebraic-differential attack [BCD+20, LWL23].

Since our parameters for performance comparison, see Section 1.2.3, have not specifically been studied in cryptanalysis papers, our compilation of the GMiMCHash cryptanalysis literature led to a de facto re-evaluation of the primitive. Instead of just recalibrating round numbers of GMiMCHash, we realized that its plain as well as proof system performance can be significantly improved by slightly tweaking the permutation. As most important change, we propose to use a degree-growth optimal power of 2 for α instead of a permutation exponent. Moreover, we propose application of an Input/Output matrix at the start respectively end of the permutation which improves Gröbner basis properties.

We call our updated variant GMiMCHash2, its specification can be found in Section 5.1.2. As a result of our modifications GMiMCHash2 is one of the most competitive primitives in plain as well as proof system performance going head to head or even beating the de facto industry standard Poseidon.

1.2.5 Efficiency Comparison across Different ZK-Proof Systems

We provide a detailed efficiency comparison of ZK-friendly primitives at both the theoretical and implementation levels. In Section 9, we derive the costs of each construction across R1CS, PLONK, and AIR, building on the arithmetization background of Section 2.2 and giving the underlying constraint and gate counts. In Section 10, we complement this analysis with concrete benchmarks for both plain evaluation and circuit performance. To make these results comparable, we evaluate the primitives at 128-bit security on a common set of instances:

- $p \approx 2^{32}$ with $t \in \{16, 24\}$ (BabyBear, KoalaBear, Mersenne31);
- $p \approx 2^{64}$ with $t \in \{8, 12\}$ (Goldilocks);
- $p \approx 2^{256}$ with $t \in \{3, 4\}$ (BN254/BLS12-381 scalar field).

Not all primitives are defined for all prime fields: `Monolith` is defined for 32-bit and 64-bit fields, pSquare-hash only for 32-bit fields, the Tip5 and XHash families only for specific primes and state sizes. Next, Skyscraper, Reinforced Concrete, Polocolo, *Grendel*, and Arion are exclusively defined over large fields. We evaluate constructions only in their defined domains. Further, as explained in Section 1.2.3, for designs with broken parameter sets we use the updated round numbers (as summarized in Tables 17 and 18), to provide a fair comparison. We evaluate the primitives in native and proof system performance, where we support all state sizes listed above in the native evaluation, we provide data only for state size of around 768 bits in the proof systems. We evaluate the discussed constructions in proof systems with different arithmetization styles to point out that *there is no single notion of ZK-friendliness.*

The Cost of Arithmetization Strategies. The simplest cost model is the Rank One Constraint System (R1CS), where cost is dictated primarily by the number of multiplications of a construction. We then use the Algebraic Intermediate Representation (AIR) where we can trade maximum constraint degree for smaller commitment size. With this added flexibility we can have different trade-offs between prover runtime, proof size and verifier runtime giving a more complex image. Finally, we will analyze the constructions when represented within the constraint system Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge (PLONK). With extensions to PLONK, custom constraints can be proven and finding the optimal arithmetization for each construction exceeds the scope of this work. Thus, we will limit ourselves to analyzing the cost of each construction under the standard three-wire PLONK gates and lookup relations to include newer constructions. PLONK is fundamentally different from the other styles as it directly prices linear layers opening a large surface for dedicated optimizations discussed in Section 9.

Efficiency Characteristics. Across the described cost models, we provide detailed insights into the arithmetization costs of each component of the analyzed constructions. We show the inverse relation between plain and proof system performance with Type-II and Type-IV being slowest in plain and compact in constraint systems while Type-III is much more efficient in plain while exhibiting very high costs in the proof systems. With the large one-time cost of lookup tables in Type-III constructions and Polocolo, we show that the amortization effects only begin to show for our most significant tests and their efficiency seems limited to very extensive batching. In light of recent attacks and updated round numbers in several Type-II primitives, we find the Type-I primitives to exhibit the most favorable characteristics. They are competitive in the proof system while forming the second best category in plain evaluations. Overall, these results demonstrate that the efficiency of an AO primitive depends on the underlying arithmetization model and the intended usage setting (i.e., batched or single evaluation).

1.3 Organization

The remainder of this paper is organized as follows.

Section 2 provides the necessary background on zero-knowledge proof systems, the arithmetizations they rely on (R1CS, PLONK, and AIR), the parameter regimes considered in practice, and why traditional hash functions are ill-suited to these settings. Section 3 provides an overview of the modes of operation relevant to ZK-friendly hashing, covering both sponge- and compression-/tree-based constructions and stating the corresponding security requirements in each case. Section 4 reviews the algebraic cryptanalytic techniques relevant to ZK-friendly primitives, covering the modeling and solving approaches that recur throughout the primitive-specific sections.

Sections 5 to 8 provide an in-depth treatment of Type-I (low degree), Type-II (CCZ-equivalent), Type-III (look-up-table-based), and Type-IV (power-residue) primitives. All four sections follow the same structure. Each begins with a definition of the treated type and its defining design strategy. The primitives are then grouped by design family and treated uniformly: for each primitive, we recall the construction, summarize the relevant third-party cryptanalysis, and, where security has been weakened, re-estimate the number of rounds required to restore the target security level. For parametrizable hash functions, we additionally provide summary tables listing the designer-specified and updated round numbers for representative instantiations targeting 128-bit security. Additional cryptanalytic results, in particular the detailed security analysis of our updated variant GMiMCHash2, are deferred to Appendices C and D.

Finally, Section 9 analyzes the theoretical proof-system cost of each construction under the arithmetization styles R1CS, PLONK, and AIR. It establishes concrete performance

metrics for each arithmetization style that estimate the expected proving time when evaluated in a concrete proof system. Section 10 reports the evaluation of our plain implementation and in-circuit benchmarks in the Gnark and Plonky3 frameworks. Finally, Section 11 provides a comparative summary of our results across cryptanalysis, theoretical, native, and proof system performance.

How to Read This Paper. This paper is deliberately detailed and therefore necessarily long and, at times, repetitive. It is designed to support both linear reading and targeted lookup, depending on the reader's goal.

- *Broad Overview.* Read linearly, skip details. Sections 2 and 3 establish the vocabulary – proof systems, arithmetization, and modes of operation – assumed throughout.

- *Design Overview.* Sections 5 to 8 treat the primitives by type; each is self-contained, so a reader interested in a single design can consult its subsection alone. Each type section opens with the design strategy that defines it before turning to the individual constructions.

- *Cryptanalysis Results.* Section 4 is the conceptual backbone and is intentionally detailed, as its techniques recur across all four type sections. The per-primitive cryptanalysis is detailed where a design has drawn significant third-party attention, most notably the Type-I and Type-II primitives, several of which underwent dedicated cryptanalysis challenges. A mapping between primitives and cryptanalysis papers is given in Appendix B.

- *Constraint System.* Section 2 gives a high-level introduction to the three arithmetizations considered in this paper – R1CS, PLONK, and AIR – each illustrated with a small worked example. The realization of the individual primitive components within these constraint systems is analyzed separately in Section 9. For each primitive, we provide a concrete arithmetization strategy and resulting constraint or gate counts.

- *Performance.* Readers interested only in the latest plain or proof-system performance can proceed directly to Section 10, consulting the relevant type section only for design choices. Where a design has been successfully attacked, these benchmarks use the updated round numbers corresponding to the minimum required against the best known attacks, in an effort to provide a fairer comparison.

Notation. Named maps and design components are set in uppercase sans-serif; examples include the hash function H, compression function C, permutation P, as well as a primitive's round function R, S-box S, lookup table T, and linear layer M.

For each primitive, t denotes the state size and R the number of rounds; the S-box exponent of a power map is α, and κ the targeted security level in bits. In sponge modes, we split the state size as $t = r + c$, where r is the rate and c the capacity; d denotes the digest length, `IV` the initialization vector, and a the arity of a compression function.

We write $\mathbb{F}_q$ for a finite field with q elements, $\mathbb{F}_p$ for a prime field, and $\mathbb{F}_p^\times$ for its multiplicative group. In general, vectors and matrices are written in bold, e.g. $\mathbf{x} \in \mathbb{F}^n$ and $\mathbf{M} \in \mathbb{F}^{n \times n}$. Vectors are indexed from 1, i.e. $\mathbf{x} = (x_1, \ldots, x_n)$, with $\mathbf{x}^\intercal$ denoting the transpose. Concatenation is written $\mathbf{x} \,\|\, \mathbf{y}$, 0^n denotes the all-zero string of length n, and $\mathbf{0}$ the zero vector. Moreover, $\mathbf{I}_{n \times n}$ and $\mathbf{0}_{n \times n}$ denote the identity and zero matrices, respectively. Dimensions may be omitted when clear from context.

2 Overview of Zero-Knowledge Protocols

In this section, we review the relevant fundamentals on zero-knowledge proofs (ZKPs). We first review the fundamentals of ZKP systems, focusing on ZK-SNARKs in Section 2.1, and survey three arithmetization paradigms: R1CS [BCG+13], classical PLONK [GWC19], and AIR [BBHR18b] (Section 2.2). We then summarize the finite fields used in modern proof systems in Section 2.3. Finally, we explain why standard hash functions such as SHA-2 and SHA-3 are ill-suited to this setting in Section 2.4.

Terminology. The traditional "SNARK-friendly" versus "STARK-friendly" terminology in large parts of the literature on ZK-friendly hashing is no longer sufficiently precise for the current landscape. Indeed, these labels mix frontend arithmetization questions with backend proof-system mechanisms, whereas the relevant efficiency metrics in our setting are determined primarily at the arithmetization layer. Accordingly, we use Arithmetization-Oriented (AO) as the umbrella term for primitives tailored to efficient realization in proof systems, and distinguish more specifically between R1CS-friendly, PLONKish-friendly, and AIR-friendly constructions whenever necessary. This terminology better reflects modern systems in which a common arithmetization may be paired with different backend protocols.

2.1 Background

A *zero-knowledge proof* (ZKP) system allows a prover $\mathcal{P}$ to convince a verifier $\mathcal{V}$ of the validity of a statement x with respect to a relation $\mathcal{R}$ (i.e., $(\mathcal{P}, \mathcal{V})$ proves that there exists a witness w such that $(x, w) \in \mathcal{R}$), without leaking any information about w beyond the validity of the statement [GMR85, GO94]. A ZKP system must satisfy three fundamental properties:

- **Completeness**: If the statement is true and $\mathcal{P}$ follows the protocol, $\mathcal{V}$ accepts with overwhelming probability.
- **Soundness**: If the statement is false, no efficient adversary $\mathcal{P}^*$ can convince $\mathcal{V}$ to accept, except with negligible probability.
- **Zero-Knowledge**: There exists an efficient simulator $\mathcal{S}$ capable of producing a transcript indistinguishable from a real interaction between $\mathcal{P}$ and $\mathcal{V}$, implying $\mathcal{V}$ learns nothing from the interaction.

In this work, we focus on *zero-knowledge succinct non-interactive arguments of knowledge* (ZK-SNARKs), which describe ZKP systems that are computationally sound (i.e., arguments), with sublinear proof size and verifier times (i.e., succinctness) [BCC+16, BCI+22]. Over the past decade, SNARK constructions have made substantial progress across multiple dimensions, including prover time [XZZ+19, GLS+23, BCF+25], proof size and verifier time [Gro16], universality and updatability of the setup [MBKM19, CHM+20], transparent setup [BCR+19, Set20, BBHR19], and field-agnostic polynomial commitments [ZCF24]. In the SNARK setting, the relation $\mathcal{R}$ is typically instantiated as the satisfiability of an arithmetic circuit over a finite field, where x specifies the values on the public wires of the circuit and w specifies the values on the private wires, so that $(x, w) \in \mathcal{R}$ holds if and only if the resulting assignment satisfies all circuit constraints [Gro16].

SNARKs have found widespread applications in practical systems [LHW+25]. For example, in privacy-preserving cryptocurrencies, a user can prove authorization to spend their funds and prove their membership in the public ledger's Merkle tree, all while keeping the specific ledger entry hidden and preventing double-spending [BCG+14]. Further, in anonymous credential systems, a user can prove possession of a valid credential without disclosing its contents or the user's identity [CL01], while in verifiable computation, a

prover can convince a verifier that a computation was executed correctly without requiring the verifier to repeat the entire computation [PHGR13].

From Algorithms to Proofs. To prove the evaluation of an algorithm (e.g., a hash evaluation), its description must first be transformed into a set of algebraic relations [GGPR13, BCG+13]. While finite-field additions and multiplications suffice in principle to represent arbitrary computations as arithmetic circuits [BCG+13], different *constraint systems* may directly support additional relations, such as custom-gate or lookup constraints [CBBZ23, ZBK+22]. The idea is that we need only a mechanism to constrain the evaluation of the basic algebraic operations, such that in composition, we can prove arbitrary computations. While there are many methods to constrain basic algebraic operations, we focus on the constraint systems R1CS [BCG+13], PLONK [GWC19, CBBZ23], and AIR [BBHR19] that have found wide application in modern ZK-SNARK proof systems.

At a high level, all three constraint systems enforce correctness through low-degree polynomial relations over a finite field. They differ in the expressiveness of their constraints, for example, quadratic relations versus higher-degree constraints, different fan-ins of operations, or differing costs of linear relations. We provide details on the cost model of these constraint systems in Section 2.2 and discuss the concrete cost of arithmetizing hash functions in Section 9. Once the computation is expressed as a set of constraints, it can be passed to the selected proof system. Given the set of constraints and a satisfying witness (i.e., inputs and intermediate results), the prover produces a succinct non-interactive proof. Such a proving system may itself combine many cryptographic components, such as polynomial commitments [KZG10] and argument protocols, each with their own performance characteristics. However, while it does not capture every prover-specific effect, the complexity of the resulting set of constraints remains a useful estimation for expected prover time, since the prover's work generally scales with the size and algebraic complexity of the given constraints [BCG+13, CBBZ23, BBHR19]. Finally, a proof is subsequently checked by the verifier algorithm against the public statement, and the verifier outputs either accept or reject.

2.2 Arithmetization in ZK-SNARKs

Below, we introduce three prevalent arithmetization paradigms used in ZK-SNARKs. For each paradigm, we present its formal definition and illustrate it using the same example. While these arithmetization styles may be instantiated over any field $\mathbb{F}$, we will restrict our introduction to the prime field $\mathbb{F}_p$.

Example 1. To make the connection with hash-function design concrete, consider one round of a permutation over $\mathbb{F}_p^2$. For an input state $\mathbf{x} = (x_1, x_2)$, the round first adds round constants, then applies the power map $x \mapsto x^3$ to both state elements, and finally applies a linear layer:

$$\mathbf{u} = ((x_1 + c_1)^3, (x_2 + c_2)^3), \qquad \mathbf{y} = \mathbf{M}\mathbf{u}, \qquad \mathbf{M} = \begin{pmatrix} 2 & 1 \\ 1 & 1 \end{pmatrix}. \tag{1}$$

Equivalently, $y_1 = 2u_1 + u_2$ and $y_2 = u_1 + u_2$. The three arithmetizations below encode exactly these relations, but assign different costs to their nonlinear and linear parts.

2.2.1 Rank-1 Constraint System (R1CS)

In R1CS, a computation is flattened into a system of equations over the prime field $\mathbb{F}_p$, where the prover must find a witness vector $\mathbf{z} \in \mathbb{F}_p^n$ (typically $\mathbf{z} = (1, \mathbf{x}, \mathbf{w})$, containing public inputs $\mathbf{x}$ and private witness $\mathbf{w}$) that satisfies

$$(\mathbf{A} \cdot \mathbf{z}) \circ (\mathbf{B} \cdot \mathbf{z}) = \mathbf{C} \cdot \mathbf{z}. \tag{2}$$

Here, $\mathbf{A}, \mathbf{B}, \mathbf{C} \in \mathbb{F}_p^{m \times n}$ are constant coefficient matrices defining the circuit logic, m is the number of constraints, and $\circ$ denotes the Hadamard product. Each row encodes one constraint

$$\langle \mathbf{a}_i, \mathbf{z} \rangle \cdot \langle \mathbf{b}_i, \mathbf{z} \rangle = \langle \mathbf{c}_i, \mathbf{z} \rangle. \tag{3}$$

Thus, the factors and result may each be arbitrary linear combinations of witness entries.

Example 2 (Example 1 in R1CS)**.** For the round in Equation (1), the prover witnesses $s_i = (x_i + c_i)^2$ for $i \in \{1, 2\}$. Since $\mathbf{M}^{-1} = \left(\begin{smallmatrix} 1 & -1 \\ -1 & 2 \end{smallmatrix}\right)$, we have $u_1 = y_1 - y_2$ and $u_2 = 2y_2 - y_1$. Hence, with $\mathbf{z} = (1, x_1, x_2, s_1, s_2, y_1, y_2)$, the complete round is represented by exactly four R1CS rows:

$$\underbrace{\begin{pmatrix} x_1 + c_1 \\ s_1 \\ x_2 + c_2 \\ s_2 \end{pmatrix}}_{\mathbf{Az}} \circ \underbrace{\begin{pmatrix} x_1 + c_1 \\ x_1 + c_1 \\ x_2 + c_2 \\ x_2 + c_2 \end{pmatrix}}_{\mathbf{Bz}} = \underbrace{\begin{pmatrix} s_1 \\ y_1 - y_2 \\ s_2 \\ 2y_2 - y_1 \end{pmatrix}}_{\mathbf{Cz}}. \tag{4}$$

The constant additions occur inside the multiplicative linear forms $\mathbf{Az}$ and $\mathbf{Bz}$, while the inverse linear layer occurs inside the output form $\mathbf{Cz}$. Thus, neither linear operation requires an additional R1CS row.

R1CS Cost. Following the classical Groth16-oriented cost model [Gro16], we use the number of multiplication constraints as the primary R1CS cost. The example above, therefore, has R1CS cost four, excluding input and output binding constraints. However, we note that in some proof systems, such as Spartan, the R1CS prover cost depends on the number of nonzero entries [Set20, §2.1, §7]. For such systems, reducing the number or density of linear relations may also improve performance.

2.2.2 PLONK Arithmetization

In this work, we restrict the PLONK analysis to the classical three-wire, degree-two gate from the original construction [GWC19]. Although modern PLONKish systems admit custom gates of different widths and degrees, their system-specific expressiveness makes a uniform theoretical comparison difficult. In PLONK arithmetization, the computation trace (i.e., all inputs and outputs of each operation) is stored in a matrix, where the cells are constrained with gates. The classical gate uses cells or wires w_L, w_R, w_O and satisfies

$$q_L w_L + q_R w_R + q_O w_O + q_M w_L w_R + q_C = 0, \tag{5}$$

where q_L, q_R, q_O, q_M, q_C are fixed field elements selecting and scaling the active wires in a given row. Across the entire trace, these row values are represented by selector polynomials. A copy-constraint permutation σ additionally ensures that different wire positions representing the same logical variable carry the same value. We retain these copy constraints in the description but do not analyze their cost separately. In a hash computation, most gate inputs reuse the outputs of preceding gates, so copy relations occur throughout the circuit, and impact is determined roughly through the number of three-wire constraints.

Example 3 (Example 1 in PLONK)**.** The running example from Equation (1) can be represented by the following six rows:

	w_L	w_R	w_O	q_L	q_R	q_O	q_M	q_C
1	x_1	x_1	s_1	c_1	c_1	-1	1	c_1^2
2	s_1	x_1	u_1	c_1	0	-1	1	0
3	x_2	x_2	s_2	c_2	c_2	-1	1	c_2^2
4	s_2	x_2	u_2	c_2	0	-1	1	0
5	u_1	u_2	y_1	2	1	-1	0	0
6	u_1	u_2	y_2	1	1	-1	0	0

, (6)

where constant addition and power map equally require 2 gates per element, but the subsequent matrix requires another gate per output element.

PLONK Cost. We use the number of classical three-wire PLONK gates as the primary cost metric. The running example, therefore, costs six PLONK gates: four for the two cubic power maps and two for the linear layer. Finding a minimum representation is generally difficult because the count depends on the chosen intermediate variables, wire placement, and straight-line program; even minimizing linear straight-line programs is hard in general [BMP13]. Consequently, the counts in this paper refer to explicit circuit constructions and are upper bounds unless stated otherwise. In contrast to the R1CS cost model, a linear layer is not generally free under the fixed three-wire PLONK gate. In particular, matrix-vector products with a dense $n \times m$ matrix require $n(m-1)$ such gates, making it important to carefully choose the matrix.

2.2.3 Algebraic Intermediate Representation (AIR)

In AIR, a computation is represented by an execution trace $T \in \mathbb{F}_p^{h \times w}$ of height h and width w. Each row represents one computation step, and each column one register. Transition constraints are polynomials $\mathsf{F}_1, \ldots, \mathsf{F}_s \in \mathbb{F}_p[X_1, \ldots, X_{2w}]$ of bounded degree that vanish on every pair of adjacent rows, namely

$$\mathsf{F}_j(T[i], T[i+1]) = 0 \qquad \forall\, j \in \{1, \ldots, s\},\ \forall\, i \in \{1, \ldots, h-1\}, \tag{7}$$

together with boundary constraints that fix selected entries of the initial and final rows.

Example 4 (Example 1 in AIR)**.** For a classical two-row encoding of Equation (1), let

$$T[1] = (x_1, x_2), \qquad T[2] = (y_1, y_2). \tag{8}$$

The round is enforced by the two transition constraints

$$\mathsf{F}_1(T[1], T[2]) = y_1 - 2(x_1 + c_1)^3 - (x_2 + c_2)^3 = 0, \tag{9}$$

$$\mathsf{F}_2(T[1], T[2]) = y_2 - (x_1 + c_1)^3 - (x_2 + c_2)^3 = 0. \tag{10}$$

The constant addition and linear layer are absorbed directly into these polynomials. This layout has height $h = 2$, width $w = 2$, two transition polynomials, and a maximum constraint degree of 3.

AIR Cost. The principal AIR parameters are the trace height h, trace width w, number of transition polynomials s, and maximum constraint degree $d_{\max}$. When constraining hash functions, the number of transition constraints is closely tied to the witnessed cells. We therefore primarily report the committed-cell count c together with $d_{\max}$, while providing context to the trade-off choices between the two parameters. Further, in the detailed performance analysis, we flatten all values required for one permutation into a single logical row. Thus, constructions like Poseidon with partial rounds do not suffer any padding issues. The committed-cell count c is then the width of that flattened row.

2.2.4 Lookup Arguments

Lookup arguments can be used to constrain a witness to be a value of a predetermined set, the lookup table. Further, an indexed lookup can be used to constrain the traditionally understood notion of a lookup table, a guaranteed input-output relation of two witnesses according to an encoded function [HACF25]. A specific lookup argument may be compatible with multiple arithmetization and commitment styles. For example, logarithmic-derivative

techniques such as LogUp [Hab22] can be incorporated into various proof systems and are used in our experiments in the R1CS, PLONK, and AIR settings. Because their concrete costs depend heavily on the lookup argument, batching strategy, and table reuse, we do not translate lookups into multiplication constraints, PLONK gates, or AIR cells. Instead, we write $k\,\mathrm{LU}_s$ for k queries to a table containing s elements. The number of queries serves as a proxy for the marginal lookup cost, while the table size serves as a proxy for preprocessing, commitment, and amortized cost. While it does make theoretical comparison with non-lookup designs harder, it avoids conflating different cost sources.

2.3 Practical Parameter Instantiations

Below we introduce several prime fields used in proof systems. Modern recursive ZK-SNARKs have shifted from large scalar fields (e.g., BN254) to smaller fields (e.g., Goldilocks, BabyBear) to optimize prover performance. Table 1 summarizes the prime values dominating the current landscape.

Table 1: Finite Fields used in ZK-SNARKs.

Field Name	Concrete Prime	Size (bits)	Instantiations
BN254 Scalar	`0x30644e72e131a029b85045b68181585d2833e84879b9709143e1f593f0000001`	254	e.g. Groth16 [BBi18]
BLS12-381 Scalar	`0x73eda753299d7d483339d80809a1d80553bda402fffe5bfeffffffff00000001`	255	e.g. Varuna [WCC+23]
Felt252	$2^{251} + 17 \cdot 2^{192} + 1$	252	e.g. STARK [GPR21]
Goldilocks	$2^{64} - 2^{32} + 1$	64	e.g. Plonky2 [Pol22]
BabyBear	$2^{31} - 2^{27} + 1$	31	e.g. Plonky3 [Pol24]
KoalaBear	$2^{31} - 2^{24} + 1$	31	
Mersenne31	$2^{31} - 1$	31	

2.4 Traditional Cryptographic Hash Functions Are Not ZK-friendly

We call hashes designed without prime field arithmetization in mind, such as SHA-256, SHA-3, and BLAKE3, *traditional*. Their cost in a proof is not a property of the hash alone: it depends on the arithmetization, the base field, available custom gates and lookups, and the proof-system backend. Nevertheless, in the prime-field R1CS, PLONK, and AIR models of Section 2.2, such designs often incur a significant overhead due to the mismatch of their native operations and the available field operations.

For example, for already constrained $x, y \in \{0, 1\} \subset \mathbb{F}_p$, where p is odd,

$$\mathrm{AND}(x, y) = xy, \qquad \mathrm{OR}(x, y) = x + y - xy, \qquad \mathrm{XOR}(x, y) = x + y - 2xy,$$
$$\mathrm{NOT}(x) = 1 - x.$$

Booleanity itself is nonlinear and must be enforced for every input by $b(b - 1) = 0$. Even in arithmetization styles that can fold linear operations into adjacent multiplicative constraints (i.e., such as R1CS or AIR) each binary operation costs a full constraint, including linear operations.

The larger cost comes from composing these operations across an entire word-oriented design. Bitwise AND, OR, and XOR require nonlinear relations, words must be decomposed and recomposed, modular word additions require carries together with Booleanity or range constraints, and shifts require truncation and fixed-zero conditions. While all of these operations can be optimized in detail using e.g., wider custom gates, lookups or better encoding strategies, the tailored design of prime-field AO hashes just generally leads to much simpler arithmetizations. Nevertheless, significant progress has been

made on optimizing traditional hash function arithmetizations. Examples include specialized hash and modular-arithmetic encodings [WOS+25] and lookup- or custom-gate techniques [ZBK+22, CBBZ23]. Recent work also applies a specialized sum-check/GKR construction to batched Keccak permutations [KKA+25]. Public BLAKE-family realizations include the BLAKE3 AIR [Pol24]. In the evaluated prime-field AIR and common backend, the Poseidon2 realization has substantially lower arithmetization cost and prover time per absorbed byte than the evaluated SHA-256, BLAKE3, and Keccak realizations.

Practical Evaluation. In Table 2, we use the public AIR arithmetizations of Plonky3 of several hash functions and one common backend configuration [Pol24]. The measured objects are batched BLAKE3 compressions with a 64-byte block size, batched SHA-256 compressions with a 64-byte block size, batched Keccak-f[1600] permutations with the 136-byte Keccak-256 sponge rate, and batched Poseidon2 permutations with a rate of 16 KoalaBear elements. These are AIR calls normalized and each row uses the largest batch not exceeding a 64 KiB target: 1 024 BLAKE3 calls (64.0 KiB), 1 024 SHA-256 calls (64.0 KiB), 341 Keccak-f[1600] calls (45.3 KiB), and 1 024 Poseidon2 calls (48.0 KiB), using a conservative encoding of three bytes per KoalaBear element. Padding, chaining, domain separation, sponge-mode constraints, and public input/output binding are excluded from the experiment. We selected Poseidon2, as it was an upstream implementation and refer the reader to Section 10.2.1, for a comparison of different AO hash functions. We report proving in ms/KiB and verification in ms/proof.

Table 2: Cost of the four AIR workloads in Plonky3 over KoalaBear with zero knowledge. All AIRs have maximum constraint degree three. Prove includes trace generation; verification is reported per packed proof, and cells/KiB uses the effective absorbed-byte rates above.

Construction	prove (ms/KiB)	verify (ms/proof)	cells/KiB
Keccak-f[1600]	102.9	25.4	475 799
BLAKE3	26.5	41.4	146 688
SHA-256	22.4	37.2	123 648
Poseidon2	1.3	12.0	5 099

Within the common AIR and backend configuration of Table 2, the cells/KiB column helps explain the prover-time gap and motivates the design of AO hash functions. These results make no claim of generality, beyond the simple fact that AO friendly hash functions permit simple arithmetizations and are thus more efficient when implemented with reasonably similar optimization effort within prime field proof systems.

3 Modes of Operation

In this section, we provide a structured overview of the modes of operation used in existing ZK-friendly hash functions. We categorize them into sponge-based constructions and their variants on the one hand, and compression functions – the building blocks of tree-based hashing – on the other. A summary of representative primitives and their supported modes is given in Table 25 of Appendix A.

3.1 Sponge Hash Function

The sponge construction is a versatile framework that transforms a fixed-length permutation into a function capable of processing arbitrary-length inputs and producing arbitrary-length

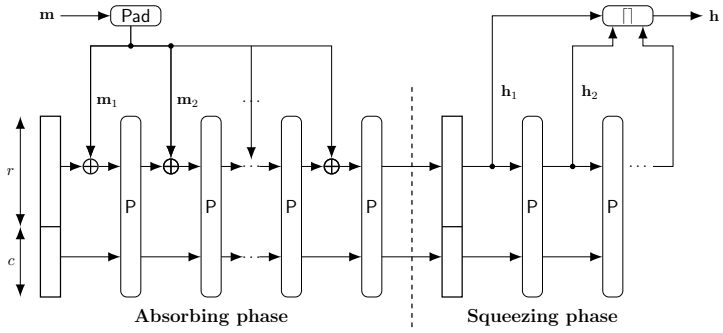


Figure 2: Sponge construction.

outputs. It is the foundation for modern hash functions like SHA-3 and many ZK-friendly primitives.

3.1.1 "Original" Sponge Hash Function

We start by recalling the original sponge hash construction proposed by Bertoni et al. [BDPA07], here reformulated over a prime field $\mathbb{F}_p$. Let $t, r, c \in \mathbb{N}$ such that $t = r + c$, where t is the state size, r is the *rate*, and c is the *capacity*. The construction uses a cryptographic permutation $\mathsf{P} : \mathbb{F}_p^t \to \mathbb{F}_p^t$ on a t-element state and consists of the following two main phases (see Figure 2):

- *Absorbing phase:* First, the input message $\mathbf{m} \in \mathbb{F}_p^*$ is padded with 10^* until its length is a multiple of r. The padded message $\mathbf{m} \,\|\, 10^*$ is then partitioned into r-element blocks $\mathbf{m}_1, \mathbf{m}_2, \ldots$. Starting from an initial state $\mathbf{x} = 0^r \,\|\, \mathtt{IV}$, where $\mathtt{IV} \in \mathbb{F}_p^c$ denotes a fixed initialization vector, each message block $\mathbf{m}_i$ is added to the outer part of the state, followed by an application of P. More formally,

$$\forall i \geq 1\colon \qquad \mathbf{x} \mapsto \mathsf{P}(\mathbf{x} + (\mathbf{m}_i \,\|\, 0^c))\,. \tag{11}$$

- *Squeezing phase:* After all message blocks are absorbed, the hash value $\mathbf{h} = \mathbf{h}_1 \,\|\, \mathbf{h}_2 \,\|\, \ldots$ is extracted from the outer state, applying P between outputs if more than r elements are required.

For a fixed-output-length hash function $\mathsf{H} : \mathbb{F}_p^* \to \mathbb{F}_p^d$, the squeezed output is simply truncated to the desired *digest* length d.

Security. The security of the sponge construction relies on the capacity c. As proven in [BDPV08],[11] it is indistinguishable from a random oracle up to $\mathcal{O}(p^{c/2})$ evaluations of the permutation P. In particular, it guarantees collision security up to $\min\{p^{d/2}, p^{c/2}\}$ queries, and pre-image as well as second pre-image security up to $\min\{p^d, p^{c/2}\}$ queries (see [AMP11, Appendix A]). A tighter bound on pre-image security is up to $\min\left\{p^d, \max\{p^{c/2}, p^{d-r}\}\right\}$ queries (see [LM22]).

3.1.2 Sponge Variants

Several variants of the sponge hash function have been proposed in the literature in order to maximize its performance. As a concrete example, Naito and Ohta [NO14] exploited

[11] Until upon recently, security proofs for sponge functions were done over $\mathbb{F}_2$ only. However, the arguments are combinatorial in nature only, so they can be generalized to arbitrary finite fields $\mathbb{F}_q$ in a straightforward fashion.

the limited control an adversary has over the initial value and squeeze phase to improve the sponge construction. In particular, they show that it is possible to increase the size of the rate of the first absorption up to roughly $r + c/2$, and the squeezing can be performed at, again, roughly $r + c/2$ at a time, without security degradation.

Here, we limit ourselves to focus on the SAFE and on the sponge-pi variants useful in the context studied in this paper.

SAFE/SAFECore. The SAFE variant [AKMQ23, KBM23] is an API framework for sponge-based hash functions tailored to prime-field-based protocols. For the goal of this paper, we limit ourselves to mention that the main difference with the "original" sponge construction concerns the padding phase:

- Let $\ell = |\mathbf{m}|$ denote the length of the message over $\mathbb{F}_p$, assumed to be known in advance, and let $\ell_1, \ell_2, ..., \ell_\lambda \in \{1, 2, \ldots, r\}$ such that $\sum_{i=1}^{\lambda} \ell_i = \ell$;
- Split the message into λ blocks $\mathbf{m}_i \in \mathbb{F}_p^{\ell_i}$ and zero-pad each block individually to rate size, $\mathbf{m}_i \,\|\, 0^{r-\ell_i} \in \mathbb{F}_p^r$;
- The initialization vector $\mathtt{IV}$ is given as a function of the block lengths $\ell_1, \ldots, \ell_\lambda$.

We refer to [AKMQ23, KBM23] for a detailed description.

sponge-pi. sponge-pi [LBM25], proposed by Lefevre et al., is a sponge variant that employs block-length-preserving padding, originally introduced in [Hir18]). Given a message $\mathbf{m} \in \mathbb{F}_p^\ell$, the padding proceeds as follows:

- if ℓ is not a multiple of the rate r, the message is padded with 10^* to complete the final rate block; this does not introduce an additional permutation evaluation;
- if ℓ is a multiple of the rate r, no padding is applied. Instead, the final absorption call is domain-separated by applying a non-cryptographic permutation to the inner part of the state; in the concrete construction, this amounts to adding a fixed non-zero constant.

3.2 Tree Hash and Compression Functions

In addition to sponge-based constructions, another widely used approach to build hash functions from fixed-size primitives is based on tree hashing and compression functions. In ZK-friendly settings over prime fields $\mathbb{F}_p$, tree-based modes are particularly attractive when hashing large inputs or when parallelism is desired.

3.2.1 Tree-Hash Function

A tree-hash function [BDPA14, DMV18] is a mode of operation that evaluates a compression function over a tree structure, typically in a bottom-up manner. In the following, we consider a binary tree hash, i.e., a tree of arity $a = 2$ (cf. Figure 3). Let

$$\mathsf{C}_{\text{leaf}}, \mathsf{C}_{\text{node}}, \mathsf{C}_{\text{root}} : \mathbb{F}_p^{2d} \to \mathbb{F}_p^d$$

be three independent compression functions, used for the leaf, internal-node, and root layers, respectively.[12] Given a message $\mathbf{m} \in \mathbb{F}_p^*$, the tree hash is evaluated as follows:

[12] These three independent compression functions can be set up via a single compression function and domain separation. For example, consider the case $p = 2$. Given $\mathsf{C} : \mathbb{F}_2^{2d+2} \to \mathbb{F}_2^d$, let $\mathsf{C}_{\text{leaf}}(\cdot) := \mathsf{C}(\cdot \,\|\, 00)$, $\mathsf{C}_{\text{node}}(\cdot) := \mathsf{C}(\cdot \,\|\, 10)$, and $\mathsf{C}_{\text{root}}(\cdot) := \mathsf{C}(\cdot \,\|\, 11)$.

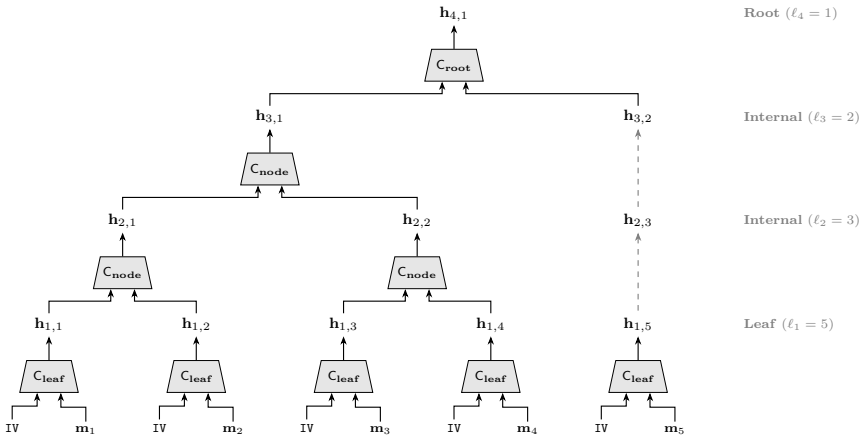


Figure 3: Example of the binary tree-hash construction.

- *Leaf Layer:* First, the input message $\mathbf{m}$ is padded with 10^* until its length is a multiple of d. The padded message $\mathbf{m} \,\|\, 10^*$ is then partitioned into ℓ_1 blocks $\mathbf{m}_1, \mathbf{m}_2, \ldots, \mathbf{m}_{\ell_1} \in \mathbb{F}_p^d$. Given an initial value $\mathtt{IV} \in \mathbb{F}_p^d$, compute

$$\forall 1 \leq i \leq \ell_1 : \qquad \mathbf{h}_{1,i} \leftarrow \mathsf{C}_{\mathrm{leaf}}(\mathtt{IV}, \mathbf{m}_i)\,; \tag{12}$$

- *Internal Node Layers:* For each layer $j > 1$, let ℓ_{j-1} denote the number of nodes in the previous layer. While $\ell_{j-1} > 2$, pair adjacent nodes and compute

$$\forall 1 \leq i \leq \left\lfloor \frac{\ell_{j-1}}{2} \right\rfloor : \qquad \mathbf{h}_{j,i} \leftarrow \mathsf{C}_{\mathrm{node}}\big(\mathbf{h}_{j-1,2i-1}, \mathbf{h}_{j-1,2i}\big)\,. \tag{13}$$

  If ℓ_{j-1} is odd, the final unpaired node is propagated unchanged; the number of nodes in layer j is given by $\ell_j = \left\lceil \frac{\ell_{j-1}}{2} \right\rceil$.

- *Root Layer:* Once only two nodes remain at layer j, compute

$$\mathbf{h}_{j+1,1} \leftarrow \mathsf{C}_{\mathrm{root}}(\mathbf{h}_{j,1}, \mathbf{h}_{j,2})\,, \tag{14}$$

  where $\mathbf{h} := \mathbf{h}_{j+1,1} \in \mathbb{F}_p^d$ is the hash value.

Security. The security of tree-based hashing modes has been studied in the indifferentiability framework by Bertoni et al. [BDPA14]. They established that if a tree hashing mode H_T satisfies three sufficient conditions – namely *tree-decodability*, *message-completeness*, and *final-node separability* – the resulting hash function is indifferentiable from an ideal monolithic random oracle. More precisely, the tree hash function is indistinguishable from a random oracle up to $\mathcal{O}(p^{d/2})$ queries made to the underlying compression functions $\mathsf{C}_{\mathrm{leaf}}$, $\mathsf{C}_{\mathrm{node}}$, $\mathsf{C}_{\mathrm{root}}$. In particular, it guarantees collision security up to $p^{d/2}$ queries, and pre-image as well as second pre-image security up to p^d queries.

Tree-Hash Functions in ZK-Protocols. For completeness, we note that in many ZK protocols, tree-hash functions are applied to data of *strictly fixed and known length* [GM20]. Consequently, the tree structure is predetermined, which implies that

- no initialization vector $\mathtt{IV}$ is required, and
- a single compression function suffices, that is, there is no need to distinguish between $\mathsf{C}_{\mathrm{leaf}}$, $\mathsf{C}_{\mathrm{node}}$, and $\mathsf{C}_{\mathrm{root}}$.

3.2.2 Compression Functions

A compression function of arity a combines $a \geq 2$ inputs, each consisting of d field elements, into a single d-element output,

$$\mathsf{C} : \left(\mathbb{F}_p^d\right)^a \to \mathbb{F}_p^d . \tag{15}$$

Compression functions form the basic building blocks of tree-hash constructions.

Several strategies can be used to construct a compression function. One option is to instantiate compression via a fixed-input sponge with capacity $c = d$, as done by many early AO designs. To compress ad input elements in a single absorption call, this requires a permutation of state size $t = ad+d$; alternatively, a narrower permutation requires multiple permutation evaluations. In either case, the inner part carries no input and is therefore unused for compression. A more efficient alternative uses the full permutation state of size $t = ad$, combining a single permutation evaluation with a feed-forward operation and a linear map that compresses the state to d elements.

Given a permutation $\mathsf{P} : \mathbb{F}_p^t \to \mathbb{F}_p^t$ with $t = ad$ and a matrix $\in \mathbb{F}_p^{d\times t}$, the compression function $\mathsf{C} : \mathbb{F}_p^t \to \mathbb{F}_p^d$ is defined as

$$\mathbf{x} \mapsto \mathsf{C}(\mathbf{x}) = \mathbf{M} \times \left(\mathsf{P}(\mathbf{x}) + \mathbf{x}\right). \tag{16}$$

Two main instantiations of $\mathbf{M}$ appear in the literature, yielding the following *compression modes*:

- Trunc_d [GKS23a]: truncates the feed-forward sum to its first d field elements, i.e. $\mathbf{M} = \begin{bmatrix}\mathbf{I}_d & \mathbf{0}_{d\times(t-d)}\end{bmatrix}$.
- $\texttt{Jive}_a$ [BBC+23, §3.2]: partitions the feed-forward sum into a blocks of d field elements and sums them coordinate-wise, i.e. $\mathbf{M} = \begin{bmatrix}\mathbf{I}_d & \dots & \mathbf{I}_d\end{bmatrix}$ (a copies).

Security. Let P be an ideal permutation, and $\mathbf{M} \in \mathbb{F}_p^{d\times t}$ a right-invertible matrix.[13] The compression mode just defined guarantees collision resistance up to $p^{d/2}$ queries and (second-)preimage resistance up to p^d queries [PGV94, BRS02, TABR26].

4 Overview of Algebraic Cryptanalytic Techniques

To attack a compression or hash function $\mathsf{H} : \mathbb{F}_q^* \to \mathbb{F}_q^d$ an adversary has to solve one of the following three problems.

Preimage: Given $\mathbf{h} \in \mathbb{F}_q^d$ find $\mathbf{x} \in \mathbb{F}_q^*$ such that $\mathsf{H}(\mathbf{x}) = \mathbf{h}$.

Second-Preimage: Given $\mathbf{y} \in \mathbb{F}_q^*$ find $\mathbf{x} \in \mathbb{F}_q^*$ such that $\mathsf{H}(\mathbf{x}) = \mathsf{H}(\mathbf{y})$.

Collision: Find $\mathbf{x}, \mathbf{y} \in \mathbb{F}_q^*$ such that $\mathbf{x} \neq \mathbf{y}$ and $\mathsf{H}(\mathbf{x}) = \mathsf{H}(\mathbf{y})$.

For security level 2^κ, a valid attack on one of these problems must perform *at most* 2^κ queries to H as well as have computational complexity *at most* equal to 2^κ. An overview of the attacks proposed in the literature against ZK-friendly hash functions is proposed in Figure 4.

This section provides an overview of the algebraic techniques used to tackle these problems for ZK-friendly hash functions. We restrict our attention to algebraic attacks, as it is the current consensus in the community that they pose the most serious threat to ZK-friendly primitives, and it is where most cryptanalytic progress has been made in

[13] A matrix $\mathbf{M} \in \mathbb{F}_p^{d\times t}$ is right-invertible if there exists $\mathbf{M}' \in \mathbb{F}_p^{t\times d}$ such that $\mathbf{M}\mathbf{M}' = \mathbf{I}_d$.

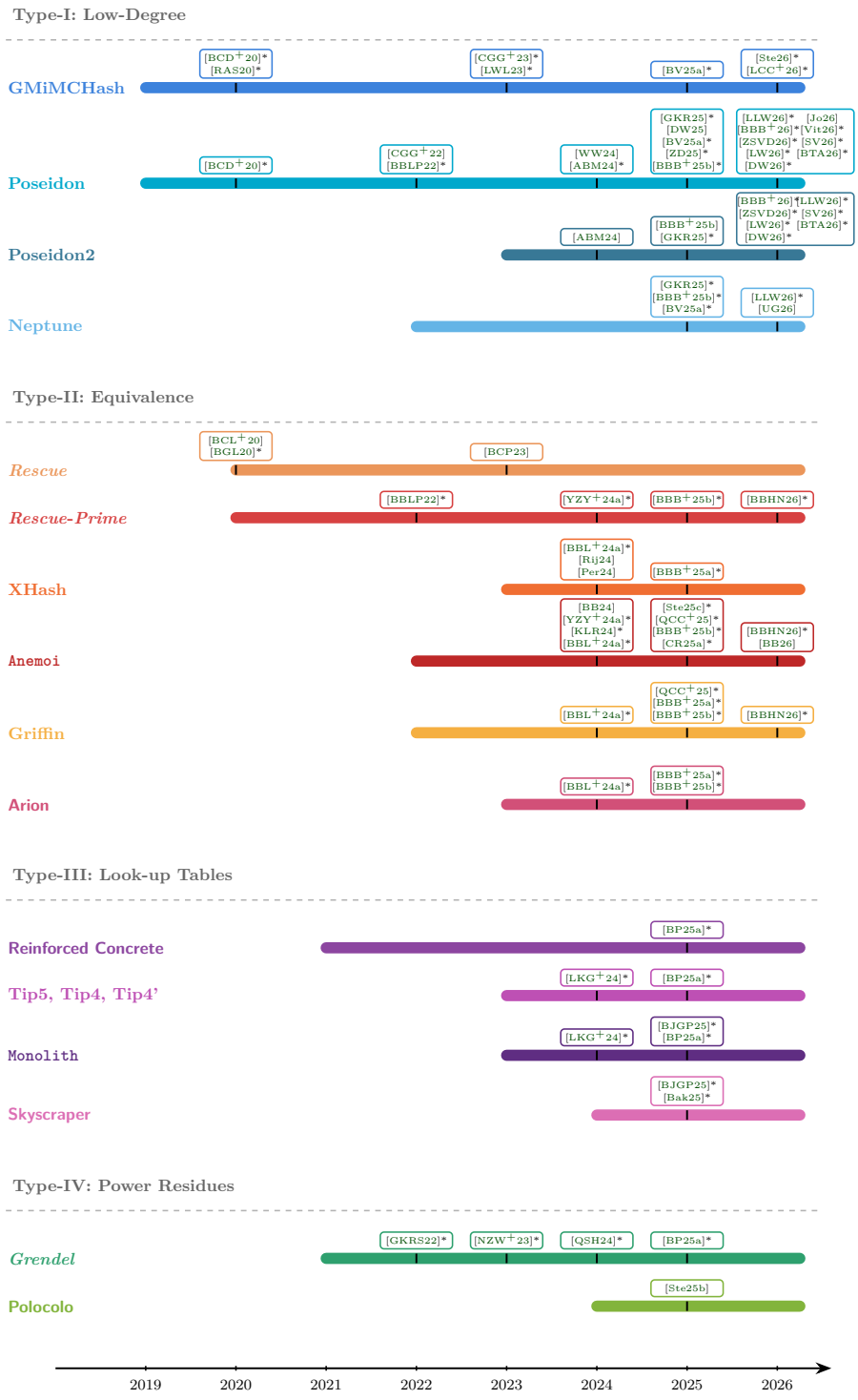


Figure 4: Cryptanalysis timeline (* denotes papers related to algebraic attacks). We omit GMiMCHash2 and pSquare-hash as no third-party cryptanalysis exists for them.

recent years. Moreover, a specific algebraic attack often applies to many members of a category at once – see, for example, the FreeLunch approach (Section 4.3.3) for Type-II primitives. Statistical attacks such as differential cryptanalysis have also been applied successfully, but they tend to be primitive-specific and generalize poorly across a category; we therefore report progress on these attacks within the individual primitive sections.

Remark 1. Throughout, we formulate the presented techniques over general finite fields $\mathbb{F}_q$, rather than specializing to prime fields $\mathbb{F}_p$, so that this section may also serve as a self-contained reference on algebraic cryptanalysis.

Notation. We write $\mathcal{O}(\cdot)$ for the usual Landau notation and $\tilde{\mathcal{O}}(\cdot)$ for its soft-O variant, which additionally suppresses factors polylogarithmic in the argument.

4.1 The Constrained-Input Constrained-Output Problem

The *Constrained-Input Constrained-Output* (CICO) problem was originally introduced by the Keccak team alongside the sponge construction [BDPVA11a, §8.2.4]. It provides a simple algebraic framework for sponge functions which encompasses the second-preimage as well as the collision-finding problem.

Definition 1 (CICO-(i, o) Problem)**.** Let $\mathbb{F}_q$ be a finite field, let $\mathsf{P} : \mathbb{F}_q^t \to \mathbb{F}_q^t$ be a permutation, let $1 \leq i, o < t$ such that $i + o \leq t$, and let $\mathbf{c}^{\text{in}} \in \mathbb{F}_q^i$ and $\mathbf{c}^{\text{out}} \in \mathbb{F}_q^o$. The CICO-$(i, o)$ problem asks to find $\mathbf{x} \in \mathbb{F}_q^{t-i} \times \{\mathbf{c}^{\text{in}}\}$ such that $\mathsf{P}(\mathbf{x}) \in \mathbb{F}_q^{t-o} \times \{\mathbf{c}^{\text{out}}\}$.

The parameters i and o denote the number of constrained input and output coordinates, respectively. When $i = o = k$, we simply write CICO-k. For a random permutation P, the expected number of solutions to the CICO-(i, o) problem is q^{t-i-o}.

Next, we discuss how a CICO solution can yield a collision or a second-preimage. Let c denote the capacity, r the rate and d the digest size in field elements. In practice, it is customary to set the initial value to $\mathtt{IV} = \mathbf{0} \in \mathbb{F}_q^c$.

Collisions. Assume that $t > 2c$. Then, the expected number of solutions to a CICO-c instance is $q^{t-2c} > 1$. Let $\mathbf{x}, \mathbf{y} \in \mathbb{F}_q^r$ be two distinct solutions to the same CICO-c instance with input constraint $\mathbf{c}^{\text{in}} = \mathtt{IV}$ and arbitrary output $\mathbf{c}^{\text{out}} = \mathbf{z}$. Then $\mathsf{P}(\mathbf{x} \,\|\, \mathtt{IV}) = \mathbf{u} \,\|\, \mathbf{z}$ and $\mathsf{P}(\mathbf{y} \,\|\, \mathtt{IV}) = \mathbf{v} \,\|\, \mathbf{z}$ with the same inner part $\mathbf{z} \in \mathbb{F}_q^c$ and outer parts $\mathbf{u}, \mathbf{v} \in \mathbb{F}_q^r$. Choosing a second pair of blocks $\mathbf{m}_x, \mathbf{m}_y \in \mathbb{F}_q^r$ such that $\mathbf{u} + \mathbf{m}_x = \mathbf{v} + \mathbf{m}_y$ equalizes the outer part, yielding a full-state collision for the sponge function based on P.

Second-Preimages. Assume that the target message consists of at least three blocks $\mathbf{m}_1, \mathbf{m}_2, \mathbf{m}_3 \in \mathbb{F}_q^r$, absorbed in this order, and write $\mathsf{P}(\mathsf{P}(\mathbf{m}_1 \,\|\, \mathtt{IV}) + (\mathbf{m}_2 \,\|\, \mathbf{0})) = \mathbf{u} \,\|\, \mathbf{a}$, where $\mathbf{u} \in \mathbb{F}_q^r$ and $\mathbf{a} \in \mathbb{F}_q^c$ denote the outer and inner parts after the second absorption step, respectively. Now suppose that $\mathbf{x} \in \mathbb{F}_q^r$ is a solution to the CICO-c problem with input constraint $\mathbf{c}^{\text{in}} = \mathtt{IV}$ and output constraint $\mathbf{c}^{\text{out}} = \mathbf{a}$, i.e., $\mathsf{P}(\mathbf{x} \,\|\, \mathtt{IV}) = \mathbf{v} \,\|\, \mathbf{a}$. Choosing a second block $\mathbf{y} \in \mathbb{F}_q^r$ such that $\mathbf{v} + \mathbf{y} = \mathbf{u} + \mathbf{m}_3$ equalizes the outer parts in the next absorption step, yielding a full-state collision with the target state obtained upon absorbing $\mathbf{m}_3$.

Preimages. The preimage problem can be viewed as a variant of the CICO-(c, d) problem in which the output constraint is applied to the first, rather than the last, d coordinates. Assuming $d \leq r$, the input is constrained by $\mathbf{c}^{\text{in}} = \mathtt{IV}$ as usual, while the output is required to lie in $\{\mathbf{c}^{\text{out}}\} \times \mathbb{F}_q^{t-d}$, where $\mathbf{c}^{\text{out}} = \mathbf{h} \in \mathbb{F}_q^d$ is the target digest. Any solution yields a preimage of $\mathbf{h}$.

4.2 Polynomial System Solving Techniques

Since any function over a finite field can be represented with polynomials, the CICO-(i, o) problem for a permutation $\mathsf{P} : \mathbb{F}_q^t \to \mathbb{F}_q^t$ can be naturally expressed as a polynomial system

$$f_j(x_1, \ldots, x_{t-i}) = \mathbf{c}_j^{\text{out}}, \qquad 1 \leq j \leq o. \tag{17}$$

Depending on the configuration (i, o), an attacker can employ various techniques such as univariate factorization, resultants or Gröbner bases to solve the system.

We first recall Gröbner bases (Section 4.2.1) since they offer the most general treatment of polynomial system solving, then we recall the resultant (Section 4.2.2), which has been used effectively to construct univariate polynomials in CICO-1 problems of Type-II primitives, and finally we recall how univariate polynomials can be efficiently factored over finite fields (Section 4.2.3).

4.2.1 Gröbner Bases

Gröbner bases are a generic toolkit to treat computational problems of polynomial ideals. They were introduced by Buchberger in his PhD thesis [Buc65]; for a general introduction to the theory we refer the reader to [CLO15]. In the following, let $\mathbb{K}$ be a field, and let $P = \mathbb{K}[x_1, \ldots, x_n]$ denote the polynomial ring in n variables.

Definition 2 (Monomial order)**.** Identify a monomial $x^{\mathbf{a}} = \prod_{i=1}^n x_i^{a_i}$ with its integer exponent vector $\mathbf{a} \in \mathbb{Z}_{\geq 0}^n$. A *monomial order* $>$ on P is a binary relation on the exponent vectors such that

(i) $>$ is a total order on $\mathbb{Z}_{\geq 0}^n$;

(ii) for $\mathbf{a}, \mathbf{b}, \mathbf{c} \in \mathbb{Z}_{\geq 0}^n$, if $\mathbf{a} > \mathbf{b}$ then $\mathbf{a} + \mathbf{c} > \mathbf{b} + \mathbf{c}$; and

(iii) $>$ is a well-ordering on $\mathbb{Z}_{\geq 0}^n$, i.e. every non-empty subset has a least element.

Let $\mathbf{a}, \mathbf{b} \in \mathbb{Z}_{\geq 0}^n$. We recall some standard monomial orders:

- *Lexicographic* (LEX): $\mathbf{a} >_{\text{LEX}} \mathbf{b}$ if the leftmost non-zero entry of $\mathbf{b} - \mathbf{a}$ is negative.
- *Reverse lexicographic* (RLEX): $\mathbf{a} >_{\text{RLEX}} \mathbf{b}$ if the rightmost non-zero entry of $\mathbf{b} - \mathbf{a}$ is negative.
- *Degree reverse lexicographic* (DRL): $\mathbf{a} >_{\text{DRL}} \mathbf{b}$ if either $\sum_{i=1}^n a_i > \sum_{i=1}^n b_i$ or $\sum_{i=1}^n a_i = \sum_{i=1}^n b_i$ and $\mathbf{a} >_{\text{RLEX}} \mathbf{b}$.
- *Weighted* ($>_{\mathbf{w}}$): given a weight vector $\mathbf{w} \in \mathbb{Q}_{>0}^n$ and a monomial order $>$, we have $\mathbf{a} >_{\mathbf{w}} \mathbf{b}$ if either $\mathbf{w}^\intercal \mathbf{a} > \mathbf{w}^\intercal \mathbf{b}$ or $\mathbf{w}^\intercal \mathbf{a} = \mathbf{w}^\intercal \mathbf{b}$ and $\mathbf{a} > \mathbf{b}$. Note that taking $\mathbf{w} = (1, \ldots, 1)^\intercal$ and $>_{\text{RLEX}}$ recovers DRL.

Given $f \in P \setminus \{0\}$, we sort the monomials of f via $>$ and call the largest one the *leading monomial* of f, denoted $\text{LM}_>(f)$. This allows us to define a $>$-Gröbner basis of an ideal[14], which can be viewed as a special generating set adapted to the chosen monomial order. In the following, we fix a monomial order $>$ on P and suppress it from the notation.

Definition 3 (Gröbner basis)**.** Let $I \subset P$ be an ideal and let $G \subset I$ be a finite generating set, i.e. $\langle G \rangle = I$. Then G is a*Gröbner basis* of I if

$$\left\langle \text{LM}(f) \;\middle|\; f \in I \right\rangle = \left\langle \text{LM}(g) \;\middle|\; g \in G \right\rangle. \tag{18}$$

In other words, the leading monomials of the elements of G generate all leading monomials arising from elements of I. We write the two leading-monomial ideals simply as $\langle \text{LM}(I) \rangle$ and $\langle \text{LM}(G) \rangle$, respectively.

[14] An ideal $I \subset P$ is a subset such that $0 \in I$, $\forall\, f \in I, g \in P$ also $f \cdot g \in I$, and $\forall\, f, g \in I$ also $f - g \in I$. For $f_1, \ldots, f_m \in P$, the ideal they generate is $\langle f_1, \ldots, f_m \rangle = \left\{ \sum_{i=1}^m g_i \cdot f_i \;\middle|\; g_i \in P \right\}$.

Given a Gröbner basis of a polynomial system, we can decide whether its number of solutions is finite.

Theorem 1 (Finiteness Theorem [CLO15, Chapter 5 §3 Theorem 6])**.** *Let $I \subset P = \mathbb{K}[x_1, \ldots, x_n]$ be an ideal and fix a monomial order on P. Consider the following statements:*

(1) For each $1 \leq i \leq n$ there is some $m_i \geq 0$ such that $x_i^{m_i} \in \langle \mathrm{LM}(I) \rangle$.

(2) Let G be a Gröbner basis of I. Then for each $1 \leq i \leq n$, there is some $m_i \geq 0$ such that $x_i^{m_i} = \mathrm{LM}(g)$ for some $g \in G$.

(3) The set $\left\{ x^{\mathbf{a}} \;\middle|\; x^{\mathbf{a}} \notin \langle \mathrm{LM}(I) \rangle \right\}$ is finite.

(4) The $\mathbb{K}$-vector space P/I is finite-dimensional.

(5) The variety $V(I) = \left\{ \mathbf{x} \in \mathbb{K}^n \;\middle|\; \forall\, f \in I \colon f(\mathbf{x}) = 0 \right\} \subset \mathbb{K}^n$ is finite.

Then (1)-(4) are equivalent and they all imply (5). Furthermore, if $\mathbb{K}$ is algebraically closed, then (1)-(5) are all equivalent.

In the following, let $\dim_{\mathbb{K}}(P/I)$ denote the dimension of the quotient ring P/I as a $\mathbb{K}$-vector space. If $\dim_{\mathbb{K}}(P/I) < \infty$, cf. *(3)* of Theorem 1, we call I *zero-dimensional* and refer to $D_I := \dim_{\mathbb{K}}(P/I)$ as the *ideal degree*.

Proposition 1 ([CLO15, Chapter 5 §3 Proposition 7])**.** *Let $I \subset P = \mathbb{K}[x_1, \ldots, x_n]$ be a zero-dimensional ideal. Then $|V(I)| \leq D_I$, and equality holds if $\mathbb{K}$ is algebraically closed and I is radical.*

Given a Gröbner basis G of a zero-dimensional ideal $I \subset P = \mathbb{K}[x_1, \ldots, x_n]$, we can directly determine D_I and thus estimate the number of solutions: a $\mathbb{K}$-vector space basis of P/I is given by the monomials not contained in $\langle \mathrm{LM}(G) \rangle = \langle \mathrm{LM}(G) \rangle$; D_I equals the number of such monomials. Moreover, the quotient-ring representation provides a way to also recover the solutions themselves.

Variety Computation. For any $g \in P$, the multiplication map $f \mapsto f \cdot g$ is $\mathbb{K}$-linear on P/I, and, with respect to a fixed basis of P/I, is represented by a *multiplication matrix* $\mathbf{M}_g \in \mathbb{K}^{D_I \times D_I}$.[15] For every point $\mathbf{x} \in V(I)$, the value $g(\mathbf{x})$ occurs as an eigenvalue of $\mathbf{M}_g$ [KR16, Proposition 6.2.1]. In particular, choosing $g = x_i$, the eigenvalues of $\mathbf{M}_{x_i}$ recover the i-th coordinates of the solutions. Since these eigenvalues are the roots of the characteristic polynomial of $\mathbf{M}_{x_i}$, the characteristic polynomials of $\mathbf{M}_{x_1}, \ldots, \mathbf{M}_{x_n}$ can be used to compute the points in $V(I)$. The application of this theory to cryptographic polynomial systems yields the following Gröbner basis attack.

Gröbner Basis Attack. Let $\mathcal{F} \subset \mathbb{F}_q[x_1, \ldots, x_n]$ model a cryptographic problem with a set of polynomials. Fix some monomial order $>$. Assuming that the ideal $I = \langle \mathcal{F} \rangle$ is zero-dimensional, proceed as follows:

(1) Compute a Gröbner basis of $I = \langle \mathcal{F} \rangle$.

(2) Compute the multiplication matrix $\mathbf{M}_{x_n}$.

(3) Compute the characteristic polynomial of $\mathbf{M}_{x_n}$.

(4) Factor the characteristic polynomial.

(5) If necessary, substitute $x_n = \alpha$ back into $\mathcal{F}$ and return to Step (1).

[15] Given a basis $\mathcal{B} = \{b_1, \ldots, b_{D_I}\}$ of P/I, the j-th column of $\mathbf{M}_g$ is obtained by multiplying b_j by g, reducing modulo I, and expressing the result again in the basis $\mathcal{B}$.

The complexity of the Gröbner basis attack is distributed across the individual steps as follows:

(1) Gröbner basis computation is very sensitive to the choice of $>$. However, with improved modeling techniques recent works [BBL+24a, BBB+25a, BP25b, Ste26] could trivialize this step for some primitives.

(2) The cost of constructing the multiplication matrix is often ignored in the cryptanalysis literature. Using generic techniques, it can be bounded by $\mathcal{O}(D_I^3)$ field operations [FGLM93].

(3) The characteristic polynomial can be computed using fast linear algebra. For example, Wiedemann's algorithm yields a complexity of $\widetilde{\mathcal{O}}(D_I^\omega)$ [Wie86, FGHR14], where $2 \leq \omega \leq 2.371552$ is a linear algebra constant [WXXZ24].

(4) Due to its particular importance for the CICO-1 problem, we discuss the complexity of univariate factorization over finite fields separately in Section 4.2.3.

Remark 2. In the cryptanalysis literature it is often proposed to perform monomial order conversion to a LEX Gröbner basis in Step (3). If an ideal is in x_n*-shape position*, i.e. if its LEX Gröbner basis is of the form

$$\left\{x_1 - f_1(x_n), \ldots, x_{n-1} - f_{n-1}(x_n), f_n(x_n)\right\}, \tag{19}$$

then the univariate polynomial $f_n(x_n)$ is the characteristic polynomial of $\mathbf{M}_{x_n}$ and can be computed via fast linear algebra [FGHR14, FM17]. The remaining LEX basis polynomials can likewise be recovered by linear algebra, making both approaches algorithmically equivalent up to logarithmic factors and implied constants.

4.2.2 Resultant

The resultant is an important tool in algebraic geometry and computer algebra. In the following, let $\mathbb{K}$ be a field, and let P denote a commutative ring with unity.

Given two univariate polynomials $f = \sum_{i=0}^{n} f_i \cdot x^i, g = \sum_{i=0}^{m} g_i \cdot x^i \in P[x]$, the *Sylvester matrix* of f and g with respect to x is defined as

$$\mathrm{Syl}(f,g,x) = \begin{pmatrix}
f_n & & & & g_m & & & \\
f_{n-1} & f_n & & & g_{m-1} & g_m & & \\
\vdots & \vdots & \ddots & & \vdots & \vdots & \ddots & \\
\vdots & \vdots & & & & \vdots & & \\
\vdots & \vdots & & f_n & g_1 & \vdots & & \\
\vdots & \vdots & & f_{n-1} & g_0 & \vdots & & \\
\vdots & \vdots & & \vdots & & g_0 & & g_0 \\
f_0 & \vdots & & \vdots & & & & \vdots \\
 & f_0 & & \vdots & & & & \vdots \\
 & & & f_0 & & & & g_0
\end{pmatrix} \in P^{(m+n)\times(m+n)}. \tag{20}$$

The *resultant* of f and g with respect to x is defined as the determinant of the Sylvester matrix, that is,

$$\mathrm{Res}(f,g,x) = \det\left(\mathrm{Syl}(f,g,x)\right). \tag{21}$$

Observe that the resultant is an element of the base ring P. If P is a field and $f, g \neq 0$, this has two important consequences:

Proposition 2 ([CLO15, Chapter 3 §6 Proposition 3 & 5]). *Given nonzero $f, g \in \mathbb{K}[x]$, the following holds:*

(i) $\operatorname{Res}(f, g, x) = 0$ *if and only if* $\gcd(f, g) \neq 1$.

(ii) There exist $A, B \in P[x]$ *such that* $\operatorname{Res}(f, g, x) = A \cdot f + B \cdot g$. *Moreover, if at least one of* f *and* g *is nonconstant, then the coefficients of* A *and* B *are integer polynomials in the coefficients of* f *and* g.

Property (ii) provides a direct mechanism for eliminating variables from a polynomial system $f_1 = \cdots = f_m = 0$ with $f_1, \ldots, f_m \in \mathbb{K}[x_1, \ldots, x_n]$.

Variable Elimination. To eliminate x_n, view $\mathbb{K}[x_1, \ldots, x_n]$ as the univariate polynomial ring in x_n over the coefficient ring $P = \mathbb{K}[x_1, \ldots, x_{n-1}]$, i.e. $\mathbb{K}[x_1, \ldots, x_n] = P[x_n]$. Choosing f_m as a pivot, compute the x_n-resultants $g_i = \operatorname{Res}(f_i, f_m, x_n) \in P$ for $1 \leq i < m$. This yields a reduced system $g_1 = \cdots = g_{m-1} = 0$ over $\mathbb{K}[x_1, \ldots, x_{n-1}]$, in which x_n has been eliminated. By construction, if $\mathbf{a} = (a_1, \ldots, a_n) \in \mathbb{K}^n$ is a solution of the original system, then $(a_1, \ldots, a_{n-1})$ is a solution of the reduced system. The converse need not hold, however, as resultant elimination may introduce solutions that do not lift to the original system.

Degree Growth and Complexity. In practice, the elimination of variables must be balanced against the resulting degree growth. If P is itself a polynomial ring and $f, g \in P[x]$, then the definition of the Sylvester matrix directly yields $\deg\big(\operatorname{Res}(f, g, x)\big) \leq \deg(f) \cdot \deg(g)$, and this bound is generically tight. Consequently, iteratively eliminating k variables can lead to a worst-case degree growth of d^{2^k}, where d bounds the degrees of the input polynomials. For systems in few variables, however, resultants can remain competitive with Gröbner basis methods. In particular, for bivariate polynomials $f, g \in \mathbb{F}_q[x, y]$, let $d_* = \max \deg_*(f), \deg_*(g)$ for $* \in x, y$ denote the maximum degree in the variable $*$. Then the resultant can be computed in $\tilde{\mathcal{O}}(d_x \cdot d_y)$ field operations [vdHL21, Vil25].

Resultant Attacks on AO Primitives. Resultant-based elimination is particularly attractive when the number of variables is small and/or the intermediate degree growth can be kept under control. In the cryptanalysis of AO primitives, Beyne et al. [BCD+20] first used resultants to solve bivariate systems arising in algebraically controlled differential attacks on GMiMCHash. See also Section 5.1.1.

Yang et al. [YZY+24a] developed the first resultant-based method for solving the CICO-1 problem of the Type-II primitives `Anemoi` and *Rescue-Prime* with S-box exponent $\alpha = 3$. They eliminate the round variables of the forward model using iterated resultants, interleaved with a cubic substitution that keeps all but two variables in each intermediate polynomial at degree at most two. This avoids the doubly exponential degree growth of naive iterated elimination and keeps the degree of the final univariate polynomial at the ideal degree D_I. Additionally, for *Rescue-Prime* they introduce a start-from-the-middle (SFTM) modeling of the CICO-1 problem, fixing all but two coordinates of an intermediate state and propagating the two remaining symbolic variables both forward and backward. The input and output constraints then yield two bivariate equations, splitting the length-R elimination chain into two chains of roughly half the length, and the degrees of the resulting bivariate polynomials multiply to the ideal degree D_I. As forming the final resultant directly is infeasible, the univariate polynomial – whose roots are the CICO-1 solutions – is reconstructed from D_I evaluations using fast Lagrange interpolation, at a cost of approximately $D_I^{1+\omega/2}$. See also Sections 6.1.1 and 6.2.

Bariant et al. [BBB+25b] generalized this construction to further Type-II primitives and replaced Yang et al.'s cubic substitution with a dedicated polynomial-division routine

applied after each resultant computation, thereby controlling the degrees of the intermediate polynomials for arbitrary α. For the complexity analysis, they adopt the weighted monomial order of the FreeLunch attack [BBL+24a] and show that the weighted degree does not increase under their reductions, which lets them rigorously bound every stage of the attack and identify the iterated resultants as the dominant cost, with total complexity only a factor $(2-1/\alpha)^n$ above the ideal degree D_I, where n is the number of variables to eliminate. Using this approach, they broke full-round instances of `Anemoi`, Arion, and Griffin in the CICO-1 setting. See also Sections 6.1.1 and 6.2 to 6.4.

Attacking the CICO-k problems via the resultant is still an active area of research. Most recently, Bak et al. [BBHN26] generalized the elimination procedure of [BBB+25b] to several free variables. They model CICO-1 from the middle (SFTM) and CICO-2 in the forward direction – both leaving two free variables – and compute the final bivariate resultant via Villard's algorithm [Vil25] instead of by interpolation, yielding the first full-round CICO-2 break of `Anemoi`. See also Sections 6.1.1, 6.2 and 6.3.

4.2.3 Univariate Root-Finding

In the final step of a Gröbner basis or resultant attack we arrive at a univariate polynomial $f \in \mathbb{F}_q[x]$ with $d = \deg(f)$. We could directly attempt to factor f, however in attacks on hash functions we are only interested in the roots of f that lie in the base field $\mathbb{F}_q$. We can filter these roots by first computing $g = \gcd(f, x^q - x)$ followed by factorization of g.

For the complexity of the GCD computation, we distinguish between the cases $d > q$ and $d \leq q$:

- If $d > q$, we directly apply a fast GCD algorithm, such as Moenck's algorithm [Moe73], with complexity

$$\mathcal{O}\Big(d \cdot \log_2(d)^2 \cdot \log_2\big(\log_2(d)\big)\Big) \tag{22}$$

  field operations.

- If $d \leq q$, Bariant et al. [BBLP22, §3.1] propose an improved routine. The first Euclidean step requires $r = x^q - x \bmod f$. Rather than computing this remainder naively by a division of cost $\tilde{\mathcal{O}}(q)$ field operations, one computes $x^q \bmod f$ using modular square-and-multiply, reducing intermediate products modulo f whenever their degrees exceed d. This requires

$$\mathcal{O}\Big(d \cdot \log_2(q) \cdot \log_2(d) \cdot \log_2\big(\log_2(d)\big)\Big) \tag{23}$$

  field operations. Combining this with the cost of computing $\gcd(f, r)$ from Equation (22) yields a total complexity of

$$\mathcal{O}\Big(d \cdot \log_2(d) \cdot \log_2\big(\log_2(d)\big) \cdot \big(\log_2(q) + \log_2(d)\big)\Big). \tag{24}$$

For the factorization of g, one may use the Kaltofen-Shoup algorithm [KS98] with complexity $\mathcal{O}\big(e^{1.815} \cdot \log_2(q)\big)$, where $e = \deg(g)$. Heuristically, however, g is expected to have only few roots in the base field $\mathbb{F}_q$, making the factorization cost negligible in practice. Accordingly, the cryptographic literature typically estimates the complexity of univariate root-finding by the preceding GCD complexity.

4.3 Algebraic Models of Arithmetization-Oriented Primitives

At the time of writing, all AO hash functions $\mathsf{H} : \mathbb{F}_q^* \to \mathbb{F}_q^d$ are permutation-based sponge constructions. The underlying permutation $\mathsf{P} : \mathbb{F}_q^t \to \mathbb{F}_q^t$ is obtained by iterating "simple"

round functions $\mathsf{R}_i : \mathbb{F}_q^t \to \mathbb{F}_q^t$ for R rounds, i.e., $\mathsf{P}(\mathbf{x}) = \mathsf{R}_R \circ \cdots \circ \mathsf{R}_1(\mathbf{x})$. This gives rise to two simple algebraic models for solving a CICO problem (cf. Definition 1), where $\mathbf{x}^{\text{in}}$ and $\mathbf{x}^{\text{out}}$ denote the input and output variable vectors, and $\mathbf{c}^{\text{in}}$ and $\mathbf{c}^{\text{out}}$ the corresponding constraint constants.[16]

- A high-degree model which represents the full function as

$$\mathsf{R}_R \circ \cdots \circ \mathsf{R}_1 \begin{pmatrix} \mathbf{x}^{\text{in}} \\ \mathbf{c}^{\text{in}} \end{pmatrix} = \begin{pmatrix} \mathbf{x}^{\text{out}} \\ \mathbf{c}^{\text{out}} \end{pmatrix}. \tag{25}$$

  Since the roots for $\mathbf{x}^{\text{out}}$ are fully parametrized by the roots of $\mathbf{x}^{\text{in}}$, we can in practice reduce to the polynomials that live in $\mathbb{F}_q[\mathbf{x}^{\text{in}}]$.

- A low-degree model introducing intermediate state variables $\mathbf{x}^{(i)}$ after each round:

$$\mathsf{R}_1 \begin{pmatrix} \mathbf{x}^{\text{in}} \\ \mathbf{c}^{\text{in}} \end{pmatrix} = \mathbf{x}^{(1)}, \qquad \mathsf{R}_i \left(\mathbf{x}^{(i-1)} \right) = \mathbf{x}^{(i)}, \qquad \mathsf{R}_R \left(\mathbf{x}^{(R-1)} \right) = \begin{pmatrix} \mathbf{x}^{\text{out}} \\ \mathbf{c}^{\text{out}} \end{pmatrix}, \tag{26}$$

  where $2 \leq i \leq R-1$.

The art of algebraic attacks lies in identifying clever transformations of these models that reduce the complexity of constructing univariate polynomials to solve for either $\mathbf{x}^{\text{in}}$ or $\mathbf{x}^{\text{out}}$.

In this section, we review several prominent techniques for reducing the complexity of the simple algebraic CICO models. In Section 4.3.1, we discuss SPN round-skipping, which allows rounds to be omitted in CICO-(i, o) problems for SPN constructions with monomial S-boxes and $t > i + o$. In Section 4.3.2, we explain how invariant subspaces can be used to linearize rounds with partial substitution layers. In Section 4.3.3, we review the FreeLunch model, a transformation of the iterative model in Equation (26) for Type-II primitives that yields Gröbner bases below the claimed security level. Finally, in Section 4.3.4, we consider a change-of-coordinates technique for computing low-degree DRL Gröbner bases, which can trivialize the Gröbner basis construction step in algebraic attacks on Feistel designs.

4.3.1 Skipping Rounds in Substitution-Permutation Networks

Suppose we are given a CICO-(c, d) problem with state size $t > c + d$, capacity $c \geq 1$, and output size $d \geq 1$; cf. Section 4.1. Then, there are $t - c$ free input variables but only d output constraints, so the corresponding polynomial system is underdetermined. We may therefore fix $t - c - d$ variables to obtain a fully determined system. The naive approach is to assign arbitrary constants to variables directly at the input. By exploiting the structure of the primitive, however, a more careful choice can reduce the complexity of the resulting system. In particular, for SPN schemes with monomial S-boxes $\mathsf{S}(x) = x^\alpha$, the variables can be fixed in such a way that one or more of the first rounds can be skipped. Such guessing strategies are therefore referred to as *round-skipping techniques*.

The first round-skip is due to Bariant et al. [BBLP22, §4.2], who exploited the structure of Poseidon and *Rescue-Prime* in the CICO-1 setting to skip the first two rounds. Since these constructions originally had no initial mixing layer, the first S-box layer could be crossed for free. The state after the next S-box layer is then restricted to a one-dimensional affine subspace whose backward propagation satisfies the input constraints. Assuming an initial mixing layer, Bak and Perrin [BP25a, Appendix D] generalized this technique from CICO-1 to CICO-(c, d): if $t \geq c + d \cdot (c + s + 1)$, the output of the first round can be restricted to a d-dimensional affine subspace, while additionally forcing s S-boxes in the second round to be inactive. For $c = d$ and $s = 0$, the condition simplifies to $t \geq c^2 + 2c$. In particular, for $c = 1$, one SPN round can be skipped whenever $t \geq 3$. Due to the importance of this generalized round-skip technique, we recapitulate it below.

[16] Depending on the mode (compression or sponge) and problem (preimage, CICO, etc.) some polynomials as well as input/output states have to be modified.

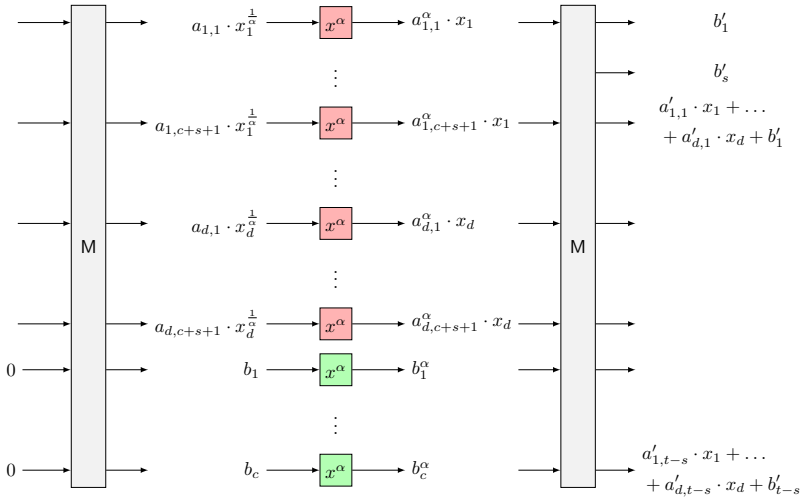


Figure 5: Generic SPN round-skip, counting each $\mathsf{M} \circ \mathsf{S}$ as one round after the initial mixing layer M. Figure is reproduced from [BP25a, Figure 6].

Let $\mathsf{M} = \left(m_{i,j}\right)_{0 \le i,j \le t-1} \in \mathbb{F}_q^{t \times t}$ be the linear layer, $\mathsf{M}^{-1} = \left(m_{i,j}^{\text{inv}}\right)_{0 \le i,j \le t-1}$ its inverse, and $\mathbf{c}^{(1)} \in \mathbb{F}_q^t$ the first round constant vector. Our goal is to find d vectors $(a_{k,1}, \ldots, a_{k,c+s+1})^\intercal$, where $1 \le k \le d$, such that for all k and $t-c+1 \le i \le t$,

$$\sum_{j=1}^{c+s+1} m_{i,(k-1)\cdot(c+s+1)+j}^{\text{inv}} \cdot a_{k,j} = 0\,. \tag{27}$$

For every i, the solution set to Equation (27) is a vector space of dimension $s+1$. Now we have to solve the homogeneous equations

$$\sum_{j=1}^{c+s+1} m_{i,(k-1)\cdot(c+s+1)+j} \cdot a_{k,j}^\alpha = 0, \qquad 1 \le i \le s, \tag{28}$$

with respect to the $a_{k,j}$'s. So, for all $1 \le k \le d$ we have s homogeneous degree α equations in $s+1$ variables. Since we need a non-trivial solution, we pick one coordinate and fix it to 1, and then solve the system. Also, for existence of a non-trivial solution to Equations (27) and (28) the following condition must be satisfied

$$t \ge c + d \cdot (c + s + 1). \tag{29}$$

Moreover, we solve for a constant vector $b = (b_1, \ldots, b_c)^\intercal$ such that

$$\sum_{j=1}^{c} m_{i,j+t-c}^{\text{inv}} \cdot b_j = \sum_{j=1}^{t} c_j^{(1)} \cdot m_{i,j}^{\text{inv}} \qquad t-c+1 \le i \le t. \tag{30}$$

Now we set the state before the first S-box layer to

$$\left(a_{1,1} \cdot x_1^{\frac{1}{\alpha}}, \ldots, a_{1,c+s+1} \cdot x_1^{\frac{1}{\alpha}}, \ldots, a_{d,1} \cdot x_d^{\frac{1}{\alpha}}, \ldots, a_{d,c+s+1} \cdot x_d^{\frac{1}{\alpha}}, b_1, \ldots, b_c\right)^\intercal. \tag{31}$$

An illustration of the round-skip is given in Figure 5.

The development of round-skipping techniques is still an active area of research. While first introduced for monomial SPN primitives such as Poseidon and *Rescue-Prime*

[BBLP22, §4], similar techniques have since been developed for primitives whose round functions are not SPNs. In particular, round-skips for the CICO-1 problems of Arion and Griffin were presented in [BBL+24a, BBL+24b, §B, §D]. For the CICO-1 problem of GMiMCHash, which is based on an expanding Feistel permutation, we present a novel round-skipping technique in Appendix D.1.4. Moreover, by exploiting the specific structure of the linear layer M, Merz and Rodríguez García [MG26] recently obtained improved round-skips for Poseidon2. Finally, we note that round-skipping techniques also extend to other modes of operation, such as compression mode.

4.3.2 Invariant Subspaces in Partial Substitution Layers

Let $\mathsf{F} : \mathbb{F}_q^t \to \mathbb{F}_q^t$ be a function. An *invariant subspace* of F is an affine subspace $V + \mathbf{a}$, given by a linear subspace $V \subseteq \mathbb{F}_q^t$ and a constant $\mathbf{a} \in \mathbb{F}_q^t$, such that $\mathsf{F}(V + \mathbf{a}) \subseteq V + \mathbf{a}$.

The existence of an invariant subspace may constitute a structural weakness of a round function. We illustrate this with the internal rounds of the Hades [GLR+20] design strategy,

$$\mathsf{R}_\mathrm{I} : \mathbb{F}_q^t \to \mathbb{F}_q^t, \qquad \mathbf{x} \mapsto \mathsf{M}_\mathrm{I}\left(x_1^\alpha, x_2, \ldots, x_t\right)^\intercal + \mathbf{c}, \tag{32}$$

where $\alpha > 1$ with $\gcd(\alpha, q-1) = 1$, $\mathsf{M}_\mathrm{I} \in \mathbb{F}_q^{t\times t}$, and $\mathbf{c} \in \mathbb{F}_q^t$ (see also Section 5.2). Since R_I applies its S-box to a single state component, it is particularly susceptible to invariant subspaces.

Affine Subspace Trails. Invariant subspaces can propagate across several consecutive rounds. An (affine) *subspace trail* of length k consists of $k+1$ linear subspaces $V_i \subseteq \mathbb{F}_q^t$ and constants $\mathbf{a}_i \in \mathbb{F}_q^t$ such that

$$\mathsf{R}_\mathrm{I}(V_i + \mathbf{a}_i) \subseteq V_{i+1} + \mathbf{a}_{i+1}, \qquad 0 \le i \le k-1. \tag{33}$$

For internal Hades rounds, affine subspace trails of length $k \le t-1$ can always be found; whether longer, or even infinite, trails exist depends crucially on the choice of M_I [BCD+20, GRS21]. In practice, M_I is chosen to preclude infinitely long subspace trails.

Such trails matter for statistical cryptanalysis, as a subspace trail over k rounds yields a differential or linear trail that activates no S-box across those k rounds. The same affine structure can also be exploited algebraically to linearize the traversed rounds, as we describe next.

Algebraic Attacks. Subspace trails can also be exploited in algebraic attacks to lower the degree of the modeled equations. In the context of AO hash functions, this was first proposed by Beyne et al. [BCD+20, §6.2] to linearize internal rounds in Poseidon polynomial systems. Suppose there exists an invariant subspace of dimension $\delta \le t$. Then, for $k \le \delta$, there exist a matrix $\mathbf{U}_1 \in \mathbb{F}_q^{\delta\times t}$ and a constant $\mathbf{a} \in \mathbb{F}_q^\delta$ such that

$$\mathbf{U}_1 \mathsf{R}_\mathrm{I}^{(k)}(\mathbf{x}) = \mathbf{U}_1\left(\mathsf{M}_\mathrm{I}^k \mathbf{x} + \mathbf{a}\right), \tag{34}$$

where $\mathsf{R}_\mathrm{I}^{(k)}$ denotes the composition of k internal rounds. In other words, on the projection $\mathbf{U}_1$ the k rounds act as the *linear* map M_I^k. Letting $\mathbf{U}_2 \in \mathbb{F}_q^{(t-\delta)\times t}$ be a matrix whose row space is complementary to that of $\mathbf{U}_1$, we obtain for all $\mathbf{x} \in \mathbb{F}_q^t$

$$\mathbf{U}_1\mathbf{y} = \mathbf{U}_1\left(\mathsf{M}_\mathrm{I}^k \mathbf{x} + \mathbf{a}\right), \qquad \mathbf{U}_2\mathbf{y} = \mathbf{U}_2\mathsf{R}_\mathrm{I}^{(k)}(\mathbf{x}), \tag{35}$$

with $\mathbf{y} = \mathsf{R}_\mathrm{I}^{(k)}(\mathbf{x})$. Guessing the $t-\delta$ nonlinear coordinates $\mathbf{U}_2\mathsf{R}_\mathrm{I}^{(k)}(\mathbf{x})$ then linearizes all k internal rounds, which allowed Beyne et al. to improve the complexity of preimage attacks. Grassi et al. [GKR25] later revisited this linearization for Poseidon2 and Neptune.

The study of invariant subspaces for partial SPNs with structured linear layers remains an active research area; for further reading we refer to [WW24, DW25, DW26, LW26]. We highlight in particular the recent extension to non-linear invariant subspaces by Li et al. [LLW26], roughly doubling the number of internal partial rounds over which degree growth can be suppressed.

4.3.3 FreeLunch Approach

Bariant et al. [BBL+24a] studied the CICO-1 problem for primitives built from inverse monomial S-boxes $x \mapsto x^{1/\alpha}$, where $\alpha > 1$ is an integer with $\gcd(\alpha, q-1) = 1$. Their core idea is to choose a *weighted monomial order* for which the polynomial model is *already* a Gröbner basis, so that the first step of a Gröbner attack becomes free. They applied this approach to `Anemoi`, Arion, Griffin and XHash8.

FreeLunch Order. Modeling the CICO-1 problem round by round and introducing a fresh variable z_i for the output of each inverse S-box (to keep the degree low) yields a *triangular* system

$$\begin{aligned} z_i^\alpha - f_i(x, z_1, \ldots, z_{i-1}) &= 0, \qquad 1 \le i \le n, \\ f_x(x, z_1, \ldots, z_n) &= 0, \end{aligned} \tag{36}$$

where n is the number of inverse S-boxes in the model, x is the single unknown input, and f_x encodes the output constraint. Whether Equation (36) is already a Gröbner basis depends on the choice of a weighted monomial order $>_{\mathbf{w}}$. The goal is to choose a weight vector $\mathbf{w} = (w_1, \ldots, w_n, w_x)^\intercal \in \mathbb{Q}_{>0}^{n+1}$ for some monomial order $>$ such that $\mathrm{LM}_{>_{\mathbf{w}}}(z_i^\alpha - f_i) = z_i^\alpha$ for every $1 \le i \le n$. It suffices to ensure that the weighted degree of z_i^α exceeds that of every monomial in f_i. Since $\deg_{\mathbf{w}}(f_i) \le w_x \deg_x(f_i) + \sum_{j=1}^{i-1} w_j \deg_{z_j}(f_i)$, this is guaranteed if $\mathbf{w}$ satisfies

$$\alpha \cdot w_i > w_x \cdot \deg_x(f_i) + \sum_{j=1}^{i-1} w_j \cdot \deg_{z_j}(f_i) \tag{37}$$

for all $1 \le i \le n$. Such a vector can be constructed iteratively: after fixing w_x, each w_i can be chosen based only on the weights of $x, z_1, \ldots, z_{i-1}$, since f_i involves only these variables. If such weights exist and additionally $\mathrm{LM}_{>_{\mathbf{w}}}(f_x) = x^{d_x}$ is a pure power of x, then the leading monomials $z_1^\alpha, \ldots, z_n^\alpha, x^{d_x}$ are pairwise coprime, implying that Equation (36) is a $>_{\mathbf{w}}$-Gröbner basis [CLO15, Ch. 2, §9, Prop. 4]. Bariant et al. call such a system a *FreeLunch system* and any order satisfying these conditions a *FreeLunch order*.

Appropriate weight vectors exist for Arion, Griffin and XHash8 [BBL+24a, §4 & §F]. For `Anemoi` they do not, since the natural output polynomial f_x has no leading monomial that is a pure power of x. As a remedy, f_x is multiplied by suitable monomials in the S-box variables z_i and reduced modulo $z_i^\alpha - f_i$, obtaining a new output polynomial whose leading monomial is a pure power of x; this enlarges the ideal degree D_I by a factor of at most $\alpha + 1$ [BBL+24a, §5].

Attack Complexity. Let D_I denote the $\mathbb{F}_q$-vector space dimension of the ideal generated by Equation (36). Once the system is known to be a Gröbner basis with respect to a weighted monomial order $>_{\mathbf{w}}$, a univariate polynomial in x can be recovered as the characteristic polynomial $\det(x \cdot \mathbf{I} - \mathbf{M}_x)$ of the multiplication matrix $\mathbf{M}_x$ of x in the quotient ring. Exploiting its structure, this polynomial can be computed in

$$\mathcal{O}\left(\left(D_I/d_x\right)^\omega \cdot d_x \cdot \log_2(d_x) \cdot \left(1 + \log_2\left(\log_2(d_x)\right)\right)\right) = \tilde{\mathcal{O}}\left(D_I^\omega / d_x^{\omega-1}\right) \tag{38}$$

finite field operations [BBL+24a, §3.2]. For large d_x this improves on generic monomial order conversion, which runs in $\tilde{\mathcal{O}}(D_I^\omega)$ [FGHR14], by a factor $d_x^{\omega-1}$. Bariant et al. did not, however, estimate the cost of *constructing* $\mathbf{M}_x$, leaving a gap in the overall complexity analysis.

A follow-up work [BBB+25b] removes this gap by dispensing with $\mathbf{M}_x$ altogether. Instead of the multiplication matrix, it solves Equation (36) by iterated resultants, exploiting the triangular structure: one eliminates z_1 from all remaining polynomials via the resultant with $z_1^\alpha - f_1(x)$, then z_2, and so on, using the relations $z_i^\alpha - f_i$ to cap each z_i-degree at α through a dedicated division routine [BBB+25b, §3.3]; see also Section 4.2.2. For a FreeLunch Gröbner basis with ideal degree D_I, the iterated resultants cost

$$\tilde{\mathcal{O}}\big(D_I \cdot \alpha^2 \cdot ((2\alpha-1)/\alpha)^{n+2}\big) \tag{39}$$

finite field operations [BBB+25b, Corollary 3]. Finally, a recent ePrint [BBB+25a] extends the FreeLunch approach via weighted Gröbner bases to the general CICO-k setting.

4.3.4 Change of Coordinates

Next we describe a technique to produce a DRL Gröbner basis with a change of coordinates. Suppose that we are given a system $f_1, \ldots, f_n \in \mathbb{F}_q[x_1, \ldots, x_n]$ where each f_i is of the form

$$f_i = g_i\left(\sum_{j=1}^{n} a_{i,j} \cdot x_j\right) + \sum_{j=1}^{n} b_{i,j} \cdot x_j + c_i, \tag{40}$$

where $a_{i,j}, b_{i,j}, c_i \in \mathbb{F}_q$ and $g_i \in \mathbb{F}_q[x]$ is univariate. Let $d_i = \deg(g_i)$, if $d_i = 1$ for some i, then we can immediately eliminate one variable in the system. So, without loss of generality we can assume that $d_i > 1$ for all i. Let us assume that $\operatorname{rank}\left(a_{i,j}\right)_{1\le i,j\le n} = n$, then we can perform the change of coordinates $\hat{x}_i = \sum_{j=1}^n a_{i,j} \cdot x_j$. After the change of coordinates our polynomials from Equation (40) become $f_i = g_i(\hat{x}_i) + \sum_{j=1}^n \hat{b}_{i,j} \cdot \hat{x}_j + c_i$, where $\hat{b}_{i,j} \in \mathbb{F}_q$. For any DRL order we then have that $\mathrm{LM}_{\mathrm{DRL}}(g_i) = \hat{x}_i^{d_i}$, so $f_1, \ldots, f_n$ is a DRL Gröbner basis with $\mathbb{F}_q$-vector space dimension $\dim_{\mathbb{F}_q}(f_1, \ldots, f_n) = \prod_{i=1}^n d_i$. We also observe that the f_i's form a FreeLunch system with respect to any variable.

Assume that $d_i = d$ for all i, then the complexity of computing the multiplication matrices $\mathbf{M}_{x_1}, \ldots, \mathbf{M}_{x_n}$ is bounded by $\mathcal{O}\left(n^2 \cdot d^{2\cdot n}\right)$ field operations [Ste26, §2.5.1.1]. In addition, computing the characteristic polynomial of $\mathbf{M}_{x_n}$ requires

$$\mathcal{O}\left(d \cdot \log_2(d)^2 \cdot \left(1 + \log_2\left(\log_2(d)\right)\right) \cdot d^{\omega\cdot(n-1)}\right) \tag{41}$$

field operations [Ste26, §2.5.4]. So, if a change of coordinates is possible, the complexity of the full Gröbner basis attack is sub-cubic in d^n.

Steiner first described this approach for the Multi-Party Computation-friendly Pseudo-Random Function Hydra [Ste25a], which is based on the Lai–Massey permutation. In his PhD thesis [Ste26, §6] he formalized the change of variables approach to construct DRL Gröbner basis for Feistel designs, as an example he studied the GMiMC family [Ste26, §6.2]. Moreover, the change of coordinates has been applied to the polynomial system of the Mersenne31 cipher mid-pSquare [BGM+25] and the Mersenne31 hash function pSquare-hash [GMBM+26, §G.5] (Section 5.3.2). In this paper we will apply it to GMiMCHash2 (Section 5.1.2).

4.4 Cryptanalysis Challenges

Due to potential applications of hash-based ZK-proofs in Blockchain ecosystems, industry showed interest in AO hash functions early on. This is most apparent with the sponsorship

of cryptanalysis challenges. The first challenge was hosted by StarkWare in 2019 [Sta19] which featured Feistel-MiMC-Hash, GMiMCHash, Poseidon, *Rescue*, Starkad and *Vision*. The challenge resulted in three survey papers [BCL+20, CBD+20, BGL20] as well as two conference papers [BCD+20, KR21].

In 2021 Ethereum hosted CICO-1 challenges [Eth21] for Feistel-MiMC-Hash, Poseidon, Reinforced Concrete and *Rescue-Prime*. During this challenge the round-skipping technique for SPNs (Section 4.3.1) was discovered by Bariant et al. and published in a journal article [BBLP22].

In 2024 Ethereum started the Poseidon initiative [Eth24] which offers rewards on improved Poseidon cryptanalysis. The challenge is still ongoing throughout 2026. At the time of writing it resulted in practical improvements of factoring univariate polynomials over FFT-friendly fields as well as improved round-skips for Poseidon2 [SV25, ZD25, ZSVD25, MG26, BBB+26].

5 ZK-Friendly Hash Functions: Type-I (Low Degree)

In this section, we describe the first class of ZK-friendly hash functions, which apply low-degree round functions to ensure low verification costs in arithmetic circuits. Representative examples of Type-I primitives include MiMC [AGR+16], GMiMC [AGP+19a], Hades-like designs (e.g. Poseidon [GKR+21], Poseidon2 [GKS23a], Neptune [GOPS22]) and generalized Feistel structures (e.g. Moirai [BP25b], pSquare-hash [GMBM+26]).

Type-I Arithmetization-Oriented. These designs typically rely on low-degree internal functions (such as S-boxes), enabling efficient verification of their results using low-degree polynomials.

Definition 4 (Type-I Arithmetization-Oriented)**.** A primitive is said to be "*Type-I Arithmetization-Oriented*" if

(i) the round function is of low polynomial degree ($\approx \log_2\left(\log_2(p)\right)$), and

(ii) the prover circuit is based on forward evaluation of the round function.

While Type-I primitives offer simplicity and often excellent performance inside ZK, the requirement for a low algebraic degree often forces a relatively high number of rounds to achieve classical cryptographic security, and this has motivated the development of more advanced types (Type-II and Type-III).

5.1 MiMC Designs

The development and cryptanalysis of AO primitives over large prime fields started in 2016 with the introduction of **Mi**nimizing **M**ultiplicative **C**omplexity (MiMC) [AGR+16] by Albrecht et al. (see also the timeline in Figure 1). Let $\mathbb{F}_p$ be a finite field, and let $\alpha > 1$ be the least odd integer such that $\gcd(\alpha, p-1) = 1$, i.e. $x \mapsto x^\alpha$ induces a power permutation of $\mathbb{F}_p$. MiMC is an iterated univariate keyed permutation over $\mathbb{F}_p$

$$\mathsf{P}(x,k) = k_R + \mathsf{R}^{(R-1)}_{k_{R-1}} \circ \cdots \circ \mathsf{R}^{(0)}_{k_0}(x), \tag{42}$$

whose round function is defined as

$$\mathsf{R}^{(i)}_{k_i}(x) = (x + k_i + c_i)^\alpha, \tag{43}$$

where $c_i \in \mathbb{F}_p$ is a constant and $k_i = k_{i-1}^\alpha$ for $i > 0$. For higher encryption throughput also a univariate Feistel cipher, called Feistel-MiMC, was proposed. Its round function is

given by the simple 2-branch keyed Feistel permutation

$$\mathsf{R}_k^{(i)} \begin{pmatrix} x_L \\ x_R \end{pmatrix} = \begin{pmatrix} x_R + (x_L + k + c_i)^\alpha \\ x_L \end{pmatrix}. \tag{44}$$

The unkeyed Feistel-MiMC permutation can also be used in sponge mode for ZK-friendly hashing. However, this simple sponge function has rate/capacity $r = c = 1$, so the maximum achievable security level of this variant scales with $\sqrt{p}$. E.g., for 128 bits of security one must have that $\log_2(p) \geq 256$. For completeness, we mention that it was also proposed to use the univariate MiMC in sponge mode by splitting the bit representation of a field element $x \in \mathbb{F}_p$ into a rate and capacity part. However, this variant was never taken into account in the subsequent literature.

While MiMC and Feistel-MiMC were the first primitives that demonstrated an increased performance over binary designs like LowMC [ARS+15] or Keccak [BDPV13] for prime field applications, their applicability is rather limited in retrospect. As already mentioned, for Feistel-MiMC in sponge mode one requires at least a 256 bit field, and only a single field element can be absorbed per permutation call. To increase throughput, Grassi et al. [AGP+19a] proposed in 2019 to utilize *generalized* Feistel structures with low multiplicative complexity.

5.1.1 GMiMC (ESORICS'19)

At ESORICS 2019, Albrecht et al. [AGP+19a] proposed the Generalized-MiMC (GMiMC) family, which extends the original MiMC design [AGR+16] into a versatile framework of block ciphers and hash functions optimized for Zero-Knowledge (ZK) proofs, secure Multi-Party Computation, and post-quantum digital signatures.

The hash function GMiMCHash is obtained by integrating the GMiMC permutation, where all subkeys are fixed to zero, into a sponge construction. The GMiMC permutation is constructed from four Feistel-based designs: an unbalanced Feistel network (UFN) with a contracting round function (CRF), one with an expanding round function (ERF, cf. Figure 6), a Nyberg generalized Feistel network (GFN), and a multi-rotating Feistel network (MR). Among these, the ERF variant is the most competitive in ZK settings, thus we focus on this variant.[17]

An unbalanced Feistel network with ERF over $\mathbb{F}_p$ can be written as

$$(x_1, \ldots, x_t)^\intercal \mapsto \left(x_2 + \mathsf{F}_i(x_1), \ldots, x_t + \mathsf{F}_i(x_1), x_1\right)^\intercal. \tag{45}$$

A circuit representation of an expanding Feistel round function is given in Figure 6. For $\text{GMiMC}_{\text{erf}}^{\pi}$ the i-th round function is defined as

$$\mathsf{F}_i(x) = (x + c_i)^\alpha, \tag{46}$$

where $\alpha > 1$ is the smallest odd integer such that $\gcd(\alpha, p-1) = 1$ and $c_i \in \mathbb{F}_p$ is a constant. The $\text{GMiMC}_{\text{erf}}^{\pi}$ permutation is then simply given by the R-fold iteration of the round function

$$\mathsf{P}(\mathbf{x}) = \mathsf{R}^{(R-1)} \circ \cdots \circ \mathsf{R}^{(0)}(\mathbf{x}). \tag{47}$$

Note that GMiMC can be instantiated over both binary fields $\mathbb{F}_{2^n}$ and prime fields $\mathbb{F}_p$, though we focus on the prime field variant.

We also note that for $t > 4$ plain and proof system performance of GMiMCHash can be improved with an alternative implementation of the ERF permutation, see Algorithm 1 in Appendix C.3.

[17] We note that CCZ-equivalence [CCZ98] between CRF and ERF Feistels can be proven with a mix of variable substitutions and invertible linear transformations, thus leading to similar security and efficiency characteristics.

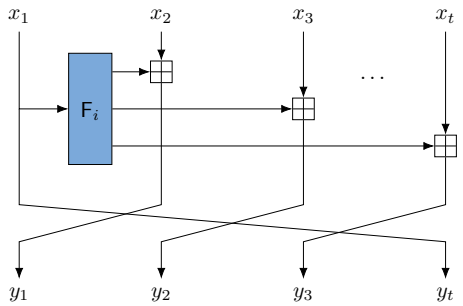


Figure 6: One round of t-branch $\mathrm{GMiMC}^{\pi}_{\mathrm{erf}}$.

The GMiMC designers provided a table [AGP+19b, Table 1] to compute the number of rounds of a GMiMC instance for security level 128 bits given $\log_2(p)$ and t. In the reference implementation[18] a Python script is included to compute the round numbers for $\alpha = 3$ given p and t. We updated the code to support arbitrary exponents α, see Appendix C.1. The parameters for $\mathrm{GMiMC}^{\pi}_{\mathrm{erf}}$ in Table 3 have been computed with the updated tool.

Table 3: (Updated) Round numbers R for GMiMCHash with a target security level of 128 bits. t denotes the state size, α the S-box exponent, and d the digest size. No round numbers require updating under the latest attacks.

(a) $\log_2(p) \approx 256$, $d = 1$.

$t\backslash\alpha$	3	5	7	11
3	331	228	190	156
4	334	231	193	159
5	337	234	196	162
8	346	243	205	171

(b) $\log_2(p) \approx 64$, $d = 4$.

$t\backslash\alpha$	3	5	7	11
8	103	78	68	60
12	115	93	93	93
16	156	156	156	156
20	235	235	235	235

(c) $\log_2(p) \approx 32$, $d = 8$.

$t\backslash\alpha$	3	5	7	11
16	158	158	158	158
24	335	335	335	335
32	579	579	579	579
40	888	888	888	888

Third-Party Cryptanalysis. GMiMC was one of the first AO primitives. During its conception it was not clear yet which field sizes will be adopted in FHE/MPC/ZK-applications. To cover all possible field and state sizes a very generic cryptanalysis was provided. At the time of writing this article, much of the original cryptanalysis of GMiMCHash is obsolete. Therefore, in the following paragraphs we discuss the most important improvements of GMiMCHash cryptanalysis.

Statistical Cryptanalysis. The GMiMC designers originally conjectured [AGP+19b, §D.1] that every truncated differential of $\mathrm{GMiMC}^{\pi}_{\mathrm{erf}}$ over $t+1$ rounds has probability $\approx (\alpha/p)^2$. Though, Beyne et al. [BCD+20, §5.2] observed that *cyclic* truncated differentials of the form

$$(0, \ldots, 0, \Delta_1, \Delta_2)^{\mathsf{T}} \xrightarrow[\text{prob. } 1/p]{\mathsf{R}^{(t)}} (0, \ldots, 0, \Delta_1 - \Delta_1', \Delta_2 + \Delta_1')^{\mathsf{T}} \tag{48}$$

can propagate through t rounds with probability p^{-1}. This trail can be generalized to more active words, see for example [BCD+20, §5.2], [CBD+20, §4.4] and Appendix C.2, and it can be made univariate [Bey23, §9, §10.2]. We can search for collisions with the generalized truncated differential in Equation (84) via *structures* [BCD+20, §5.2]

$$\mathcal{S}_a = \{(a_0, \ldots, a_{t-k-1}, x_1, \ldots, x_k)^{\mathsf{T}} \mid x_1, \ldots, x_k \in \mathbb{F}_p\}, \tag{49}$$

[18] https://extgit.isec.tugraz.at/krypto/gmimc-implementation

where $a_0, \ldots, a_{t-k-1} \in \mathbb{F}_p$ are arbitrary but fixed within $\mathcal{S}_a$ Any distinct $\mathbf{x}, \mathbf{y} \in \mathcal{S}_a$ satisfy the input difference of Equation (84). The number of unique pairs in the structure $\mathcal{S}_a$ is given by $\frac{p^k \cdot (p^k - 1)}{2}$. Moreover, finding a pair that satisfies the output difference of Equation (84) can be done in complexity $\mathcal{O}\left(p^k\right)$ since we just have to evaluate the unique p^k elements of $\mathcal{S}_a$ and compare the first $t - k$ entries. Taking the capacity and the digest size of GMiMCHash into account, for $l \geq 1$ we can then cycle the truncated differential for up to $t - c + (l-1) \cdot t + t - d - l = (l+1) \cdot t - c - d - k$ many rounds with probability p^{-l}. Moreover, we can search up to $2^\kappa \cdot p^{-k}$ many structures. So, we expect to find $\gtrapprox p^{2 \cdot k} \cdot p^{-l} \cdot 2^\kappa \cdot p^{-k} = 2^\kappa \cdot p^{k-l}$ many collisions with structures. For security level 128 bits and a full codebook search, i.e. $k = c/2$, we then expect to find collisions up to $\left\{\begin{matrix} 5 \cdot t - 10, & \log_2(p) \approx 64, \\ 9 \cdot t - 20, & \log_2(p) \approx 32 \end{matrix}\right\}$ many rounds. To the best of our knowledge, for small prime fields this is currently the best attack on GMiMCHash over most parameter choices.

Chen et al. [CGG+23] studied links between differential, impossible differential, integral, linear and zero-correlation distinguishers for $\text{GMiMC}^{\pi}_{\text{erf}}$. With these links the then known impossible differential, integral and zero-correlation distinguishers could be improved by up to 3 rounds.

Algebraically Controlled Differential Attacks. Beyne et al. [BCD+20, §5.3] proposed to *symbolically* propagate the truncated differential from Equation (48) in forward (resp. backward) direction to construct a bivariate polynomial system, whose roots yield a pair $\mathbf{x}, \mathbf{y} \in \mathbb{F}_p^t$ which satisfies the input/output difference of Equation (48). The bivariate system solving complexity is negligible, and according to Beyne et al. up to $4 \cdot t - 2$ rounds can be covered via this inside-out approach.

Li et al. [LWL23] generalized the algebraically controlled differential attack to more variables. According to their analysis with k variables up to $k \cdot (t-1) + t - c$ many rounds can be attacked by solving k polynomials of degree $\leq \alpha^{2 \cdot k - 1}$. According to Li et al.'s estimation the complexity of solving the polynomial system is bounded by $\mathcal{O}\left(\left(\frac{\alpha^{2 \cdot k - 2 - \alpha^{k-1}}}{\frac{2}{k-1}} + 1\right)^{\omega} + \left(\frac{\alpha^{2 \cdot k - \alpha^k}}{\frac{2}{k}} + 1\right)^{\omega}\right)$. For $\alpha = 3, 5, 7$ and $\omega = 2$ we can then solve up to $k = 4, 3, 3$ many variables below 128 bits. With $\alpha = 3$ and $t \in \{8, 12, 16, 24\}$ we can then attack up to $R \in \{32, 52, 68, 108\}$ rounds respectively (see also [LWL23, Table 1]).

Algebraic-Differential Attacks. Beyne et al. [BCD+20] and Li et al. [LWL23, §4.1] proposed a hybrid algebraically controlled differential attack where the number of rounds is split as $R = R_1 + R_2$. For the first R_1 rounds one uses the algebraically controlled differential attack to find input pairs with input difference $(0, \ldots, 0, *, \ldots, *, 0, \ldots, 0)^\intercal$ and output difference $(0, \ldots, 0, *, \ldots, *)^\intercal$ after R_1 rounds. Since the output difference satisfies Equation (84) we can propagate it another $t \cdot l$ rounds with probability p^{-l}. For k variables, Li et al. propose $R_1 = t \cdot (k-1) - k - c$ and to solve the system $\approx p$ times, which yields $\approx p^2$ pairs. The differential is then propagated for another $R_2 = 2 \cdot t + t - d - k$ rounds. So, we expect to find 1 collision for $R = t \cdot (k+2) - c - d - 2 \cdot k$. The complexity of this attack is then upper bounded by the system solving complexity, Li et al. estimated it as $\mathcal{O}\left(2 \cdot p \cdot \left(\frac{\alpha^{2 \cdot k - 2 - \alpha^{k-1}}}{\frac{2}{k-1}} + 1\right)^{\omega}\right)$. For $\log_2(p) \approx 64$, $\alpha = 3$, $\omega = 2$ and $k = 4$ the system solving complexity becomes 2^{111} bits and up to $R = 32$ ($t = 8$) and $R = 56$ ($t = 12$) rounds can be attacked. For $\log_2(p) \approx 32$, $\alpha = 3$, $\omega = 2$ and $k = 5$ the system solving complexity becomes 2^{117} bits and up to $R = 86$ ($t = 16$) and $R = 124$ ($t = 24$) rounds can be attacked.

Algebraic Cryptanalysis. Roy et al. [RAS20] studied the exponential degree growth of (keyed) GMiMC permutations. Their findings can also be leveraged to accurately predict the polynomial degree of GMiMCHash over 256 bit fields.

Beyne et al. [BCD+20, §4.1] developed a zero-sum distinguisher for $\text{GMiMC}^{\pi}_{\text{erf}}$ based on the integral property to distinguish up to $3 \cdot t - 4 + \lfloor \log_{\alpha}(p-2) \rfloor$ rounds from a random permutation. Such an integral distinguisher can be extended to a distinguisher on GMiMCHash with a slightly less number of rounds. (They also developed a MitM distinguisher covering more rounds which is not applicable to sponge functions.)

Beyne and Verbauwhede [BV25a] generalized techniques to study degree growth of functions over $\mathbb{F}_2^t$ to $\mathbb{F}_p^t$. With their techniques one can accurately predict the polynomial degree of iterated functions over $\mathbb{F}_p^t$ during the saturation phase. This enables the currently best degree growth estimation of $\text{GMiMC}^{\pi}_{\text{erf}}$ over 32 bit fields, see for example Figure 24 in Appendix D.1.3.

Li et al. [LCC+26] developed dedicated tools for GMiMC to further advance the capabilities of integral distinguishers. In particular, over smaller fields their analysis provides better distinguishers on $\text{GMiMC}^{\pi}_{\text{erf}}$ than Beyne and Verbauwhede.

In his PhD thesis Steiner [Ste26, §6.2] developed DRL Gröbner bases for the GMiMC family via a linear change of coordinates, see also Section 4.3.4. However, the change of coordinates is only successful for certain round numbers [Ste26, Example 6.2.4]. In particular, simultaneous DRL Gröbner bases for the CICO and preimage problems are only available for $t = 4$.

5.1.2 GMiMCHash2

In retrospect, the cryptanalysis of GMiMCHash suffers from three problems.

(I) The original differential cryptanalysis was too optimistic, because there exist truncated differential trails over t rounds with probability $\approx q^{-1}$ for any x^{α}, see Equations (48) and (84).

(II) For $\log_2(p) \approx 256$, sponge round numbers were based on a univariate Meet-in-the-Middle attack on a cipher. However, univariate Meet-in-the-Middle attacks do not exist for sponge mode.

(III) For $\log_2(p) \approx 32, 64$ and fixed R, it is in general not possible to construct simultaneous CICO and preimage DRL Gröbner bases.

Therefore, we propose an updated variant GMiMCHash2 which addresses these problems.

As a new non-linear function, we propose to use $\alpha = 2^k$, which offers optimal degree growth with k multiplications. Also, the round constant is now added into the state before computation of x^{α}. This change permits slightly more efficient circuits in Plonk arithmetic. The circuit of the updated round function is depicted in Figure 7.

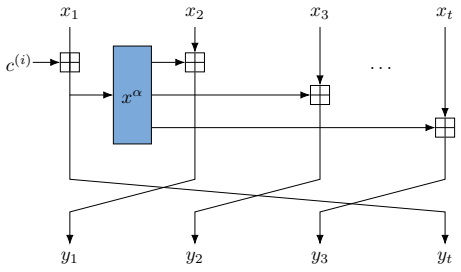


Figure 7: $\text{GMiMC}^{\pi}_{\text{erf}}2$ round function $\mathsf{R}^{(i)}$.

Moreover, we propose the application of an Input/Output matrix $\mathsf{M}_{\text{IO}} \in \mathbb{F}_p^{t \times t}$ at the start and the end of the permutation. Then, the $\text{GMiMC}^{\pi}_{\text{erf}}2$ permutation is defined as

$$\mathsf{P}(\mathbf{x}) = \mathsf{M}_{\text{IO}} \mathsf{R}^{(R)} \circ \cdots \circ \mathsf{R}^{(1)} (\mathsf{M}_{\text{IO}} \mathbf{x}) . \tag{50}$$

Analogously to $\text{GMiMC}^{\pi}_{\text{erf}}$, we can construct an improved $\text{GMiMC}^{\pi}_{\text{erf}}2$ circuit for $t > 4$. It is presented in Algorithm 2 in Appendix D.2.

For GMiMCHash2, over 256 bit fields we propose sponge mode only[19] and for 32/64 bit fields we propose compression and sponge mode. The round numbers for GMiMCHash2 are chosen to have a sufficient security margin against the state of the art of GMiMCHash cryptanalysis. Over 32 bit fields the dominating attack is truncated differential cryptanalysis based on the generalized differential of Beyne et al. [BCD+20], see Equation (48) and Appendix C.2. Over 64 bit fields, the dominating attacks are again truncated differentials as well as Gröbner basis attacks. Over 256 bit fields, the dominating attack is univariate polynomial system solving. Full details on our security analysis are given in Appendix D.1. In Table 4 we presented the updated round numbers for GMiMCHash2.[20] Configurations (t, α) which offer the best tradeoff between plain/proof system performance are highlighted in bold.

Table 4: Round numbers R for GMiMCHash2 with a target security level of 128 bits. t denotes the state size, α the S-box exponent, and d the digest size. Bold values indicate the best plain/proof-system performance tradeoff for the respective state size.

(a) $\log_2(p) \approx 256$, $d = 1$.

$t \backslash \alpha$	2	4	8
3	168	90	**63**
4	172	92	**64**
5	175	95	**75**
8	184	104	**80**

(b) $\log_2(p) \approx 64$, $d = 4$.

$t \backslash \alpha$	2	4	8
8	168	**88**	64
12	168	**96**	72
16	176	**112**	96
20	180	**120**	120

(c) $\log_2(p) \approx 32$, $d = 8$.

$t \backslash \alpha$	2	4	8
16	**176**	176	176
24	**264**	264	264
32	**352**	352	352
40	**440**	440	440

The Input/Output matrix should be chosen so that a DRL Gröbner can be computed with a change of coordinates (Section 4.3.4), but with minimal impact on overall performance. As concrete instances, we propose the following circulant matrices $\mathsf{M}_{\text{IO}} = \text{circ}(\mathbf{v}_{t,c})$. For ease of notation, let $\mathbf{e}_i$ denote the i-th unit vector of $\mathbb{F}_p^t$.

- For $c = 1$: $\mathbf{v}_{t,c} = \mathbf{e}_1$.
- For $t \in \{8, 16\}$ and $c = r = \frac{t}{2}$ as well as compression mode: $\mathbf{v}_{t,\frac{t}{2}} = \mathbf{e}_1 + 2 \cdot \mathbf{e}_{\frac{t}{2}+1}$.
- For $t \in \{12, 24\}$ and $c = \frac{t}{3}$: $\mathbf{v}_{t,\frac{t}{3}} = \mathbf{e}_1 + 2 \cdot \left(\mathbf{e}_{\frac{t}{3}+1} + \mathbf{e}_{\frac{t}{2}+1}\right)$.
- For $t \in \{16, 32\}$ and $c = \frac{t}{4}$: $\mathbf{v}_{t,\frac{t}{4}} = \mathbf{e}_1 + 2 \cdot \left(\mathbf{e}_{\frac{t}{8}+1} + \mathbf{e}_{\frac{t}{4}+1} + \mathbf{e}_{\frac{t}{2}+1}\right)$.
- For $t \in \{20, 40\}$ and $c = \frac{t}{5}$: $\mathbf{v}_{t,\frac{t}{5}} = \mathbf{e}_1 + 2 \cdot \left(\mathbf{e}_{\frac{t}{10}+1} + \mathbf{e}_{\frac{t}{5}+1} + \mathbf{e}_{\frac{t}{2}+1} + \mathbf{e}_{\frac{t}{4}+\frac{t}{2}+1}\right)$.

5.2 Hades-Like Designs

The HADES design strategy, introduced by Grassi et al. [GLR+20], generalizes the classical SPN paradigm to MPC/ZK-friendly permutations by alternating rounds with full and partial S-box layers:

- an initial R_{f_1} rounds in which S-boxes are applied to all state elements;
- R_{P} middle rounds where only a single S-box is used in each round, while the remaining state elements pass through unchanged;

[19] Compression mode with $d = 1$ admits a univariate MitM attack, thus over 256 bit field compression mode would need approximately twice the number of rounds as sponge mode. Thus, if one wishes to digest t elements with one $\text{GMiMC}^{\pi}_{\text{erf}}2$ call it is more efficient to use a $\text{GMiMC}^{\pi}_{\text{erf}}2$ sponge with state size $t + 1$.

[20] All round numbers are the smallest multiple of t, for which R exceeds our estimations in Appendix D.1.

- a final R_{f_2} rounds again applying S-boxes to the full state.

The total number of rounds with full S-box layers is denoted by $R_{\mathrm{F}} = R_{\mathrm{f}_1} + R_{\mathrm{f}_2}$. We will refer to SPN rounds which apply the S-box to all elements as *full* or *external rounds*, and to SPN rounds with a single S-box as *partial* or *internal rounds*.

The strategy was first instantiated in HadesMiMC [GLR+20] a family of MPC-friendly block ciphers, which extends the original MiMC construction by adopting the Hades round structure. Building on this approach, the Poseidon permutation [GKR+21] refines the design and has since become the most widely deployed Hades-like permutation in practical ZK protocols.

5.2.1 Poseidon (USENIX'21), Poseidon2 (Africacrypt'23)

Poseidon, proposed by Grassi et al. [GKR+21] at USENIX 2021, is a sponge-based hash function over the prime field $\mathbb{F}_p$, where $p \approx 2^n > 2^{31}$. Targeting security levels of 80, 128, and 256 bits, Poseidon (Poseidon-80, -128, and -256) is commonly instantiated over the scalar fields of elliptic curves like BLS12-381, BN254, and Ed25519.

The internal permutation, Poseidon^π, follows the Hades design strategy [GLR+20], and consists of $R_{\mathrm{F}} = 2 \cdot R_{\mathrm{f}}$ full rounds and R_{P} partial rounds. Its round functions consists of the following two components:

- The *S-box* is defined as an (α)-power permutation $(x \mapsto x^\alpha)$ over a prime field $(\mathbb{F}_p)$, where $(\alpha \geq 3)$ is the smallest integer satisfying $\gcd(\alpha, p-1) = 1$.
- The *affine layer* includes the addition of round constants and multiplication of the state by an MDS matrix $\mathsf{M} \in \mathbb{F}_p^{t \times t}$.

A circuit representation of Poseidon^π is depicted in Figure 8a.

Poseidon2 was introduced by Grassi et al. [GKS23a] at AFRICACRYPT 2023 and supports two modes of operation, namely sponge and compression, depending on the application. The internal permutation, Poseidon2^π, is a permutation over $\mathbb{F}_p^t$, where p is as in Poseidon^π and $t \in \{2, 3, 4, \ldots, 4 \cdot t', \ldots, 24\}$ for $t' \in \mathbb{N}$. Compared to Poseidon^π, four differences arise:

- Only $t \in \{2, 3, 4, 8, 12, 16, 20, 24\}$ are possible;
- Different matrices $\mathsf{M}_{\mathrm{E}} \in \mathbb{F}_p^{t \times t}$ and $\mathsf{M}_{\mathrm{I}} \in \mathbb{F}_p^{t \times t}$ are applied for the external and internal layers respectively. Also, the matrices do not need to be MDS anymore;
- $\mathsf{M}_{\mathrm{E}} \in \mathbb{F}_p^{t \times t}$ is applied before the first S-box application;
- Only a single field element is added as a constant (noted `RC` in Figure 8b) in each internal round.

A graphical overview of the differences between Poseidon^π and Poseidon2^π is given in Figure 8.

Concrete matrix instantiations for Poseidon2 were proposed in [GKS23a, §5.1, 5.2]. If $4 \mid t$, then the external matrix M_{E} follows the linear layer of Griffin [GHR+23, §5.1]

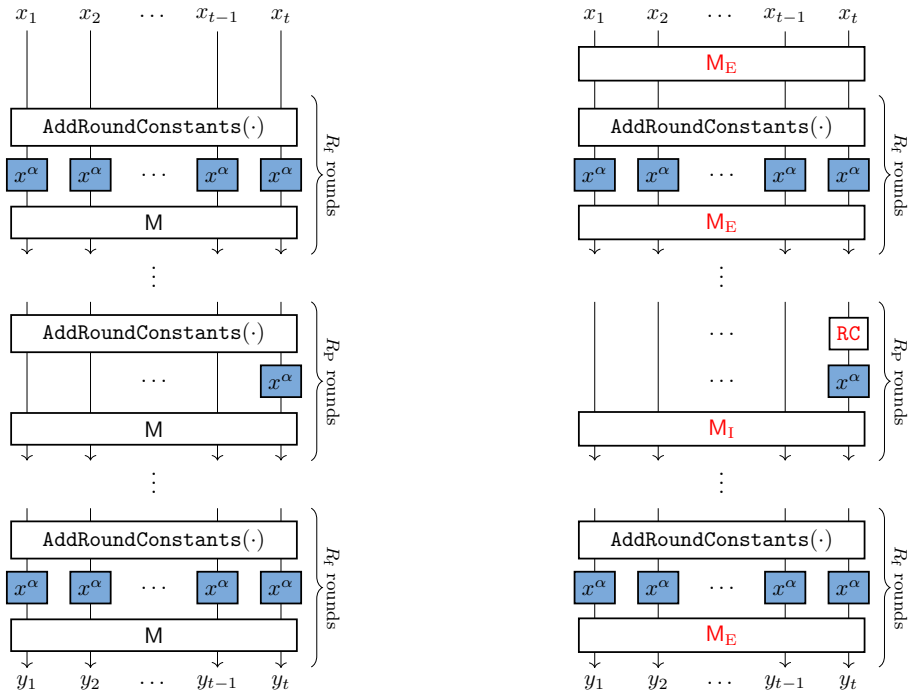
$$\mathsf{M}_4 = \begin{pmatrix} 5 & 7 & 1 & 3 \\ 4 & 6 & 1 & 1 \\ 1 & 3 & 5 & 7 \\ 1 & 1 & 4 & 6 \end{pmatrix} \in \mathbb{F}_p^{4 \times 4}, \qquad \mathsf{M}_{\mathrm{E}} = \mathrm{circ}\left(2 \cdot \mathsf{M}_4, \mathsf{M}_4, \ldots, \mathsf{M}_4\right) \in \mathbb{F}_p^{t \times t}, \tag{51}$$

where M_4 is the matrix $\mathsf{M}_{4,4}^{8,4}$ from [DL18]. For the internal layers matrices of the form

$$\mathsf{M}_\mathrm{I} = \begin{pmatrix} \mu_0 & 1 & \cdots & 1 \\ 1 & \mu_1 & \cdots & 1 \\ \vdots & \vdots & \ddots & \vdots \\ 1 & 1 & \cdots & \mu_{t-1} \end{pmatrix} \tag{52}$$

were recommended. The coefficients $\mu_0, \ldots, \mu_{t-1} \in \mathbb{F}_p \setminus \{0, 1\}$ have to be chosen so that no infinitely long subspace trail through the internal rounds exists. Finally, if $t = 2, 3$ one can use an MDS matrix $\mathsf{M}_\mathrm{E} = \mathsf{M}_\mathrm{I}$, for example $\mathrm{circ}\,(2, 1)$ and $\mathrm{circ}\,(2, 1, 1)$.

We note that constant additions in internal rounds of the original Poseidon$^\pi$ can also be reduced to a single non-zero field element per round. We discuss this reduction in Appendix E.1.



(a) Poseidon$^\pi$. **(b)** Poseidon2$^\pi$.

Figure 8: Poseidon$^\pi$ and Poseidon2$^\pi$ permutations with differences highlighted in red.

The Poseidon and Poseidon2 designers provided tables [GKR+21, Table 2], [GKS23b, Table 1] with sample round numbers. In addition, the reference implementations[21] of both variants contain `SageMath` [Sag26] scripts to compute R_F and R_P given a prime q and state size t. In Table 5 we present round numbers for security level 128 bits, which have been generated with the scripts. For all considered parameters the scripts produced identical round numbers. Poseidon has been proposed for sponge mode and Poseidon2 has been proposed for compression as well as sponge mode.

Third-Party Cryptanalysis. Like GMiMC, the Poseidon design tried to cover many possible field and state sizes with its cryptanalysis. Unsurprisingly, a dedicated analysis of concrete Poseidon/Poseidon2 parameters often improved upon the designer's initial

[21] https://extgit.isec.tugraz.at/krypto/hadeshash, https://github.com/HorizenLabs/poseidon2

Table 5: Partial round numbers R_P for Poseidon and Poseidon2 with a target security level of 128 bits. t denotes the state size, α the S-box exponent, and d the digest size. All instances use $R_F = 8$. No round numbers require updating under the latest attacks.

(a) $\log_2(p) \approx 256$, $d = 1$.

$t\backslash\alpha$	3	5	7	11
3	83	56	46	37
4	84	56	46	37
5*	84	56	46	37
8	84	57	47	37

* Poseidon only.

(b) $\log_2(p) \approx 64$, $d = 4$.

$t\backslash\alpha$	3	5	7	11
8	41	27	22	17
12	42	27	22	18
16	42	27	22	18
20	42	27	22	18

(c) $\log_2(p) \approx 32$, $d = 8$.

$t\backslash\alpha$	3	5	7	11
16	21*	14	13	13
24	23	22	21	21
32†	31	30	30	30
40†	38	38	38	37

* $R_P = 20$ for KoalaBear.
† Poseidon only.

analysis. Despite its slightly outdated cryptanalysis, to the best of our knowledge no full round Poseidon/Poseidon2 instances with security level 128 bits have been broken yet.

Invariant Subspaces. The internal rounds of Poseidon and Poseidon2 are vulnerable to invariant subspaces, also see Section 4.3.2. Beyne et al. [BCD+20, §6.2] proposed the linearization of internal rounds in Poseidon polynomial systems via invariant subspaces, being able to improve the complexity of preimage attacks. The attack exploited a weakness in the original design of Poseidon, in which the only condition on the matrix was of being MDS. This allows the existence of invariant subspace trail for *any* number of internal rounds. The problem has been later on solved by Grassi et al. [GRS21], who defined extra conditions on the linear layer to address this issue.

Grassi et al. [GKR25] revisited the linearization of internal rounds for Poseidon2. They worked under the strengthened assumption that an invariant subspace can cover at most $t-1$ internal rounds. They analyzed the polynomial models of sponge and compression mode. Their findings suggest that the original Poseidon/Poseidon2 cryptanalysis overas well as underestimates the complexity of algebraic attacks.

More recently, Li et al. [LLW26] showed how to extend this attack in the case of non-linear invariant subspaces, with the result that the nonlinear subspace model can analyze approximately twice as many internal partial rounds as the linear subspace model considered in ToSC 2025.

Statistical Cryptanalysis. Keller and Rosemarin [KR21] studied the number of active S-boxes in differential trails through internal Poseidon rounds. For this purpose, they extended a computational tool of Bar-On et al. [BDD+15] to characteristic $p > 2$. According to their analysis, for small t MDS matrices activate more S-boxes than estimated by the Poseidon designers. E.g., for $t = 6$ and $R_P = 84$ the designers estimated that at least 32 S-boxes are active whereas Keller and Rosemarin improved this lower bound to 60.

Algebraic Cryptanalysis. For the CICO-1 problem Bariant et al. [BBLP22] introduced the round-skipping technique for SPNs, see also Section 4.3.1. For Poseidon one can skip 2 rounds and reduce to CICO-1 a univariate polynomial of degree $\alpha^{R_F+R_P-2}$. For Poseidon2 one can skip 1 round and reduce CICO-1 to a univariate polynomial of degree $\alpha^{R_F+R_P-1}$. Moreover, they developed an improved routine to compute $\gcd(f, x^p - x)$ which runs in complexity $\tilde{\mathcal{O}}(\deg(f) \cdot \log_2(p))$ if $\deg(f) \leq p$.

Ashur et al. [ABM24] performed Gröbner basis experiments with the F4 algorithm [Fau99] for small-scale Poseidon/Poseidon2 instances. Their findings indicate that computing DRL Gröbner bases for Poseidon is easier than for a generic regular sequence in the same variables and degrees. Their extrapolations [ABM24, Table 2] reveal that the Poseidon cryptanalysis overestimates the complexity of computing a DRL Gröbner basis.

Though, the security of Poseidon instances with security level 128 bits is not affected.

Beyne and Verbauwhede [BV25a] generalized techniques to study degree growth of permutations over $\mathbb{F}_2^t$ to $\mathbb{F}_p^t$. With their techniques one can accurately predict the polynomial degree of iterated functions over $\mathbb{F}_p^t$ during the saturation phase. This enables the currently best degree growth estimation of Poseidon/Poseidon2 over 32 bit fields. Moreover, Beyne and Verbauwhede presented the first non-trivial degree growth estimation of inverse partial rounds [BV25a, Fig. 9], [BV25b, §D].

Bak et al. [BBB+25a] constructed Poseidon/Poseidon2 Gröbner bases with respect to a weighted monomial order. The Gröbner bases can take round-skipping techniques into account, and they confirm the ideal degree conjecture $D = \alpha^{r \cdot R_\mathrm{F} + R_\mathrm{P}}$ due to Ashur et al. [ABM24]. According to the complexity analysis of Bak et al. [BBB+25a, Table 2] monomial order conversion dominates the complexity of a Gröbner basis attack on Poseidon/Poseidon2.

Merz and Rodríguez García [MG26] improved round-skipping techniques for Poseidon2 by taking the circulant block matrix structure of M_E (Equation (51)) into account. round-skips for various Poseidon2 parameter sets were given in [MG26, Table 4, 5]. In the most favorable cases, the improved techniques skip two rounds and linearize some S-boxes in the third round. Combined with Bak et al.'s Gröbner bases [BBB+25a] this constitutes the current state of the art of Gröbner basis cryptanalysis of Poseidon2.

Motivated by Ethereum Poseidon cryptanalysis challenges, Zhao et al. [ZSVD26] attacked the Poseidon CICO-1 problem over Number Theoretic Transform (NTT) friendly prime fields with the Graeffe transform. Since NTT-based polynomial arithmetic is parallelizable this allows speedups in practical settings compared to computing the GCD with the field equation.

We also mention that Ethereum Foundation (EF) Poseidon CICO-2 challenges have been solved with state-of-the art bivariate resultant techniques [BBB+26, BTA26] and improved linearization techniques of full/partial rounds [SV26]. Moreover, independent works of Vitto [Vit26] and Jo [Jo26] showed that attacks on full Poseidon can be set up in the case in which the attacker can freely choose the MDS matrix that instantiates the external full rounds.

5.2.2 Neptune (ToSC'22)

Another variant of Poseidon is Neptune, which has been proposed by Grassi et al. [GOPS22] at ToSC 2022. It serves as the internal permutation within a sponge hash function, defined over the finite field $\mathbb{F}_p^t$ with a prime $p > 2^{63}$ and an even integer $t = 2t' \in \{2, 4, \ldots, 24\}$. The Neptune permutation consists of $R_\mathrm{F} = R_{\mathrm{f}_1} + R_{\mathrm{f}_2}$ full rounds, and R_P partial rounds, as illustrated Figure 9a. We summarize the key differences between Neptune$^\pi$ and Poseidon$^\pi$:

- The state size t must be an even number.
- Different matrices $\mathsf{M}_\mathrm{E} \in \mathbb{F}_p^{t \times t}$ and $\mathsf{M}_\mathrm{I} \in \mathbb{F}_p^{t \times t}$ are applied for the external and internal layers respectively.
- The external matrix M_E is applied before the first external round.
- In external rounds the state $\mathbf{x} = (x_1, \ldots, x_t)^\intercal \in \mathbb{F}_p^t$ is interpreted as

$$\mathbf{x} = \begin{pmatrix} \mathbf{x}_u \\ \mathbf{x}_l \end{pmatrix} = \begin{pmatrix} x_1 & x_3 & \ldots & x_{t-1} \\ x_2 & x_4 & \ldots & x_t \end{pmatrix} \in \mathbb{F}_p^{2 \times \frac{t}{2}}. \tag{53}$$

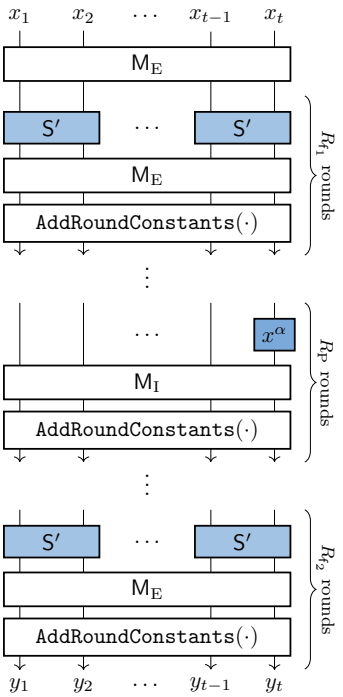
  - The S-box S of the external rounds is a collection of t' permutations of $\mathsf{S}' : \mathbb{F}_p^2 \to \mathbb{F}_p^2$ applied on the columns of $\mathbf{x}$, i.e.,

$$\mathsf{S}(\mathbf{x}) = \big(\mathsf{S}'(x_1, x_2), \mathsf{S}'(x_3, x_4), \ldots, \mathsf{S}'(x_{t-1}, x_t)\big). \tag{54}$$

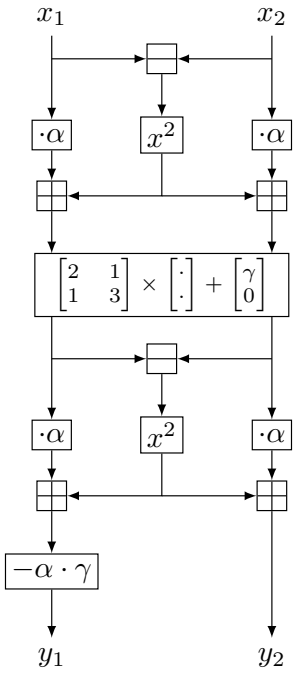
The external S-box S' is based on the Lai–Massey scheme, its circuit representation is depicted in Figure 9b where $\alpha, \gamma \in \mathbb{F}_p^\times$.

– The external linear layer $\mathsf{M}_\mathrm{E} \in \mathbb{F}_p^t$ consists of two MDS matrices $\mathsf{M}, \mathsf{M}' \in \mathbb{F}_p^{\frac{t}{2} \times \frac{t}{2}}$ which are applied row-wise on $\mathbf{x}$, i.e.,

$$\mathsf{M}_\mathrm{E}(\mathbf{x}) = \begin{pmatrix} \mathsf{M}\mathbf{x}_u \\ \mathsf{M}'\mathbf{x}_l \end{pmatrix}. \tag{55}$$



(a) NEPTUNE$^\pi$.



(b) NEPTUNE external S-box, $\alpha, \gamma \in \mathbb{F}_p^\times$.

Figure 9: NEPTUNE.

For the matrices M, M' Grassi et al. required that

(i) $\mathsf{M} \neq \mu \cdot \mathsf{M}'$ for $\mu \in \mathbb{F}_p^\times$, and

(ii) $\mathsf{M}_{i,j} \neq \mathsf{M}'_{i,j}$ for all $1 \leq i, j \leq \frac{t}{2}$.

However, as identified by Urani and Grassi [UG26] these conditions are not sufficient to guarantee exponential degree growth 4^R for the full rounds. Let $\mu^{(1)}_{2\cdot i,j} = \mathsf{M}_{i,j}$ and $\mu^{(1)}_{2\cdot i+1,j} = \mathsf{M}'_{i,j}$, recursively we then define

$$\mu^{(r)}_{2\cdot i,j} = \sum_{k=1}^{\frac{t}{2}} \mu^{(1)}_{2\cdot i,k} \cdot \left(\mu^{(r-1)}_{2\cdot k,j} - \mu^{(r-1)}_{2\cdot k+1,j}\right)^4, \tag{56}$$

$$\mu^{(r)}_{2\cdot i+1,j} = \sum_{k=1}^{\frac{t}{2}} \mu^{(1)}_{2\cdot i+1,k} \cdot \left(\mu^{(r-1)}_{2\cdot k,j} - \mu^{(r-1)}_{2\cdot k+1,j}\right)^4. \tag{57}$$

To achieve exponential degree growth the following additional condition must be satisfied [UG26, Proposition 1]

(iii) $\mu_{2\cdot i,j}^{(r-1)} - \mu_{2\cdot i+1,k}^{(r-1)} \neq 0$, $\mu_{2\cdot i,j}^{(r)} \neq 0$ and $\mu_{2\cdot i+1,j}^{(r)} \neq 0$ for all $1 < r \leq R_{\mathrm{F}}$ and all $1 \leq i, j \leq \frac{t}{2}$.

The partial rounds on the other hand follow the Poseidon strategy, but for the internal matrix $\mathsf{M}_{\mathrm{I}} \in \mathbb{F}_p^{t\times t}$ Neptune allows arbitrary invertible matrices which do not admit infinitely long subspace trails. For efficiency reasons the Neptune designers propose to use internal matrices like in Poseidon2, see Equation (52). We also recall that addition of constant vectors within the internal rounds can be reduced to a single constant addition per internal round, see Appendix E.1.

Neptune has been proposed for sponge mode only. For the round numbers, the Neptune designers propose to use $R_{\mathrm{f}_1} = 4$ and $R_{\mathrm{f}_2} = 2$ for all instances. For the internal rounds they proposed the following formula [GOPS22, §7.2] for security level κ:

$$R_{\mathrm{P}} = \left\lceil 1.125 \cdot \left\lceil \log_\alpha(2) \cdot \left(\min\left\{\kappa, \log_2(p)\right\} - 6 \right) + 3 + t + \log_\alpha(t) \right\rceil \right\rceil . \tag{58}$$

In Table 6 we summarize the round numbers for Neptune.

Table 6: Partial round numbers R_{P} for Neptune with a target security level of 128 bits. t denotes the state size, α the S-box exponent, and d the digest size. All instances use $R_{\mathrm{f}_1} = 4$ and $R_{\mathrm{f}_2} = 2$. No round numbers require updating under the latest attacks.

(a) $\log_2(p) \approx 256$, $d = 1$.

$t\backslash\alpha$	3	5	7	11
4	97	69	59	49
8	102	74	63	54
12	107	79	68	59
16	112	84	72	63

(b) $\log_2(p) \approx 64$, $d = 4$.

$t\backslash\alpha$	3	5	7	11
8	57	43	38	33
12	61	48	42	38
16	67	52	48	42
20	71	57	52	48

Third-Party Cryptanalysis. Invariant subspace cryptanalysis for Poseidon/Poseidon2 is also applicable to Neptune. Grassi et al. [GKR25] investigated Neptune and Poseidon polynomials system where the first k internal rounds have been linearized with an invariant subspace. A key difference between Neptune and Poseidon is that Neptune allows a Start-in-the-Middle (SitM) polynomial model. Since the S-box S' is a composition of two quadratic Lai–Massey functions, see Figure 9b, both S' and S'^{-1} have polynomial degree 4. Thus, also the inverse of an external round R_{E} has degree 4. Via a SitM modeling this property can be utilized to reduce polynomial degrees in Neptune polynomial systems [GKR25, §6]. Via the dedicated subspace trail analysis Grassi et al. could significantly improve upon Neptune's original Gröbner basis complexity estimations [GKR25, Figure 7]. Moreover, they demonstrated that Neptune 's security in compression mode cannot be reduced to the CICO problem in sponge mode. Even if Neptune was proposed in sponge mode only (hence, this result does not have a practical impact on the security of Neptune), to the best of their knowledge, this marks the first concrete example where preimage finding is easier than solving the corresponding CICO problem, highlighting the need to consider the mode of operation in security analysis. The result by Grassi et al. was recently improved by Li et al. [LLW26], who extended the attack to non-linear invariant subspaces. In particular, their non-linear subspace model can analyze approximately twice as many internal partial rounds as the linear subspace model.

Like for Poseidon, Beyne and Verbauwhede's [BV25a] techniques can be used to study the degree growth of inverse partial rounds [BV25a, Fig. 9], [BV25b, §D] during the saturation phase.

For the CICO-$t/2$ problem Bak et al. [BBB+25a] developed a Neptune Gröbner basis with respect to a weight order. The ideal degree of Neptune in CICO-$t/2$ is given by $2^t \cdot \alpha^{R_{\mathrm{P}}}$. Moreover, they developed a Gröbner basis for Neptune in compression mode.

Urani and Grassi [UG26] revealed the existence of weak external linear layers M_{E} which fail to achieve uniform degree 4 across all components of an external round. They also developed a simple matrix criterion [UG26, Proposition 1] that ensures uniform degree growth.

5.3 Generalized Feistel Designs

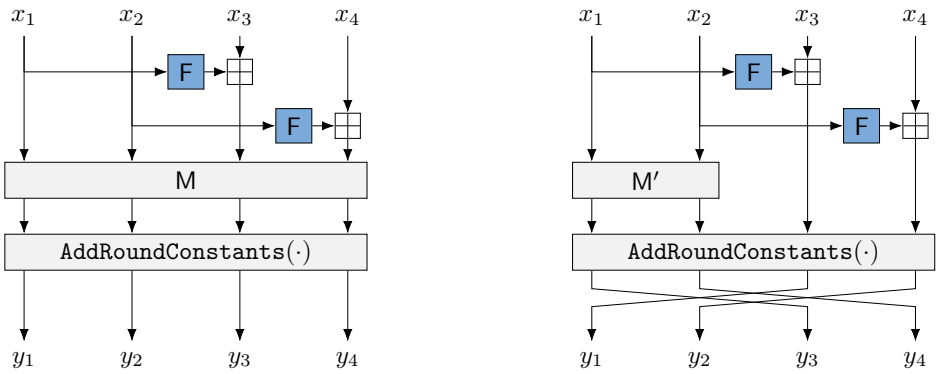
Beside GMiMC, generalized Feistel structures for AO hashing have been reinvestigated in two recent ePrints by Boeuf and Perrin [BP25b] and Grassi et al. [GMBM+26].

5.3.1 Moirai

Motivated by the progress in algebraic cryptanalysis Boeuf and Perrin introduced a performance metric [BP25b, §3], called *ALFOMs* (Algebraic Figures of Merit), which aims to quantify R1CS/AIR cost in terms of the ideal degree of a primitive. They introduced a high-level family of Feistel permutations [BP25b, §5], called Moirai, which has optimal R1CS/AIR cost in terms of the ideal degree according to their analysis. Let $t \geq 2$ be an even integer, and let $\mathsf{F} : \mathbb{F}_p \to \mathbb{F}_p$ be a function. As Feistel layer, the Moirai round function applies

$$\begin{pmatrix} x_1 \\ \vdots \\ x_t \end{pmatrix} \mapsto \begin{pmatrix} \{x_i\}_{1 \leq i \leq t/2} \\ \{x_{t/2+j} + \mathsf{F}(x_j)\}_{1 \leq j \leq t/2} \end{pmatrix}. \tag{59}$$

For the linear layer, Moirai specifies two choices: Either the state is mixed with a full MDS matrix $\mathsf{M} \in \mathbb{F}_p^{t \times t}$ or only $\frac{t}{2}$ many elements are mixed with an MDS matrix $\mathsf{M}' \in \mathbb{F}_p^{\frac{t}{2} \times \frac{t}{2}}$ and a swap is performed to exchange the two blocks. A Moirai permutation first applies the linear layer followed by R iterations of the round function. High-level circuits of the full and half MDS round functions are depicted in Figure 10.



(a) Full MDS. **(b)** Half MDS.

Figure 10: High level circuits of Moirai.

For the function F, Boeuf and Perrin propose three variants:

(i) $\mathsf{F}(x) = x^{\alpha}$ (Clotho variant),

(ii) $\mathsf{F}(x) = x^{\frac{1}{\alpha}}$ if $\gcd(\alpha, p-1) = 1$ (Lachesis variant), and

(iii) $\mathsf{F}(x) = (x^{\alpha} - \theta)^{-1}$ (Atropos variant).

A dedicated Gröbner basis analysis is performed for these variants. In particular, for $\mathsf{F}(x) = x^\alpha$ the authors construct a DRL Gröbner basis by hand. We note that this is an alternative to the Gröbner basis construction via a change of coordinates (Section 4.3.4).

However, no cryptanalysis against other attack vectors was performed, and no concrete instances were proposed. Therefore, we omit the Moirai family from our performance comparison in Sections 9 and 10.

5.3.2 pSquare-hash

When cryptographic algorithms are deployed on embedded devices it is important to protect them against physical attacks like fault injection or power analysis. A standard countermeasure against physical attacks is masking, which represents secret data as multiple random shares and performs cryptographic computations on those shares rather than on the secret data itself. For symmetric cryptography, a series of recent works [MMMS23, CMM+23, GMM+24, FMM+24, MSS24, BGM+25] presents the case that masking over a prime field $\mathbb{F}_p$ provides a better performance/security trade-off than standard masking over $\mathbb{F}_2$. Though, prime field masking requires new symmetric primitives which are native in $\mathbb{F}_p$. In particular, the Mersenne prime numbers $\mathrm{M7} = 2^7 - 1$ and $\mathrm{M31} = 2^{31} - 1$ are well-suited candidates for performant prime field masking on 8/32-bit processor architectures. So far, the suite of prime field masking friendly ciphers consists of `AES-prime` [MMMS23] (SPN, M7), small-pSquare [GMM+24] (tweakable Feistel, M7) and mid-pSquare [BGM+25] (tweakable Feistel, M31). The latest installment in this line of research is the *tweakable hash function* pSquare-hash [GMBM+26] for applications in post-quantum hash-based signature schemes.

The latest generation of hash-based signature schemes [Hül13, BHK+19, HK22] relies on a tweakable hash function

$$\mathsf{TH} : \mathcal{P} \times \mathcal{T} \times \mathcal{M} \to \mathcal{H}, \tag{60}$$

which maps a public parameter $p \in \mathcal{P}$, a public tweak $t \in \mathcal{T}$ and a secret message $m \in \mathcal{M}$ to the hash value $h \in \mathcal{H}$. In the standardized variants of SPHINCS$^+$ [NIS25] a tweakable hash function is obtained by concatenation of public parameter, tweak and message and hashing them together with SHA-2/3. A natural performance improvement in environments where masking is required is therefore the separation between public (public parameter, tweak) and secret (message) inputs to minimize the total number of masked operations.

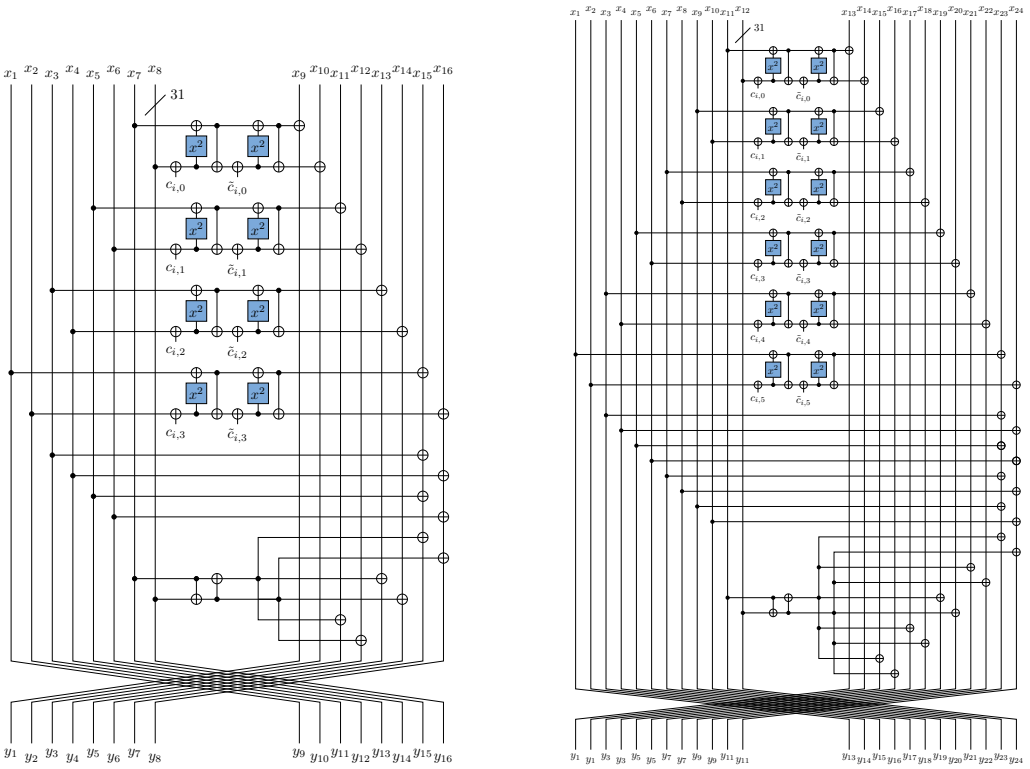
pSquare-hash is a Feistel hash function for M31. Since pSquare-hash tries to minimize the total number of arithmetic operations for masking efficiency it is at the same time arithmetization-friendly over 32-bit prime fields. In pSquare-hash the public values of a hash-based signature scheme are digested in a tweak schedule, whose outputs are added every 4 rounds to the state of the Feistel permutation much like a classic block cipher. While in this work we only focus on the untweaked variant of pSquare-hash, it is still worthwhile mentioning that the tweaked variant might also be of interest for the aggregation of hash-based signatures in a ZK proof, see e.g. [KCLM22, DKKW25].

pSquare-hash is defined for $\log_2(p) \approx 32$ and $t \in \{16, 24\}$. Its round function depicted in Figure 11 is a modified version of Berger et al.'s [BMT14] "Extended Generalized Feistel Network". In addition, pSquare-hash applies an Input/Output matrix

$$\mathbf{M}_{\mathrm{IO}} = \begin{pmatrix} 1 & 1 \\ 2 & 1 \end{pmatrix} \otimes \mathbf{I}_{\frac{t}{2}\times\frac{t}{2}} = \begin{pmatrix} \mathbf{I}_{\frac{t}{2}\times\frac{t}{2}} & \mathbf{I}_{\frac{t}{2}\times\frac{t}{2}} \\ 2\cdot\mathbf{I}_{\frac{t}{2}\times\frac{t}{2}} & \mathbf{I}_{\frac{t}{2}\times\frac{t}{2}} \end{pmatrix}. \tag{61}$$

The (untweaked) pSquare-hash permutation is then defined as

$$\mathsf{P}(\mathbf{x}) = \mathbf{M}_{\mathrm{IO}}\mathsf{R}^{(R-1)} \circ \cdots \circ \mathsf{R}^{(0)}(\mathbf{M}_{\mathrm{IO}}\mathbf{x}). \tag{62}$$



(a) $t = 16$. **(b)** $t = 24$.

Figure 11: pSquare-hash circuits, where $\tilde{c}_{i,j} \coloneqq c'_{i,j} - c_{i,j} \in \mathbb{F}_p$. Courtesy of the pSquare-hash designers.

pSquare-hash has been proposed for compression and sponge mode. For both supported state sizes $t \in 16, 24$ with digest size $d = 8$, pSquare-hash uses $R = 52$ rounds to target 128-bit security [GMBM+26, §4.5].

Third-Party Cryptanalysis. Due to its recency, no third-party cryptanalysis for pSquare-hash is available yet.

6 ZK-Friendly Hash Functions: Type-II (CCZ-Equivalence)

In this section we turn to the second class of ZK-friendly hash functions, whose design strategies leverage *CCZ-equivalence* to decouple the cost of evaluating a primitive from the cost of verifying a claimed evaluation inside an arithmetic circuit. Representative examples of Type-II primitives include the extended *Marvellous* family (e.g., *Rescue* [AAB+20], *Rescue-Prime* [SAD20], XHash [ABK+26]), `Anemoi` [BBC+23], Griffin [GHR+23], and Arion [CRST26].

Type-II Arithmetization-Oriented. The defining feature of a Type-II round is that it is built from a component of *high* algebraic degree that is nonetheless *CCZ-equivalent* to a *low*-degree function, through which the input-output relation can be verified. Since the number of constraints in the proof system is governed by the degree of the *verification* rather than that of the evaluation, a Type-II primitive reaches a high algebraic degree in far fewer rounds than a Type-I primitive, which relies on a genuinely low-degree round

function and therefore needs many rounds to resist algebraic attacks. The asymmetry is paid for in native evaluation: the map that is cheap to *verify* may be comparatively expensive to *compute*.

Definition 5 (CCZ-Equivalence [CCZ98])**.** Let $\mathsf{F}, \mathsf{G} : \mathbb{F}_q^m \to \mathbb{F}_q^m$ be two functions, and let $\Gamma_{\mathsf{F}} = \{(x, \mathsf{F}(x)) \mid x \in \mathbb{F}_q^m\}$ denote the graph of F. We say that F and G are *CCZ-equivalent* if there exists an affine permutation $\mathsf{L} : (\mathbb{F}_q^m)^2 \to (\mathbb{F}_q^m)^2$ such that $\Gamma_{\mathsf{F}} = \mathsf{L}(\Gamma_{\mathsf{G}})$.

Definition 6 (Type-II Arithmetization-Oriented)**.** A primitive is said to be "*Type-II Arithmetization-Oriented*" if

(i) round functions $\mathsf{R}^{(i)} : \mathbb{F}_q^t \to \mathbb{F}_q^t$ admit CCZ-equivalent functions $\mathsf{V}^{(i)} : \mathbb{F}_q^t \to \mathbb{F}_q^t$ with $\deg\left(\mathsf{V}^{(i)}\right) \ll \deg\left(\mathsf{R}^{(i)}\right)$, and

(ii) the prover circuit is based on the CCZ-equivalent functions $\mathsf{V}^{(i)}$.

All functions in a CCZ-equivalence class share identical resistance to differential and linear cryptanalysis, so these properties may be analyzed on any single representative; crucially, however, CCZ-equivalence does *not* preserve algebraic degree, which is what makes a low-degree verification representative useful. All Type-II primitives considered here exploit the same equivalence: the inverse power map $x \mapsto x^{1/\alpha}$ has near-maximal degree, yet its functional inverse $x \mapsto x^{\alpha}$ has minimal multiplicative complexity, and a permutation is CCZ-equivalent to its inverse, so $y = x^{1/\alpha}$ is verified by the single constraint $y^{\alpha} = x$. More generally, the CCZ-equivalent verification map need not be the inverse, or even a permutation. In `Anemoi`, for example, the high-degree S-box $\mathsf{F}_o : \mathbb{F}_q^2 \to \mathbb{F}_q^2$ is CCZ-equivalent to a low-degree verification function F_c, allowing $(u, v) = \mathsf{F}_o(x, y)$ to be verified as $(x, u) = \mathsf{F}_c(y, v)$, even though F_c is not a permutation.

6.1 *Marvellous* Designs

The *Marvellous* design strategy was introduced by Aly et al. [AAB+20] at FSE 2020, in response to the attacks [ACG+19] on the earlier MARVELLOUS designs JARVIS and FRIDAY [AD18]. It underlies a family of SNARK/STARK-friendly hash functions; over prime fields these include *Rescue* [AAB+20], *Rescue-Prime* [SAD20] and *Rescue-Prime Optimized* [AKM+22]. The recent XHash [ABK+26] derives from and extends the framework.

A *Marvellous* permutation is a Substitution-Permutation Network whose R rounds are each built from two SPN steps. A step is a standard SPN layer – an S-box applied to every state element, a multiplication by an MDS matrix, and a round-constant addition – and the two steps of a round differ only in which S-box they use. Writing α for the smallest exponent with $\gcd(\alpha, p-1) = 1$, one step applies $x \mapsto x^{\alpha}$ to each state element and the other one its functional inverse $x \mapsto x^{1/\alpha}$. Since one map is low-degree in the forward direction and the other low-degree in the backward direction, a single *Marvellous* round reaches high algebraic degree in both directions. Through appropriate *folding*, two neighboring steps collapse into a single degree-α transition constraint, since traversing $x^{1/\alpha}$ backward is just evaluating x^{α}, a concrete instance of the CCZ asymmetry. For *Rescue* and its variants *Rescue-Prime* and *Rescue-Prime Optimized*, see Section 6.1.1.

XHash [ABK+26] extends the *Marvellous* framework with a third, wider S-box: a power map over the cubic extension field $\mathbb{F}_{p^3}$ that mixes three base-field elements at once, at low per-word degree. More aggressive variants additionally apply the expensive high-degree S-box only partially, relying on the extension-field S-box to spread high degree across the rest. For details, see Section 6.1.2.

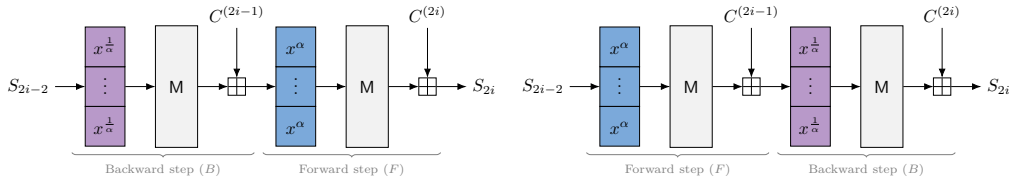
6.1.1 *Rescue* (ToSC'20)

Rescue [AAB+20] is a *Marvellous* instance defined over a prime field $\mathbb{F}_p$. It is originally specified as a block cipher with a dedicated key schedule, but turned into an (unkeyed) permutation by fixing the master key to zero, so that the round keys reduce to fixed round constants. Used in a sponge construction, this permutation yields an AO hash function; we describe the permutation itself in the following.[22]

Let $\alpha \geq 3$ be the smallest integer coprime to $p-1$. A round of *Rescue* is built from four operations: two S-box layers applying $x \mapsto x^\alpha$ and $x \mapsto x^{1/\alpha}$ respectively to each state element, multiplication by an MDS matrix $\mathsf{M} \in \mathbb{F}^{t \times t}$, and the addition of round constants. These are grouped into two steps:

- the first step applies the $x \mapsto x^{1/\alpha}$ layer, then M, then a round-constant addition; it is denoted (B)-step, as $x^{1/\alpha}$ has low degree in the *backward* direction;
- the second step applies the $x \mapsto x^\alpha$ layer, then M, then a round-constant addition; it is denoted (F)-step, as x^α has low degree in the *forward* direction.

A single *Rescue* round is visualized in Figure 12a. The *Rescue* permutation is obtained by an initial round-constant addition followed by R iterations of the round function. The round number is chosen as $R = 2 \cdot \max(\ell_0, \ell_1, 5)$, i.e. twice the longest attack reach (a 100% security margin), where ℓ_0 and ℓ_1 bound the rounds broken by differential and Gröbner-basis attacks, respectively, and the constant 5 serves as a sanity factor.



(a) *Rescue*: (BF) round. **(b)** *Rescue-Prime*: (FB) round.

Figure 12: One round of *Rescue*/*Rescue-Prime*, consisting of two SP-steps: (B), with low algebraic degree when evaluated in the backward direction, and (F), with low algebraic degree in the forward direction.

Rescue-Prime. *Rescue-Prime* [SAD20] is a family of ZK-friendly sponge hash functions proposed by Szepieniec et al. in 2020, building upon the *Rescue-XLIX* permutation, a simplified version of the *Rescue* permutation. Presented as a standard specification rather than a new design, and parameterized over a prime field with $\log_2 p \geq 32$ at a security level $80 \leq \kappa \leq 512$ [SAD20, Section 2], it introduces three changes to *Rescue*:

- *Flipped S-box order.* The two S-box layers are swapped so that the (F)-step comes first, making folding apply uniformly across the permutation rather than requiring special treatment at the first and last layers. See Figure 12b.
- *Simplified round constants.* Instead of reusing the block cipher's key schedule, the constants are expanded directly from a seed via SHAKE-256, giving a transparent, nothing-up-my-sleeve derivation and a single reference instantiation. Additionally, the initial round-constant addition present in *Rescue* (inherited from its block-cipher structure) is dropped, so the permutation is simply R iterations of the round function.

[22] Originally provided instance generation tool: https://github.com/KULeuven-COSIC/Marvellous.

- *Reduced security margin.* The margin is lowered from 100% to 50%. Whereas *Rescue* sets $R = 2 \cdot \max(\ell_0, \ell_1, 5)$ with ℓ_0, ℓ_1 the rounds reachable by differential and Gröbner-basis attacks, the envelope $\log_2 p \geq 32$, $80 \leq \kappa \leq 512$ ensures the Gröbner-basis attack dominates (i.e. $\ell_1 \geq \ell_0$), so the differential term drops out and $R = \lceil 1.5 \cdot \max(5, \ell_1) \rceil$. The reduced margin is justified by the lack of cryptanalytic progress against *Rescue* since its publication.

***Rescue-Prime Optimized* (RPO).** *Rescue-Prime Optimized* [AKM+22], proposed by Ashur et al. in 2022, specifies two concrete *Rescue-Prime*-based hash functions tailored for native hashing in STARK virtual machines such as Miden [Thr]. Both instances, targeting 128- and 160-bit security with state sizes $t = 12$ and $t = 16$ respectively, are defined over the Goldilocks field $\mathbb{F}_p$ with $p = 2^{64} - 2^{32} + 1$ (so that $\alpha = 7$), matching the VM's native field. The design departs from *Rescue-Prime* in three implementation-level aspects. First, it replaces the original MDS matrix with a circulant MDS matrix that admits fast matrix–vector multiplication. Second, it reorders the operations in each half-round as MDS multiplication, round-constant injection, and S-box application. This ordering enables a folding strategy that avoids multiplication by the inverse MDS matrix, for which the chosen circulant structure does not provide an equally efficient implementation. Finally, both instances use $R = 7$ rounds, one round fewer than prescribed by *Rescue-Prime*'s formula.

A subsequent variant, RPO-M31 [AT24], targets the Mersenne31 prime $p = 2^{31} - 1$ (so that $\alpha = 5$), with $t = 24$, capacity 8, and $R = 7$ rounds as before, claiming 124-bit security. The truncated circulant MDS matrix and the terminal linear layer are shared with the XHash instances over the same field. Note that this specification has been withdrawn in favor of the merged XHash paper [ABK+23], which however only covers the XHash instances over Mersenne31, leaving [AT24] the sole specification of RPO-M31.

Third-party Cryptanalysis. The cryptanalysis efforts on *Rescue*/*Rescue-Prime* concentrate mainly on algebraic attacks and solving the CICO problem.

Invariant Subspaces. In [BCP23] the authors show that in *Rescue*, the alternation between the S-box x^α and its inverse $x^{1/\alpha}$ allows, for certain round constants, the creation of chains of one-dimensional affine subspaces that make the permutation affine over these subspaces after several rounds. This explains the high differential uniformity already observed in previous work [BCL+20] on small instances of *Rescue*. The authors then exploit this same principle to deliberately construct two backdoored primitives that reuse the *Rescue* structure: the Stir hash function and the Snare block cipher, both facilitating the solving of CICO problem. Note that the proportion of weak constants is negligible and unlikely to occur in practice.

Algebraic Cryptanalysis. Let us concentrate on the CICO-k problem. Introducing one variable per α-inversion and fixing the $t - 2k$ surplus variables (the free input coordinates beyond the k kept symbolic) always yields a square system of $t \cdot R$ equations of degree α, hence an ideal degree of at most α^{tR}. Since no initial MDS-matrix multiplication is applied, the k input constraints propagate through the first monomial S-box layer at no cost. This leaves a square system of $t \cdot (R-1) + k$ equations of degree α, whose ideal degree attains the Bézout bound $\alpha^{t(R-1)+k}$ [BGL20, Theorem 11]. For *Rescue-Prime*, Bariant et al. [BBLP22, §4.4] showed how to skip the second S-box layer as well for t large enough (generalized to CICO-k by Bak and Perrin [BP25a], see Section 4.3.1): starting from an appropriate k-dimensional affine subspace after the first round yields a square system of $t \cdot (R-1)$ equations of degree α, hence an ideal degree of at most $\alpha^{t(R-1)}$, independently

of k.[23] Combining this modeling with Gröbner basis techniques, and estimating the cost as D_I^3, they improved the best known cryptanalysis of the reduced-round *Rescue-Prime* instances of the Ethereum Foundation bounties with $t = 2$ and $t = 3$, solving the $R = 4$, $t = 3$ instance in practice. The resulting complexities nevertheless remained above the claimed security levels.

A more recent line of work replaces the Gröbner basis machinery by resultant computations, focusing on the CICO-1 problem in *Rescue-Prime*. For $t = \alpha = 3$, Yang et al. [YZY+24a] eliminate the auxiliary variables in the forward modeling by iterated resultants, combined with a cubic substitution that keeps the degrees of all but two variables in each intermediate polynomial at most two, and recover the final univariate polynomial by fast Lagrange interpolation, whose degree is bounded by the previously observed ideal degree $D_I = 3^{3(R-1)}$. They further introduce the start-from-the-middle (SFTM) modeling, where the degrees-of-freedom are used to fix $t - 2$ coordinates approximately in the middle of the permutation, the two remaining symbolic variables being propagated in both directions, so that both the input and the output constraints become equations. This forgoes the skip of the second S-box layer, but it splits the elimination into two chains of half the length, ending in two bivariate polynomials whose degrees multiply to the ideal degree $D_I = \alpha^{t(R-1)+k} = 3^{3R-2}$ of this modeling. Since directly computing the resultant exceeds the available memory, the final univariate polynomial is instead reconstructed by fast Lagrange interpolation from D_I evaluations, each obtained as a Sylvester determinant. This step costs approximately $D_I^{1+\omega/2}$, dominates the overall attack complexity, and can at least be parallelized. For the challenge instances with $t = \alpha = 3$, they estimate that SFTM modeling increasingly outperforms forward modeling as the number of rounds grows; see [YZY+24b, Tables 2 and 4]. In practice at $R = 4$, both modelings improve on the previous attack, by a factor of about 290 and 115 respectively; at $R = 5$, only the SFTM modeling proves feasible, yielding the first practical attack on that instance. Both instances remain above their claimed security level.[24]

Bariant et al. [BBB+25b] start from the forward modeling and follow the same elimination path, but replace the cubic substitution by a dedicated reduction algorithm applied to the output of every multivariate multiplication, so that the intermediate polynomials stay reduced throughout and no separate substitution step is needed. Adopting the weighted monomial ordering of the FreeLunch attack [BBL+24a] – there used to certify that the modeling is already a Gröbner basis, here merely as an analysis tool – they observe that the weighted degree cannot increase under these reductions, which lets them control the otherwise unwieldy degree in the input variable and obtain rigorous complexity estimates for every step of the attack, including the construction of the system, whose cost [YZY+24a] assumed to be negligible; the iterated resultants can thereby be identified as the dominant step. Their analysis holds for arbitrary t and α, and yields a total cost of $\tilde{O}((2\alpha-1)^n) = \tilde{O}(\alpha^{c_\alpha n})$ with $c_\alpha = \log_\alpha(2\alpha-1)$, that is, an overhead of only $(2-1/\alpha)^n$ over the ideal degree $D_I = \alpha^n$, where $n = t(R-1)$ is the number of α-inversions left to model after the round-skip. For $t = \alpha = 3$, they improve the best practical attack on $R = 5$ by a factor of about 240 and mount the first practical attack on $R = 6$ (albeit over a prime of reduced size ≈ 55 bits, against the ≈ 64 bits of the challenge instance), and for $t = \alpha = 3$ with $\log_2 q \approx 512$ they are the first to break a full-round *Rescue-Prime* instance, namely the one claiming 512 bits of security, while the 256-bit level is left with essentially no margin.

Bak et al. [BBHN26] combine the SFTM modeling of [YZY+24a] with the elimination machinery of [BBB+25b], which they first generalize from one free variable to several in

[23] Note that bounds were verified experimentally for CICO-1 with $\alpha = 3$, the systems behaving generically [BBLP22, §4.4]: $D_I = 3^{3(R-1)}$ for $t = 3$, where the first two SPN steps can be skipped, and $D_I = 3^{2R-1}$ for $t = 2$, where only the first SPN step can be skipped.

[24] [YZY+24a] also claim to "find a 5-round collision which was originally thought to be 'hard' in the Ethereum Foundation challenge", but do not provide further details.

order to accommodate the two variables of the middle state. Writing $R = R_{in} + R_{out}$ for the position of the middle state, one is thus left with two polynomials in two variables of respective total degrees $d_{in} = \alpha^{t(R_{in}-1)+k}$ and $d_{out} = \alpha^{tR_{out}}$, whose resultant is now obtained from an adaptation of Villard's algorithm in $\tilde{O}(d_{in}^2 + d_{out}^{2-1/\omega} d_{in})$ for $d_{in} \geq d_{out}$, instead of by interpolation. As this cost is asymmetric while only the product $d_{in} d_{out} = D_I$ is fixed, it is minimized for the unbalanced choice $d_{in} = D_I^{(2\omega-1)/(3\omega-1)}$, that is, by placing a fraction $(2\omega-1)/(3\omega-1)$ of the rounds before the middle state for $2 \leq \omega \leq 3$, and the attack then runs in $\tilde{O}(D_I^{(4\omega-2)/(3\omega-1)}) = \tilde{O}(\alpha^{c_\omega(n+1)})$ for $c_\omega = \frac{4\omega-2}{3\omega-1}$ and the same $n = t(R-1)$ as above. For $t = \alpha = 3$, they improve the best practical attack on $R = 6$ by a factor of about 2.3, without reaching beyond that round number. Further, they claim theoretical attacks for several concrete instances with $t = 3$ for $\alpha = 3$ they reach $2^{112}/2^{215}$ at the designed 18/35 rounds for a target security level of 128/256 bits; for $\alpha \in \{5, 7\}$ only the 256-bit instances fall, at 2^{232} and 2^{247}, whereas the 128-bit ones survive, the estimate landing exactly on the claim for $\alpha = 5$ and slightly above it for $\alpha = 7$.

New Estimation of the Number of Rounds. For CICO-k with $k \geq 2$, no competitive attack on *Rescue-Prime* is known. All attacks discussed above target the CICO-1 problem. We expect that [BBHN26] outperforms [BBB+25c] whenever $R > 1 + \frac{c_\omega}{t \cdot (c_\alpha - c_\omega)}$. Since $1.2 \leq c_\omega \leq 1.25$ for $2 \leq \omega \leq 3$ and $1.25 < c_\alpha < 1.5$ for $\alpha \in \{3, 5, 7, 11\}$, we have $c_\alpha - c_\omega > 0$ throughout. As the threshold increases with c_ω and decreases with c_α, $R > 1 + \frac{1.2}{t \cdot (1.25-1.2)} = 1 + \frac{24}{t}$, that is $R \geq 14, 10, 7, 6$ for $t = 2, 3, 4, 5$. This is satisfied by all instances considered here, so that the required number of rounds is dictated by [BBHN26] alone. In particular, we require $\alpha^{c_\omega(t(R-1)+1)} \geq 2^\kappa$, yielding

$$R \;\geq\; 1 + \frac{1}{t}\left(\frac{\kappa}{c_\omega \log_2 \alpha} - 1\right). \tag{63}$$

The new round numbers (for a conservative choice of $\omega = 2$) are given in Table 7a. Where the round number is superseded by the attack of [BBHN26], we state the new requirement and give the original one in parentheses.

Table 7: (Updated) Round numbers R for *Rescue-Prime* with a target security level of 128 bits. t denotes the state size, α the S-box exponent, and d the digest size. If the round number has been updated, the original value is given in parentheses; the updated values in bold are reported without an additional security margin.

(a) $\log_2(p) \approx 256$, $d = 1$.

$t\backslash\alpha$	3	5	7	11
2	**35** (27)	**24** (20)	**20** (17)	**16** (15)
3	**24** (18)	**16** (14)	**14** (12)	11
4	**18** (14)	**13** (11)	**11** (9)	**9** (8)
5	**15** (12)	**10** (9)	**9** (8)	8

(b) $\log_2(p) \approx 64$, $d = 4$.

$t\backslash\alpha$	3	5	7	11
8	8	8	8	8
12	8	8	8	8
16	8	8	8	8
20	8	8	8	8

(c) $\log_2(p) \approx 32$, $d = 8$.

$t\backslash\alpha$	3	5	7	11
16	8	8	8	8
24	8	8	8	8
32	8	8	8	8
40	8	8	8	8

6.1.2 XHash (DCC'26)

The XHash family [ABK+23, ABK+26], proposed by Ashur et al. in DCC 2026, is a collection of STARK-friendly sponge hash functions obtained as a combination of the *Marvellous* design strategy with a new type of S-boxes defined over cubic field extensions. The underlying permutations are defined for two state sizes, and the authors propose concrete primes for the instantiations:

- a state of $t = 12$ elements, giving the permutation XHash12 and its more aggressive variant XHash8; the proposed instantiation uses the Goldilocks prime $p = 2^{64} - 2^{32} + 1$,

yielding the sponge hash functions XHash12-Goldilocks and XHash8-Goldilocks with rate $r = 8$ and capacity $c = 4$ with $\approx$ 128-bit security;

- a state of $t = 24$ elements, giving the permutation XHash24 and its more aggressive variant XHash16; the proposed instantiation uses the Mersenne-31 prime $p = 2^{31} - 1$, yielding the sponge hash functions XHash24-M31 and XHash16-M31 with rate $r = 16$ and capacity $c = 8$ with $\approx$ 124-bit security.

Let α be the smallest integer ≥ 3 such that $x \mapsto x^\alpha$ is a permutation of *both* the base field $\mathbb{F}_p$ and its cubic extension $\mathbb{F}_{p^3}$; that is, $\gcd(\alpha, p-1) = 1$ and $\gcd(\alpha, p^3-1) = 1$.[25] The permutation alternates two kinds of rounds: an (FB)-round, of similar shape as a *Rescue-Prime Optimized* round, and a novel $(P3)$-round operating over the cubic extension field:

- An (FB)-round consists of an (F)-step and a (B)-step: (F) applies constants, M, and then $x \mapsto x^\alpha$, while (B) applies M, constants, and then $x \mapsto x^{1/\alpha}$. In the aggressive variants, the full (B)-step is replaced by a partial one, the (B')-step, in which the inverse S-box is applied to only two out of every three state elements, the remaining third being left unchanged:[26]

$$x_i \longmapsto \begin{cases} x_i, & i \equiv 2 \pmod 3, \\ x_i^{1/\alpha}, & \text{otherwise}, \end{cases} \qquad 1 \leq i \leq t\,.$$

  It is the following $(P3)$-round that raises the degree of the skipped elements again.

- The $(P3)$-round adds round constants and then applies the extension-field S-box. The t state elements are grouped into $t/3$ consecutive triples; via the isomorphism $\mathbb{F}_{p^3} \cong \mathbb{F}_p^3$, each triple $(x_{3i-2}, x_{3i-1}, x_{3i})$, for $1 \leq i \leq t/3$, is identified with an element $X_i \in \mathbb{F}_{p^3}$, mapped through $X \mapsto X^\alpha$, and decomposed back into three elements of $\mathbb{F}_p$ ($t/3 = 4$ triples for $t = 12$, and 8 for $t = 24$). Unlike (F) and (B), this round uses no MDS matrix: the $\mathbb{F}_{p^3}$ power map already mixes each triple's three coordinates non-linearly.

The permutation ends with a finalization step (MC), consisting of multiplication by M followed by round-constant addition. The proposed instances alternate three (FB)-rounds with three $(P3)$-rounds, for a total of 6 rounds: $(FB)\,(P3)\,(FB)\,(P3)\,(FB)\,(P3)\,(MC)$. In the aggressive variants, (B) is replaced by (B'). See Figure 13 for the XHash-Goldilocks instances.

Third-Party Cryptanalysis. All third-party cryptanalysis targets the two Goldilocks permutations of the 2023 ePrint [ABK+23], XHash8 and XHash12; the M31 instances added later[AT24, ABK+26] have been analyzed only by the designers. Rijmen [Rij24] mostly confirmed the designers' differential analysis, adding a single-round saturation distinguisher and a probability-1 higher-order differential. His main objection (§2.4) is that "XHash12 activates at least as many S-boxes as XHash8, hence is at least as secure" does not follow. Following the *Algebraic FreeLunch* [BBL+24a] approach – which showed that the Gröbner basis step can be free, and found this applicable to the aggressive XHash8 with its partial S-box layer, though only for CICO-1 and far beyond brute force – Perrin [Per24] moves the security claim to the variable-elimination step, whose complexity is governed by

[25] For the proposed instantiations this gives $\alpha = 7$ for the Goldilocks and $\alpha = 5$ for Mersenne31.

[26] In the permutation's name the numeral records how many $x \mapsto x^{1/\alpha}$ S-boxes are applied: the full variants XHash12 and XHash24 apply one to every state element (12 of 12, resp. 24 of 24), the aggressive variants XHash8 and XHash16 apply them to only two out of every three (8 of 12, resp. 16 of 24).

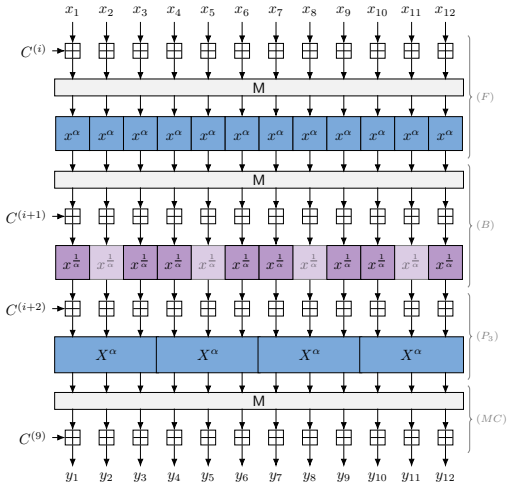


Figure 13: Two rounds, (FB) and $(P3)$, of XHash-Goldilocks. (MC) denotes the finalization step. While the conservative variant, XHash12, applies a full S-box layer in the (B) step, the more aggressive variant, XHash8, applies only a partial one.

the ideal degree D_I. To lower-bound it he introduces *b-almost basal* (b-AB) systems[27] and states the *Monotonous Ideal Degree Conjecture* (MIDC): the ideal degree is at least the product of the degrees of the leading terms in the basal part, i.e., the non-basal equations do not decrease the number of solutions. Note that Perrin is now a coauthor, and this line of argumentation was added to the published version [ABK+26] as part of its algebraic security rationale. The basal part is exactly the equations introducing a new variable at an α-th root, the remaining ones encode the output constraints. Each (FB)-round contributes one such equation of degree α per inverse S-box, while the $(P3)$-rounds introduce no new variables and thus no basal equations. Thus, over R rounds, the MIDC yields $D_I \geq \alpha^{\frac{R}{2} \cdot n_S}$, where n_S denotes the number of inverse S-boxes per round: $n_S = t$ for the full (B) layer of the classical variants, and $n_S = \frac{2t}{3}$ for the partial (B') layer of the aggressive ones. With $R = 6$, this gives $D_I \geq 7^{3 \cdot 12} = 7^{36}$ for XHash12 and $D_I \geq 5^{3 \cdot 24} = 5^{72}$ for XHash24, against $D_I \geq 7^{3 \cdot 8} = 7^{24}$ for XHash8 and $D_I \geq 5^{3 \cdot 16} = 5^{48}$ for XHash16. In the CICO-1 setting, XHash8 and XHash16 admit a sharper bound: the partial inverse layer lets the last equation have leading monomial $x_1^{\alpha^R}$ [ABK+26, §G.2.1], so the basal product gains a factor α^R, giving $D_I \geq 7^{30}$ and $D_I \geq 5^{54}$. This fails for the full inverse layer of XHash12 and XHash24, and beyond CICO-1.

The *Algebraic CheapLunch* makes the MIDC provable for the aggressive XHash variants in the CICO-$(t-k, k)$ setting whenever $n_S + k \leq t$, or equivalently $k \leq \frac{t}{3}$, under a genericity assumption and without round-skips [BBB+25a, Proposition 2]. Its *CheapLunch systems* are b-AB systems with one extra condition, arranged by the weighted order: the top-weight parts of the k non-basal output equations must lie in $\mathbb{F}[x_1, \ldots, x_k]$. A Gröbner basis of the non-basal part, which can be computed comparatively cheaply in $\mathbb{F}[x_1, \ldots, x_k]$, then extends to a Gröbner basis of the full ideal after adjoining the basal part.[28] The leading monomials of the basal polynomials are preserved, yielding the MIDC. Weighted Bézout adds a matching *upper* bound, $D_I \leq \alpha^{\frac{R}{2} n_S + Rk}$, i.e. 7^{24+6k} for XHash8 and 5^{48+6k} for

[27] A zero-dimensional system of n polynomials $f_1, \ldots, f_n$ in $x_1, \ldots, x_n$ is b-AB if, under some monomial order, the leading term of f_i is exactly $x_i^{d_i}$ for some $d_i > 0$ and $i \leq b$. Since a system with pairwise coprime leading terms is automatically a Gröbner basis, these first b equations – the *basal part* – make a b-AB system a "partial Gröbner basis"; n-AB then means a genuine Gröbner basis, i.e. a FreeLunch system.

[28] The Gröbner basis step is therefore cheap rather than free, so the cost of the attack may be dominated by the subsequent change of ordering, governed by D_I.

XHash16, tight in the (scaled-down) experiments of [BBB+25a], which cover XHash8 only. Considering the round-skip trick described in [BP25a, Appendix D], where the first (F) layer and, in addition, $s \leq \frac{t}{k} - k - 2$ S-boxes of the first (B') layer can be skipped, the CICO-k problem becomes noticeably easier for small k: for XHash8 this covers the entire (B') layer at $k = 1$ and two of its S-boxes at $k = 2$, and for XHash16 all 16 at $k = 1$, eight at $k = 2$, three at $k = 3$ and the (F) layer alone at $k = 4$, with nothing left beyond $k = 2$ resp. $k = 4$. Alternatively to the full skip, the input space can be reparametrized so that, after the first S-box layer, the state is affine in $s = \min(k, \lfloor t/k \rfloor - 2)$ of the input variables, whose weight may then be multiplied by α [BBB+25a, Appendix D.1]. As the weighted degrees of the system are unaffected, this increases only the denominator of the weighted Bézout bound, lowering it to $D_I \leq \alpha^{\frac{R}{2} n_S + Rk - s}$. In the preimage scenario $k = c = \frac{t}{3}$ this leaves $s = 1$, hence $D_I \leq 7^{47}$ for XHash8 and $D_I \leq 5^{95}$ for XHash16.

New Estimation of the Number of Rounds. The XHash authors do take the MIDC into account: in [ABK+26, §5.2.2] they argue that a Gröbner-basis attack would require both the bound on D_I to be tight and an FGLM-like algorithm of strictly sub-quadratic complexity $D_I^{1.5}$. Under that assumption, κ bits of security require $R \geq \frac{4\kappa}{3 \cdot n_S \cdot \log_2 \alpha}$, so that for $\kappa = 128$ one needs $R \geq 7.60$ for XHash8, 5.07 for XHash12, 4.59 for XHash16 and 3.06 for XHash24. Notably, XHash8 exceeds the specified $R = 6$. Using the sharper CICO-1 bound, available only for the aggressive variants, lowers this to $R \geq 6.08$ for XHash8 and 4.08 for XHash16, leaving the former still marginally short of the claimed level – what the authors themselves describe as barely an attack. The CheapLunch analysis does not worsen this picture but repairs it: at the preimage parameter $k = c = \frac{t}{3}$ the ideal degree is not merely bounded but attained in the experiments, and as no full round-skip is available there, the weight adjustment costs a single factor α, leaving $2^{197.9}$ for XHash8 at $R = 6$, even under the $D_I^{1.5}$ assumption.

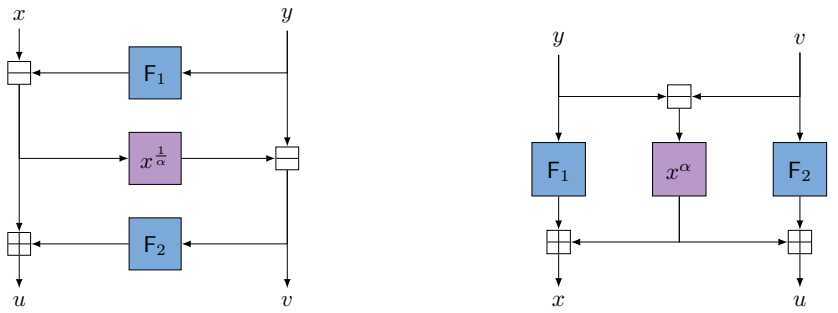
6.2 `Anemoi` (CRYPTO'23)

`Anemoi` is a family of ZK-friendly permutations proposed by Bouvier et al. [BBC+23] at CRYPTO 2023. It is defined over $\mathbb{F}_q$ for any $q \in \{p, 2^n\}$ with $\lceil \log_2 q \rceil \geq 10$ and operates on an even state of $t = 2t'$ field elements. On the design side, the paper introduces two reusable components: a novel S-box called `Flystel`, detailed below, and a new compression mode called `Jive` (see Section 3.2.2). Building on these, the designers propose two functions: the hash function `AnemoiSponge` and the compression function `AnemoiJive`. They further give concrete instantiations over the BN254 and BLS12-381 scalar fields, with $t = 2$ in both cases aiming for 127 bits of security. In the following, we focus on the prime-field variant of `Anemoi`.

The `Flystel` S-box. The `Flystel` is a novel S-box over $\mathbb{F}_q^2$ based on the butterfly structure [LTYW18]. It has two CCZ-equivalent forms: the open-`Flystel` F_o, a permutation used in the round function for (high-degree) evaluation, and the closed-`Flystel` F_c, a function used for (low-degree) verification in a circuit. They are defined as follows:

$$
\begin{aligned}
(u, v) = \mathsf{F}_o(x, y) &= \left(x - \mathsf{F}_1(y) + \mathsf{F}_2(y - \mathsf{E}^{-1}(x - \mathsf{F}_1(y))),\ y - \mathsf{E}^{-1}(x - \mathsf{F}_1(y))\right), \\
(x, u) = \mathsf{F}_c(y, v) &= \left(\mathsf{E}(y - v) + \mathsf{F}_1(y),\ \mathsf{E}(y - v) + \mathsf{F}_2(v)\right),
\end{aligned}
$$

where, over a prime field $\mathbb{F}_p$, the low-degree functions are $\mathsf{F}_1(x) = gx^2 + g^{-1}$, $\mathsf{F}_2(x) = gx^2$, and $\mathsf{E}(x) = x^\alpha$, with g a generator of $\mathbb{F}_p \setminus \{0\}$ and α a positive integer such that $\gcd(\alpha, p - 1) = 1$. Circuit representations are given in Figure 14.



(a) Open `Flystel` permutation F_o. **(b)** Closed `Flystel` function F_c.

Figure 14: The `Flystel` structure.

The `Anemoi` Permutation. The `Anemoi` permutation (see Figure 15) follows a classical SPN structure that iterates R rounds. As its state size $t = 2t'$ is always even, it is convenient to arrange the state into a $2 \times t'$ matrix

$$(x_1, x_2, \cdots, x_{t'}, y_1, y_2, \cdots, y_{t'}) = \begin{pmatrix} x_1 & x_2 & \cdots & x_{t'} \\ y_1 & y_2 & \cdots & y_{t'} \end{pmatrix},$$

whose t' columns are the *branches* of the permutation. Each round applies the following operations:

- First, round constants are added to each state element.
- The linear layer then applies two diffusion sub-layers:
  - each of the two *rows* is multiplied by a $t' \times t'$ MDS matrix;[29]
  - each *column* is transformed by the pseudo-Hadamard matrix $[2, 1; 1, 1]$.
- Finally, the nonlinear layer applies the open-`Flystel` F_o independently to every column of the state matrix:

$$(x_i, y_i) \mapsto \mathsf{F}_o(x_i, y_i) \quad \forall i \in \{1, 2, \ldots, t'\}.$$

After the last round, one additional linear layer is applied. Independent of the mode of operation, the number of rounds is set as $R \geq \max\{8, \min\{5, 1 + t'\} + R'\}$, where 8 is the minimum needed to resist statistical attacks, R' is the number of rounds required to resist algebraic attacks, and $\min\{5, 1 + t'\}$ is a state-size-dependent security margin capped at 5 rounds.

Remark 3. Concretely, the designers set $R' = 2 + \min\{r \in \mathbb{N} \mid \mathcal{C}_{\text{alg}}(r) \geq 2^{\kappa}\}$, where κ is the targeted security level, $\mathcal{C}_{\text{alg}}(r)$ is the complexity of computing a Gröbner basis for the CICO-t' problem on r rounds of the permutation (with a lower bound on the solving degree extrapolated from small-scale experiments on $t' = 1$) and the two extra rounds account for a second algebraic modeling whose complexity could not be estimated reliably. Note, however, that the modes do not fix this particular setting: `AnemoiSponge` leaves the split between rate and capacity free, and $\texttt{AnemoiJive}_b$ allows b-to-1 compression for any b dividing the state size, so the relevant CICO instance may differ from the analyzed one.

[29] The MDS matrices applied to the two rows differ by a word rotation, thereby breaking the column structure imposed by the S-box layer. For $t' \leq 4$, they are built from the minimum-addition matrices of Duval and Leurent [DL18].

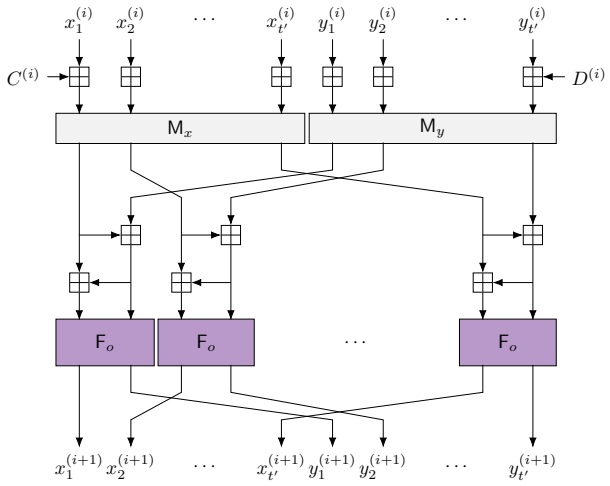


Figure 15: One round of `Anemoi`.

Third-Party Cryptanalysis. As for *Rescue*, most third-party cryptanalysis is of algebraic nature, concentrating on the CICO problem, i.e., the sponge setting.

Linear Cryptanalysis. On the statistical side, progress has been made on the linear properties of the `Flystel`. The `Anemoi` designers conjectured that the absolute value of the Walsh transform of the `Flystel` is upper bounded by $q \cdot \log(q)$ [BBC+22, Conjecture 1]. With methods from algebraic geometry the stronger bound $(\alpha - 1) \cdot q$ was independently proven by Steiner [Ste25c] and Beyne and Bouvier [BB24] (generalized to other Feistel structures in [BB26]).

Algebraic Cryptanalysis. In algebraic cryptanalysis, central is the ideal degree D_I, which the designers conjectured $D_I = (\alpha + 2)^R$ for the CICO-1 problem when $t = 2$ [BBC+22, Conjecture 2]. Koschatko et al. [KLR24] remain in this setting but analyze all three steps of a Gröbner basis attack. By assessing each step experimentally or through theoretical bounds, they obtain the *six worlds* and tighten the theoretical estimates for the ideal degree using the multihomogeneous Bézout bound. They conclude that the designers' round-number derivation does not always suffice: for the second modeling approach, $\alpha = 3$ and $\kappa = 256$ require at least 41 rather than 37 rounds.

The importance of considering all steps of a Gröbner basis attack is underlined by the FreeLunch approach of Bariant et al. [BBL+24a], in which the first step may come for free using a suitable monomial ordering. For `Anemoi`, however, no such *FreeLunch order* exists for the direct modeling, so the system is replaced by one generating a subideal, obtained through a chain of explicit reductions whose cost is only heuristically bounded. The univariate polynomial is then computed as the characteristic polynomial of a multiplication matrix, which by exploiting the block structure of that matrix costs $\tilde{O}(u \cdot \alpha^{\omega R})$ field operations, where u is the degree of the leading term in the free variable and $\alpha^R u$ the degree of the subideal, provably exceeding D_I by at most a factor of roughly α. For $\alpha = 3$ and $\omega = 2.81$, this gives 2^{118} at $R = 21$ and 2^{203} at $R = 37$, the first estimates below the claimed security levels of $\kappa = 128$ and $\kappa = 256$. They also provide the first practical attack, solving the CICO-1 problem for $R = 7$ rounds over a 55-bit prime in about two days and 42 GB of memory [BBL+24a, Table 7].

Campa and Roy [CR25a] lift the restriction to two branches, considering the CICO -t' problem for any $t' \geq 1$ in odd characteristic. They introduce a new model better suited to multiple branches and explicitly construct a (weighted) DRL Gröbner basis for

$\alpha \in \{3, 5, 7, 11\}$. This makes the first step provably polynomial in R and t', and settles [BBC+22, Conjecture 2], showing that $D_I = (\alpha + 2)^{Rt'}$ and proving the shape form of the ideal assumed in earlier work. For the second step they compare three methods, and settle on the eigenvalue method realized by Wiedemann's algorithm, which is cheaper and independent of ω, at the cost of a conjecture on the sparsity of the multiplication matrix. Only single-branch instances fall below their claimed security level [CR25a, Table 2]: for $t' = 1$ and $\alpha = 3$, the estimates are 2^{111} at $R = 21$ against $\kappa = 128$ and 2^{196} at $R = 37$ against $\kappa = 256$; for $\alpha = 5$, they obtain 2^{253} at $R = 37$ against $\kappa = 256$. All instances with $t' \geq 2$ remain above their claimed security level. For $\alpha = (3, 5, 7, 11)$, they experimentally reach $R = (8, 6, 5, 4)$ at $t' = 1$ and $R = (3, 3, 2, 2)$ at $t' = 2$, and for $t' = 3$ a single instance with $\alpha = 3$ and $R = 2$ [CR25a, Tables 7 to 9]. For $\alpha = 3$ and $t' = 1$ this improves [BBL+24a] by one round, though over a 31-bit prime.

The resultant-based line abandons the Gröbner basis machinery altogether, but keeps the modeling with one variable and one equation per α-inversion, as in the designers' second modeling, yielding a quasi-triangular system. Yang et al. [YZY+24a] consider CICO-1 for $t = 2$ over $\mathbb{F}_p$ with $\alpha = 3$ and apply their SFTM modeling to halve the elimination chain: since there are no coordinates left to fix, the middle state after $\lfloor R/2 \rfloor$ rounds is taken fully symbolic and propagated in both directions. Eliminating the round variables by iterated resultants, interleaved with the cubic substitution, leaves two bivariate polynomials whose degrees they observe experimentally to be $(\alpha + 2)^{\lfloor R/2 \rfloor}$ and $(\alpha + 2)^{\lceil R/2 \rceil}$, so that their product is again the ideal degree D_I. The final univariate polynomial, of degree D_I, is then reconstructed by fast Lagrange interpolation, which dominates the attack at about $D_I^{1+\omega/2}$, but is trivially parallelizable. For $R = 21$ an d $\omega = 2.376$ this gives 2^{110}, and in practice they improve the $R = 7$ attack of [BBL+24a] by a factor of about 56 and reach $R = 8$ for the first time over the same 55-bit prime in ≈ 10 hours.

Bariant et al. [BBB+25b] return to the forward modeling in the CICO-1 setting for arbitrary t and α, but replace the cubic substitution by a custom reduction applied after every multiplication inside the resultant computation. For complexity analysis, they adopt the weighted ordering of the FreeLunch attack [BBL+24a] and observe that the weighted degree cannot increase under their reductions. Consequently, they can rigorously bound all steps, including the previously neglected generation of the system, with the iterated resultants emerging as the dominant step at total cost of $\tilde{O}\left(d_P(h)(2\alpha - 1)^n\right)$, where $d_P(h) \leq \left(\frac{\alpha+2}{\alpha}\right)^R$ is the weighted degree of the final equation and n the number of α-inversions left to model. For $t' = 1$ no round can be bypassed and $n = R$, whereas for $t' \geq 2$ a suitable choice of input leaves all but one component of the first non-linear layer constant, so that $n = t'(R - 1) + 1$. In both cases the degree of the final univariate polynomial, and with it the ideal degree, is bounded by $D_I \leq \alpha^n d_P(h)$, that is, by $(\alpha+2)^R$ for $t' = 1$ – the value conjectured by the designers – and by $(\alpha + 2)^R \alpha^{(t'-1)(R-1)}$ in general. Since $d_P(h)$ occurs both in the cost and in D_I, it cancels, and the attack costs $\tilde{O}\left(D_I\,(2 - 1/\alpha)^n\right)$, that is, an overhead of only $(2 - 1/\alpha)^n$ over the ideal degree, whatever the growth rate of the weighted degree. Consequently, every `Anemoi` instance with $t' = 1$ falls short of its claimed security level: for $\alpha = 3, 5, 7, 11$ and $\omega = 2.81$, the 128-bit instances cost $2^{80}, 2^{96}, 2^{103}, 2^{111}$ and the 256-bit instances $2^{130}, 2^{157}, 2^{170}, 2^{187}$, improving on the FreeLunch estimates by 38–87 and 73–171 bits respectively, with larger gains for larger α. Wider states fare better: at 128 bits, the claim is met from $t' = 4$ for $\alpha = 3$, from $t' = 3$ for $\alpha = 5, 7$, and from $t' = 2$ for $\alpha = 11$; at 256 bits, within the tabulated range $t' \leq 6$, only $t' = 6$ survives for $\alpha = 5, 7$ and $t' = 4$ for $\alpha = 11$, with no surviving $\alpha = 3$ instance. In practice, for $\alpha = 3$ and $t' = 1$ over the same 55-bit prime, they solve CICO-1 on $R = 8$ rounds in 85 seconds, before reaching $R = 9$, 10 and 11 for the first time, the last one in under two days.

In [QCC+25], the authors apply their Mixed-Integer Quadratic Constraint Programming (MIQCP) tool to `Anemoi` (with S-boxes of degree $\alpha = 3$) to automatically find the most

efficient system of equations to break via a Gröbner basis attack. Up to 4 rounds, the tool finds attack complexities lower than the lower bound announced by the `Anemoi` designers, showing that their security estimate was too optimistic for this number of rounds. Beyond that, the results remain incomplete as the model could not be solved within an 12-hour time limit.

Bak et al. [BBHN26] generalize the reduction procedure of [BBB+25b] from a single free variable to several, so that iterated resultants successively eliminate the n temporary variables modeling the α-inversions and return a square system in the free variables alone. Notably, with at least two free variables, the entire elimination runs in time quasi-linear in its output size, in contrast to the $(2-1/\alpha)^n$ overhead of the univariate case [BBHN26, Proposition 6]. The final step, however, restricts the considered scenarios to at most two free variables: the bivariate resultant is computed using an adaptation of Villard's algorithm in $\tilde{O}(d_1^2 + d_2^{2-1/\omega} d_1)$, where $d_1 \geq d_2$ are the respective total degrees, rather than via interpolation. For `Anemoi`, the authors consider the CICO-t' problem for $t' \in \{1, 2\}$ and arbitrary values of α:[30]

- CICO-1 for $t = 2$, using the SFTM model. Writing $R = R_{in} + R_{out}$ for the position of the middle state, one is left with two polynomials in two variables of respective total degrees $d_{in} = (\alpha+2)^{R_{in}}$ and $d_{out} = (\alpha+2)^{R_{out}}$. As the cost of computing the final bivariate resultant is asymmetric while the product $d_{in} d_{out} = D_I = (\alpha+2)^R$ is fixed, it is minimized for the unbalanced choice $d_{in} = D_I^{(2\omega-1)/(3\omega-1)}$, that is, by placing a fraction $(2\omega-1)/(3\omega-1)$ of the rounds before the middle state. The attack then runs in $\tilde{O}((\alpha+2)^{c_\omega R}) = \tilde{O}(D_I^{c_\omega})$ with $c_\omega = \frac{4\omega-2}{3\omega-1}$, against $\tilde{O}(D_I(2-1/\alpha)^R)$ in [BBB+25b]. This attack outperforms the best previous one for $\alpha \in \{3, 5, 7, 11\}$ and $2 \leq \omega \leq 2.81$, though only marginally so for $\alpha = 11$: for $\alpha = 3, 5, 7$ and $\omega = 2.81$, the 128-bit instances cost $2^{70}, 2^{84}, 2^{90}$ and the 256-bit instances $2^{117}, 2^{141}, 2^{154}$, improving on the previous estimates by 10–13 and 13–16 bits respectively, with larger gains for larger α. They practically reach $R = 11$ rounds, which it solves in 33 hours against 46, without reaching beyond that round number.

- CICO-2 for $t = 4$, using the forward model. The two free variables are now input coordinates, the two others being fixed to zero, so that both output constraints are eliminated over all $n = 2R$ temporary variables and the two resulting curves have the same total degree $(\alpha(\alpha+2))^R = \sqrt{\alpha^n D_I}$, for $D_I = (\alpha+2)^{2R}$. The attack then runs in $\tilde{O}((\alpha(\alpha+2))^{(3-1/\omega)R}) = \tilde{O}(D_I^{c_{\alpha,\omega}})$ with $c_{\alpha,\omega} = \frac{3\omega-1}{2\omega} \cdot (1 + \log_{\alpha+2} \alpha)$, against $O(D_I^\omega)$ in [CR25b]. Whether this is an improvement depends on ω, and holds only up to the logarithmic factors that the $\tilde{O}$ hides: at $\omega = 2.81$ it does for every $\alpha \in \{3, 5, 7, 11\}$, but for none at $\omega = 2$. At $\omega = 2.81$, for $\alpha = 3$, this brings the $R = 22$ instance claiming 256 bits down to 2^{237}, the first full-round break of `Anemoi` in the CICO-2 setting, while the 128-bit instances and the larger α stay above their claim.

Additionally, for the first time, they analyze `Anemoi` in `Jive` mode rather than the CICO problem, giving a preimage attack on the 4-to-2 compression function. Starting from the `Flystel` of the $\lceil R/2 \rceil$-th round with a middle state of the form $(0, x_1, 0, x_2)$ leaves the two middle α-inversions inactive, saving one round of inversions, so that $n = 2(R-1)$.

[30] In the CICO-k problem for permutations of minimal width $t = 2k$, the free variables required by the solving step – k of them in the forward model, $2k$ from the middle – exhaust the available degrees of freedom, so that no round can be bypassed. For wider states, however, the round-skipping idea of [BBB+25b] carries over to the forward model: parametrizing the input by a k-dimensional affine subspace rather than by $(x_1, \ldots, x_k, 0, \ldots, 0)$, one can keep (at most) $s \leq t' - k$ of the t' `Flystel`s of the first round constant, each removing one temporary variable, so that $n = t'R - s \geq t'(R-1) + k$. Starting from the middle, the degrees of freedom of the middle state might be exploitable in a similar way, the trivialized `Flystel`s being then distributed over the two layers adjacent to it, though we are not aware of any analysis of this.

The feed-forward of `Jive` combines the input and output reached by propagation into two equations, on which the bivariate resultant proceed as above. Notably, the cost of this attack is lower than a CICO-2 attack on the underlying permutation by a factor close to $(1+2/\alpha)^{(3-1/\omega)R/2}$, which at $\omega = 2.81$ amounts to 2^{14}, 2^9 and 2^6 for $\alpha = 3, 5, 7$ at the respective $R = 14, 14, 13$ rounds of the 128-bit instances, and to 2^{21}, 2^{14} and 2^{10} at the $R = 22, 22, 21$ rounds of the 256-bit ones.

New Estimation of the Number of Rounds. With a capacity of d field elements, the preimage attack scenario relevant for the sponge mode is CICO-d, so that the number of rounds required to reach a security level of κ bits depends on both d and t'. We take $\omega = 2$, the most favorable value for the attacker, and neglect logarithmic factors. Three cases admit an estimate.

- CICO-1 with $t' = 1$, where the SFTM attack of [BBHN26] costs $\tilde{O}(D_I^{c_\omega})$, $D_I = (\alpha+2)^R$ is proven in [CR25a], and $c_\omega = \frac{4\omega-2}{3\omega-1} = \frac{6}{5}$ for $\omega = 2$:

$$R \geq \frac{\kappa}{c_\omega \log_2(\alpha+2)}.$$

- CICO-1 with $t' > 1$, where the forward attack of [BBB+25b] costs $\tilde{O}(D_I(2-1/\alpha)^n)$ with $n = t'(R-1)+1$, and assuming $D_I = (\alpha+2)^R \alpha^{(t'-1)(R-1)}$:

$$R \geq \frac{\kappa + (t'-1)\left[\log_2 \alpha + \log_2(2-1/\alpha)\right]}{\log_2(\alpha+2) + (t'-1)\log_2 \alpha + t' \log_2(2-1/\alpha)}.$$

- CICO-t' with $t' > 1$, where we assume the Gröbner basis attack of [CR25a] to cost $O(D_I^\omega)$ and $D_I = (\alpha+2)^{t'R}$ is proven there:

$$R \geq \frac{\kappa}{\omega t' \log_2(\alpha+2)}.$$

The remaining case $2 \leq d < t'$ is not covered by any analysis, and we leave those entries as suggested by the designers. The updated round numbers for some Sponge instances with $\kappa = 128$ are given in Table 8. We omit the compression setting: the only analysis of `Anemoi` in `Jive` mode is the preimage attack of [BBHN26].

Table 8: (Updated) Round numbers R for `Anemoi` with a target security level of ≈ 128 bits, where $t = 2t'$ denotes the state size and α the exponent used in the `Flystel` S-box. If the round number has been updated, the original value is given in parentheses; the updated values in bold are reported without an additional security margin.

(a) $\log_2(p) \approx 256$, $d = 1$.

$t\backslash\alpha$	3	5	7	11
2	**46** (21)	**38** (21)	**34** (20)	**29** (19)
4	**25** (14)	**20** (14)	**17** (13)	**15** (13)
6	**18** (12)	**14** (12)	12	11
8	**14** (12)	12	11	11

(b) $\log_2(p) \approx 64$, $d = 4$.

$t\backslash\alpha$	3	5	7	11
8	12	12	11	11
12	10	10	10	10
16	10	10	9	9
20	9	9	9	9

(c) $\log_2(p) \approx 32$, $d = 8$.

$t\backslash\alpha$	3	5	7	11
16	10	10	9	9
24	9	9	9	9
32	9	9	8	8
40	8	8	8	8

6.3 Griffin (CRYPTO'23)

The hash function Griffin has been proposed by Grassi et al. [GHR+23] at CRYPTO 2023. Its design combines an expanding Horst structure with a *Rescue*-like SPN design.

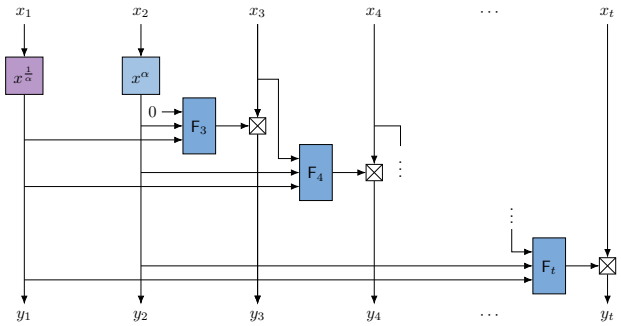


Figure 16: Non-linear layer of Griffin-π.

Horst. The Horst structure replaces the addition operation in the Feistel structure with a multiplication operation while retaining the original addition operation. Specifically, the transformation $(x, y) \mapsto (y + \mathsf{F}(x), x)$ becomes $(x, y) \mapsto (y \cdot \mathsf{G}(x) + \mathsf{F}(x), x)$. Note that this modification breaks the original reversibility of the Feistel structure, and thus, a special function G that never returns zero (hence, $\mathsf{G}(x) \neq 0$ for all x) is needed to maintain the reversibility of the Horst structure. The Horst scheme just defined over $\mathbb{F}_q^2$ can also be generalized to $\mathbb{F}_q^t$ for any $t \geq 2$.

The Griffin Permutation. Griffin supports both sponge and compression modes, instantiated by the internal permutation Griffin-π defined over $\mathbb{F}_p^t$ for prime $p > 2^{63}$ (i.e., $\lceil \log_2(p) \rceil > 63$) and integer $t \in \{3, 4t'\}$, i.e., t is either 3 or a multiple of 4.

The round function of Griffin-π follows an SPN structure and consists, in each round, of three steps: the addition of a round constant, a nonlinear layer, and the same matrix is applied to the input and in every round.

- The *non-linear layer* of Griffin-π $(x_1, \ldots, x_t) \mapsto (y_1, \ldots, y_t)$ is defined as follows:

$$y_i = \begin{cases} x_1^{1/\alpha}, & \text{if } i = 1, \\ x_2^{\alpha}, & \text{if } i = 2, \\ x_3 \cdot \big((\mathsf{L}_i(y_1, y_2, 0))^2 + \alpha_2 \cdot \mathsf{L}_i(y_1, y_2, 0) + \beta_2\big), & \text{if } i = 3, \\ x_i \cdot \big((\mathsf{L}_i(y_1, y_2, x_{i-1}))^2 + \alpha_i \cdot \mathsf{L}_i(y_1, y_2, x_{i-1}) + \beta_i\big), & \text{otherwise.} \end{cases} \tag{64}$$

  where $\alpha \geq 3$ is the smallest integer co-prime with $p - 1$ (i.e., $\gcd(\alpha, p - 1) = 1$), and $\mathsf{L}_i : \mathbb{F}_p^3 \to \mathbb{F}_p$ is defined as a linear combination of its inputs of the form $\mathsf{L}_i(z_1, z_2, z_3) = \gamma_i \cdot z_1 + z_2 + z_3$. A representation of the non-linear layer is depicted in Figure 16, where for simplicity we denote by F_i the functions given in the last lines of Equation (64).

- The *affine layer*, defined via a matrix multiplication and a constant addition. The linear layer is designed differently depending on the state size: it is an MDS matrix for $t = 3$ or $t = 4$, while for $t = 4 \cdot t' \geq 8$ it is an invertible matrix with non-zero entry and branch number $4 + t'$ (see [GHR+23] for details).

Third-Party Cryptanalysis. At CRYPTO 2024, Bariant et al. [BBL+24a] applied the FreeLunch framework to solve the underlying CICO system of Griffin. They reported the expected time complexity based on the extrapolation of the dominating `polyDet` step. Combining the FreeLunch attack with a trick to bypass 3 rounds, they manage to give

an explicit solution for the CICO problem on 7 out of 10 rounds, when $t = 12$, $\alpha = 3$ and $p \approx 55$ bits, in less than 4 hours.

At CRYPTO 2025, Bariant et al. [BBB+25b] reported the precise complexity of all steps based on the improved resultant attack (the authors used $\log_2(p) \approx 256$ in their computations). They show complexities for solving CICO problem lower than the claims for $\alpha = 3, 5$ and $t \in \{3, 4t'\}$. The only tested case reaching the claimed level in their table is $d = 5$, $t = 3$. They also manage to improve the previous attack by one round on the same parameters. The complexity of the overall attack is given by $\tilde{O}\left(D_I \cdot \alpha^2((2\alpha-1)/\alpha)^{R-b+2}\right)$ where the ideal degree is $D_I = d_b\alpha^{R-b}(2\alpha+1)^{R-b}$ and the values d_b and b are driven by the number of rounds that can be skipped:

$$(b, d_b) = \begin{cases} (1,1), & t = 3,4, \\ (2,3), & t = 8, \\ (3, 6\alpha+3), & t \geq 12. \end{cases}$$

Additionally, Bak et al. [BBB+25a] proposed a new algebraic model targeting CICO-k instances with $k > 1$. While they derived an upper bound of $(\alpha^2 \cdot 3^k)^R$ for the ideal degree, experimental results demonstrated a gap between theory and practice; consequently, the authors put forward a tighter conjectured bound of $\alpha \cdot (\alpha \cdot (3^k - 1) + 1))^R$ as an open problem.

The same year, Qin et al.[QCC+25] presented an automated tool to transform the problem of finding an optimal modeling into a Mixed-Integer Quadratic Constraint Programming (MIQCP) model. Their experiments show lower solving complexities up to 8 rounds when $t = 3$ and up to 5 rounds when $t = 4, 8$.

More recently, Bak et al. [BBHN26] proposed a generalization of the variable elimination procedure used in [BBB+25b]. Combined with efficient bivariate solving techniques, they provide a generalization of the resultant attack leading to the first full-round breaks in the CICO-2 setting for some versions of Griffin with 128-bit security (i.e. when $t \geq 12$ and $\alpha \in \{3, 5, 7\}$. Also considering round-skipping techniques, they obtained an attack with overall complexity

$$\tilde{O}\left((\alpha^n d_Q)^{3-1/\omega}\right), \text{ where } (n, d_Q) = \begin{cases} (R, (2\alpha+1)^R), & t = 3,4, \\ (R-1, 3\cdot(2\alpha+1)^{R-1}), & t = 8, \\ (R-2, (2\alpha+1)^{R-1}), & t = 12, \\ (R-2, 3\cdot(2\alpha+1)^{R-2}), & t \geq 16. \end{cases}$$

New Estimation of the Number of Rounds. As for `Anemoi`, we consider CICO-d complexities for preimage attack scenario when the capacity is d field elements. We take $\omega = 2$, and omit logarithmic factors. Note that for the instances studied in this paper, only CICO-1 and CICO-4 are relevant here.

- CICO-1: following the improved resultant attack from [BBB+25b], the number of rounds must re-evaluated as

$$R \geq \frac{\kappa - \log_2(d_b \cdot (2\alpha-1)^2)}{\log_2(4\alpha^2-1)} + b, \text{ where } (b, d_b) = \begin{cases} (1,1), & t = 3,4, \\ (2,3), & t = 8, \\ (3, 6\alpha+3), & t \geq 12, \end{cases}$$

- CICO-d, $d > 1$: The CheapLunch attack [BBB+25a] has a complexity of $\tilde{O}(D_I^\omega)$. Given the two bounds proposed for the ideal degree, this gives

$$R \geq \frac{\kappa}{\omega \cdot \log_2(\alpha^2 \cdot 3^d)}$$

with the proven bound and

$$R \geq \frac{\kappa}{\omega \cdot \log_2(\alpha \cdot (\alpha(3^d - 1) + 1))}$$

with the conjectured one. Note that for $d = 4$ those bounds do not exceed 8 rounds, that is the minimum number of rounds for any instances of Griffin.

The updated round numbers for some Sponge instances with $\kappa = 128$ are given in Table 9.

Table 9: (Updated) Round numbers R for Griffin with a target security level of 128 bits. t denotes the state size, α the S-box exponent, and d the digest size. If the round number has been updated, the original value is given in parentheses; the updated values in bold are reported without an additional security margin.

(a) $\log_2(p) \approx 256$, $d = 1$.

$t\backslash\alpha$	3	5	7	11
3	**26** (16)	**20** (12)	**17** (11)	**15** (10)
4	**26** (14)	**20** (11)	**17** (10)	**15** (9)
8	**26** (11)	**21** (9)	**18** (8)	**16** (8)
12	**27** (10)	**21** (9)	**19** (8)	**16** (8)

(b) $\log_2(p) \approx 64$, $d = 4$.

$t\backslash\alpha$	3	5	7	11
8	11	9	8	8
12	10	9	8	8
16	10	9	8	8
20	10	9	8	8

6.4 Arion (ToSC'26)

Arion [RST23, CRST26] was first proposed by Roy et al. in preprint on arXiv. The first variant of Arion [RST23] was successfully attacked with the FreeLunch approach [BBL+24a] (Section 4.3.3) and resultants [BBB+25b]. The designers acknowledged these attacks and updated Arion parameters accordingly [CRST26].

Generalized Triangular Dynamical System (GTDS). The non-linear layer of Arion is a based on a generalization of Horst. For $t = 2$ it is given by $(x_1, x_2) \mapsto \big(\mathsf{H}_1(x_2) \cdot \mathsf{G}(x_1) + \mathsf{F}(x_2), \mathsf{H}_2(x_2)\big)$, where H_1 and H_2 are univariate permutations, F is a polynomial without zeros in $\mathbb{F}_q$ and G is an arbitrary function. This paradigm can be generalized to any $t \geq 2$. Roy and Steiner [RS24] called such a generalized Horst non-linear layer *Generalized Triangular Dynamical System* (GTDS) and studied its cryptanalytic properties.

The Arion Permutation. Arion supports both sponge and compression modes, instantiated by the internal permutation Arion^π defined over $\mathbb{F}_p^t$ for BLS12-381/BN254 scalar fields (i.e., $\lceil \log_2(p) \rceil \approx 256$) and integer $3 \leq t \leq 12$.

The Arion^π applies an affine layer at start (with constant set to $\mathbf{0}$) followed by R alternating iterations of the non-linear and affine layers.

- Let $\alpha_1, \alpha_2 \in \mathbb{Z}_{>1}$ be integers such that $\gcd(\alpha_i, p-1) = 1$, i.e. both define power permutations $x \mapsto x^{\alpha_i}$ over $\mathbb{F}_p$. The *non-linear* layer of Arion^π is defined as [CRST26, Definition 1]:

$$y_i = \begin{cases} x_t^{\frac{1}{\alpha_2}}, & \text{if } i = t, \\ x_i^{\alpha_1} \cdot \mathsf{G}_i\left(\sum_{j=i+1}^{t}(x_j + y_j)\right) + \mathsf{H}_i\left(\sum_{j=i+1}^{t}(x_j + y_j)\right), & \text{otherwise,} \end{cases} \tag{65}$$

where

$$\begin{aligned} \mathsf{G}_i(x) &= x^2 + \alpha_{i,1} \cdot x + \alpha_{i,2}, \\ \mathsf{H}_i(x) &= x^2 + \beta_i \cdot x, \end{aligned} \tag{66}$$

Table 10: Round numbers for Arion [CRST26, Table 1] with a target security level of 128 bits, $\log_2(q) \approx 256$, $\alpha_1 = 5$ and $\alpha_2 = 257$ and capacity and digest $c = d = 1$. t denotes the state size.

t	3	4	5	6	8	9	10	12
R	11	10	9	9	8	8	8	7

and the constants $\alpha_{i,1}, \alpha_{i,2}, \beta_i \in \mathbb{F}_p$ are chosen such that $\alpha_{i,1}^2 - 4\alpha_{i,2}$ is a quadratic non-residue in $\mathbb{F}_p$. A circuit representation of the non-linear layer is depicted in Figure 17. For simplicity again the F_i's denote the functions from the last line of Equation (66).

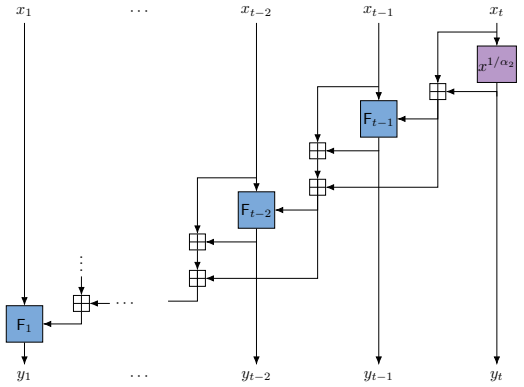


Figure 17: The non-linear layer of Arion.

- The *affine layer* in Arion$^\pi$ [CRST26, Definition 3] applies a $x \mapsto \mathrm{swap}(1, \ldots, t)\,\mathbf{x} + \mathbf{c}$, where $\mathbf{c} \in \mathbb{F}_p^t$ is a constant and $\mathrm{swap}(1, \ldots, t)$ is the matrix that has $(1, \ldots, t)$ in its first row, and in the i-th row $(i > 1)$ it swaps the $(i-1)$-th and i-th elements of $(1, \ldots, t)$, i.e.

$$\mathrm{swap}(1, \ldots, t) = \begin{pmatrix} 1 & 2 & 3 & \ldots & t-1 & t \\ 2 & 1 & 3 & \ldots & t-1 & t \\ 1 & 3 & 2 & \ldots & t-1 & t \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 1 & 2 & 3 & \ldots & t & t-1 \end{pmatrix} \in \mathbb{F}_p^{t \times t}. \tag{67}$$

  An efficient algorithm for matrix-vector products with swap was given in [CRST26, Algorithm 1]. Round constants of Arion$^\pi$ are generated by rejection sampling of the binary expansion of the fraction part of π, see [CRST26, §A].

Round numbers for Arion are summarized in Table 10.

Third-Party Cryptanalysis. Note that the third-party analysis presented here applies to the old version of Arion [RST23]. The new version [CRST26], which contains nearly twice as many rounds, has not yet been analyzed due to its recent release.

Due to similarities in the construction of Griffin and Arion, the FreeLunch framework, proposed by Bariant et al. [BBL+24a] at CRYPTO 2024, also applied to Arion. Combining this approach by a trick to bypass the first round and avoid the first inversion, they challenged the security of full round instances of Arion and α-Arion (the aggressive variant with less rounds). They showed that the algebraic attack complexity can be as low as 2^{98}

for the weakest instance of Arion ($\alpha_1 = 3$ and $t = 8$) and as low as 2^{83} for the weakest instance of α-Arion ($\alpha_1 = 5$ and $t = 3$).

At CRYPTO 2025, Bariant et al. [BBB+25b] further extended the analysis using the improved resultant-based approach. The complexity of their overall attack is given by $\tilde{O}\left(D_I \cdot \alpha_2^2((2\alpha_2 - 1)/\alpha_2)^{R+1}\right)$ where the ideal degree is $D_I = 3\alpha_1(\alpha_2(2^{t-1}(\alpha_1 + 1) - \alpha_1))^{R-1}$. This led them to show that almost all full variants of Arion and all full variants of α-Arion do not reach the security level of 128 bits. They reported complexities for solving the CICO problem on instances with $\alpha = 3, 5$, $\alpha_2 = 121$ and $t \in \{3, 4, 5, 6, 8\}$. The only tested cases reaching the claimed security level are $(\alpha_1, t) \in \{(3, 3), (3, 4), (5, 3)\}$.

The CheapLunch approach proposed by Bak et al. [BBB+25a] also targeted CICO-k (with $k > 1$) instances of Arion. However, as for Griffin, the underlying algebraic system is not regular and they could not apply their approach directly. Still, they provided a theoretic upper bound for the ideal degree of $\frac{\alpha_2^R\left(\prod_{j=1}^{k} \deg(\overline{Q}_j)\right)}{\prod_{j=1}^{R} w_j}$ where the $\overline{Q}_j$ are simplified polynomials obtained by linear combination of polynomials Q_j from the CICO-k system and such that $\deg(\overline{Q}_j) = (2^{t-1} \cdot (\alpha_1 + 1) - \alpha_1)^{R-1} \cdot (2^{t-j}(\alpha_1 + 1) - \alpha_1)$.

7 ZK-Friendly Hash Functions: Type-III (Look-up Tables)

In this section, we turn to the third type of ZK-friendly hash functions, which correspond to more recent designs that rely on so-called *Split-and-Lookup* (S&L) S-boxes. Notable representatives of this type include Reinforced Concrete [GKL+22], Tip5 [SLST23], Tip4 [Sal23], `Monolith` [GKL+24], and Skyscraper [BGK+25a, BGK+25b].

Type-III Arithmetization-Oriented. The core idea is to harden algebraic attacks through a domain switch: rather than applying a single S-box directly to a large field element, the element is decomposed/split into smaller chunks (belonging to a smaller field or ring). These chunks are then processed individually by small nonlinear maps, typically realized through lookup tables, before the outputs are recombined into a large field element. By mixing different algebraic structures, these constructions can achieve their target security level with substantially fewer rounds. Together with the efficient realization of S&L operations, this results in highly performant designs. However, as we will see, the decomposition layer also introduces exploitable structure that facilitates hybrid statistical-algebraic attacks, most notably rebound attacks.

Definition 7 (Decomposition)**.** Let $s_1, \ldots, s_m$ be positive integers satisfying $\prod_{i=1}^{m} s_i > p$, and define the base offsets $b_i := \prod_{j=i+1}^{m} s_j$, with $b_m = 1$ corresponding to the empty product. The *decomposition map*

$$\mathsf{Decomp} : \mathbb{F}_p \to \mathbb{Z}_{s_1} \times \cdots \times \mathbb{Z}_{s_m}, \qquad x \mapsto (x_1, \ldots, x_m), \tag{68}$$

identifies $x \in \mathbb{F}_p$ by its canonical representative $0 \le x < p$ and expands it in mixed-radix form as $x = \sum_{i=1}^{m} x_i b_i$, where the digits $x_i \in \mathbb{Z}_{s_i}$ are determined by

$$x_m = x \bmod s_m, \qquad x_i = \frac{x - \sum_{j>i} x_j b_j}{b_i} \bmod s_i\,. \tag{69}$$

Definition 8 (Composition)**.** Let p, $s_1, \ldots, s_m$, and $b_1, \ldots, b_m$ be as in Definition 7. Since every integer in $\{0, \ldots, p-1\}$ admits a unique mixed-radix expansion with radices $s_1, \ldots, s_m$, this map is injective. Its inverse on the image of Decomp is given by the *composition map*

$$\mathsf{Comp} : \mathbb{Z}_{s_1} \times \cdots \times \mathbb{Z}_{s_m} \to \mathbb{F}_p, \qquad (x_1, \ldots, x_m) \mapsto \sum_{i=1}^{m} x_i b_i \mod p\,. \tag{70}$$

Definition 9 (Type-III Arithmetization-Oriented)**.** A primitive is said to be "*Type-III Arithmetization-Oriented*" if

(i) round functions $\mathsf{R}^{(i)} : \mathbb{F}_q^t \to \mathbb{F}_q^t$ apply *Split-and-Lookup* (S&L) S-boxes of the form

$$x \xmapsto{\mathsf{Decomp}} (x_1, \ldots, x_m) \xmapsto{\mathsf{T}_1,\ldots,\mathsf{T}_m} (y_1, \ldots, y_m) \xmapsto{\mathsf{Comp}} y,$$

where the maps T_i are non-linear maps over small domains, and

(ii) the prover circuit implements Split-and-Lookup S-boxes via look-up table arguments.

The S&L construction was originally introduced in Reinforced Concrete [GKL+22] under the name `Bar`. Accordingly, we use `Bar` throughout to refer to this construction and distinguish it from other S-box constructions.

7.1 Reinforced Concrete (CCS'22)

The hash function Reinforced Concrete (RC) has been introduced by Grassi et al. [GKL+22] at CCS 2022. It is the first design proposed for ZK protocols based on look-up tables. This primitive is defined over $\mathbb{F}_p$ with different prime-field elements: two are the scalar fields of BN254 and BLS12-381, and the third (-ST) is a specially crafted prime chosen to enable very high performance for non-elliptic-curve-based proof systems.

RC follows a sponge construction whose internal permutation combines outer rounds to prevent statistical attacks and an inner round to resist algebraic ones. The permutation consists of a 7-round SPN-like structure with a word size of 3. Its linear layer is given by `Concrete`, while its nonlinear layer integrates both the low-degree `Bricks` and the high-degree `Bars` S-boxes (see Figure 18).

- The `Concrete` function is composed of a 3×3 MDS matrix multiplication (specifically the circulant matrix $(2, 1, 1)$) and the addition of round constants.
- The `Bricks` function is a nonlinear permutation over $\mathbb{F}_p^3$, same construction as Horst

$$\texttt{Bricks}(x_1, x_2, x_3) = \big(x_1^\alpha,\ x_2\big(x_1^2 + \lambda_1 x_1 + \beta_1\big),\ x_3\big(x_2^2 + \lambda_2 x_2 + \beta_2\big)\big),$$

where $\lambda_1, \lambda_2, \beta_1, \beta_2 \in \mathbb{F}_p$ are such that $\lambda_i^2 - 4\beta_i$ is not a square in $\mathbb{F}_p$. The exponent α is chosen such that $\gcd(\alpha,\ p-1) = 1$; in particular, the design sets $\alpha = 5$.

- The `Bars` function is a permutation defined over $\mathbb{F}_p^3$:

$$\texttt{Bars}(x_1, x_2, x_3) = \big(\texttt{Bar}(x_1),\ \texttt{Bar}(x_2),\ \texttt{Bar}(x_3)\big).$$

The map $\texttt{Bar} : \mathbb{F}_p \to \mathbb{F}_p$ is constructed by decomposing a field element into smaller digits, applying independent S-boxes, and recombining:

$$\texttt{Bar} = \mathsf{Comp} \circ \mathsf{SBox} \circ \mathsf{Decomp}.$$

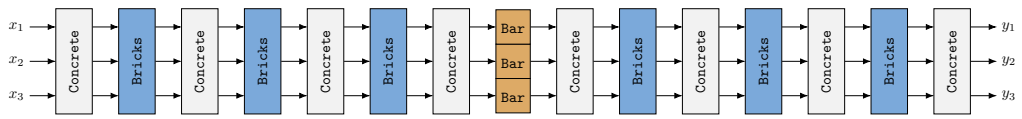


Figure 18: The permutation of Reinforced Concrete.

Third-Party Cryptanalysis. Bak and Perrin [BP25a] studied the cryptographic properties of Split-and-Lookup components, including the `Bars` layer of Reinforced Concrete. Their analysis provides efficient methods to compute Fourier coefficients and linear approximation probabilities of S&L S-boxes, and identifies linear approximations with higher probabilities than those considered in the original design analysis. For Reinforced Concrete, the improvement comes from taking into account not only the fixed points induced by the decomposition, but also the fixed points of the small MiMC permutation used inside the buckets. This raises the best known lower bounds on the number of solutions of linear approximations to about $2^{140.6}$, $2^{132.6}$, and $2^{88.6}$ for the BLS12-381, BN254, and ST prime-field instances, respectively. These results are component-level observations and do not lead to an attack on the full 7-round Reinforced Concrete permutation.

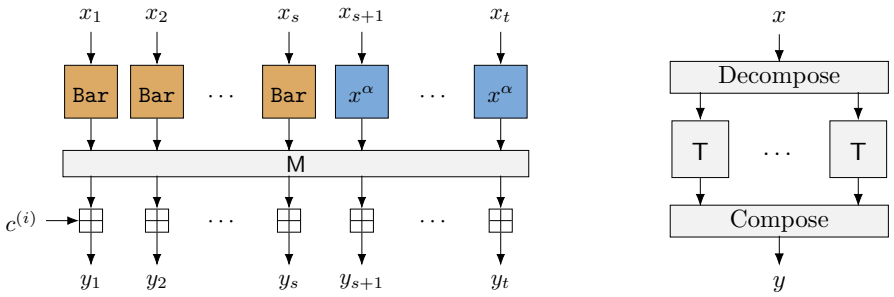
7.2 Tip5 Variants

The hash function Tip5, proposed by Szepieniec et al. [SLST23] in 2023, is designed for efficient computation over the Goldilocks field $\mathbb{F}_p$ with modulus $p = 2^{64} - 2^{32} + 1$. It is a sponge construction based on the Tip5 permutation.

The Tip5 permutation employs a 5-round SPN on a state of 16 field elements, as shown in Figure 19. Each round consists of three steps: a nonlinear layer, a linear layer, and the addition of round constants.

- The *non-linear layer* is composed of two distinct types of S-boxes:
  - the S&L S-box `Bar`, is taken from a lookup table,
  - the second S-box S is the classical power function $\mathsf{S}(x) = x^\alpha$ (here $\alpha = 7$).

  Of the 16 branches in the state, the first 4 are subjected to `Bar`, whereas the remaining 12 are processed by S.
- The *affine layer* is defined by multiplying the state by an $m \times m$ circulant MDS matrix M, and by the addition with the round constant.



(a) One round of the Tip5 permutation. **(b)** Split-and-lookup.

Figure 19: The Tip5 permutation.

Tip4 and Tip4'. Tip4 and Tip4' are two variants of Tip5 proposed by Salen [Sal23] in 2023, designed over the same field, but offer improved performance.

The structure of inner permutations of Tip4 and Tip4' are similar to that of Tip5, with the exception that Tip4' employs a smaller circulant MDS matrix in its diffusion layer, which is identical to the matrix used in the 128-bit security instantiation of *Rescue-Prime* [AKM+22]. While Tip5 is targeting a security level of 160 bits, Tip4 and Tip4' achieve 128 bits of security.

Besides, the main differences between Tip4, Tip4' and Tip5 lie in their parameter choices, including state size, sponge rate, sponge capacity, and digest size. We summarize the parameters in Table 11

Table 11: Tip5 family parameters.

Variant	t	r	d	$\#S\&L$	R
Tip5	16	10	5	4	5
Tip4	16	12	4	4	5
Tip4'	12	8	4	4	5

Third-party Cryptanalysis. The first third-Party cryptanalysis of Tip5, Tip4, and Tip4' was given by Liu et al. [LKG+24]. They developed an efficient algorithm to analyze the differential behavior of the Split-and-Lookup S-box. This yields collision attacks on 3-round Tip5, Tip4, and Tip4', practical SFS collision attacks on 3-round instances of all three variants, and an SFS collision attack on 4-round Tip4.

Bak and Perrin [BP25a] further studied the S&L component from the linear-cryptanalysis perspective. They identified linear approximations of the Tip5 S-box with substantially higher probability than previously accounted for, and proposed a round-skipping technique that leads to a freestart partial-preimage attack on 4-round Tip5. This can be turned into a collision attack on a modified 4-round Tip5 instance with reduced capacity. Note that the same method does not directly apply to the actual Tip5 parameters, nor to Tip4 and Tip4', because of their digest/capacity constraints.

7.3 `Monolith` (ToSC'24)

`Monolith` has been proposed by Grassi et al. [GKL+24] at ToSC 2024. `Monolith` supports two finite field options for $\mathbb{F}_p$: the Goldilocks field with $p = 2^{64} - 2^{32} + 1$ (with the state consisting of $t = 8$ or $t = 12$ elements), and the Mersenne31 field with $p = 2^{31} - 1$ (with the state consisting of $t = 16$ or $t = 24$ elements).

The `Monolith` permutation employs a 6-round SPN structure, with each round comprising the nonlinear layers "`Bars`" and "`Bricks`", the linear layer "`Concrete`", and a round constant addition layer, as shown in Figure 20. The names of the components of a `Monolith` round are inspired by those of Reinforced Concrete [GKL+22]. However some changes have been made compared to Reinforced Concrete. For instance, the `Bricks` layer no longer has multiplications between branches; Moreover, there is more than only one `Bars` layer.

- The "`Bars`" layer is defined as follows,

$$\texttt{Bars}(x_1, x_2, \ldots, x_t) := \texttt{Bar}(x_1) \parallel \cdots \parallel \texttt{Bar}(x_u) \parallel x_{u+1} \parallel \cdots \parallel x_t,$$

  where `Bar` is based on the `Kintsugi`[31] strategy.

- The "`Bricks`" operation is a Type-III Feistel network defined by $(y_1, y_2, y_3, \ldots, y_t) := (x_1, x_2 + x_1^2, x_3 + x_2^2, \ldots, x_t + x_{t-1}^2)$.

- The "`Concrete`" layer is defined by multiplication with a circulant MDS matrix.

In Table 12 we present the parameters of `Monolith`.

[31] `Kintsugi` is a Japanese technique where broken pottery is repaired using gold, the same concept applies here by breaking down a state, modifying each piece individually, and recombining them.

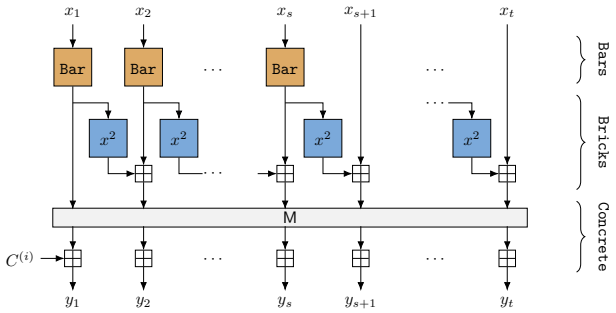


Figure 20: The `Monolith` permutation.

Table 12: `Monolith` parameters.

Field	t	r	d	u	R
Mersenne31	16	8	8	8	6
Mersenne31	24	16	8	8	6
Goldilocks	8	4	4	4	6
Goldilocks	12	8	4	4	6

Third-Party Cryptanalysis. The first third-party analysis of `Monolith` was given by Liu et al. [LKG+24], using techniques similar to those developed for the Tip5 family. They showed practical collision attacks on 2-round `Monolith`-64 and `Monolith`-31, and obtained semi-free-start collision attacks on 3-round `Monolith`-64. Bak and Perrin [BP25a] further analyzed the linear properties of the `Monolith` Split-and-Lookup S-box, identifying higher-probability linear approximations than those considered by the designers. The authors also propose replacing the S-box of `Monolith` with simple single-cycle bit permutations, which would significantly reduce the maximum linear probability (2^{-48} against $2^{-22.24}$ for `Monolith`-64 and 2^{-23} against $2^{-12.64}$ for `Monolith`-31) whilst improving the efficiency of the R1CS arithmetization (approximately 4 times fewer constraints). Note, however, that these new S-boxes are less resistant to differential attacks. Finally, Bak et al. [BJGP25] introduced a probabilistic polynomial-system-solving approach, in which high-degree S&L layers are replaced by high-probability linear approximations and the resulting low-degree systems are used in a rebound-style attack. Applied to `Monolith`, this yields limited-birthday distinguishers for round-reduced instances and efficient attacks on reduced compression-mode variants.

7.4 Skyscraper-v2 (TCHES'25)

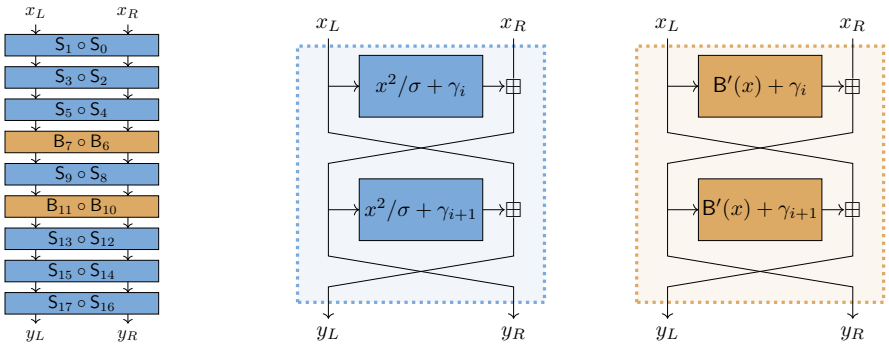
Skyscraper has been proposed by Bouvier et al. [BGK+25a, BGK+25b] at TCHES 2025. Soon after its publication, a distinguisher on full Skyscraper was found by Bak [Bak25], as explained in the following. Skyscraper's designers adjusted the number of rounds, proposing Skyscraper-v2. Here, we limit ourselves to consider Skyscraper-v2.

Skyscraper-v2 can be operated as a sponge or a 2-to-1 compression function, as it is friendly to large (256-bit and higher) primes. The Skyscraper-v2 permutation is defined over the state $(x_L, x_R) \in \mathbb{F}_{p^n}^2$ and consists of 18 rounds:

$$\begin{aligned}(x_L, x_R) \mapsto & \mathsf{S}_{17} \circ \mathsf{S}_{16} \circ \mathsf{S}_{15} \circ \mathsf{S}_{14} \circ \mathsf{S}_{13} \circ \mathsf{S}_{12} \circ \mathsf{B}_{11} \circ \mathsf{B}_{10} \circ \mathsf{S}_9 \\ & \circ \mathsf{S}_8 \circ \mathsf{B}_7 \circ \mathsf{B}_6 \circ \mathsf{S}_5 \circ \mathsf{S}_4 \circ \mathsf{S}_3 \circ \mathsf{S}_2 \circ \mathsf{S}_1 \circ \mathsf{S}_0(x_L, x_R),\end{aligned}$$

which combines two types of transformations: a low-degree *Square* operation and a high-degree *Bars* operation (see Figure 21). In particular:

- each Square round S_i is a Feistel transformation using a squaring-based round function:
$$(x_L, x_R) \mapsto \big(x_R + (x_L)^2/\sigma + \gamma_i,\ x_L\big), \tag{71}$$
where $x \mapsto x^2/\sigma$ is computed in the prime field (σ is a Montgomery constant) and γ_i is a round constant;
- the Bars round B_i has a similar Feistel structure:
$$(x_L, x_R) \mapsto \big(x_R + \mathsf{B}'(x_L) + \gamma_i,\ x_L\big), \tag{72}$$
where B' is a high-degree nonlinear transformation defined over $\mathbb{F}_{p^n}$. More specifically, the function B' operates in four conceptual steps:
  - *Decomp.* The input is split into 8-bit chunks.
  - *Rot.* The chunks are cyclically shifted to improve diffusion.
  - *S-box.* A nonlinear 8-bit transformation is applied independently to each chunk.
  - *Comp.* The chunks are combined back into a field element.



(a) Overview. **(b)** Details of Square. **(c)** Details of Bar.

Figure 21: Overview of Skyscraper.

Third-Party Cryptanalysis. Bak [Bak25] presented a practical rebound distinguisher on the original 10-round Skyscraper permutation, which results in practical collisions on the 9-round compression function for a non-negligible proportion of the instances. The attack exploits the weak differential properties of the `Bars` layer: the middle square rounds are handled algebraically in the inbound phase, while high-probability differentials through the `Bars` layers are used in the outbound phase.

Bak et al. [BJGP25] later generalized this idea using probabilistic polynomial system solving and described it as a rootfinding-aided rebound attack, giving a full-round distinguisher for the 10-round version of Skyscraper-v1, while the attack is not known to extend to the 18-round Skyscraper-v2 version.

8 ZK-Friendly Hash Functions: Type-IV (Power Residues)

The final type we discuss is Type-IV, which correspond to those that rely on *Power-Residues* S-boxes. Notable representatives of this type include *Grendel* [Sze21] and Polocolo [HHL+25]. The core idea is to use residue characters as a source of high-degree nonlinearity, while allowing faster evaluation.

Type-IV Arithmetization-Oriented. In Type-IV primitives, the bulk of the nonlinearity is realized by power-residue characters over prime fields. Before proposing a formal definition, we recall the notion of power residues.

Definition 10. Let $p \geq 3$ be a prime, and let $m \geq 2$ be an integer that divides $p-1$ (equivalently, $p-1$ is a multiple of m).

(1) The m-th power residue symbol is defined as

$$\left(\frac{x}{p}\right)_m := x^{\frac{p-1}{m}}.$$

(2) Let $f, g \in \mathbb{F}_q[x]$, the power residue S-box of (f, g, m) is defined as

$$\mathsf{S}(x) = f(x) \cdot g\left(\left(\frac{x}{p}\right)_m\right).$$

Let $g \in \mathbb{F}_p^\times$ be a generator of the multiplicative group, then any $x \in \mathbb{F}_p^\times$ can be represented as $x = g^{k \cdot m + r}$, where $k, r \in \mathbb{Z}$ such that $0 \leq r < m$. Thus, the m-th power residue maps x into the subgroup $\mathcal{G} \subset \mathbb{F}_p^\times$ of order m.

Given this definition, we can formally define the Type-IV Arithmetization-Oriented schemes.

Definition 11 (Type-IV Arithmetization-Oriented)**.** A primitive is said to be "*Type-IV Arithmetization-Oriented*" if

(i) round functions $\mathsf{R}^{(i)} : \mathbb{F}_p^t \to \mathbb{F}_p^t$ apply power residue S-boxes with subgroup order m_i, and

(ii) the prover circuit requires a subgroup membership circuit for the multiplicative group $\mathcal{G} \subset \mathbb{F}_p^\times$ of order m_i.

Power residue S-boxes offer an attractive security/proof-system-performance trade-off. They are inherently of high degree, so they achieve fast degree growth. While in Gröbner attacks degrees can be reduced by guessing the inputs of power residue symbols, this comes at the cost of a significant combinatorial overhead. On the other hand, their verification circuits can be implemented via low degree polynomials. As example for the latter we refer to the circuit of the *Grendel* S-box in Lemma 1.

8.1 *Grendel*

In [Sze21], Szepieniec proposes a new design based on an SPN construction over $\mathbb{F}_p^t$, where the S-box uses the Legendre symbol $\left(\frac{x}{p}\right)_2$. The round function of *Grendel* is depicted in Figure 22. Each round consists of the following steps:

- The bijective *S-box* $\mathsf{S} : \mathbb{F}_p \to \mathbb{F}_p$ is defined as

$$\mathsf{S}(x) := x^\alpha \cdot \left(\frac{x}{p}\right)_2 = x^{\alpha + \frac{p-1}{2}}$$

  for $\alpha \geq 2$ being the smallest integer that satisfies $\gcd(\alpha + \frac{p-1}{2}, p-1) = 1$ (e.g., $\alpha = 2$ for $p = 3 \mod 4$).

- The *affine layer* is defined as an affine transformation $\mathsf{LinLayer}(x) = \mathsf{M} \times x + c$, where $\mathsf{M} \in \mathbb{F}_p^{t \times t}$ is an MDS matrix and c is a round constant vector.

The number of rounds of *Grendel* is set to the smallest number that prevents all possible attacks given in [Sze21, Sect. 5.7] plus an additional 25% on top of security margin (we refer to the original paper for more details).

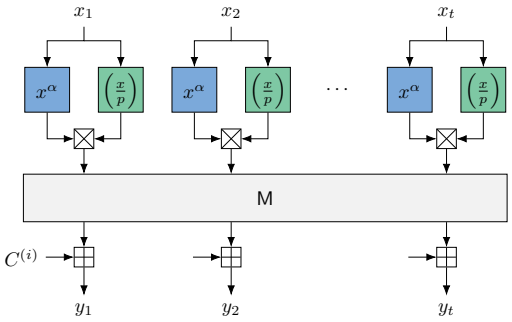


Figure 22: One round of *Grendel*.

Third-Party Cryptanalysis. Soon after its publication, the full *Grendel* has been broken by Grassi et al. [GKRS22]. Given the system of equations that defines a (preimage/collision) attack against the primitive, the idea of the attack consists in guessing all the Legendre symbols in order to simplify the equations. Once a solution to this simplified system of equations is found, the attacker must double check that it is also a solution for the real system of equations. If this is not the case, the process is repeated. The attack is faster than brute force due to the fact that, while the Legendre function is of high degree (namely, $(p-1)/2$), its domain consists of only 3 elements (basically 2, as the output zero can only happen for a zero input). Hence, this leads to guess $t \cdot (R-1)+1$ Legendre symbols per R rounds (actually, only one for first and for the last round, assuming $c = d = 1$), and to find a root of an equation of degree α^R.

The attack has been improved in [NZW+23, QSH24], in which the authors make use of the trick from [BBLP22] to bypass the first two rounds of the permutation in order to reduce both the number of guesses and the degree of the polynomial to solve. This leads to guessing $t \cdot (R-2)$ Legendre symbols and to solve a polynomial of degree α^{R-2}.

Later on, Bak and Perrin showed in [BP25a, Appendix E] how to generalize the round-skipping strategy in such a way to guess less Legendre symbols in the third round. Overall, this leads to guessing $t \cdot (R-3)+3$ Legendre symbols and to solve a polynomial of degree α^{R-2}.

New Estimation of the Round Numbers. For the CICO-1 problem of *Grendel*, the attack of Bak and Perrin [BP25a, §E] skips the first two rounds and linearizes $t-3$ S-boxes in the third round. In order to solve the CICO-1 problem over 256 bit fields, an attacker has to do $2^{t\cdot(R-3)+3}$ probabilistic guesses of Legendre symbols and factor a univariate polynomial of degree α^{R-2}. Therefore, for security 2^κ against the CICO-1 problem *Grendel* requires that at least

$$2^{t\cdot(R-3)+3} \cdot \alpha^{R-2} \geq 2^\kappa \qquad \Longleftrightarrow \qquad R \geq \frac{\kappa + 2 \cdot \log_2(\alpha) + 3 \cdot t - 3}{t + \log_2(\alpha)}. \tag{73}$$

This number of rounds guarantees security against the other attacks as well.

In Table 13 we present the minimum required round numbers of *Grendel* for various t and α.

8.2 Polocolo (EUROCRYPT'25)

The hash function Polocolo was proposed by Ha et al. [HHL+25] at EUROCRYPT 2025. It is constructed using the sponge construction instantiated with the permutation Polocolo$^\pi$, which is defined over $\mathbb{F}_p^t$ for $\mathbb{F}_p$ being the scalar field of the BLS12 and BN254 elliptic curves and $t \in \{3, 4, \ldots, 8\}$.

Table 13: (Updated) Round numbers R for *Grendel* with a target security level of 128 bits and $p \approx 2^{256}$. t denotes the state size, and α the S-box exponent. If the round number has been updated, the original value is given in parentheses; the updated values in bold are reported without an additional security margin.

$t\backslash\alpha$	2	3	4	5
2	**45** (41)	**38** (32)	**34** (27)	**32** (24)
3	**34** (28)	**30** (21)	**28** (18)	**27** (16)
4	**28** (21)	**26** (16)	**24** (14)	**23** (12)

$\mathsf{Polocolo}^\pi$ follows an SPN structure with R rounds, consisting of the following steps in each round:

- The *S-box* is defined via the power residue method as the bijective map $\mathsf{S} : \mathbb{F}_p \to \mathbb{F}_p$ defined as following. Let ϱ be a generator of the multiplication group $\mathbb{F}_p \setminus \{0\}$. Given $m = 2^l$ divisor of $p-1$, then

$$\mathsf{S}(x) := \begin{cases} 0, & \text{if } x = 0, \\ \varrho^{-qm+rm+\sigma(r)} & \text{if } x = \varrho^{qm+r}, \text{ where } 0 \le q < \frac{p-1}{m} \text{ and } 0 \le r < m\,, \end{cases}$$

  for σ being a permutation on $\{0, 1, \ldots, m-1\}$. In particular, if $x \neq 0$, then

$$\mathsf{S}(x) = x^{-1} \cdot \mathsf{T}\left[\left(\frac{x}{p}\right)_m\right] \qquad \text{where} \qquad \left(\frac{x}{p}\right)_m = \varrho^{\frac{r(p-1)}{m}}$$

  and where T is a lookup table defined via the permutation σ.

- The *affine layer* is defined as an affine transformation $\mathsf{LinLayer}(x) = \mathsf{M} \times x + c$, where $\mathsf{M} \in \mathbb{F}_p^{t \times t}$ is an MDS matrix and c is a round constant vector. Polocolo utilizes a heuristic approach to find MDS matrices that minimize the number of addition gates in the Plonk setting. Limiting ourselves to the case of small state sizes ($t = 3, 4$), the following specific low-addition MDS matrices are used:

$$\mathsf{M}_3 = \begin{pmatrix} 2 & 1 & 1 \\ 1 & 2 & 1 \\ 1 & 1 & 2 \end{pmatrix}, \qquad \text{and} \qquad \mathsf{M}_4 = \begin{pmatrix} 5 & 7 & 1 & 3 \\ 4 & 6 & 1 & 1 \\ 1 & 3 & 5 & 7 \\ 1 & 1 & 4 & 6 \end{pmatrix}.$$

The number of rounds of Polocolo is set to $R = 6$ when the permutation size is $t = 3$, and to $R = 5$ for permutation sizes $t \in \{4, 5, 6, 7, 8\}$.

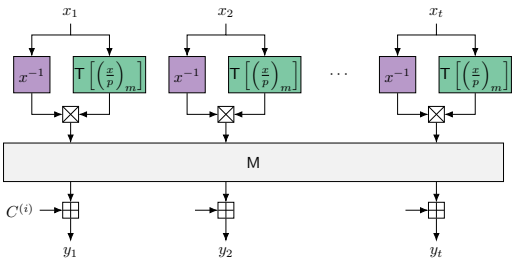


Figure 23: One round of Polocolo.

Third-Party Cryptanalysis. At the time of writing there is only one third-party work on Polocolo by Steiner [Ste25b] which proves a conjecture on Polocolo's S-box correlation in linear cryptanalysis.

9 Theoretical Proof System Performance

Throughout this section, for a power map $x \mapsto x^\alpha$, we denote by $\ell(\alpha)$ the minimum number of multiplications, including squarings, necessary to evaluate the power map. We count lookup constraints separately: LU_s denotes one lookup into a table of size s, so that $k\,\mathrm{LU}_s$ denotes k such lookup constraints. For partially lookup-based layers, u denotes the number of state elements per round to which the split-and-lookup S-box is applied. For constructions with Bars, R_B denotes the number of Bar rounds. For Skyscraper, η denotes the chunk bit-length. To avoid an overlap with the Type-III digit count m, we write τ for the parameter denoted by m in the definition of Polocolo.

9.1 General Arithmetization Strategies

In this subsection, we collect generic strategies that are relevant across different arithmetization styles. We consider power maps, Split-and-Lookup, power-residue S-boxes, and strategies for non-linear layers that build on top of them.

9.1.1 Power Maps and Inverse Power Maps

The simplest S-box of AO primitives is the power map $y = x^\alpha$. Usually, α is chosen as the smallest value such that $\gcd(\alpha, p-1) = 1$ to ensure invertibility, or $\alpha = 2$ in case invertibility comes through the structure (e.g., a Feistel). For general α, there is no closed formula for finding $\ell(\alpha)$. Often, the generic square-and-multiply approach with a cost of $\lfloor \log_2 \alpha \rfloor + \mathrm{hw}(\alpha) - 1$ is used. Given that the degree doubles at most per multiplication, we have

$$\lceil \log_2 \alpha \rceil \leq \ell(\alpha) \leq \lfloor \log_2 \alpha \rfloor + \mathrm{hw}(\alpha) - 1.$$

Thus, we can trivially check whether square-and-multiply is optimal for a given α. Only Arion uses power maps that are not optimal under square-and-multiply. As given in [CRST26, Tab. 1], the deviating instances are $\alpha \in \{125, 123, 121\}$, where $\ell(\alpha) = 9$ instead of $(11, 11, 10)$, respectively.

In Type-II constructions, it is common to employ the inverse power map $y = x^{1/\alpha}$, where α is a small value such that $\gcd(\alpha, p-1) = 1$. In a proof system, this map can be constrained by $y^\alpha = x$, equivalently leading to $\ell(\alpha)$ multiplication constraints or rows.

The Cubic Extension-Field Power Map. XHash (Section 6.1.2) additionally applies the power map $X \mapsto X^\alpha$ over $\mathbb{F}_{p^3}$ in its $(P3)$-rounds. Let $h \in \mathbb{F}_p[x]$ be irreducible with $\deg(h) = 3$. Then $\mathbb{F}_{p^3} \cong \mathbb{F}_p[x]/(h)$, so we can deduce $\mathbb{F}_p$-constraints for this map from univariate polynomial arithmetic. Let $\mathcal{M}(d)$ denote the complexity of multiplying two polynomials of degree $< d$; modular reduction of a degree $< 2d$ polynomial by a degree-d polynomial can also be done in $\mathcal{O}(\mathcal{M}(d))$ field operations, see, e.g., [vzGG13, §9.1]. With a recursive Fast Fourier Transform (FFT) algorithm for polynomial multiplication, $\mathcal{M}(d) \in \mathcal{O}\big(d \log_2(d) \log_2(\log_2(d))\big)$, see [vzGG13, Theorem 8.23]. The XHash designers estimated the constraint complexity of this power map by at most $\lceil \log_2(\alpha)\mathcal{M}(d) \rceil$ $\mathbb{F}_p$-constraints [ABK+26, §6.1].

9.1.2 Composite Non-linear Layers

The GMiMCHash Feistel Layer. In each round, GMiMCHash computes $y = (x_1 + c_i)^\alpha$ and adds y to the remaining $t-1$ state words. Thus, its non-linear layer applies one power map and $t-1$ affine branch updates per round, for a total of R power maps.

The GMiMCHash2 Feistel Layer. GMiMCHash2 uses the same Feistel application with the power map $x \mapsto x^{2^k}$, but first adds the round constant to the active state word and retains it in the state. It likewise applies one power map and $t-1$ affine branch updates in each of its R rounds.

The Lai–Massey Pair-wise S-box. Each NEPTUNE full round applies $t/2$ pair-wise Lai–Massey S-boxes, each comprising two squares and affine operations. The full-round layers therefore contain $R_\mathrm{F}t/2$ such S-boxes.

The pSquare-hash Feistel S-box. Each pSquare-hash round applies $t/4$ two-round Feistel S-boxes, whose only non-linearity is two squarings. The permutation therefore contains $Rt/4$ such S-boxes.

The Closed `Flystel` Layer. Each `Anemoi` round applies $t/2$ closed `Flystel` equations, replacing the open `Flystel` inverse power by the forward relation $(y-v)^\alpha$ and two squares. The permutation therefore contains $Rt/2$ closed `Flystel` applications.

The Griffin Non-linear Layer. Each GRIFFIN round applies one inverse and one forward power map, followed by $t-2$ Horst branches $y_i = x_i\mathsf{G}_i(\mathsf{L}_i)$ with quadratic G_i. Across its R rounds, GRIFFIN thus applies $2R$ power maps and $R(t-2)$ Horst branches.

The Arion Non-linear Layer. Each Arion round applies one inverse power map with exponent α_2 and $t-1$ triangular branches using $x_i^{\alpha_1}$ and two quadratics that share a common term in the running sum. Across its R rounds, Arion thus applies R inverse power maps and $R(t-1)$ triangular branches.

The Reinforced Concrete `Bricks` Layer. Each non-Bar Reinforced Concrete round applies $y_1 = x_1^\alpha$ and $y_i = x_i(x_{i-1}^2 + \lambda_{i-1}x_{i-1} + \beta_{i-1})$. The `Bricks` layer occurs $R - R_\mathrm{B}$ times.

The `Monolith` `Bricks` Layer. Each `Monolith` round applies the `Bricks` layer $y_1 = x_1$ and $y_i = x_i + x_{i-1}^2$. The permutation therefore contains $R(t-1)$ square-and-add branches.

The Skyscraper Square and Feistel Layers. Each of the R Skyscraper rounds is a Feistel step over two $\mathbb{F}_{p^n}$ branches, where $n = t/2$; R_B rounds use Bars and the remaining $R - R_\mathrm{B}$ use the Square map $x \mapsto x^2/\sigma$, where σ is the Montgomery constant.

9.1.3 The Split-and-Lookup Paradigm

In Type-III constructions, we first split each state element into chunks, evaluate lookup relations on the individual components, and then recompose the output. Let $s_1, \ldots, s_m$ be the digit radices and $b_i = \prod_{j=i+1}^{m} s_j$ their place values. A prover can use chunks $x_i \in [0, s_i)$ satisfying

$$x = \sum_{i=1}^{m} b_i x_i \mod p.$$

The lookup constraints verify both the table relations $y_i = \mathsf{L}(x_i)$ and the well-formedness of the chunks, while the decomposition constraint binds the input chunks to x. However, we need a separate canonicity check that, given

$$x \equiv x + rp \equiv \sum_{i=1}^{m} b_i x_i \mod p,$$

verifies that $r = 0$. For composition, we need only a linear combination of the output chunks; whether it can be folded into the subsequent constraint depends on the arithmetization. The corresponding costs are given separately for R1CS in Section 9.2.2 and for PLONK in Section 9.4.2.

In the following, we give the canonicity checks for the fields and constructions considered here. When using range constraints, we write rchk_b for one constraint to bit-width b.

Canonicity Checks. For canonicity checks, we may split the words into higher and lower parts, denoted by $y = y_{\mathrm{hi}}S + y_{\mathrm{lo}}$ and similarly $p = p_{\mathrm{hi}}S + p_{\mathrm{lo}}$. The following cases cover the constructions considered here.

- *BN254 and BLS12-381, Skyscraper and Reinforced Concrete.* For Skyscraper, we use a lexicographic comparison between the halves of the input word and the prime. We first check whether $y_{\mathrm{hi}} = p_{\mathrm{hi}}$ and range constrain either $p_{\mathrm{lo}} - 1 - y_{\mathrm{lo}}$ or $p_{\mathrm{hi}} - 1 - y_{\mathrm{hi}}$, depending on the result.

  Reinforced Concrete uses three 4-column tables instead of a separate arithmetic canonicity check. The table T_1 contains tuples (x_i, z_i, y_i, c_i) that bind the digits to their range, their output digit, and to carry values for the canonicity check. Writing $(v_1, \ldots, v_m) = \mathrm{Decomp}(p-1)$ and taking p' as the largest prime not exceeding $\min_i v_i$, its size is $|T_1| = p' + \sum_{i=1}^{m}(s_i - p' + 1)$, which is 1637 for the BLS12-381 instance and 1509 for the BN254 instance. Table T_2 is 16 elements long and constrains z_i, while T_3 contains 23 elements and constrains the well-formedness of each 4-element window of the canonicity carry elements c_i.

- *Goldilocks, the Tip5 family, and* `Monolith`*.* All of these constructions decompose into clean byte chunks. For $p = 2^{64} - 2^{32} + 1 = (2^{32} - 1)2^{32} + 1$, canonicity only requires treating the boundary case $y_{\mathrm{hi}} = 2^{32} - 1$. With witnessed variables w, e, the two relations $w\big(y_{\mathrm{hi}} - (2^{32} - 1)\big) = e$ and $y_{\mathrm{lo}}(1 - e) = 0$ accept exactly the canonical representations.

- *Mersenne31,* `Monolith`*.* For $p = 2^{31} - 1$ and radices $(128, 256, 256, 256)$, there is exactly one non-canonical digit vector: the representation of p. It suffices to provide a hint w and constrain

  $$w\big((p_{\mathrm{hi}} - y_{\mathrm{hi}}) + (p_{\mathrm{lo}} - y_{\mathrm{lo}})\big) = 1.$$

  This excludes the single non-canonical representation.

9.1.4 Power-Residue S-boxes

As with the inverse power map, power-residue S-boxes can be accelerated with hints. Both *Grendel* and Polocolo witness the relevant residue class: *Grendel* uses a specialized lookup-free system for quadratic residues, whereas Polocolo uses a general lookup-based system.

The *Grendel* S-box. In the *Grendel* paper [Sze21, §2] the following constraint system for the Legendre symbol $l = \left(\frac{x}{p}\right)_2$ was proposed:

$$\hat{l} \cdot \left(\hat{l}+1\right) \cdot \left(\hat{l}-1\right) = 0, \tag{74}$$

$$\hat{l} \cdot \left(\hat{l}-1\right) \cdot \left(b^2 - g \cdot \hat{x}\right) + \left(\hat{l}+1\right) \cdot \left(b^2 - \hat{x}\right) = 0, \tag{75}$$

where $g \in \mathbb{F}_p^\times$ is a quadratic non-residue and $\hat{l}, b, \hat{x}$ variables. The system is complete but not sound: for example, $\hat{l} = 0$ and $b = \hat{x} = 1$ satisfy the equations although $\hat{l} \neq \left(\frac{\hat{x}}{p}\right)_2$.

Instead, the following system is complete and sound for the S-box output.

Lemma 1. *Let p be an odd prime, let $\alpha \in \mathbb{Z}_{\geq 1}$ be such that $\gcd\left(\alpha + \frac{p-1}{2}, p-1\right) = 1$, let $\mathsf{S}(x) = x^\alpha \cdot \left(\frac{x}{p}\right)_2$, and let $g \in \mathbb{F}_p$ be a quadratic non-residue modulo p. The following polynomials are a complete and sound constraint system for $\mathsf{S}(x) = y$:*

$$\begin{aligned} y &= x^\alpha \cdot l, \\ (l+1) \cdot \left(b^2 - x\right) &= 0, \\ (l-1) \cdot \left(b^2 - g \cdot x\right) &= 0. \end{aligned}$$

Proof. For completeness, set $l = b = 0$ when $x = y = 0$. If x is a quadratic residue, set $l = 1$ and $b = \pm\sqrt{x}$; otherwise, set $l = -1$ and $b = \pm\sqrt{gx}$.

For soundness, first let $x \neq 0$. If $l \notin \{-1, 1\}$, the last two equations imply $b^2 = x = gx$, a contradiction. For $l = 1$, the second equation shows that x is a square and $y = x^\alpha$; for $l = -1$, the third shows that x is a non-square and $y = -x^\alpha$. If $x = 0$, the first equation directly gives $y = 0$. □

For $\alpha = 2$, the permutation condition forces $p \equiv 3 \pmod 4$: if $p \equiv 1 \pmod 4$, both $2 + (p-1)/2$ and $p - 1$ are even. Hence -1 is a quadratic non-residue and $x\left(\frac{x}{p}\right)_2$ is always a square. We can use a witness $v^2 = x\left(\frac{x}{p}\right)_2$, check the residue class, and reuse it in $y = xv^2$, reducing the S-box constraints.

The Polocolo S-box. Let ϱ, σ, and T be as in Section 8.2, and write τ for the parameter denoted there by m. For $0 \leq j < \tau$, define

$$\mathsf{G}_j = \varrho^j, \qquad \mathsf{V}_j = \varrho^{-j}\mathsf{T}\left[\varrho^{j(p-1)/\tau}\right] = \varrho^{j\tau+\sigma(j)}, \qquad \mathsf{G}_\tau = \mathsf{V}_\tau = 0.$$

To constrain $y = \mathsf{S}(x)$, witness j, w, e, look up (j, G_j) and (j, V_j), and impose

$$w^\tau \mathsf{G}_j = x, \qquad w^\tau e = 1, \qquad y = e\mathsf{V}_j,$$

where w^τ is computed by repeated squaring. For $x = \varrho^{q\tau+r} \neq 0$, these constraints force $j = r$ and give $y = \varrho^{-q\tau+r\tau+\sigma(r)} = \mathsf{S}(x)$. If $x = 0$, then $w^\tau \neq 0$ forces the unique zero row $j = \tau$, and hence $y = 0$. Thus, the system is complete and sound.

9.2 R1CS

In R1CS arithmetization the dominant metric in proof systems implementations is the number of multiplications necessary to represent a primitive. It follows that

- For Type-I primitives (Section 5), the R1CS constraints coincide with the multiplicative complexity of the forward evaluation.

- For Type-II primitives (Section 6), we use the power map and inverse power map strategy from Section 9.1.1. As the only exception, XHash additionally applies the power map $X \mapsto X^\alpha$ over $\mathbb{F}_{p^3}$; we discuss its R1CS circuit in Section 9.2.1.
- For Type-III primitives (Section 7) we count the quadratic constraints and lookup constraints separately. The field-dependent constraints needed for a sound split-and-lookup S-box are discussed in Section 9.1.3.
- For Type-IV primitives (Section 8), we use the complete and sound power-residue systems from Section 9.1.4.

For inverse power maps, the counts below assume that the equality $y^\alpha = x$ is folded into the last multiplication constraint. In Table 14 we summarize the formulae for R1CS constraints and evaluate them for sample parameters.

9.2.1 R1CS Costs for the XHash Cubic Extension-Field Power Map

Using the polynomial representation from Section 9.1.1, we claim that the generic bound given there inaccurately measures the R1CS circuit complexity of XHash. For non-recursive FFT polynomial multiplication, we have to find an adequate n-th root of unity in $\mathbb{F}_p$, and the number of multiplications in the FFT domain scales with n.

FFT Constraints for the Cubic Extension-Field Power Map. Let $f, g \in \mathbb{F}_p[x]$ be two polynomials with $\deg(f) + \deg(g) < n$, and let $\omega \in \mathbb{F}_p$ be a primitive n-th root of unity, i.e. $\omega^n = 1$ and $\omega^{\frac{n}{q}} \neq 1$ for all prime divisors $q \mid n$. Then, the product $f \cdot g$ can be computed via the discrete Fourier transform, see for example [vzGG13, §8.2]. Due to the convolution rule in the Fourier domain [vzGG13, Lemma 8.11], the product $f \cdot g$ can then be computed with two n-point Fourier transforms, n products in $\mathbb{F}_p$, and one inverse Fourier transform.

We can use this approach to construct R1CS constraints for the power map $X \mapsto X^\alpha$ over $\mathbb{F}_{p^3}$. Let $h = h_1 + h_2 \cdot x + h_3 \cdot x^2 + x^3 \in \mathbb{F}_p[x]$ be such that $\mathbb{F}_{p^3} \cong \mathbb{F}_p[x]/(h)$, and let $f = f_1 + f_2 \cdot x + f_3 \cdot x^2$ and $g = g_1 + g_2 \cdot x + g_3 \cdot x^2$ be two more polynomials in $\mathbb{F}_p[x]$. So, to compute $f \cdot g$ with the Fourier transform we need at least a 5-th primitive root of unity. For Mersenne31 such a root does not exist, however, for $n = 6$ we can choose $\omega_6 = 1513477736$. For Goldilocks we can pick $n = 5$ and $\omega_5 = 211587555138949697$. Since the Fourier transform is a linear operation, in R1CS we only need to account for multiplications in the Fourier domain, so we need 6 (Mersenne31) and 5 (Goldilocks) R1CS constraints, respectively, to compute $f \cdot g$. To compute $f \cdot g \bmod h$ we apply the following identities

$$x^3 \equiv -(h_1 + h_2 \cdot x + h_3 \cdot x^2) \mod (h), \tag{76}$$

$$x^4 \equiv x \cdot x^3 \equiv h_1 \cdot h_3 + (h_2 \cdot h_3 - h_1) \cdot x + (h_3^2 - h_2) \cdot x^2 \mod (h). \tag{77}$$

So, we can multiply field elements in $\mathbb{F}_{p^3}$ with the polynomial representation $\mathbb{F}_p[x]/(h)$ with 6 respectively 5 R1CS constraints.

Thus, we can implement the extension-field power map with $\ell(\alpha) \cdot \begin{cases} 5, & \text{Goldilocks,} \\ 6, & \text{Mersenne31} \end{cases}$ R1CS constraints.

9.2.2 Split-and-Lookup S-boxes

We use the generic split-and-lookup constraints from Section 9.1.3. In the R1CS profile, the decomposition equality costs one constraint, while the linear recomposition may be folded into the subsequent constraint. Per split-and-lookup application, this gives the following costs.

Table 14: Number of R1CS constraints. Every formula includes the t constraints that bind the permutation outputs. The round numbers used in the evaluated formulae are taken from Tables 17 and 18.

Primitive	Formula	$\log_2(p) \approx 32$, $t = 24$, $\alpha = 3/5/7$	$\log_2(p) \approx 64$, $t = 12$, $\alpha = 7$	$\log_2(p) \approx 256$, $t = 4$, $\alpha = 5$
GMiMCHash	$\ell(\alpha) \cdot R + t$	694/1029/1364	384	697
GMiMCHash2	$\ell(\alpha) \cdot R + t$	288 ($\alpha = 2$)	204 ($\alpha = 4$)	196 ($\alpha = 8$)
Neptune	$t \cdot R_F + \ell(\alpha) \cdot R_P + t$	–	228	232
Poseidon	$\ell(\alpha) \cdot (t \cdot R_F + R_P) + t$	454/666/876	356	268
Poseidon2				
pSquare-hash	$\frac{t}{2} \cdot R + t$	648	–	–
Anemoi	$(\ell(\alpha) + 2) \cdot \frac{t}{2} \cdot R + t$	456/564/672	372	204
Arion	$\big((\ell(\alpha_1) + 2) \cdot (t-1) + \ell(\alpha_2)\big) \cdot R + t$	–	–	244 ($\alpha_2 = 257$)
Griffin	$2 \cdot (\ell(\alpha) + t - 2) \cdot R + t$	–	236	204
Rescue		–	–	–
Rescue-Prime	$2 \cdot \ell(\alpha) \cdot t \cdot R + t$	792/1176/1560	780	316
Rescue-Prime Optimized		–	–	–
XHash	$R \cdot \ell(\alpha) \cdot t \cdot \begin{Bmatrix} \frac{5}{3}, & \text{XHash8}, \\ \frac{11}{6}, & \text{XHash12}, \\ \frac{11}{6}, & \text{XHash16}, \\ 2, & \text{XHash24} \end{Bmatrix} + t$	$-/\begin{Bmatrix} 816, & \text{XHash16}, \\ 888, & \text{XHash24} \end{Bmatrix}/-$	$\begin{Bmatrix} 492, & \text{XHash8}, \\ 540, & \text{XHash12} \end{Bmatrix}$	–
Monolith	$R \cdot (t - 1 + c_p \cdot u) + t + R \cdot u \cdot \sum_{i=1}^{m} \text{LU}_{s_i}, \quad c_p = \begin{cases} 2, & \text{Mersenne31}, \\ 3, & \text{Goldilocks} \end{cases}$	$258 + 144\,\text{LU}_{256} + 48\,\text{LU}_{128}$	$150 + 192\,\text{LU}_{256}$	–
Reinforced Concrete	$(R - R_B) \cdot (\ell(\alpha) + 2(t-1)) + R_B \cdot t + t + R_B \cdot t \cdot \big(m\,\text{LU}_{\lvert T_1 \rvert} + \lceil \frac{m-1}{3} \rceil\,\text{LU}_{23} + \lceil \frac{m}{4} \rceil\,\text{LU}_{16}\big)$	–	–	$48 + 81\,\text{LU}_{1637} + 27\,\text{LU}_{23} + 21\,\text{LU}_{16}$ (BLS12-381), $48 + 81\,\text{LU}_{1509} + 27\,\text{LU}_{23} + 21\,\text{LU}_{16}$ (BN254), ($t = 3$)
Skyscraper	$4 \cdot \frac{t}{2} \cdot R_B + (R - R_B) \cdot \begin{cases} 1, & t = 2, \\ 2, & t = 4 \end{cases} + t + \frac{t}{2} \cdot R_B \cdot \big(m\,\text{LU}_{2^n} + \text{rchk}_{m\eta/2}\big)$	–	–	$64 + 256\,\text{LU}_{256} + 8\,\text{rchk}_{128}$
Tip5/Tip4/Tip4'	$R \cdot (3u + \ell(\alpha)(t-u)) + t + R \cdot u \cdot m\,\text{LU}_{256}$	–	$\begin{cases} 316 + 160\,\text{LU}_{256}, & \text{Tip5, Tip4 } (t = 16), \\ 232 + 160\,\text{LU}_{256}, & \text{Tip4' } (t = 12) \end{cases}$	–
Grendel	$R \cdot t \cdot (\ell(\alpha) + 4 - \varepsilon) + t$, $\varepsilon = 1$ if $\alpha = 2$ and 0 otherwise	–	–	452/628/648
Polocolo	$R \cdot t \cdot (\log_2(\tau) + 3) + t + 2 \cdot R \cdot t\,\text{LU}_{\tau+1}$	–	–	$244 + 40\,\text{LU}_{513}$

- *Skyscraper:* 4 R1CS constraints, one range constraint $\text{rchk}_{m\eta/2}$, and $m\text{LU}_{2^\eta}$ lookup constraints. The additional constraints implement the lexicographic comparison with the prime.
- *Reinforced Concrete:* one R1CS decomposition constraint, together with $m\text{LU}_{|T_1|} + \left\lceil \frac{m-1}{3} \right\rceil \text{LU}_{23} + \left\lceil \frac{m}{4} \right\rceil \text{LU}_{16}$. The lookup tables also enforce the carry-based canonicity check.
- *Monolith:* 3 R1CS constraints over Goldilocks and 2 over Mersenne31, together with $\sum_{i=1}^{m} \text{LU}_{s_i}$. The difference comes from the two-product Goldilocks boundary check versus excluding the single non-canonical Mersenne-31 representation.
- *The Tip5 family:* 3 R1CS constraints and $m\text{LU}_{256}$ lookup constraints over Goldilocks. It uses the same Goldilocks canonicity check as `Monolith`.

The resulting full-primitive costs are given in Table 14.

9.2.3 Power-Residue S-boxes

We use the generic systems from Section 9.1.4. Per S-box application, their R1CS costs are as follows.

- *Grendel:* $\ell(\alpha) + 4$ constraints. For $\alpha = 2$, the cheaper residuosity check reduces this to 4 constraints.
- *Polocolo:* $\log_2(\tau) + 3$ multiplication constraints and $2\,\text{LU}_{\tau+1}$: repeated squaring computes w^τ, followed by the three multiplication relations.

Linear layers are free in the R1CS profile; the full-primitive costs are given in Table 14.

9.2.4 Type-I Primitives

GMiMCHash. Each of the R Feistel layers applies one power map at a cost of $\ell(\alpha)$ constraints; the branch updates and cyclic shifts are affine and therefore free. Including t output constraints, the permutation costs $R\ell(\alpha) + t$ constraints.

GMiMCHash2. Each of the R Feistel layers applies one power map at a cost of $\ell(\alpha)$ constraints; the branch updates, cyclic shifts, and M_{IO} layers are affine and therefore free. Including t output constraints, the permutation costs $R\ell(\alpha) + t$ constraints.

Neptune. The pair-wise Lai–Massey S-boxes in the full rounds cost tR_F constraints, and the power maps in the partial rounds cost $\ell(\alpha)R_\text{P}$ constraints; the linear layers are free. Including t output constraints, the permutation costs $tR_\text{F} + \ell(\alpha)R_\text{P} + t$ constraints.

Poseidon. The full-round S-box layers cost $\ell(\alpha)tR_\text{F}$ constraints, and the partial-round S-box layers cost $\ell(\alpha)R_\text{P}$ constraints; the linear layers are free. Including t output constraints, the permutation costs $\ell(\alpha)(tR_\text{F} + R_\text{P}) + t$ constraints.

Poseidon2. The full-round S-box layers cost $\ell(\alpha)tR_\text{F}$ constraints, and the partial-round S-box layers cost $\ell(\alpha)R_\text{P}$ constraints; the external and internal linear layers are free. Including t output constraints, the permutation costs $\ell(\alpha)(tR_\text{F} + R_\text{P}) + t$ constraints.

pSquare-hash. Each round applies $t/4$ two-round Feistel S-boxes at a cost of two constraints each, for $tR/2$ constraints across the non-linear layers; the feed-forward and linear layers are free. Including t output constraints, the permutation costs $tR/2 + t$ constraints.

9.2.5 Type-II Primitives

Rescue, Rescue-Prime, & Rescue-Prime Optimized. The forward and inverse S-box layers each apply t power maps per round, costing $2\ell(\alpha)tR$ constraints in total; the linear layers are free. Including t output constraints, each permutation costs $2\ell(\alpha)tR + t$ constraints.

`Anemoi`. Each round applies $t/2$ closed `Flystel` layers at a cost of $\ell(\alpha) + 2$ constraints each; the linear layers are free. Including t output constraints, the permutation costs $(\ell(\alpha) + 2)tR/2 + t$ constraints.

XHash. The (FB)-rounds cost $Rt\ell(\alpha)$ constraints for XHash12 and XHash24, and $5Rt\ell(\alpha)/6$ for XHash8 and XHash16. The $(P3)$-rounds cost $Rt\ell(\alpha)\xi_p/6$, where $\xi_p = 5$ over Goldilocks and $\xi_p = 6$ over Mersenne31; the linear layers are free. Including t output constraints, the permutation costs $R\ell(\alpha)t$ times 5/3, 11/6, 11/6, or 2 for XHash8, XHash12, XHash16, or XHash24, respectively, plus t.

Griffin. In each round, the forward and inverse power maps cost $2\ell(\alpha)$ constraints, while the $t - 2$ Horst branches cost $2(t - 2)$ constraints; the linear layers are free. Including t output constraints, the permutation costs $2(\ell(\alpha) + t - 2)R + t$ constraints.

Arion. In each round, the $t - 1$ triangular branches cost $(\ell(\alpha_1) + 2)(t - 1)$ constraints, and the inverse power map costs $\ell(\alpha_2)$ constraints; the linear layer is free. Including t output constraints, the permutation costs $((\ell(\alpha_1) + 2)(t - 1) + \ell(\alpha_2))R + t$ constraints.

9.2.6 Type-III Primitives

Reinforced Concrete. The $R - R_{\mathrm{B}}$ `Bricks` layers cost $(R - R_{\mathrm{B}})(\ell(\alpha) + 2(t - 1))$ constraints. The Bars contribute $R_{\mathrm{B}}t$ arithmetic constraints and $R_{\mathrm{B}}t(m\,\mathrm{LU}_{|T_1|} + \lceil (m-1)/3 \rceil\ \mathrm{LU}_{23} + \lceil m/4 \rceil\ \mathrm{LU}_{16})$ lookup constraints; the Concrete layers are free. Including t output constraints, the permutation cost is the sum of these layer costs and t.

`Monolith`. The `Bricks` layers cost $R(t - 1)$ constraints. The Bars contribute $c_p uR$ arithmetic constraints and $Ru \sum_{i=1}^{m} \mathrm{LU}_{s_i}$ lookup constraints, where $c_p = 2$ over Mersenne31 and $c_p = 3$ over Goldilocks; the Concrete layers are free. Including t output constraints, the permutation costs $R(t - 1 + c_p u) + t + Ru \sum_{i=1}^{m} \mathrm{LU}_{s_i}$.

Skyscraper. The Bars contribute $4tR_{\mathrm{B}}/2$ arithmetic constraints and $tR_{\mathrm{B}}(m\,\mathrm{LU}_{2^\eta} + \mathrm{rchk}_{m\eta/2})/2$ lookup and range constraints. The Square layers cost $R - R_{\mathrm{B}}$ constraints for $t = 2$ and, using the two-product extension-field squaring, $2(R - R_{\mathrm{B}})$ for $t = 4$; the Feistel layers are free. Including t output constraints, the permutation cost is the sum of these layer costs and t.

Tip5, Tip4, & Tip4'. In each round, the u split-and-lookup S-boxes contribute $3u$ arithmetic constraints and $um\,\mathrm{LU}_{256}$ lookup constraints, while the remaining $t - u$ power maps cost $\ell(\alpha)(t - u)$ constraints; the linear layers are free. Including t output constraints, the permutation costs $R(3u + \ell(\alpha)(t - u)) + t + Rum\,\mathrm{LU}_{256}$.

9.2.7 Type-IV Primitives

Grendel. Each of the Rt S-boxes costs $\ell(\alpha) + 4 - \varepsilon$ constraints, where $\varepsilon = 1$ for $\alpha = 2$ and 0 otherwise; the linear layers are free. Including t output constraints, the permutation costs $Rt(\ell(\alpha) + 4 - \varepsilon) + t$ constraints.

Polocolo. Each of the Rt S-boxes costs $\log_2(\tau) + 3$ arithmetic constraints and $2\,\mathrm{LU}_{\tau+1}$ lookup constraints; the linear layers are free. Including t output constraints, the permutation costs $Rt(\log_2(\tau) + 3) + t + 2Rt\,\mathrm{LU}_{\tau+1}$.

9.3 AIR

As explained in Section 2.2, an AIR arithmetization of a primitive is determined by its maximum degree and chosen trace layout (i.e., the choice of height and width of the trace table). The layout may be a single wide row or multiple narrower rows. Additionally, intermediate values can be committed to reduce the degree of the constraint polynomials at the cost of additional cells. These choices trade off prover time, verifier time, and proof size. To simplify our performance discussion, we assume a padding-free trace layout. In practice, this may be achieved by arithmetizing a construction into a single main-trace row. For the layouts considered here, the number and structure of the constraints are closely linked to the committed cells. We therefore limit the comparison to cell count and maximum degree, which are reliable predictors of performance for these layouts. Further, the exact number of cells depends on the maximum degree, which, in theory, can be chosen freely, and the cell count can decrease as the degree increases. Increasing the degree can have a multiplicative impact on prover performance because the maximum degree constrains the minimum FRI blowup. In a FRI proof [BBHR18a], each polynomial interpolated from a trace column of length 2^k is evaluated through a low-degree extension on a domain of size 2^{k+b}, where $B = 2^b$ is the FRI blowup factor. The required blowup grows approximately linearly with the maximum constraint degree, subject to rounding and proof-system-specific overhead such as hiding. A higher degree can therefore pay off when the reduction in committed cells compensates for the larger blowup. To provide concrete numbers, we restrict the comparison to maximum degrees two and three where both layouts are available. These are representative low-degree layouts rather than an exhaustive optimization over all degrees, and higher-degree layouts may also be competitive. We split the data into the number of witnessed cells c and the number of abstracted lookup constraints LU_s, both in relation to the chosen maximum degree $d_{\max}$ of the arithmetization. The cell count includes the input and output cells, every committed intermediate value, and all auxiliary witnesses or hints used by the arithmetic constraints. The cost of LU depends to a large degree on the concrete choice of lookup argument employed. Thus, we keep the notation abstract as a detailed treatment of different lookup styles would go beyond the scope of this section.

9.3.1 Power Maps and Inverse Power Maps

Consider a power map $y = x^\alpha$ as in Section 9.1.1 and fix the maximum permitted constraint degree $d_{\max}$. An addition chain for α can be split by committing intermediate powers of x: each committed value adds one cell, but resets its degree to one in all subsequent constraints. Let $r_\alpha(d_{\max})$ denote the number of intermediate cells in a chosen degree-$d_{\max}$ decomposition. Counting the input and output cells, its local AIR cost is $c_\alpha(d_{\max}) = 2 + r_\alpha(d_{\max})$ cells.

For example, decompositions of x^7 may use three, one, or no intermediate cells for $d_{\max} = 2, 3, 7$, respectively. For $d_{\max} = 3$, one may constrain $z = x^3$ and then $y = z^2x$. This example shows that cell reduction does not behave linearly, and that the optimal choice of $d_{\max}$ is construction-dependent. An inverse power map $y = x^{1/\alpha}$ has the same AIR footprint: the prover witnesses y and constrains the forward relation $y^\alpha = x$ with the

same degree-$d_{\max}$ decomposition. For cost accounting in Table 15, we let

$$a_{\alpha,d_{\max}} = 1 + r_\alpha(d_{\max}) = \begin{cases} i, & (\alpha, d_{\max}) = (2^i, 2), \\ 2, & (\alpha, d_{\max}) = (3, 2), \\ 3, & (\alpha, d_{\max}) = (5, 2), \\ 4, & (\alpha, d_{\max}) = (7, 2), \\ 9, & (\alpha, d_{\max}) = (257, 2), \\ 1, & (\alpha, d_{\max}) = (3, 3), \\ 2, & \alpha \in \{5, 7\},\ d_{\max} = 3, \\ 6, & (\alpha, d_{\max}) = (257, 3) \end{cases} \tag{78}$$

be the number of committed cells contributed by one power-map or inverse power-map application.

The XHash ($P3$)-Round. For the cubic extension-field power map from Section 9.1.1, we use the following arithmetization in AIR. In AIR, the linear operations required for exponentiation in the extension field can all be folded. Thus, the arithmetization is analogous to the general power map, and with $d_{\max} = 3$, both the Goldilocks instance with $\alpha = 7$ and the Mersenne31 instance with $\alpha = 5$ require coefficient-wise commitments $H = X^{(\alpha-1)/2}$ and $Y = H^2 X$. The full ($P3$)-round incurs $2t$ cells.

9.3.2 Split-and-Lookup S-boxes

We use the generic split-and-lookup relations from Section 9.1.3. Since AIR folds linear relations into the constraints, the input decomposition and output composition can be folded into the adjacent constraints. Each lookup binds the input and output chunks to its lookup relation and canonicity. Given that, we additionally need one cell to bind the canonicity of Mersenne31, and two cells for Goldilocks. For the decomposition and canonicity, we require a maximum degree of only $d_{\max} = 2$, incentivizing the low-degree S-boxes of e.g. `Monolith`. For the individual constructions, the AIR-specific costs are thus:

- *Skyscraper:* $c = 2m + 3$ cells, one range constraint $\mathrm{rchk}_{m\eta/2}$, and $m\mathrm{LU}_{2^\eta}$. Besides the chunks, the layout commits an equality witness, an inverse witness, and the selected half-word difference used by the lexicographic comparison with p. Across the full permutation, this component is applied $\frac{t}{2} R_{\mathrm{B}}$ times.
- *Reinforced Concrete:* $c = 4m$ cells, together with

  $$m\mathrm{LU}_{|T_1|} + \left\lceil \frac{m-1}{3} \right\rceil \mathrm{LU}_{23} + \left\lceil \frac{m}{4} \right\rceil \mathrm{LU}_{16}.$$

  The four cells per digit are the tuple (x_i, z_i, y_i, c_i), the latter two lookup families reuse the committed z_i and c_i cells. The tables enforce canonicity, and the affine output recomposition is folded into the subsequent `Concrete` layer. The permutation contains tR_{B} such applications.
- *`Monolith`:* $c = 2m + a$, where a is 1 for Mersenne31 and 2 for Goldilocks. The lookup cost is $\sum_{i=1}^{m} \mathrm{LU}_{s_i}$. Concretely, this gives $c = 9$ over Mersenne31 with radices $(128, 256, 256, 256)$, and $c = 18$ over Goldilocks with eight byte chunks. The degree is two, matching the `Bricks` layer, and the full permutation applies the component uR times.
- *The Tip5 family:* $c = 2m + 2 = 18$ cells and $m\mathrm{LU}_{256} = 8\mathrm{LU}_{256}$ in the Goldilocks field. Each variant applies four such S-boxes in each of its R rounds. Requiring only degree 2, these relations do not raise the maximum degree of the surrounding power-map layout.

9.3.3 Power-Residue S-boxes

For *Grendel*, the degree-three layout uses the residue checks from Lemma 1. At degree two, we replace them with

$$l^2 = 1, \qquad 2b^2 = \big((1+g) + (1-g)l\big)x$$

which, together with the power-map relation and multiplication by l, are complete and sound for the S-box output, including at $x = 0$.

9.3.4 Type-I Primitives

GMiMCHash. At degree $d_{\max} \in \{2, 3\}$, each of the R power-map Feistel layers costs $a_{\alpha,d_{\max}}$ cells; the branch updates and cyclic shifts are folded into these layers. Including the $2t$ input and output cells, the permutation costs $2t + a_{\alpha,d_{\max}} R$ cells.

GMiMCHash2. At degree two, each of the R power-map Feistel layers costs $a_{\alpha,2}$ cells; the branch updates, cyclic shifts, and M_{IO} layers are folded into these layers. Including the $2t$ input and output cells, the permutation costs $2t + a_{\alpha,2} R$ cells.

Neptune. The pair-wise Lai–Massey S-box has degree four; committing its first square $(x-y)^2$ reduces the constraint degree to two and gives three cells per pair. Thus, at degree $d_{\max} \in \{2, 3\}$, each full-round Lai–Massey layer costs $3t/2$ cells, and each partial-round power-map and linear layer costs $t + a_{\alpha,d_{\max}} - 1$ cells. Including the t initial-state cells, the permutation costs $t + 3tR_{\mathrm{F}}/2 + (t + a_{\alpha,d_{\max}} - 1)R_{\mathrm{P}}$ cells.

Poseidon. At degree $d_{\max} \in \{2, 3\}$, the full-round S-box layers cost $a_{\alpha,d_{\max}} tR_{\mathrm{F}}$ cells and the partial-round S-box layers cost $a_{\alpha,d_{\max}} R_{\mathrm{P}}$ cells; the linear layers are folded into the S-box constraints. Including the t initial-state cells, the permutation costs $t + a_{\alpha,d_{\max}}(tR_{\mathrm{F}} + R_{\mathrm{P}})$ cells.

Poseidon2. At degree $d_{\max} \in \{2, 3\}$, the full-round S-box layers cost $a_{\alpha,d_{\max}} tR_{\mathrm{F}}$ cells and the partial-round S-box layers cost $a_{\alpha,d_{\max}} R_{\mathrm{P}}$ cells; the external and internal linear layers are folded into the S-box constraints. Including the t initial-state cells, the permutation costs $t + a_{\alpha,d_{\max}}(tR_{\mathrm{F}} + R_{\mathrm{P}})$ cells.

pSquare-hash. At degree two, the two-round Feistel S-box layers cost $tR/2$ cells; the feed-forward and linear layers are folded into these cells. Including the $2t$ input and output cells, the permutation costs $2t + tR/2$ cells.

9.3.5 Type-II Primitives

***Rescue*, *Rescue-Prime*, & *Rescue-Prime Optimized*.** At degree three, the paired forward- and inverse-power-map layers cost $t(2a_{\alpha,3} - 1)$ cells per round, with their linear layers folded into the same cells. Including the t initial-state cells, the permutation costs $t + tR(2a_{\alpha,3} - 1)$ cells for *Rescue-Prime* and *Rescue-Prime Optimized*; the endpoint half-rounds of *Rescue* cost another t cells in total, for a total of $2t + tR(2a_{\alpha,3} - 1)$.

`Anemoi`. At degree $d_{\max} \in \{2, 3\}$, each closed-`Flystel` layer costs $a_{\alpha,d_{\max}} + 1$ cells: two output cells and $a_{\alpha,d_{\max}} - 1$ intermediate power-map cells. Its linear layer is folded into those cells. Including the $2t$ input and output cells, the permutation costs $2t + tR(a_{\alpha,d_{\max}} + 1)/2$ cells.

XHash. At degree three, each (FB)-round costs $t(2a_{\alpha,3}-1)$ cells and each cubic-extension $(P3)$-round costs $2t$ cells; the linear and finalization layers are folded into these cells. Since XHash12 and XHash24 contain $R/2$ rounds of each kind, including the t initial-state cells their permutations cost $t + Rt(2a_{\alpha,3}+1)/2$ cells.

Griffin. At degree two, the forward and inverse power maps and the $t-2$ Horst branches cost $2(a_{\alpha,2}+t-2)$ cells per round; at degree three, they cost $2a_{\alpha,3}+t-2$ cells per round. The linear layers are folded into these cells. Including the $2t$ input and output cells, the permutation costs $2t + 2R(a_{\alpha,2}+t-2)$ cells at degree two and $2t + R(2a_{\alpha,3}+t-2)$ cells at degree three.

Arion. At degree two, the $t-1$ triangular branches cost $(t-1)(a_{\alpha_1,2}+2)$ cells per round and the inverse power map costs $a_{\alpha_2,2}$; at degree three, these costs are $(t-1)(a_{\alpha_1,3}+1)$ and $a_{\alpha_2,3}$. The linear layer is folded into these cells. Including the $2t$ input and output cells, the permutation costs $2t + R((t-1)(a_{\alpha_1,2}+2) + a_{\alpha_2,2})$ cells at degree two and $2t + R((t-1)(a_{\alpha_1,3}+1) + a_{\alpha_2,3})$ cells at degree three.

9.3.6 Type-III Primitives

Reinforced Concrete. At degree two, each `Bricks` layer costs $a_{\alpha,2}+3$ cells, while at degree three it costs $a_{\alpha,3}+2$ cells. Each Bar layer costs $4mt$ cells and incurs $t(m\,\mathrm{LU}_{|T_1|} + \lceil (m-1)/3 \rceil\ \mathrm{LU}_{23} + \lceil m/4 \rceil\ \mathrm{LU}_{16})$; the Concrete layers are folded into the adjacent constraints. Including the $2t$ input and output cells, the permutation costs $2t + (R-R_{\mathrm{B}})(a_{\alpha,2}+3) + 4mtR_{\mathrm{B}}$ cells at degree two and $2t + (R-R_{\mathrm{B}})(a_{\alpha,3}+2) + 4mtR_{\mathrm{B}}$ cells at degree three, together with $tR_{\mathrm{B}}(m\,\mathrm{LU}_{|T_1|} + \lceil (m-1)/3 \rceil\ \mathrm{LU}_{23} + \lceil m/4 \rceil\ \mathrm{LU}_{16})$.

`Monolith`. At degree two, each `Bricks` layer costs t cells, while its u Bar applications cost $u(2m+1)$ cells over Mersenne31 or $u(2m+2)$ cells over Goldilocks, together with $u\sum_{i=1}^{m}\mathrm{LU}_{s_i}$; the Concrete layers are folded into these cells. Including the t initial-state cells, the permutation costs $t+R(t+u(2m+1))$ cells over Mersenne31 or $t+R(t+u(2m+2))$ cells over Goldilocks, together with $Ru\sum_{i=1}^{m}\mathrm{LU}_{s_i}$.

Skyscraper. At degree two, each Square-and-Feistel layer costs $t/2$ cells, while each Bar-and-Feistel layer costs $t(2m+3)/2$ cells and $t(m\,\mathrm{LU}_{2^\eta} + \mathrm{rchk}_{m\eta/2})/2$. Including the $2t$ input and output cells, the permutation costs $2t + t(R-R_{\mathrm{B}})/2 + tR_{\mathrm{B}}(2m+3)/2$ cells, together with $tR_{\mathrm{B}}(m\,\mathrm{LU}_{2^\eta} + \mathrm{rchk}_{m\eta/2})/2$.

Tip4'. At degree $d_{\max} \in \{2,3\}$, the u split-and-lookup S-boxes in each round cost $u(2m+2)$ cells and $um\,\mathrm{LU}_{256}$, while the remaining $t-u$ power maps cost $(t-u)a_{\alpha,d_{\max}}$ cells; the linear layers are folded into these cells. Including the $2t$ input and output cells, the permutation costs $2t + R(u(2m+2) + (t-u)a_{\alpha,d_{\max}})$ cells, together with $Rum\,\mathrm{LU}_{256}$.

9.3.7 Type-IV Primitives

***Grendel*.** At degree $d_{\max} \in \{2,3\}$, each power-residue S-box costs $a_{\alpha,d_{\max}}+3$ cells: the power-map cells, residue witness l, square-root witness b, and the product by l. The affine layers are folded into the S-box constraints. Including the t initial-state cells, the permutation costs $t + Rt(a_{\alpha,d_{\max}}+3)$ cells.

Polocolo. At degree two, each power-residue S-box costs $\log_2(\tau) + 6$ cells and $2\,\mathrm{LU}_{\tau+1}$; at degree three, it costs $\log_2(\tau) + 5$ cells and the same two lookups. The linear layers are folded into the S-box constraints. Including the t initial-state cells, the permutation costs $t + Rt(\log_2(\tau) + 6)$ cells at degree two or $t + Rt(\log_2(\tau) + 5)$ cells at degree three, together with $2Rt\,\mathrm{LU}_{\tau+1}$.

9.4 PLONK

In PLONK arithmetization, we must represent a permutation via quadratic polynomials. In particular, a three-wire constraint, see Equation (5), corresponds to a function $(x, y, z) \mapsto c_m \cdot x \cdot y + c_x \cdot x + c_y \cdot y + c_z \cdot z + c$, where $c_m, c_x, c_y, c_z, c \in \mathbb{F}_p$ are constants. In Table 16 we summarize the formulae for PLONK constraints and evaluate them for sample parameters. In the following, we discuss the PLONK-specific costs of the individual components; the common power map strategies are given in Section 9.1.1.

9.4.1 Constraining Affine Transformations

For the discussion of matrix arithmetization, we will denote by M_I the identity matrix, by M_J the all-1 matrix, and by $\mathrm{diag}(\mu)$ the diagonal matrix defined by vector μ.

Generic Matrix-Vector Products. Following Equation (5), it is common to constrain linear functions through linear straight-line programs with components of the form $w = \alpha u + \beta v + \gamma$, with variables $u, v \in \mathbb{F}_p$ and constants $\alpha, \beta, \gamma \in \mathbb{F}_p$. Finding the shortest linear straight-line program for an arbitrary matrix is hard in general [BMP13]. Thus, we describe the cost of the generic approach here and outline more efficient solutions where available, while making no claim of optimality.

Generic Matrix Splits. Constructions such as Neptune and `Anemoi` split the state into two blocks and apply the matrix block-wise. The constraint count is then significantly reduced to the sum of the costs of constraining the smaller matrix multiplications. For the dense matrices of external Neptune rounds, this yields $t(t/2 - 1)$ constraints, whereas for `Anemoi`, the cost depends on its dedicated matrices.

The $\mathsf{M}_J + \mathrm{diag}(\mu)$ Matrix. For the internal matrix M_I (Equation (52)), we first compute $\sigma = \sum_{i=1}^{t} x_i$, at a cost of $t - 1$ constraints. Then the i-th matrix-vector product entry is given by $y_i = \sigma + (\mu_i - 1) \cdot x_i$, requiring one constraint per output element. In total, this amounts to $2t - 1$ constraints. This also covers the $\mathsf{M}_J + \mathsf{M}_I$ matrix used by the $t = 3$ external layer of Poseidon2 and the Concrete layer of Reinforced Concrete.

Constraining Matrices of Partial Rounds. In the Hades design strategy, employed by Poseidon, Poseidon2, and Neptune, the S-box of inner layers is only applied to a single element of the state. As outlined in Section E.2, we can fold the majority of the cost of matrix evaluation in the partial rounds into an initial base shift. The last matrix of the first full-round block will become dense with a cost of $t^2 - t$. Then all partial-round matrices can be constrained using $2t - 2$ PLONK constraints.

Duval-Leurent and Block Circulant Matrices. Both Poseidon2 and Griffin use the M_4 Duval–Leurent matrix [DL18, Figure 13] for several of their instances. In both constructions, the M_4 matrix is applied when $4 \mid t$; for $t > 4$, it is used in the block-circulant matrix $\mathsf{M} = \mathrm{circ}(2\mathsf{M}_4, \mathsf{M}_4, \ldots, \mathsf{M}_4)$. The M_4 matrix is specifically designed to permit an

Table 15: Single-row AIR costs. The committed-cell count c and abstract lookup queries are reported separately; the evaluated columns instantiate c. The layouts use one logical row per permutation call and no next-row access. Round numbers are taken from Tables 17 and 18; the fixed seven-round *Rescue-Prime Optimized* instances use their specified round number.

Primitive	$d_{\max}$	Cells c	Lookup queries	Evaluated cells c, $\log_2(p) \approx 32$, $t = 24$, $\alpha = 3/5/7$	Evaluated cells c, $\log_2(p) \approx 64$, $t = 12$, $\alpha = 7$	Evaluated cells c, $\log_2(p) \approx 256$, $t = 4$, $\alpha = 5$
GMiMCHash	2, 3	$2t + a_{\alpha,d_{\max}} R$	–	$\begin{cases} 718/1053/1388, & d_{\max} = 2, \\ 383/718/718, & d_{\max} = 3 \end{cases}$	$\begin{cases} 396, & d_{\max} = 2, \\ 210, & d_{\max} = 3 \end{cases}$	$\begin{cases} 701, & d_{\max} = 2, \\ 470, & d_{\max} = 3 \end{cases}$
GMiMCHash2	2	$2t + a_{\alpha,2} R$	–	312 ($\alpha = 2$)	216 ($\alpha = 4$)	200 ($\alpha = 8$)
Neptune	2, 3	$t + \frac{3t}{2} R_{\mathrm{F}} + (t + a_{\alpha,d_{\max}} - 1) R_{\mathrm{P}}$	–	–	$\begin{cases} 750, & d_{\max} = 2, \\ 666, & d_{\max} = 3 \end{cases}$	$\begin{cases} 448, & d_{\max} = 2, \\ 380, & d_{\max} = 3 \end{cases}$
Poseidon Poseidon2	2, 3	$t + a_{\alpha,d_{\max}} (t R_{\mathrm{F}} + R_{\mathrm{P}})$	–	$\begin{cases} 454/666/876, & d_{\max} = 2, \\ 239/452/450, & d_{\max} = 3 \end{cases}$	$\begin{cases} 484, & d_{\max} = 2, \\ 248, & d_{\max} = 3 \end{cases}$	$\begin{cases} 268, & d_{\max} = 2, \\ 180, & d_{\max} = 3 \end{cases}$
pSquare-hash	2	$2t + \frac{t}{2} R$	–	672 ($\alpha = 2$)	–	–
Anemoi	2, 3	$2t + \frac{tR}{2}(a_{\alpha,d_{\max}} + 1)$	–	$\begin{cases} 372/480/588, & d_{\max} = 2, \\ 264/372/372, & d_{\max} = 3 \end{cases}$	$\begin{cases} 324, & d_{\max} = 2, \\ 204, & d_{\max} = 3 \end{cases}$	$\begin{cases} 168, & d_{\max} = 2, \\ 128, & d_{\max} = 3 \end{cases}$
Arion	2, 3	$\begin{cases} 2t + R\big((t-1)(a_{\alpha_1,2} + 2) + a_{\alpha_2,2}\big), & d_{\max} = 2, \\ 2t + R\big((t-1)(a_{\alpha_1,3} + 1) + a_{\alpha_2,3}\big), & d_{\max} = 3 \end{cases}$	–	–	–	$\begin{cases} 248, & d_{\max} = 2, \\ 158, & d_{\max} = 3 \end{cases}$ ($\alpha_2 = 257,\ a_{257,2} = 9,\ a_{257,3} = 6$)
Griffin	2, 3	$\begin{cases} 2t + 2R(a_{\alpha,2} + t - 2), & d_{\max} = 2, \\ 2t + R(2a_{\alpha,3} + t - 2), & d_{\max} = 3 \end{cases}$	–	–	$\begin{cases} 248, & d_{\max} = 2, \\ 136, & d_{\max} = 3 \end{cases}$	$\begin{cases} 208, & d_{\max} = 2, \\ 128, & d_{\max} = 3 \end{cases}$
Rescue	3	$t + tR\big(2a_{\alpha,3} - 1\big) + \begin{cases} t, & \textit{Rescue}, \\ 0, & \textit{Rescue-Prime}, \textit{Rescue-Prime Optimized} \end{cases}$	–	–	–	–
Rescue-Prime				216/600/600	300	160
Rescue-Prime Optimized				– / 528 / –	264	–
XHash (XHash12, XHash24)	3	$t + \frac{Rt}{2}(2a_{\alpha,3} + 1)$	–	– / 384 / –	192	–
Monolith	2	$t + R\left(t + u \cdot \begin{cases} 2m + 1, & \text{Mersenne31}, \\ 2m + 2, & \text{Goldilocks} \end{cases}\right)$	$Ru \sum_{i=1}^{m} \mathrm{LU}_{s_i}$	600 ($\alpha = 2$)	516 ($\alpha = 2$)	–
Reinforced Concrete	2, 3	$\begin{cases} 2t + (R - R_{\mathrm{B}})(a_{\alpha,2} + 3) + 4mtR_{\mathrm{B}}, & d_{\max} = 2, \\ 2t + (R - R_{\mathrm{B}})(a_{\alpha,3} + 2) + 4mtR_{\mathrm{B}}, & d_{\max} = 3 \end{cases}$	$tR_{\mathrm{B}} \left(m\,\mathrm{LU}_{\lvert T_1 \rvert} + \lceil \frac{m-1}{3} \rceil\,\mathrm{LU}_{23} + \lceil \frac{m}{4} \rceil\,\mathrm{LU}_{16}\right)$	–	–	$\begin{cases} 366, & d_{\max} = 2, \\ 354, & d_{\max} = 3 \end{cases}$ ($t = 3,\ \alpha = 5$)
Skyscraper	2	$2t + \frac{t}{2}(R - R_{\mathrm{B}}) + \frac{t}{2} R_{\mathrm{B}} (2m + 3)$	$\frac{t}{2} R_{\mathrm{B}} \left(m\,\mathrm{LU}_{2^n} + \mathrm{rchk}_{mn/2}\right)$	–	–	572 ($\alpha = 2$)
Tip4'	2, 3	$2t + R\big(u(2m + 2) + (t - u) a_{\alpha,d_{\max}}\big)$	$Rum\,\mathrm{LU}_{256}$	–	$\begin{cases} 544, & d_{\max} = 2, \\ 464, & d_{\max} = 3 \end{cases}$	–
Grendel	2, 3	$t + Rt(a_{\alpha,d_{\max}} + 3)$	–	–	–	$\begin{cases} 556, & d_{\max} = 2, \\ 464, & d_{\max} = 3 \end{cases}$
Polocolo	2, 3	$\begin{cases} t + Rt(\log_2(\tau) + 6), & d_{\max} = 2, \\ t + Rt(\log_2(\tau) + 5), & d_{\max} = 3 \end{cases}$	$2Rt\,\mathrm{LU}_{\tau+1}$	–	–	$\begin{cases} 304, & d_{\max} = 2, \\ 284, & d_{\max} = 3 \end{cases}$

efficient straight-line linear program. Following [GHR+23, §7.3] it amounts to the cost of

$$\begin{Bmatrix} 2 \cdot t, & t = 4, \\ 3 \cdot t, & t = 8, \\ 4t - 4, & t \geq 12 \end{Bmatrix} \tag{79}$$

PLONK three-wire constraints.

9.4.2 Constraining Non-linear Layers

The common power-map strategy and the definitions of the composite non-linear layers are given in Sections 9.1.1 and 9.1.2; their PLONK costs are assembled in the construction paragraphs below. We retain here only the PLONK-specific extension-field and split-and-lookup realizations.

Constraints for the Cubic Extension-Field Power Map. We consider the power map $X \mapsto X^\alpha$ over $\mathbb{F}_{p^3}$ used in the $(P3)$-rounds of XHash. For the small extension degree considered here, directly evaluating the circuit is more efficient than an FFT-based realization.

For the evaluated costs, we choose the irreducible binomials $h(x) = x^3 + 5$ over Mersenne31 and $h(x) = x^3 - 2$ over Goldilocks. Writing $h(x) = x^3 - \nu$, let $f = f_1 + f_2 \cdot x + f_3 \cdot x^2$ and $g = g_1 + g_2 \cdot x + g_3 \cdot x^2$. Then

$$f \cdot g = f_1 g_1 + \nu(f_3 g_2 + f_2 g_3) + (f_2 g_1 + f_1 g_2 + \nu f_3 g_3)x + (f_1 g_3 + f_3 g_1 + f_2 g_2)x^2. \tag{80}$$

This polynomial product requires 9 $\mathbb{F}_p$-products and 6 $\mathbb{F}_p$-sums for a total of 15 PLONK constraints. For the case of $f = g$ (i.e., squaring), we require 6 products and 3 sums for a total of 9 constraints. With the generic square-and-multiply approach we require for an α of 3, 5, and 7, exactly $8t$, $11t$, and $16t$ constraints, respectively. However, when fully expanding the power map for each exponent, there may be representations with lower constraint counts.

The Split-and-Lookup Paradigm. We again use the generic constraints from Section 9.1.3. With the standard three-wire gate, decomposition and recomposition cost $m-1$ constraints each. Per split-and-lookup application, this gives the following costs. For Skyscraper, this costs $2m + 7$ PLONK constraints, one range constraint $\mathrm{rchk}_{m\eta/2}$, and $m\mathrm{LU}_{2^\eta}$ lookup constraints. For Reinforced Concrete, it costs $2m - 2$ PLONK constraints, together with $m\mathrm{LU}_{|T_1|} + \lceil \frac{m-1}{3} \rceil \mathrm{LU}_{23} + \lceil \frac{m}{4} \rceil \mathrm{LU}_{16}$. For `Monolith`, it costs $2m$ PLONK constraints and $\sum_{i=1}^{m} \mathrm{LU}_{s_i}$; the Tip5 family similarly costs $2m$ PLONK constraints and $m\mathrm{LU}_{256}$ over Goldilocks. The resulting full-primitive costs are given in Table 16.

9.4.3 Type-I Primitives

The common power-map strategy and composite-layer definitions are given in Sections 9.1.1 and 9.1.2; the affine costs are given in Section 9.4.1.

GMiMCHash. A direct Feistel update costs $t - 1$ constraints per round. Viewing the non-S-box branches as accumulators instead caps this at three sliding-window constraints per round, with a one-time closing cost of $t - 4$ when $t \geq 4$. Hence the non-linear Feistel layers cost $R(\ell(\alpha) + t - 1)$ constraints for $t \leq 4$, and $R(\ell(\alpha) + 3) + t - 4$ for $t \geq 4$; the cyclic shift is free. Only the first and last round-constant additions require constraints. Including these two constraints and t output assertions, the permutation costs $R(\ell(\alpha) + t - 1) + t + 2$ for $t \leq 4$, and $R(\ell(\alpha) + 3) + 2t - 2$ for $t \geq 4$.

Table 16: Number of PLONK three-wire constraints. The round numbers for the evaluated formulae are taken from Tables 17 and 18.

Primitive	Formula	$\log_2(p) \approx 32$, $t = 24$, $\alpha = 3/5/7$	$\log_2(p) \approx 64$, $t = 12$, $\alpha = 7$	$\log_2(p) \approx 256$, $t = 4$, $\alpha = 5$
GMiMCHash	$\begin{Bmatrix} R\cdot(\ell(\alpha)+t-1)+t+2, & t\le 4,\\ R\cdot(\ell(\alpha)+3)+2\cdot t-2, & t\ge 4 \end{Bmatrix}$	1721/2056/2391	673	1392
GMiMCHash2	$\begin{Bmatrix} R\cdot(\ell(\alpha)+t-1)+t+1, & t<4,\ \mathsf{M_{IO}}=\mathsf{M}_I,\\ R\cdot(\ell(\alpha)+t-1)+3\cdot t, & t<4,\ \mathsf{M_{IO}}\neq\mathsf{M}_I,\\ R\cdot(\ell(\alpha)+3)+2\cdot t-3, & t\ge 4,\ \mathsf{M_{IO}}=\mathsf{M}_I,\\ R\cdot(\ell(\alpha)+3)+4\cdot t-4, & t\ge 4,\ \mathsf{M_{IO}}\neq\mathsf{M}_I \end{Bmatrix}$	1148 ($\alpha = 2$)	524 ($\alpha = 4$)	389 ($\alpha = 8$)
Neptune	$\frac{9t}{2}R_{\mathrm{F}} + \ell(\alpha)R_{\mathrm{P}} + t(R_{\mathrm{F}}+1)\cdot\left(\frac{t}{2}-1\right) + \min\left\{t(t-1)R_{\mathrm{P}},\ (2t-1)R_{\mathrm{P}},\ (2t-2)R_{\mathrm{P}} + \frac{t(t-1)}{2}\right\} + t + \begin{Bmatrix} t\cdot R_{\mathrm{F}}, & t=2,\\ 0, & t\ge 4 \end{Bmatrix}$	–	1890	758
Poseidon	$\ell(\alpha)\cdot(t\cdot R_{\mathrm{F}} + R_{\mathrm{P}}) + t\cdot(t-1)\cdot R_{\mathrm{F}} + (2\cdot t-2)\cdot R_{\mathrm{P}} + 2\cdot t$	5952/6118/6282	2036	704
Poseidon2	$\ell(\alpha)(tR_{\mathrm{F}} + R_{\mathrm{P}}) + (R_{\mathrm{F}}+1)G_{\mathrm{ext}} + \min\left\{(2t-1)R_{\mathrm{P}},\ (2t-2)R_{\mathrm{P}} + t(t-1) - G_{\mathrm{ext}}\right\} + t, G_{\mathrm{ext}} = \begin{Bmatrix} 2, & t=2,\\ 2t-1, & t=3,\\ 8, & t=4,\\ 24, & t=8,\\ 4t-4, & t\ge 12 \end{Bmatrix}$	2363/2528/2691	1386	680
pSquare-hash	$R\cdot(3\cdot t-4)+3\cdot t$	3608	–	–
Anemoi	$R\cdot\frac{t}{2}\cdot(\ell(\alpha)+5) + (R+1)\cdot\left(2\cdot G_{t/2}+t\right) + t, \quad G_{t/2} = \begin{Bmatrix} 0, & t/2=1,\\ 2, & t/2=2,\\ 5, & t/2=3,\\ 8, & t/2=4,\\ \frac{t}{2}\cdot\left(\frac{t}{2}-1\right), & t/2\ge 5 \end{Bmatrix}$	3660/3768/3876	1344	492
Arion	$R\cdot\left((\ell(\alpha_1)+5)\cdot(t-1) + \ell(\alpha_2) - 1 + G_M\right) + t, \quad G_M = \begin{Bmatrix} 5, & t=3,\\ 4\cdot t-4, & t\ge 4 \end{Bmatrix}$	–	–	444
Griffin	$R\cdot(4\cdot t + 2\cdot\ell(\alpha) - 9 - \varepsilon) + (R+1)\cdot G_M + t, \quad G_M = \begin{Bmatrix} 2\cdot t-1, & t=3,\\ 8, & t=4,\\ 24, & t=8,\\ 4\cdot t-4, & t\ge 12 \end{Bmatrix}, \quad \varepsilon = \begin{Bmatrix} 1, & t\le 4,\\ 0, & t\ge 8 \end{Bmatrix}$	–	784	412
Rescue *Rescue-Prime* *Rescue-Prime Optimized*	$2\cdot R\cdot t\cdot(\ell(\alpha)+t-1) + t + \begin{Bmatrix} t, & \textit{Rescue},\\ 0, & \textit{Rescue-Prime},\ \textit{Rescue-Prime Optimized} \end{Bmatrix}$	9624/10008/10392	2892	628
XHash	$R\ell(\alpha)\cdot\begin{Bmatrix} t, & \text{XHash12, XHash24,}\\ \frac{5\cdot t}{6}, & \text{XHash8, XHash16} \end{Bmatrix} + \frac{R}{2}\frac{t}{3}\ell_{p^3}(\alpha) + (R+1)\cdot t(t-1) + \frac{Rt}{3} + t, \quad \ell_{p^3}(\alpha) = 9\cdot\lfloor\log_2\alpha\rfloor + 15\cdot(\mathrm{hw}(\alpha)-1)$	5160/5088 ($t = 24$, $\alpha = 5$)	1824/1776 ($t = 12$)	–
Monolith	$(R+1)\cdot t\cdot(t-1) + R\cdot(2\cdot m\cdot u + 2\cdot(t-1)) + t + R\cdot u\cdot\sum_{i=1}^{m}\mathrm{LU}_{s_i}$	$4548 + 144\,\mathrm{LU}_{256} + 48\,\mathrm{LU}_{128}$	$1452 + 192\,\mathrm{LU}_{256}$	–
Reinforced Concrete	$(R-R_{\mathrm{B}})\cdot(\ell(\alpha)+2(t-1)) + R_{\mathrm{B}}t(2m-2) + (R+1)\cdot(2t-1) + t + R_{\mathrm{B}}t\left(m\,\mathrm{LU}_{\lvert T_1\rvert} + \left\lceil\frac{m-1}{3}\right\rceil\mathrm{LU}_{23} + \left\lceil\frac{m}{4}\right\rceil\mathrm{LU}_{16}\right)$	–	–	$241 + 81\,\mathrm{LU}_{1637} + 27\,\mathrm{LU}_{23} + 21\,\mathrm{LU}_{16}$ (BLS12-381), $241 + 81\,\mathrm{LU}_{1509} + 27\,\mathrm{LU}_{23} + 21\,\mathrm{LU}_{16}$ (BN254), ($t = 3$)
Skyscraper	$\frac{t}{2}\cdot R_{\mathrm{B}}\cdot(2\cdot m+7) + (R-R_{\mathrm{B}})\cdot\begin{Bmatrix} 1, & t=2,\\ 5, & t=4 \end{Bmatrix} + \frac{t}{2}R + t + \frac{t}{2}\cdot R_{\mathrm{B}}\cdot\left(m\,\mathrm{LU}_{2^\eta} + \mathrm{rchk}_{m\cdot\eta/2}\right)$	–	–	$678 + 256\,\mathrm{LU}_{256} + 8\,\mathrm{rchk}_{128}$
Tip5/Tip4/Tip4'	$R\cdot(2mu + \ell(\alpha)(t-u) + t(t-1)) + t + Rum\,\mathrm{LU}_{256}$	–	$\begin{Bmatrix} 1776 + 160\,\mathrm{LU}_{256}, & \text{Tip5, Tip4 } (t=16),\\ 1152 + 160\,\mathrm{LU}_{256}, & \text{Tip4' } (t=12) \end{Bmatrix}$	–
Grendel	$R\cdot t\cdot(\ell(\alpha)+t+3-\varepsilon) + t, \quad \varepsilon = \begin{Bmatrix} 1, & \alpha=2,\\ 0, & \alpha\ge 3 \end{Bmatrix}$	–	–	924
Polocolo	$Rt(\log_2(\tau)+3) + (R+1)\cdot\begin{Bmatrix} 5, & t=3\\ 8, & t=4\\ 13, & t=5\\ 17, & t=6\\ 24, & t=7\\ 31 & t=8 \end{Bmatrix} + t, \quad 2Rt\,\mathrm{LU}_{\tau+1},$	–	–	$292 + 40\,\mathrm{LU}_{513}$

GMiMCHash2. The non-linear Feistel layers have the same cost as for GMiMCHash, and the cyclic shift is free. The two M_{IO} applications cost $2t$ constraints when it is non-identity; if $\mathsf{M}_{\mathrm{IO}} = \mathsf{M}_I$, one round-constant addition instead requires a constraint. Including t output assertions, the permutation costs $R(\ell(\alpha) + t - 1) + t + 1$ for $t < 4$ and $\mathsf{M}_{\mathrm{IO}} = \mathsf{M}_I$, $R(\ell(\alpha) + t - 1) + 3t$ for $t < 4$ and $\mathsf{M}_{\mathrm{IO}} \neq \mathsf{M}_I$, $R(\ell(\alpha) + 3) + 2t - 3$ for $t \geq 4$ and $\mathsf{M}_{\mathrm{IO}} = \mathsf{M}_I$, and $R(\ell(\alpha) + 3) + 4t - 4$ for $t \geq 4$ and $\mathsf{M}_{\mathrm{IO}} \neq \mathsf{M}_I$.

Poseidon. The S-box layers cost $\ell(\alpha)(tR_{\mathrm{F}} + R_{\mathrm{P}})$ constraints, the external layers cost $t(t-1)R_{\mathrm{F}}$, and the folded internal layers cost $(2t-2)R_{\mathrm{P}}$. Only the initial round-constant addition costs t constraints; all others are absorbed. With t output assertions, the permutation costs $\ell(\alpha)(tR_{\mathrm{F}} + R_{\mathrm{P}}) + t(t-1)R_{\mathrm{F}} + (2t-2)R_{\mathrm{P}} + 2t$.

Poseidon2. The S-box layers cost $\ell(\alpha)(tR_{\mathrm{F}} + R_{\mathrm{P}})$ constraints and the external layers cost $(R_{\mathrm{F}} + 1)G_{\mathrm{ext}}$; for $t = 2$, each external matrix costs 2 constraints. The internal layers cost the minimum of $(2t-1)R_{\mathrm{P}}$ and $(2t-2)R_{\mathrm{P}} + t(t-1) - G_{\mathrm{ext}}$. All round constants are absorbed. Including t output assertions, the permutation costs $\ell(\alpha)(tR_{\mathrm{F}} + R_{\mathrm{P}}) + (R_{\mathrm{F}} + 1)G_{\mathrm{ext}} + \min\{(2t-1)R_{\mathrm{P}}, (2t-2)R_{\mathrm{P}} + t(t-1) - G_{\mathrm{ext}}\} + t$.

Neptune. After folding the affine map internal to each pair-wise Lai–Massey S-box into the surrounding constraints, the full-round non-linear layers cost $9tR_{\mathrm{F}}/2$ constraints. The partial-round power maps cost $\ell(\alpha)R_{\mathrm{P}}$, and the external layers cost $t(R_{\mathrm{F}} + 1)(t/2 - 1)$. The internal layers cost the minimum of $t(t-1)R_{\mathrm{P}}$, $(2t-1)R_{\mathrm{P}}$, and $(2t-2)R_{\mathrm{P}} + t(t-1)/2$. Including t output assertions, the permutation costs $9tR_{\mathrm{F}}/2 + \ell(\alpha)R_{\mathrm{P}} + t(R_{\mathrm{F}} + 1)(t/2 - 1) + \min\{t(t-1)R_{\mathrm{P}}, (2t-1)R_{\mathrm{P}}, (2t-2)R_{\mathrm{P}} + t(t-1)/2\} + t$. Round constants are absorbed for $t \geq 4$, but add tR_{F} constraints for $t = 2$.

pSquare-hash. The Feistel S-boxes cost $3tR/2$ constraints, the feed-forward costs $tR/2$, the matrix M costs $(t-4)R$, and the two M_{IO} applications cost $2t$. Round constants are incorporated into the Feistel constraints. Including t output assertions, the permutation costs $R(3t-4) + 3t$.

9.4.4 Type-II Primitives

The common power-map strategy and composite-layer definitions are given in Sections 9.1.1 and 9.1.2; the PLONK-specific extension-field power map and affine costs are given above.

***Rescue*, *Rescue-Prime*, & *Rescue-Prime Optimized*.** The forward and inverse S-box layers cost $2Rt\ell(\alpha)$ constraints, and the dense linear layers cost $2Rt(t-1)$. All round constants are absorbed for *Rescue-Prime* and *Rescue-Prime Optimized*; the initial constants cost t constraints for *Rescue*. Including t output assertions, the permutation costs $2Rt(\ell(\alpha) + t - 1) + t$, plus another t for *Rescue*.

`Anemoi`. The closed `Flystel` layers cost $Rt(\ell(\alpha)+5)/2$ constraints. Each linear layer comprises two block-split matrix products costing $2G_{t/2}$ constraints and a Pseudo-Hadamard transform costing t constraints, where $G_{t/2}$ is $0, 2, 5, 8$ for half-state sizes $1, 2, 3, 4$, respectively, and $(t/2)(t/2-1)$ thereafter. All round constants are absorbed. Including t output assertions, the permutation costs $Rt(\ell(\alpha) + 5)/2 + (R+1)(2G_{t/2} + t) + t$.

XHash. The (FB)-rounds cost $Rt\ell(\alpha)$ constraints for XHash12 and XHash24, and $5Rt\ell(\alpha)/6$ for XHash8 and XHash16. The $(P3)$-rounds cost $Rt\ell_{p^3}(\alpha)/6$, and the dense linear layers cost $(R+1)t(t-1)$. Only the $(P3)$-round constants require extra constraints,

costing $Rt/3$. Including t output assertions, the permutation cost is the applicable (FB) cost plus $Rt\ell_{p^3}(\alpha)/6 + (R+1)t(t-1) + Rt/3 + t$.

Griffin. The non-linear layers cost $R(4t + 2\ell(\alpha) - 9 - \varepsilon)$ constraints after sharing one constraint with the linear layer when $t \leq 4$, where $\varepsilon = 1$ for $t \leq 4$ and 0 otherwise. For $t = 3$, the MDS matrix costs $G_M = 5$ constraints per application; the larger instances use the Duval–Leurent matrices described above. The $R+1$ linear layers therefore cost $(R+1)G_M$. All round constants are absorbed. Including t output assertions, the permutation costs $R(4t + 2\ell(\alpha) - 9 - \varepsilon) + (R+1)G_M + t$.

Arion. The non-linear layers cost $R((\ell(\alpha_1) + 5)(t-1) + \ell(\alpha_2) - 1)$ constraints. The matrix costs $G_M = 5$ constraints per application if $t = 3$ and $G_M = 4t - 4$ constraints if $t \geq 4$, so the linear layers cost RG_M. All round constants are absorbed. Including t output assertions, the permutation costs $R((\ell(\alpha_1) + 5)(t-1) + \ell(\alpha_2) - 1 + G_M) + t$.

9.4.5 Type-III Primitives.

The composite non-linear layers and generic split-and-lookup relations are defined in Sections 9.1.2 and 9.1.3; their PLONK-specific lookup and affine costs are given above.

Reinforced Concrete. The `Bricks` layers cost $(R - R_{\mathrm{B}})(\ell(\alpha) + 2(t-1))$ constraints. The Bars cost $R_{\mathrm{B}}t(2m-2)$ arithmetic constraints and $R_{\mathrm{B}}t(m\,\mathrm{LU}_{|T_1|} + \lceil (m-1)/3 \rceil\ \mathrm{LU}_{23} + \lceil m/4 \rceil\ \mathrm{LU}_{16})$; the Concrete layers cost $(R+1)(2t-1)$. All round constants are absorbed. Including t output assertions, the permutation cost is the sum of these layer costs and t.

`Monolith`. Materializing each square and subsequent addition separately, the `Bricks` layers cost $2R(t-1)$ constraints. The Concrete layers cost $(R+1)t(t-1)$, while the Bars cost $2muR$ arithmetic constraints and $Ru\sum_{i=1}^{m}\mathrm{LU}_{s_i}$. All round constants are absorbed. Including t output assertions, the permutation costs $(R+1)t(t-1) + R(2mu + 2(t-1)) + t + Ru\sum_{i=1}^{m}\mathrm{LU}_{s_i}$.

Skyscraper. The Bars cost $tR_{\mathrm{B}}(2m+7)/2$ arithmetic constraints and $tR_{\mathrm{B}}(m\,\mathrm{LU}_{2^\eta} + \mathrm{rchk}_{m\eta/2})/2$. The Square layers cost $R - R_{\mathrm{B}}$ constraints for $t = 2$ and, using the two-product extension-field squaring, $5(R - R_{\mathrm{B}})$ for $t = 4$; the Feistel additions cost $tR/2$. All round constants are absorbed. Including t output assertions, the permutation cost is the sum of these layer costs and t.

Tip5, Tip4, & Tip4'. In each round, the u split-and-lookup S-boxes cost $2mu$ arithmetic constraints and $um\,\mathrm{LU}_{256}$ lookup constraints, while the remaining $t - u$ power maps cost $\ell(\alpha)(t-u)$ constraints. The dense linear layer costs $t(t-1)$ constraints per round. All round constants are absorbed. Including t output assertions, the permutation costs $R\big(2mu + \ell(\alpha)(t-u) + t(t-1)\big) + t$ arithmetic constraints and $Rum\,\mathrm{LU}_{256}$ lookup constraints.

9.4.6 Type-IV Primitives

The shared power-residue systems are given in Section 9.1.4; the affine costs are given in Section 9.4.1.

Grendel. The S-box layers cost $Rt(\ell(\alpha) + 4 - \varepsilon)$ constraints, where $\varepsilon = 1$ for $\alpha = 2$ and 0 otherwise, and the dense linear layers cost $Rt(t-1)$. All round constants are absorbed. Including t output assertions, the permutation costs $Rt(\ell(\alpha) + t + 3 - \varepsilon) + t$.

Polocolo. The S-box layers cost $Rt(\log_2(\tau)+3)$ arithmetic constraints and $2Rt\,\mathrm{LU}_{\tau+1}$. The dedicated linear programs cost $5, 8, 13, 17, 24, 31$ constraints per application for state sizes $3, 4, 5, 6, 7, 8$, respectively. All round constants are absorbed. Including t output assertions, the permutation costs $Rt(\log_2(\tau)+3)$ plus $R+1$ times the applicable linear-layer cost, plus t and the lookup constraints.

10 Performances

We performed the plain and proof system experiments on an AMD Ryzen 9 7900X with 12 physical cores and 24 hardware threads, and 124 GiB of available system memory, running Linux 6.8.0. The round numbers for all instances are chosen based on the cryptanalysis above. In Tables 17 and 18, we give an overview of the concrete instances and associated rounds used in our plain and proof system performance evaluations. For Hades designs, entries of the form $a+b$ give $R_\mathrm{F}+R_\mathrm{P}$.

Table 17: Round numbers over 256-bit and 64-bit prime fields at a 128-bit security level. BLS12-381 and BN254 yield identical round numbers for every construction listed here and are therefore merged into a single column group. Each cell displays the most recent round number, with the original number in parentheses if updated. The power map α fixed for each field.

	BLS12-381 / BN254 $\alpha=5$		Goldilocks $\alpha=7$	
Construction	$t=3$	$t=4$	$t=8$	$t=12$
GMiMCHash	228	231	68	93
GMiMCHash2†	63	64	88	96
Neptune	—	$6+69$	$6+38$	$6+42$
Poseidon	$8+56$	$8+56$	$8+22$	$8+22$
Poseidon2	$8+56$	$8+56$	$8+22$	$8+22$
`Anemoi`	—	20 (14)	11	10
Arion	11	10	—	—
Griffin	20 (14)	20 (11)	8	8
Rescue-Prime	16 (14)	13 (11)	8	8
XHash12	—	—	—	6
`Monolith`	—	—	6	6
Reinforced Concrete	7	—	—	—
Skyscraper	—	18	—	—
Tip4'	—	—	—	5
Grendel	24 (14)	23 (12)	—	—
Polocolo	6	5	—	—

† GMiMCHash2 only has $\alpha \in \{2, 4, 8\}$ and instead uses $\alpha = 8$ over BLS12-381/BN254 and $\alpha = 4$ over Goldilocks.

10.1 Plain Implementations

For our benchmarks of the discussed constructions, we implement them in the same ecosystem and aim to provide a similar level of optimization. For field arithmetic over the BN254 and BLS12-381 scalar we employ the `arkworks` library,[32] while arithmetic over the

[32] https://arkworks.rs

Table 18: Round numbers over 31-bit prime fields at a 128-bit security level. Each cell displays the most recent round number, with the original number in parentheses if updated. α fixed for each field is stated in the header.

	Mersenne31 $\alpha = 5$		KoalaBear $\alpha = 3$		BabyBear $\alpha = 7$	
Construction	$t = 16$	$t = 24$	$t = 16$	$t = 24$	$t = 16$	$t = 24$
GMiMCHash	158	335	158	335	158	335
GMiMCHash2[†]	176	264	176	264	176	264
Poseidon	$8 + 14$	$8 + 22$	$8 + 20$	$8 + 23$	$8 + 13$	$8 + 21$
Poseidon2	$8 + 14$	$8 + 22$	$8 + 20$	$8 + 23$	$8 + 13$	$8 + 21$
pSquare-hash	52	52	52	52	52	52
`Anemoi`	10	9	10	9	9	9
Rescue-Prime	8	8	8	8	8	8
XHash24	—	6	—	—	—	—
`Monolith`	6	6	6	6	6	6

[†] GMiMCHash2 only has $\alpha \in \{2, 4, 8\}$ and $\alpha = 2$ throughout.

Goldilocks field, Mersenne31, BabyBear, and KoalaBear fields is provided by Plonky3.[33] To ensure a fair comparison, all primitives share the same underlying field implementations, so that observed differences reflect the algebraic properties of each design. We benchmark a single-threaded call to each permutation and report the median of 100 runs in Tables 19 to 21. Whenever the cryptanalysis led to an updated round number, we provide data for the originally proposed round numbers in parenthesis.

Table 19: Native permutation timings over BLS12-381 and BN254. Median of 100 runs, with units given per column. For cells whose round count was updated, the original round count data is given in parentheses.

	BLS12-381		BN254	
Construction	$t = 3$ (µs)	$t = 4$ (µs)	$t = 3$ (µs)	$t = 4$ (µs)
GMiMCHash	15.97	16.28	15.30	15.52
GMiMCHash2	6.88	7.07	6.73	6.86
Neptune	—	9.92	—	9.58
Poseidon	15.64	23.29	15.01	22.55
Poseidon2	4.85	8.75	4.75	8.38
`Anemoi`	—	281.47 (197.08)	—	276.20 (193.35)
Arion	83.55	80.61	81.34	78.33
Griffin	139.49 (97.78)	140.66 (77.36)	137.11 (95.95)	137.99 (76.02)
Rescue-Prime	336.98 (294.77)	366.70 (310.22)	330.20 (288.91)	359.00 (303.79)
Reinforced Concrete	1.85	—	1.86	—
Skyscraper	—	1.24	—	1.21
Grendel	1 035.60 (582.27)	1 329.62 (663.94)	1 027.13 (574.02)	1 317.73 (658.93)
Polocolo	257.74	286.55	231.62	258.11

Results. For native performance, we see the common trend between the different types of algebraic hash functions. Type-III, with its non-algebraic S-boxes brings forth the most efficient constructions. We see `Monolith` as the fastest construction in Goldilocks and Mersenne31 following Table 20. While `Monolith` is also applicable to KoalaBear and

[33] https://github.com/Plonky3/Plonky3

Table 20: Native permutation timings over Goldilocks and Mersenne31. Median of 100 runs, with units given per column. For cells whose round count was updated, the original round count data is given in parentheses.

	Goldilocks		Mersenne31	
Construction	$t = 8$ (ns)	$t = 12$ (ns)	$t = 16$ (ns)	$t = 24$ (ns)
GMiMCHash	643.65	899.67	1 025.32	2 438.32
GMiMCHash2	1 122.36	1 380.10	3 085.00	6 836.36
Neptune	1 217.86	2 321.44	—	—
Poseidon	2 474.61	4 724.94	4 950.55	13 876.18
Poseidon2	747.34	1 041.64	967.95	1 609.88
pSquare-hash	—	—	933.04	1 358.01
`Anemoi`	10 577.82	14 545.89	10 416.86	14 677.40
Griffin	2 165.65	2 301.60	—	—
Rescue-Prime	15 434.27	24 066.67	20 106.18	32 416.82
XHash8	—	8 665.51	—	—
XHash12	—	11 331.89	—	—
XHash16	—	—	—	9 212.96
XHash24	—	—	—	11 110.01
`Monolith`	253.85	392.60	410.29	834.83
Tip4'	—	1 270.46	—	—

BabyBear, we do not provide an implementation as the decomposition is prime dependent and the original publication provided a clear reference only for Goldilocks and Mersenne31, else we believe it would also be the fastest choice in Table 21. In Table 19, we see that Reinforced Concrete and Skyscraper are best performing across primes and state sizes. Type-I primitives are second in native performance with GMiMCHash, Poseidon2, or pSquare-hash being the fastest, non Type-III choice across all primes and state sizes. On the other end of the spectrum, Type-II and Type-IV build on quite expensive S-boxes that permit an efficient representation in the proof system yielding the least efficient constructions when it comes to native performance. We further see, that the gap between the fastest and slowest constructions widens with the size of the prime. This effect is expected and due to the Type-II and Type-IV S-boxes (i.e., being inversions or power-residue computation) scaling much worse with the field size.

Table 21: Native permutation timings over KoalaBear and BabyBear. Median of 100 runs, with units given per column. For cells whose round count was updated, the original round count data is given in parentheses.

	KoalaBear		BabyBear	
Construction	$t = 16$ (µs)	$t = 24$ (µs)	$t = 16$ (µs)	$t = 24$ (µs)
GMiMCHash	1.17	2.53	1.88	3.96
GMiMCHash2	2.22	3.57	2.22	3.60
Poseidon	7.73	18.56	6.16	17.63
Poseidon2	1.07	1.55	1.17	1.90
pSquare-hash	1.19	1.77	1.19	1.77
`Anemoi`	15.02	21.05	14.72	22.83
Rescue-Prime	27.40	44.41	29.92	48.12

10.2 Proof System Performance

We now measure the discussed constructions inside two different proof systems: Plonky3, covering small-prime fields, and Gnark, covering the large-prime fields. We test each construction with instances and round numbers defined in Tables 17 and 18, evaluating only those that are explicitly defined for a particular setting. As a result, Arion, Skyscraper, and Polocolo appear only in the Gnark tables, whereas pSquare-hash, XHash, Tip4', and `Monolith` appear only in the Plonky3 tables. We acknowledge the following gaps in our implementation. For `Monolith`, the decomposition is prime dependent, and we follow only the defined instances in the original paper, even though the construction generalizes. On the Gnark side, there is no support for multi-column lookup relations, and we have thus not included Reinforced Concrete. *Grendel* was excluded due to the S-box evaluation being orders of magnitude slower in the plain evaluation and preliminary implementation was dominated by what we deemed an implementation artifact.

10.2.1 Plonky3 Benchmarks

For the Plonky3[34] benchmarks, we prove many permutation calls per primitive as a univariate STARK over FRI, with 12 threads. Every configuration is zero-knowledge (hiding FRI) and parameterized to achieve soundness above 100 bits according to the internal soundness bounds. We use the AIR cost model and the degree-dependent FRI blowup described in Section 9.3, setting the blowup to the minimum value permitted by the maximum degree of each construction. For our experiments, we compare one state size per field, $t = 24$ over KoalaBear and BabyBear and $t = 12$ over Goldilocks, at the lowest exponent that permits a permutation (for constructions where this is relevant). We do not compare performance under Mersenne31, since at the time of writing, there is no straight-forward support for the ZK pipeline yet. For all constructions, we vectorize the single-call layout from Section 9.3 by placing 8 independent permutation calls side by side in one physical row, yielding 2^{14} rows and 2^{17} permutation calls; the reported cell counts are normalized per call. For the lookup-based designs, we have only a single permutation per row due to the relatively larger widths.

Table 22: Cost of proving 2^{14} trace rows. Cells are prover-committed cells per call, prove is witness generation and proving, and verify is verification. Every construction batches 2^{17} permutation calls per proof, except `Monolith` and Tip4' 2^{14}.

		KoalaBear ($t = 24$)			BabyBear ($t = 24$)			Goldilocks ($t = 12$)		
construction	α	cells	prove ms	verify ms	cells	prove ms	verify ms	cells	prove ms	verify ms
GMiMCHash	3/7/7	718	9334.84	50.35	718	18764.75	34.10	210	11911.04	22.99
GMiMCHash2	2/2/4	312	4439.13	29.20	312	4479.42	29.49	216	5985.59	32.65
Neptune	7		–			–		750	18457.65	79.73
Poseidon	3/7/7	239	7776.11	18.15	450	12688.47	25.32	248	14297.90	24.62
Poseidon2	3/7/7	239	7378.80	17.88	450	12156.01	24.81	248	13599.50	24.97
pSquare-hash	–	672	8943.26	48.33	672	8824.24	48.85		–	
`Anemoi`	7		–			–		204	12215.22	22.45
Griffin	7		–			–		136	9315.94	18.86
Rescue-Prime	3/7/7	216	7536.70	17.19	600	16617.34	30.59	480	26609.42	37.43
XHash	7		–			–		192	11836.57	21.65
`Monolith`	–		–			–		950.1	4123.17	31.23
Tip4'	7		–			–		574.1	6276.39	19.10

[34] https://github.com/Plonky3/Plonky3

Results. As shown in Table 22, the prover and verifier times broadly follow the cell–degree trade-off described in Section 9.3. However, we make no claim of optimality as tuning the many different parameters might yield performance improvements for the individual designs. The best example of a different prover–verifier trade-off is the Goldilocks instances of GMiMCHash2 and GMiMCHash, where the cell count is nearly equivalent, but the higher degree leads to significantly cheaper verification for GMiMCHash. Finally, it is important to reiterate that the numbers for Tip4' and `Monolith` are per proof, for a trace that incurs 8 times fewer permutation calls. Roughly extrapolating their proof time yields for `Monolith` (33 s) and for Tip4' (51 s).

10.2.2 Gnark Benchmarks

For the Gnark benchmarks, Go's default parallelism was used. The implementation was compiled with Go 1.26.5 and uses gnark v0.15.0 and gnark-crypto v0.20.1. All circuits operate over the scalar field of BLS12-381. We consider the two proving styles of Gnark: R1CS with Groth16 and PLONK with the implemented sparse constraint system. The results should serve as a comparison within each category, since the R1CS pipeline is significantly more mature. As for the data, we distinguish between prover time, verifier time, and one-time circuit-specific setup time. However, we exclude unrelated one-time setups such as the trusted key generation ceremony. Further, we separate the results into two settings. First in Table 23, we display the costs associated with proving a single permutation call. Second, in Table 24, we provide data for a front-to-back proof of a Merkle-tree evaluation with 2^{11} leaf nodes.

Results. In both settings, the newly proposed GMiMCHash2 gives the smallest arithmetization and lowest prover time. More generally, Type-I and Type-II primitives exhibit better performance characteristics than Type-III and Type-IV. As discussed in Section 9, PLONK and R1CS arithmetization most crucially diverges in the cost of transformations. As a result, a hash function such as e.g., Neptune performs significantly worse within PLONK due to the matrices and the additional linear cost in its Lai-Massey S-box. Regarding prover time, we see a rough relation between constraints and prover time. Outliers: there are lookup-based constructions with a high per-proof setup cost, and the construction of Type-II and Polocolo incurs a high cost at hint generation. Due to the high cost of evaluation, we had to limit repetitions to 100 and to 5 runs for the full suite, making the already low absolute counts for Verify less reliable. We report it anyway to show that verifier time is in a similar ballpark, irrespective of the construction proven. Finally, we must add that lookup-based designs are dominated in cost by the lookups themselves. Thus, these numbers and their positioning relative to other designs may vary significantly depending on the lookup argument used.

Table 23: Cost of a single permutation call inside a circuit, over BLS12-381, with $t = 4$. Prove and verify are averaged over 100 runs.

Construction	Constraints	Prove (ms)	Verify (ms)	Circuit setup (ms)
R1CS				
GMiMCHash	697	6.94	0.77	50.9
GMiMCHash2	196	2.99	0.75	11.3
Neptune	232	3.59	0.78	15.0
Poseidon	268	4.14	0.78	17.6
Poseidon2	268	3.83	0.76	16.0
Anemoi	244	4.53	0.77	13.6
Arion	254	3.40	0.76	14.7
Griffin	224	3.49	0.78	12.1
Rescue-Prime	368	5.63	0.79	16.1
Skyscraper	1 425	17.76	1.55	132.6
Polocolo	1 553	21.51	1.49	120.4
PLONK				
GMiMCHash	1 392	25.26	1.84	40.1
GMiMCHash2	389	9.48	1.81	7.3
Neptune	758	15.85	1.87	17.3
Poseidon	704	15.12	1.79	14.9
Poseidon2	732	15.36	1.81	14.1
Anemoi	492	11.25	1.84	9.0
Arion	444	10.16	1.81	7.7
Griffin	412	10.75	1.81	8.3
Rescue-Prime	628	16.65	1.83	14.3
Skyscraper	5 290	73.69	1.94	84.9
Polocolo	4 041	53.48	1.85	61.5

Table 24: Cost of proving a 2:1 Merkle root over 2 048 leaves (2 047 node hashes) inside a single circuit, over BLS12-381, with $t = 4$. Each node is a compression or a sponge, in the mode its designers define. Prove and verify are averaged over 5 runs.

Construction	Constraints	Prove (s)	Verify (ms)	Circuit setup (s)
R1CS				
GMiMCHash	1 420 618	4.30	0.80	52.02
GMiMCHash2	393 057	1.19	0.91	12.87
Neptune	470 081	2.03	0.82	23.55
Poseidon	528 127	1.90	1.86	19.41
Poseidon2	540 409	1.85	0.86	20.25
Anemoi	496 738	2.84	0.82	18.78
Arion	1 024 984	3.35	0.83	37.04
Griffin	450 341	1.97	0.83	12.74
Rescue-Prime	1 484 077	7.16	1.86	38.10
Skyscraper	923 055	5.41	1.54	78.79
Polocolo	1 433 830	7.93	2.56	50.44
PLONK				
GMiMCHash	2 839 190	24.21	2.01	26.74
GMiMCHash2	786 049	5.90	2.03	6.80
Neptune	1 539 345	12.35	2.10	14.48
Poseidon	1 416 525	11.91	1.98	13.93
Poseidon2	1 488 170	11.96	1.94	13.70
Anemoi	1 011 220	7.83	2.09	7.34
Arion	1 807 503	13.12	2.75	13.68
Griffin	833 130	6.79	2.07	7.15
Rescue-Prime	2 552 611	27.65	1.95	26.76
Skyscraper	3 873 626	28.70	2.11	27.87
Polocolo	2 101 185	29.17	2.07	25.73

11 Conclusion and Outcomes

We conclude with some observations and outcomes based on the analysis proposed in this paper, focusing mainly on the security and on the performance aspects.

Security & Cryptanalysis. Type-I primitives have been subject to the most extensive and diverse cryptanalysis. While initial cryptanalysis of early primitives, like Poseidon and GMiMCHash, often turned out to be too optimistic, Type-I primitives stood the test of time relatively well. Of the parameters considered in this work, none failed to achieve its security claim. If at all, Type-I primitives require just a modest recalibration of their parameters to reinstall security margins. We have done such a recalibration with GMiMCHash2, by taking the best attacks on GMiMCHash into account, we could even reduce round numbers among many configurations.[35]

Type-II primitives received the most cryptanalysis attention for the CICO-1 problem over 256 bit fields. For this parameter regime, resultant-based techniques have identified many primitives as vulnerable. On the other hand, Type-II has seen almost no attention over 32 and 64 bit fields. At the time of writing, we believe that the security problems of Type-II primitives with respect to attacks other than algebraic are not clear yet. So, we encourage more efforts by the community.

Type-III primitives have enjoyed relatively little cryptanalysis so far. While recent works have exploited high probability differentials and linear approximations of S&L S-boxes, it is not clear yet whether these techniques can be improved further or whether they hit a barrier after a certain number of rounds. Moreover, S&L S-boxes have seen no Gröbner basis cryptanalysis beyond the primitive papers.

Type-IV primitives have only been introduced for 256 bit fields, so the best attacks are algebraic in nature. Quickly after its introduction, *Grendel* has already been identified as vulnerable to polynomial system solving. However, for the later primitive Polocolo it is not clear yet whether dedicated algebraic techniques can improve upon the state of the art.

Plain Performance. Due to their algebraic simplicity and low polynomial degrees, Type-I primitives enjoy decent plain performance. Even if designers tried to identify cheap linear layers with good cryptanalytic properties, their main bottleneck is the linear layers, as it is clear by comparing, for example, the performance of Poseidon and Poseidon2. Exceptions to this paradigm are GMiMCHash and GMiMCHash2. Their round functions are so cheap to implement that they can compensate for a large number of rounds. As seen in our performance evaluation, GMiMCHash2 is one of the most performant primitives, so we encourage the community to find other "lightweight" round function candidates for AO hash functions.

On the other hand, Type-II primitives apply a high degree S-box $x^{\frac{1}{\alpha}}$ per round, and these operations dominate plain performance. For the parameters considered in this paper, Type-II primitives are at least an order of magnitude slower than Type-I primitives, and no implementation strategies are yet known to narrow this gap.

The clear winners of plain performance are Type-III primitives whose S&L S-boxes are based on binary functions. These designs are an order of magnitude faster than Type-I. In particular, `Monolith` almost closes the plain performance gap between AO hash functions and SHA-3. Type-IV primitives require the evaluation of a power residue $x^{\frac{p-1}{m}}$, where $m \ll p$. So, this high-degree monomial evaluation dominates plain performance, and they are orders of magnitude slower than Type-I. Again, no implementation strategies are known yet to narrow this gap.

[35] In the GMiMC paper, cryptanalysis was done for block ciphers, and then reused for the hash function. Since attacks on sponge function differ to block ciphers, a dedicated analysis leads to a better calibration of the round numbers.

Proof System Performance. For proof system performance, Type-I primitives are the most consistently competitive: GMiMCHash2 gives the lowest prover times in both our AIR and large-field benchmarks, while Poseidon and Poseidon2 remain strong across settings. While efficient choices of linear layers might benefit some PLONK arithmetizations, the dominant cost is still multiplicative complexity. Type-II designs can match or improve on these constructions in individual arithmetizations, but the round increases required by recent attacks have narrowed their advantage. While theoretically the advantage is clear, the same per-round cost as a Type-I construction with fewer rounds, the advantage is diminished by the substantial plain processing cost of generating the witness values required for the backward evaluation. For Type-III and Type-IV designs, the high costs of decomposition, lookups, auxiliary witnesses, and hint generation lead to long prover times. Though for lookup-based designs, a more efficient lookup argument might change the perspective. Finally, among the evaluated instances, simple Type-I designs provide the most robust proof-system performance, whereas rankings within and across the remaining types remain backend- and workload-dependent.

Conclusion. 10 years after the introduction of MiMC, the field of AO hash functions has seen exciting developments, including new design approaches for round functions and novel algebraic techniques for cryptanalysis. At the end of our survey, we can finally address the question of which design approaches have upheld their promises and which have failed.

At the state of the art, Type-I primitives provide the best performance trade-off for plain/proof systems. Moreover, for the parameters considered in this work, plain SPNs (Poseidon, Poseidon2) and a plain Feistel permutation (GMiMCHash) have withstood cryptanalysis efforts so far. Moreover, our updated primitive GMiMCHash2 turned out to be one of the best primitives in both plain and proof-system performance.

Type-II primitives like `Anemoi`, Arion, and Griffin, on the other hand, appeared to be very strong in proof system performance. Though the resultant-based attacks on these primitives required substantial round number increases, which worsened their plain performance and almost closed the proof system performance gap to Type-I primitives.

Among novel design strategies, Type-III primitives with lookup tables based on binary functions appear to be the most promising direction for improving Type-I primitives. In terms of plain performance, `Monolith` and Skyscraper were able to almost close the performance gap to SHA-3. However, on the proof system side, the results are mixed. In theory, Type-III primitives require relatively few arithmetic gates; however, in actual benchmarks, we observed that the S&L S-boxes significantly impact proof system performance. Cryptanalysis of S&L S-boxes is a challenging task requiring novel techniques. Thus, we encourage more cryptanalysis efforts to increase confidence in the security of these primitives.

Type-IV primitives enjoy similar fates to Type-II and Type-III primitives. Their plain performance bottleneck is the power residues, and on the proof system side, theoretical performance advantages do not translate into running times.

Thus, as conclusion relatively simple adaptions of design strategies over binary fields, like Poseidon, Poseidon2 and GMiMCHash2, emerged as the most competitive primitives, while novel approaches like Type-II and Type-IV did not uphold their improved proof system performance promises.

Acknowledgment. Hailun Yan is supported by the National Key Research and Development Program of China (2024YFB4504800), the National Cryptologic Science Foundation of China (2025NCSF01012), and the National Natural Science Foundation of China (62532013 and 62202444). Lorenzo Grassi was supported by the European Research Council (ERC), grant number 101160608 "SYMPZON". Views and opinions expressed are, however, those of the author(s) only and do not necessarily reflect those of the European Union or the

European Research Council. Neither the European Union nor the granting authority can be held responsible for them.

References

[AAB+20] Abdelrahaman Aly, Tomer Ashur, Eli Ben-Sasson, Siemen Dhooghe, and Alan Szepieniec. Design of symmetric-key primitives for advanced cryptographic protocols. *IACR Trans. Symm. Cryptol.*, 2020(3):1–45, 2020. doi:10.13154/tosc.v2020.i3.1-45.

[ABK+23] Tomer Ashur, Amit Singh Bhati, Al Kindi, Mohammad Mahzoun, and Léo Perrin. XHash: Efficient STARK-friendly hash function. Cryptology ePrint Archive, Report 2023/1045, 2023. URL: https://eprint.iacr.org/2023/1045.

[ABK+26] Tomer Ashur, Amit Singh Bhati, Al Kindi, Mohammad Mahzoun, Léo Perrin, and Sundas Tariq. The XHash family for ZK-friendly hash functions. *DCC*, 94(132), 2026. doi:10.1007/s10623-026-01875-1.

[ABM24] Tomer Ashur, Thomas Buschman, and Mohammad Mahzoun. Algebraic cryptanalysis of the HADES design strategy: Application to Poseidon and Poseidon2. In Tianqing Zhu and Yannan Li, editors, *ACISP 24, Part II*, volume 14896 of *LNCS*, pages 225–244. Springer, Singapore, July 2024. doi:10.1007/978-981-97-5028-3_12.

[ACG+19] Martin R. Albrecht, Carlos Cid, Lorenzo Grassi, Dmitry Khovratovich, Reinhard Lüftenegger, Christian Rechberger, and Markus Schofnegger. Algebraic cryptanalysis of STARK-friendly designs: Application to MARVELlous and MiMC. In Steven D. Galbraith and Shiho Moriai, editors, *ASIACRYPT 2019, Part III*, volume 11923 of *LNCS*, pages 371–397. Springer, Cham, December 2019. doi:10.1007/978-3-030-34618-8_13.

[AD18] Tomer Ashur and Siemen Dhooghe. MARVELlous: a STARK-friendly family of cryptographic primitives. Cryptology ePrint Archive, Report 2018/1098, 2018. URL: https://eprint.iacr.org/2018/1098.

[AGP+19a] Martin R. Albrecht, Lorenzo Grassi, Léo Perrin, Sebastian Ramacher, Christian Rechberger, Dragos Rotaru, Arnab Roy, and Markus Schofnegger. Feistel structures for MPC, and more. In Kazue Sako, Steve Schneider, and Peter Y. A. Ryan, editors, *ESORICS 2019, Part II*, volume 11736 of *LNCS*, pages 151–171. Springer, Cham, September 2019. doi:10.1007/978-3-030-29962-0_8.

[AGP+19b] Martin R. Albrecht, Lorenzo Grassi, Leo Perrin, Sebastian Ramacher, Christian Rechberger, Dragos Rotaru, Arnab Roy, and Markus Schofnegger. Feistel structures for MPC, and more. Cryptology ePrint Archive, Report 2019/397, 2019. URL: https://eprint.iacr.org/2019/397.

[AGR+16] Martin R. Albrecht, Lorenzo Grassi, Christian Rechberger, Arnab Roy, and Tyge Tiessen. MiMC: Efficient encryption and cryptographic hashing with minimal multiplicative complexity. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *ASIACRYPT 2016, Part I*, volume 10031 of *LNCS*, pages 191–219. Springer, Berlin, Heidelberg, December 2016. doi:10.1007/978-3-662-53887-6_7.

[AKM+22] Tomer Ashur, Al Kindi, Willi Meier, Alan Szepieniec, and Bobbin Threadbare. Rescue-Prime Optimized. Cryptology ePrint Archive, Report 2022/1577, 2022. URL: https://eprint.iacr.org/2022/1577.

[AKMQ23] JP Aumasson, Dmitry Khovratovich, Bart Mennink, and Porçu Quine. SAFE: Sponge API for field elements. Cryptology ePrint Archive, Report 2023/522, 2023. URL: https://eprint.iacr.org/2023/522.

[AMP11] Elena Andreeva, Bart Mennink, and Bart Preneel. Security reductions of the second round SHA-3 candidates. In Mike Burmester, Gene Tsudik, Spyros S. Magliveras, and Ivana Ilic, editors, *ISC 2010*, volume 6531 of *LNCS*, pages 39–53. Springer, Berlin, Heidelberg, October 2011. doi:10.1007/978-3-642-18178-8_5.

[ARS+15] Martin R. Albrecht, Christian Rechberger, Thomas Schneider, Tyge Tiessen, and Michael Zohner. Ciphers for MPC and FHE. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part I*, volume 9056 of *LNCS*, pages 430–454. Springer, Berlin, Heidelberg, April 2015. doi:10.1007/978-3-662-46800-5_17.

[AT24] Tomer Ashur and Sundas Tariq. RPO-M31 and XHash-M31: Efficient hash functions for circle STARKs. Cryptology ePrint Archive, Report 2024/1635, 2024. URL: https://eprint.iacr.org/2024/1635.

[Bak25] Antoine Bak. A practical distinguisher on the full Skyscraper permutation. Cryptology ePrint Archive, Report 2025/102, 2025. URL: https://eprint.iacr.org/2025/102.

[BB24] Tim Beyne and Clémence Bouvier. Linear approximations of the Flystel construction. Cryptology ePrint Archive, Report 2024/1465, 2024. URL: https://eprint.iacr.org/2024/1465.

[BB26] Tim Beyne and Clémence Bouvier. Exponential sums in linear cryptanalysis. *J. Cryptol.*, 39(2):16, Feb 2026. doi:10.1007/s00145-026-09575-8.

[BBB+25a] Antoine Bak, Augustin Bariant, Aurélien Boeuf, Pierre Briaud, Morten Øygarden, and Atharva Phanse. The algebraic CheapLunch: Extending FreeLunch attacks on arithmetization-oriented primitives beyond CICO-1. Cryptology ePrint Archive, Report 2025/2040, 2025. URL: https://eprint.iacr.org/2025/2040.

[BBB+25b] Augustin Bariant, Aurélien Boeuf, Pierre Briaud, Maël Hostettler, Morten Øygarden, and Håvard Raddum. Improved resultant attack against arithmetization-oriented primitives. In Yael Tauman Kalai and Seny F. Kamara, editors, *CRYPTO 2025, Part V*, volume 16004 of *LNCS*, pages 335–367. Springer, Cham, August 2025. doi:10.1007/978-3-032-01901-1_11.

[BBB+25c] Augustin Bariant, Aurélien Boeuf, Pierre Briaud, Maël Hostettler, Morten Øygarden, and Håvard Raddum. Improved resultant attack against arithmetization-oriented primitives. Cryptology ePrint Archive, Report 2025/259, 2025. URL: https://eprint.iacr.org/2025/259.

[BBB+26] Antoine Bak, Augustin Bariant, Aurélien Boeuf, Maël Hostettler, and Guilhem Jazeron. Claiming bounties on small scale Poseidon and Poseidon2 instances using resultant-based algebraic attacks. IACR Cryptology ePrint Archive, Paper 2026/150, 2026. URL: https://eprint.iacr.org/2026/150.

[BBC+22] Clémence Bouvier, Pierre Briaud, Pyrros Chaidos, Léo Perrin, and Vesselin Velichkov. Anemoi: Exploiting the link between arithmetization-orientation and CCZ-equivalence. Cryptology ePrint Archive, Report 2022/840, 2022. URL: https://eprint.iacr.org/2022/840.

[BBC+23] Clémence Bouvier, Pierre Briaud, Pyrros Chaidos, Léo Perrin, Robin Salen, Vesselin Velichkov, and Danny Willems. New design techniques for efficient arithmetization-oriented hash functions: Anemoi permutations and Jive compression mode. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part III*, volume 14083 of *LNCS*, pages 507–539. Springer, Cham, August 2023. doi:10.1007/978-3-031-38548-3_17.

[BBHN26] Antoine Bak, Augustin Bariant, Maël Hostettler, and Vincent Neiger. Resultants meet resultant: Improving CICO-1 and CICO-2 attacks on ZK-friendly permutations. Cryptology ePrint Archive, Report 2026/1281, 2026. URL: https://eprint.iacr.org/2026/1281.

[BBHR18a] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast reed-solomon interactive oracle proofs of proximity. In Ioannis Chatzigiannakis, Christos Kaklamanis, Dániel Marx, and Donald Sannella, editors, *ICALP 2018*, volume 107 of *LIPIcs*, pages 14:1–14:17. Schloss Dagstuhl, July 2018. doi:10.4230/LIPIcs.ICALP.2018.14.

[BBHR18b] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. Cryptology ePrint Archive, Report 2018/046, 2018. URL: https://eprint.iacr.org/2018/046.

[BBHR19] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable zero knowledge with no trusted setup. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part III*, volume 11694 of *LNCS*, pages 701–732. Springer, Cham, August 2019. doi:10.1007/978-3-030-26954-8_23.

[BBi18] Jordi Baylina, Marta Bellés, and iden3. circom and snarkjs: Circuit Compiler and zk-SNARK Implementation. https://github.com/iden3/circom, 2018.

[BBL+24a] Augustin Bariant, Aurélien Boeuf, Axel Lemoine, Irati Manterola Ayala, Morten Øygarden, Léo Perrin, and Håvard Raddum. The algebraic FreeLunch: Efficient Gröbner basis attacks against arithmetization-oriented primitives. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part IV*, volume 14923 of *LNCS*, pages 139–173. Springer, Cham, August 2024. doi:10.1007/978-3-031-68385-5_5.

[BBL+24b] Augustin Bariant, Aurélien Boeuf, Axel Lemoine, Irati Manterola Ayala, Morten Øygarden, Léo Perrin, and Håvard Raddum. The algebraic Freelunch: Efficient Gröbner basis attacks against arithmetization-oriented primitives. Cryptology ePrint Archive, Report 2024/347, 2024. URL: https://eprint.iacr.org/2024/347.

[BBLP22] Augustin Bariant, Clémence Bouvier, Gaëtan Leurent, and Léo Perrin. Algebraic attacks against some arithmetization-oriented primitives. *IACR Trans. Symm. Cryptol.*, 2022(3):73–101, 2022. doi:10.46586/tosc.v2022.i3.73-101.

[BBS99] Eli Biham, Alex Biryukov, and Adi Shamir. Cryptanalysis of Skipjack reduced to 31 rounds using impossible differentials. In Jacques Stern, editor, *EUROCRYPT'99*, volume 1592 of *LNCS*, pages 12–23. Springer, Berlin, Heidelberg, May 1999. doi:10.1007/3-540-48910-X_2.

[BBS05] Eli Biham, Alex Biryukov, and Adi Shamir. Cryptanalysis of Skipjack reduced to 31 rounds using impossible differentials. *Journal of Cryptology*, 18(4):291–311, September 2005. doi:10.1007/s00145-005-0129-3.

[BCC+16] Jonathan Bootle, Andrea Cerulli, Pyrros Chaidos, Jens Groth, and Christophe Petit. Efficient zero-knowledge arguments for arithmetic circuits in the discrete log setting. In Marc Fischlin and Jean-Sébastien Coron, editors, *EUROCRYPT 2016, Part II*, volume 9666 of *LNCS*, pages 327–357. Springer, Berlin, Heidelberg, May 2016. doi:10.1007/978-3-662-49896-5_12.

[BCD+20] Tim Beyne, Anne Canteaut, Itai Dinur, Maria Eichlseder, Gregor Leander, Gaëtan Leurent, María Naya-Plasencia, Léo Perrin, Yu Sasaki, Yosuke Todo, and Friedrich Wiemer. Out of oddity - new cryptanalytic techniques against symmetric primitives optimized for integrity proof systems. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part III*, volume 12172 of *LNCS*, pages 299–328. Springer, Cham, August 2020. doi:10.1007/978-3-030-56877-1_11.

[BCF+25] Martijn Brehm, Binyi Chen, Ben Fisch, Nicolas Resch, Ron D. Rothblum, and Hadas Zeilberger. Blaze: Fast SNARKs from interleaved RAA codes. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part IV*, volume 15604 of *LNCS*, pages 123–152. Springer, Cham, May 2025. doi:10.1007/978-3-031-91134-7_5.

[BCG+13] Eli Ben-Sasson, Alessandro Chiesa, Daniel Genkin, Eran Tromer, and Madars Virza. SNARKs for C: Verifying program executions succinctly and in zero knowledge. In Ran Canetti and Juan A. Garay, editors, *CRYPTO 2013, Part II*, volume 8043 of *LNCS*, pages 90–108. Springer, Berlin, Heidelberg, August 2013. doi:10.1007/978-3-642-40084-1_6.

[BCG+14] Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. Zerocash: Decentralized anonymous payments from Bitcoin. In *2014 IEEE Symposium on Security and Privacy*, pages 459–474. IEEE Computer Society Press, May 2014. doi:10.1109/SP.2014.36.

[BCI+22] Nir Bitansky, Alessandro Chiesa, Yuval Ishai, Rafail Ostrovsky, and Omer Paneth. Succinct non-interactive arguments via linear interactive proofs. *Journal of Cryptology*, 35(3):15, July 2022. doi:10.1007/s00145-022-09424-4.

[BCL+20] Tim Beyne, Anne Canteaut, Gregor Leander, María Naya-Plasencia, Léo Perrin, and Friedrich Wiemer. On the security of the Rescue hash function. Cryptology ePrint Archive, Report 2020/820, 2020. URL: https://eprint.iacr.org/2020/820.

[BCP23] Aurélien Boeuf, Anne Canteaut, and Léo Perrin. Propagation of subspaces in primitives with monomial sboxes: Applications to Rescue and variants of the AES. *IACR Trans. Symm. Cryptol.*, 2023(4):270–298, 2023. doi:10.46586/tosc.v2023.i4.270-298.

[BCR+19] Eli Ben-Sasson, Alessandro Chiesa, Michael Riabzev, Nicholas Spooner, Madars Virza, and Nicholas P. Ward. Aurora: Transparent succinct arguments for R1CS. In Yuval Ishai and Vincent Rijmen, editors, *EUROCRYPT 2019, Part I*, volume 11476 of *LNCS*, pages 103–128. Springer, Cham, May 2019. doi:10.1007/978-3-030-17653-2_4.

[BDD+15] Achiya Bar-On, Itai Dinur, Orr Dunkelman, Virginie Lallemand, Nathan Keller, and Boaz Tsaban. Cryptanalysis of SP networks with partial nonlinear layers. In Elisabeth Oswald and Marc Fischlin, editors, *EUROCRYPT 2015, Part I*, volume 9056 of *LNCS*, pages 315–342. Springer, Berlin, Heidelberg, April 2015. doi:10.1007/978-3-662-46800-5_13.

[BDPA07] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Sponge Functions. ECRYPT Hash Workshop 2007, May 2007. URL: http://www.csrc.nist.gov/pki/HashWorkshop/Public_Comments/2007_May.html.

[BDPA14] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Sufficient conditions for sound tree and sequential hashing modes. *Int. J. Inf. Sec.*, 13(4):335–353, 2014. doi:10.1007/S10207-013-0220-Y.

[BDPV08] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. On the indifferentiability of the Sponge construction. In Nigel P. Smart, editor, *EUROCRYPT 2008*, volume 4965 of *LNCS*, pages 181–197. Springer, Berlin, Heidelberg, April 2008. doi:10.1007/978-3-540-78967-3_11.

[BDPV13] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Keccak. In Thomas Johansson and Phong Q. Nguyen, editors, *EUROCRYPT 2013*, volume 7881 of *LNCS*, pages 313–314. Springer, Berlin, Heidelberg, May 2013. doi:10.1007/978-3-642-38348-9_19.

[BDPVA11a] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Cryptographic Sponge Functions, 2011. SHA-3 competition (round 3). URL: https://keccak.team/files/CSF-0.1.pdf.

[BDPVA11b] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. The Keccak Reference, 2011. URL: https://keccak.team/files/Keccak-reference-3.0.pdf.

[Bey23] Time Beyne. *A geometric approach to symmetric-key cryptanalysis*. PhD thesis, Katholieke Universiteit Leuven, 2023. URL: https://lirias.kuleuven.be/4079947.

[BGK+25a] Clémence Bouvier, Lorenzo Grassi, Dmitry Khovratovich, Katharina Koschatko, Christian Rechberger, Fabian Schmid, and Markus Schofnegger. Skyscraper: Fast hashing on big primes. *IACR TCHES*, 2025(2):743–780, 2025. doi:10.46586/tches.v2025.i2.743-780.

[BGK+25b] Clémence Bouvier, Lorenzo Grassi, Dmitry Khovratovich, Katharina Koschatko, Christian Rechberger, Fabian Schmid, and Markus Schofnegger. Skyscraper-v2: Fast hashing on big primes. Cryptology ePrint Archive, Report 2025/058, 2025. URL: https://eprint.iacr.org/2025/058.

[BGL20] Eli Ben-Sasson, Lior Goldberg, and David Levit. STARK friendly hash – survey and recommendation. Cryptology ePrint Archive, Report 2020/948, 2020. URL: https://eprint.iacr.org/2020/948.

[BGM+25] Brieuc Balon, Lorenzo Grassi, Pierrick Méaux, Thorben Moos, François-Xavier Standaert, and Matthias Johann Steiner. mid-pSquare: Leveraging the strong side-channel security of prime-field masking in software. *IACR TCHES*, 2025(4):486–519, 2025. doi:10.46586/tches.v2025.i4.486-519.

[BHK+19] Daniel J. Bernstein, Andreas Hülsing, Stefan Kölbl, Ruben Niederhagen, Joost Rijneveld, and Peter Schwabe. The SPHINCS+ signature framework. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 2129–2146. ACM Press, November 2019. doi:10.1145/3319535.3363229.

[BJGP25] Antoine Bak, Guilhem Jazeron, Pierre Galissant, and Léo Perrin. Attacking split-and-lookup-based primitives using probabilistic polynomial system solving applications to round-reduced Monolith and full-round Skyscraper. *IACR Trans. Symm. Cryptol.*, 2025(3):337–367, 2025. doi:10.46586/tosc.v2025.i3.337-367.

[BMP13] Joan Boyar, Philip Matthews, and René Peralta. Logic minimization techniques with applications to cryptology. *Journal of Cryptology*, 26(2):280–312, April 2013. doi:10.1007/s00145-012-9124-7.

[BMT14] Thierry P. Berger, Marine Minier, and Gaël Thomas. Extended generalized Feistel networks using matrix representation. In Tanja Lange, Kristin Lauter, and Petr Lisonek, editors, *SAC 2013*, volume 8282 of *LNCS*, pages 289–305. Springer, Berlin, Heidelberg, August 2014. doi:10.1007/978-3-662-43414-7_15.

[Bou23] Clémence Bouvier. *Cryptanalysis and design of symmetric primitives defined over large finite fields*. Theses, Sorbonne Université, November 2023. doi:10.70675/3fe7be65z224az47abz9b6bza8de293ae5e7.

[BP25a] Antoine Bak and Léo Perrin. On the security of split-and-lookup-based ZK-friendly primitives. *IACR Trans. Symm. Cryptol.*, 2025(2):87–123, 2025. doi:10.46586/tosc.v2025.i2.87-123.

[BP25b] Aurélien Boeuf and Léo Perrin. ALFOMs and the moirai: Quantifying the performance/security tradeoff for ZK-friendly hash functions. Cryptology ePrint Archive, Report 2025/1920, 2025. URL: https://eprint.iacr.org/2025/1920.

[BR14] Andrey Bogdanov and Vincent Rijmen. Linear hulls with correlation zero and linear cryptanalysis of block ciphers. *DCC*, 70(3):369–383, 2014. doi:10.1007/s10623-012-9697-z.

[BRS02] John Black, Phillip Rogaway, and Thomas Shrimpton. Black-box analysis of the block-cipher-based hash-function constructions from PGV. In Moti Yung, editor, *CRYPTO 2002*, volume 2442 of *LNCS*, pages 320–335. Springer, Berlin, Heidelberg, August 2002. doi:10.1007/3-540-45708-9_21.

[BRW26] Benedikt Bünz, Ron Rothblum, and William Wang. Flock: Fast proving for batch boolean computations. Cryptology ePrint Archive, Paper 2026/1329, 2026. URL: https://eprint.iacr.org/2026/1329.

[BS91] Eli Biham and Adi Shamir. Differential cryptanalysis of DES-like cryptosystems. In Alfred J. Menezes and Scott A. Vanstone, editors, *CRYPTO'90*, volume 537 of *LNCS*, pages 2–21. Springer, Berlin, Heidelberg, August 1991. doi:10.1007/3-540-38424-3_1.

[BSV07] Thomas Baignères, Jacques Stern, and Serge Vaudenay. Linear cryptanalysis of non binary ciphers. In Carlisle M. Adams, Ali Miri, and Michael J. Wiener, editors, *SAC 2007*, volume 4876 of *LNCS*, pages 184–211. Springer, Berlin, Heidelberg, August 2007. doi:10.1007/978-3-540-77360-3_13.

[BTA26] Amit Singh Bhati, Sundas Tariq, and Tomer Ashur. From round skipping to s-box skipping: Attacking Poseidon's partial layer via subspace restriction. Cryptology ePrint Archive, Paper 2026/1692, 2026. URL: https://eprint.iacr.org/2026/1692.

[Buc65] Bruno Buchberger. *Ein Algorithmus zum Auffinden der Basiselemente des Restklassenringes nach einem nulldimensionalen Polynomideal*. PhD thesis, Universität Innsbruck, 1965.

[BV25a] Tim Beyne and Michiel Verbauwhede. Integral cryptanalysis in characteristic p. In Goichiro Hanaoka and Bo-Yin Yang, editors, *ASIACRYPT 2025, Part I*, volume 16245 of *LNCS*, pages 66–96. Springer, Singapore, December 2025. doi:10.1007/978-981-95-5018-0_3.

[BV25b] Tim Beyne and Michiel Verbauwhede. Integral cryptanalysis in characteristic *p*. Cryptology ePrint Archive, Report 2025/932, 2025. URL: https://eprint.iacr.org/2025/932.

[CBBZ23] Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. HyperPlonk: Plonk with linear-time prover and high-degree custom gates. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part II*, volume 14005 of *LNCS*, pages 499–530. Springer, Cham, April 2023. doi:10.1007/978-3-031-30617-4_17.

[CBD+20] Anne Canteaut, Tim Beyne, Itai Dinur, Maria Eichlseder, Gregor Leander, Gaëtan Leurent, María Naya-Plasencia, Léo Perrin, Yu Sasaki, Yosuke Todo, and Friedrich Wiemer. Report on the Security of STARK-friendly Hash Functions (Version 2.0), 2020. URL: https://inria.hal.science/hal-02883253.

[CCZ98] Claude Carlet, Pascale Charpin, and Victor A. Zinoviev. Codes, bent functions and permutations suitable for DES-like cryptosystems. *DCC*, 15(2):125–156, 1998. doi:10.1023/A:1008344232130.

[CGG+22] Carlos Cid, Lorenzo Grassi, Aldo Gunsing, Reinhard Lüftenegger, Christian Rechberger, and Markus Schofnegger. Influence of the linear layer on the algebraic degree in SP-networks. *IACR Trans. Symm. Cryptol.*, 2022(1):110–137, 2022. doi:10.46586/tosc.v2022.i1.110-137.

[CGG+23] Shiyao Chen, Chun Guo, Jian Guo, Li Liu, Meiqin Wang, Puwen Wei, and Zeyu Xu. Towards the links of cryptanalytic methods on MPC/FHE/ZK-friendly symmetric-key primitives. *IACR Trans. Symm. Cryptol.*, 2023(2):132–175, 2023. doi:10.46586/tosc.v2023.i2.132-175.

[CHM+20] Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Psi Vesely, and Nicholas P. Ward. Marlin: Preprocessing zkSNARKs with universal and updatable SRS. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part I*, volume 12105 of *LNCS*, pages 738–768. Springer, Cham, May 2020. doi:10.1007/978-3-030-45721-1_26.

[CL01] Jan Camenisch and Anna Lysyanskaya. An efficient system for non-transferable anonymous credentials with optional anonymity revocation. In Birgit Pfitzmann, editor, *EUROCRYPT 2001*, volume 2045 of *LNCS*, pages 93–118. Springer, Berlin, Heidelberg, May 2001. doi:10.1007/3-540-44987-6_7.

[CLO15] David A. Cox, John Little, and Donal O'Shea. *Ideals, Varieties, and Algorithms: An Introduction to Computational Algebraic Geometry and Commutative Algebra*. Undergraduate Texts in Mathematics. Springer International Publishing, 4 edition, 2015. doi:10.1007/978-3-319-16721-3.

[CMM+23] Gaëtan Cassiers, Loïc Masure, Charles Momin, Thorben Moos, and François-Xavier Standaert. Prime-field masking in hardware and its soundness against low-noise SCA attacks. *IACR TCHES*, 2023(2):482–518, 2023. doi:10.46586/tches.v2023.i2.482-518.

[CR25a] Luca Campa and Arnab Roy. Gröbner basis cryptanalysis of Anemoi. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part I*, volume 15601 of *LNCS*, pages 303–332. Springer, Cham, May 2025. doi:10.1007/978-3-031-91107-1_11.

[CR25b] Luca Campa and Arnab Roy. Groebner basis cryptanalysis of Anemoi. Cryptology ePrint Archive, Report 2025/814, 2025. URL: https://eprint.iacr.org/2025/814.

[CRST26] Luca Campa, Arnab Roy, Matthias Johann Steiner, and Stefano Trevisani. Arion: Arithmetization-Oriented Hashing for Zero-Knowledge Proof Systems, 2026. Personal communication by the authors, To appear in IACR Trans. Symm. Cryptol. 2026(3). URL: https://eprint.iacr.org/2026/1842.

[DKKW25] Justin Drake, Dmitry Khovratovich, Mikhail A. Kudinov, and Benedikt Wagner. Hash-based multi-signatures for post-quantum ethereum. *CiC*, 2(1):13, 2025. doi:10.62056/aey7qjp10.

[DL18] Sébastien Duval and Gaëtan Leurent. MDS matrices with lightweight circuits. *IACR Trans. Symm. Cryptol.*, 2018(2):48–78, 2018. doi:10.13154/tosc.v2018.i2.48-78.

[DMV18] Joan Daemen, Bart Mennink, and Gilles Van Assche. Sound hashing modes of arbitrary functions, permutations, and block ciphers. *IACR Trans. Symm. Cryptol.*, 2018(4):197–228, 2018. doi:10.13154/tosc.v2018.i4.197-228.

[DP25] Benjamin E. Diamond and Jim Posen. Succinct arguments over towers of binary fields. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part IV*, volume 15604 of *LNCS*, pages 93–122. Springer, Cham, May 2025. doi:10.1007/978-3-031-91134-7_4.

[DP26] Benjamin E. Diamond and Jim Posen. Polylogarithmic proofs for multilinears over binary towers. In Joan Daemen and Emmanuel Thomé, editors, *EUROCRYPT 2026, Part VII*, volume 16547 of *LNCS*, pages 3–32. Springer, Cham, May 2026. doi:10.1007/978-3-032-25336-1_1.

[DW25] Ee Duan and Wenling Wu. Invariant Subspace of the P-SPN Structure with a Class of Linear Layer Matrix. *Chin. J. Electron.*, 34(5):1545–1558, 2025. doi:10.23919/cje.2024.00.239.

[DW26] Ee Duan and Wenling Wu. Security analysis of the P-SPN structure with a class of linear layer matrix against invariant subspace attack. *DCC*, 94(3):54, Feb 2026. doi:10.1007/s10623-025-01796-5.

[Eth21] Ethereum Foundation. ZK hash bounties. https://www.zkhashbounties.info/, 2021. Accessed 2026-05-27.

[Eth24] Ethereum Foundation. Poseidon initiative 2024-2026. https://www.poseidon-initiative.info/, 2024. Accessed 2026-05-27.

[Fau99] Jean-Charles Faugère. A new efficient algorithm for computing Gröbner bases (F4). *J. Pure Appl. Algebra*, 139(1):61–88, 1999. doi:10.1016/S0022-4049(99)00005-5.

[FGHR14] Jean-Charles Faugère, Pierrick Gaudry, Louise Huot, and Guénaël Renault. Sub-cubic change of ordering for gröbner basis: a probabilistic approach. In *Proceedings of the 39th International Symposium on Symbolic and Algebraic Computation*, ISSAC '14, page 170–177, New York, NY, USA, 2014. Association for Computing Machinery. doi:10.1145/2608628.2608669.

[FGLM93] Jean-Charles Faugère, Patrizia Gianni, Daniel Lazard, and Teo Mora. Efficient Computation of Zero-dimensional Gröbner Bases by Change of Ordering. *J. Symb. Comput.*, 16(4):329–344, 1993. doi:10.1006/jsco.1993.1051.

[FM17] Jean-Charles Faugère and Chenqi Mou. Sparse FGLM algorithms. *J. Symb. Comput.*, 80:538–569, 2017. doi:10.1016/j.jsc.2016.07.025.

[FMM+24] Sebastian Faust, Loïc Masure, Elena Micheli, Maximilian Orlt, and François-Xavier Standaert. Connecting leakage-resilient secret sharing to practice: Scaling trends and physical dependencies of prime field masking. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part IV*, volume 14654 of *LNCS*, pages 316–344. Springer, Cham, May 2024. doi:10.1007/978-3-031-58737-5_12.

[GGPR13] Rosario Gennaro, Craig Gentry, Bryan Parno, and Mariana Raykova. Quadratic span programs and succinct NIZKs without PCPs. In Thomas Johansson and Phong Q. Nguyen, editors, *EUROCRYPT 2013*, volume 7881 of *LNCS*, pages 626–645. Springer, Berlin, Heidelberg, May 2013. doi:10.1007/978-3-642-38348-9_37.

[GHR+23] Lorenzo Grassi, Yonglin Hao, Christian Rechberger, Markus Schofnegger, Roman Walch, and Qingju Wang. Horst meets fluid-SPN: Griffin for zero-knowledge applications. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part III*, volume 14083 of *LNCS*, pages 573–606. Springer, Cham, August 2023. doi:10.1007/978-3-031-38548-3_19.

[GKL+22] Lorenzo Grassi, Dmitry Khovratovich, Reinhard Lüftenegger, Christian Rechberger, Markus Schofnegger, and Roman Walch. Reinforced Concrete: A fast hash function for verifiable computation. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 1323–1335. ACM Press, November 2022. doi:10.1145/3548606.3560686.

[GKL+24] Lorenzo Grassi, Dmitry Khovratovich, Reinhard Lüftenegger, Christian Rechberger, Markus Schofnegger, and Roman Walch. Monolith: Circuit-friendly hash functions with new nonlinear layers for fast and constant-time implementations. *IACR Trans. Symm. Cryptol.*, 2024(3):44–83, 2024. doi:10.46586/tosc.v2024.i3.44-83.

[GKR+21] Lorenzo Grassi, Dmitry Khovratovich, Christian Rechberger, Arnab Roy, and Markus Schofnegger. Poseidon: A new hash function for zero-knowledge proof systems. In Michael Bailey and Rachel Greenstadt, editors, *USENIX Security 2021*, pages 519–535. USENIX Association, August 2021. URL: https://www.usenix.org/conference/usenixsecurity21/presentation/grassi.

[GKR25] Lorenzo Grassi, Katharina Koschatko, and Christian Rechberger. Poseidon and Neptune: Gröbner basis cryptanalysis exploiting subspace trails. *IACR Trans. Symm. Cryptol.*, 2025(2):34–86, 2025. doi:10.46586/tosc.v2025.i2.34-86.

[GKRS22] Lorenzo Grassi, Dmitry Khovratovich, Sondre Rønjom, and Markus Schofnegger. The Legendre symbol and the modulo-2 operator in symmetric schemes over $\mathbb{F}_p^n$: Preimage attack on full Grendel. *IACR Trans. Symm. Cryptol.*, 2022(1):5–37, 2022. doi:10.46586/tosc.v2022.i1.5-37.

[GKS23a] Lorenzo Grassi, Dmitry Khovratovich, and Markus Schofnegger. Poseidon2: A faster version of the Poseidon hash function. In Nadia El Mrabet, Luca De Feo, and Sylvain Duquesne, editors, *AFRICACRYPT 23*, volume 14064 of *LNCS*, pages 177–203. Springer, Cham, July 2023. doi:10.1007/978-3-031-37679-5_8.

[GKS23b] Lorenzo Grassi, Dmitry Khovratovich, and Markus Schofnegger. Poseidon2: A faster version of the Poseidon hash function. Cryptology ePrint Archive, Report 2023/323, 2023. URL: https://eprint.iacr.org/2023/323.

[GLR+20] Lorenzo Grassi, Reinhard Lüftenegger, Christian Rechberger, Dragos Rotaru, and Markus Schofnegger. On a generalization of substitution-permutation networks: The HADES design strategy. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part II*, volume 12106 of *LNCS*, pages 674–704. Springer, Cham, May 2020. doi:10.1007/978-3-030-45724-2_23.

[GLS+23] Alexander Golovnev, Jonathan Lee, Srinath T. V. Setty, Justin Thaler, and Riad S. Wahby. Brakedown: Linear-time and field-agnostic SNARKs for R1CS. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part II*, volume 14082 of *LNCS*, pages 193–226. Springer, Cham, August 2023. doi:10.1007/978-3-031-38545-2_7.

[GM20] Aldo Gunsing and Bart Mennink. Collapseability of Tree Hashes. In Jintai Ding and Jean-Pierre Tillich, editors, *Post-Quantum Cryptography - 11th International Conference, PQCrypto 2020*, volume 12100 of *LNCS*, pages 524–538. Springer, 2020. doi:10.1007/978-3-030-44223-1_28.

[GMBM+26] Lorenzo Grassi, Mario Marhuenda-Beltrán, Thorben Moos, Fabian Schmid, Matthias Johann Steiner, and Hailun Yan. pSquare-hash: A Family of Tweakable Hash Functions for Physically Secure PQ Signatures. Cryptology ePrint Archive, Report 2026/1129, 2026. URL: https://eprint.iacr.org/2026/1129.

[GMM+24] Lorenzo Grassi, Loïc Masure, Pierrick Méaux, Thorben Moos, and François-Xavier Standaert. Generalized Feistel ciphers for efficient prime field masking. In Marc Joye and Gregor Leander, editors, *EUROCRYPT 2024, Part III*, volume 14653 of *LNCS*, pages 188–220. Springer, Cham, May 2024. doi:10.1007/978-3-031-58734-4_7.

[GMR85] Shafi Goldwasser, Silvio Micali, and Charles Rackoff. The knowledge complexity of interactive proof-systems (extended abstract). In *17th ACM STOC*, pages 291–304. ACM Press, May 1985. doi:10.1145/22145.22178.

[GO94] Oded Goldreich and Yair Oren. Definitions and properties of zero-knowledge proof systems. *Journal of Cryptology*, 7(1):1–32, December 1994. doi:10.1007/BF00195207.

[GOPS22] Lorenzo Grassi, Silvia Onofri, Marco Pedicini, and Luca Sozzi. Invertible quadratic non-linear layers for MPC-/FHE-/ZK-friendly schemes over $\mathbb{F}_p^n$: Application to Poseidon. *IACR Trans. Symm. Cryptol.*, 2022(3):20–72, 2022. doi:10.46586/tosc.v2022.i3.20-72.

[GPR21] Lior Goldberg, Shahar Papini, and Michael Riabzev. Cairo – a Turing-complete STARK-friendly CPU architecture. Cryptology ePrint Archive, Report 2021/1063, 2021. URL: https://eprint.iacr.org/2021/1063.

[Gro16] Jens Groth. On the size of pairing-based non-interactive arguments. In Marc Fischlin and Jean-Sébastien Coron, editors, *EUROCRYPT 2016, Part II*, volume 9666 of *LNCS*, pages 305–326. Springer, Berlin, Heidelberg, May 2016. doi:10.1007/978-3-662-49896-5_11.

[GRS21] Lorenzo Grassi, Christian Rechberger, and Markus Schofnegger. Proving resistance against infinitely long subspace trails: How too choose the linear layer. *IACR Trans. Symm. Cryptol.*, 2021(2):314–352, 2021. doi:10.46586/tosc.v2021.i2.314-352.

[GWC19] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. PLONK: Permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive, Report 2019/953, 2019. URL: https://eprint.iacr.org/2019/953.

[Hab22] Ulrich Haböck. Multivariate lookups based on logarithmic derivatives. Cryptology ePrint Archive, Report 2022/1530, 2022. URL: https://eprint.iacr.org/2022/1530.

[HACF25] Hossein Hafezi, Gaspard Anthoine, Matteo Campanelli, and Dario Fiore. SoK: Lookup table arguments. Cryptology ePrint Archive, Report 2025/1876, 2025. URL: https://eprint.iacr.org/2025/1876.

[HHL+25] Jincheol Ha, Seongha Hwang, Jooyoung Lee, Seungmin Park, and Mincheol Son. Polocolo: A ZK-friendly hash function based on S-boxes using power residues. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part IV*, volume 15604 of *LNCS*, pages 303–332. Springer, Cham, May 2025. doi:10.1007/978-3-031-91134-7_11.

[Hir18] Shoichi Hirose. Sequential hashing with minimum padding. *Cryptogr.*, 2(2):11, 2018. doi:10.3390/CRYPTOGRAPHY2020011.

[HK22] Andreas Hülsing and Mikhail A. Kudinov. Recovering the tight security proof of SPHINCS+. In Shweta Agrawal and Dongdai Lin, editors, *ASIACRYPT 2022, Part IV*, volume 13794 of *LNCS*, pages 3–33. Springer, Cham, December 2022. doi:10.1007/978-3-031-22972-5_1.

[Hül13] Andreas Hülsing. W-OTS+ - shorter signatures for hash-based signature schemes. In Amr Youssef, Abderrahmane Nitaj, and Aboul Ella Hassanien, editors, *AFRICACRYPT 13*, volume 7918 of *LNCS*, pages 173–188. Springer, Berlin, Heidelberg, June 2013. doi:10.1007/978-3-642-38553-7_10.

[Jo26] Sunghyeon Jo. Midpoint reset: A full-round Poseidon collision from an adaptively chosen MDS matrix. Cryptology ePrint Archive, Paper 2026/1760, 2026. URL: https://eprint.iacr.org/2026/1760.

[KBM23] Dmitry Khovratovich, Mario Marhuenda Beltrán, and Bart Mennink. Generic security of the SAFE API and its applications. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part VIII*, volume 14445 of *LNCS*, pages 301–327. Springer, Singapore, December 2023. doi:10.1007/978-981-99-8742-9_10.

[KCLM22] Irakliy Khaburzaniya, Konstantinos Chalkias, Kevin Lewi, and Harjasleen Malvai. Aggregating and thresholdizing hash-based signatures using STARKs. In Yuji Suga, Kouichi Sakurai, Xuhua Ding, and Kazue Sako, editors, *ASIACCS 22*, pages 393–407. ACM Press, May / June 2022. doi:10.1145/3488932.3524128.

[KKA+25] Marcin Kostrzewa, Matthew Klein, Ara Adkins, Grzegorz Świrski, and Wojciech Żmuda. Keccacheck: towards a SNARK friendly keccak. Cryptology ePrint Archive, Report 2025/1764, 2025. URL: https://eprint.iacr.org/2025/1764.

[KLR24] Katharina Koschatko, Reinhard Lüftenegger, and Christian Rechberger. Exploring the six worlds of Gröbner basis cryptanalysis: Application to Anemoi. *IACR Trans. Symm. Cryptol.*, 2024(4):138–190, 2024. doi:10.46586/tosc.v2024.i4.138-190.

[Knu95] Lars R. Knudsen. Truncated and higher order differentials. In Bart Preneel, editor, *FSE'94*, volume 1008 of *LNCS*, pages 196–211. Springer, Berlin, Heidelberg, December 1995. doi:10.1007/3-540-60590-8_16.

[KR16] Martin Kreuzer and Lorenzo Robbiano. *Computational Linear and Commutative Algebra*. Springer International Publishing, Cham, 1 edition, 2016. doi:10.1007/978-3-319-43601-2.

[KR21] Nathan Keller and Asaf Rosemarin. Mind the middle layer: The HADES design strategy revisited. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part II*, volume 12697 of *LNCS*, pages 35–63. Springer, Cham, October 2021. doi:10.1007/978-3-030-77886-6_2.

[KS98] Erich Kaltofen and Victor Shoup. Subquadratic-Time Factoring of Polynomials over Finite Fields. *Math. Comput.*, 67(223):1179–1197, 1998. doi:10.1090/S0025-5718-98-00944-2.

[KZG10] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In Masayuki Abe, editor, *ASIACRYPT 2010*, volume 6477 of *LNCS*, pages 177–194. Springer, Berlin, Heidelberg, December 2010. doi:10.1007/978-3-642-17373-8_11.

[LBM25] Charlotte Lefevre, Mario Marhuenda Beltrán, and Bart Mennink. To pad or not to pad? Padding-free arithmetization-oriented sponges. *IACR Trans. Symm. Cryptol.*, 2025(1):97–137, 2025. doi:10.46586/tosc.v2025.i1.97-137.

[LCC+26] Muzhou Li, Jiamin Cui, Longzheng Cui, Kai Hu, Chao Niu, and Meiqin Wang. Towards Better Integral Distinguishers over Fp Based on Exact Coefficients of Monomials. *IEEE Trans. Inf. Theory*, pages 1–1, 2026. doi:10.1109/TIT.2026.3661587.

[LHW+25] Junkai Liang, Daqi Hu, Pengfei Wu, Yunbo Yang, Qingni Shen, and Zhonghai Wu. SoK: Understanding zk-SNARKs: The gap between research and practice. In Lujo Bauer and Giancarlo Pellegrino, editors, *USENIX Security 2025*, pages 2085–2104. USENIX Association, August 2025. URL: https://www.usenix.org/conference/usenixsecurity25/presentation/liang-sok.

[LKG+24] Fukang Liu, Katharina Koschatko, Lorenzo Grassi, Hailun Yan, Shiyao Chen, Subhadeep Banik, and Willi Meier. Opening the blackbox: Collision attacks on round-reduced Tip5, Tip4, Tip4' and Monolith. *IACR Trans. Symm. Cryptol.*, 2024(4):97–137, 2024. doi:10.46586/tosc.v2024.i4.97-137.

[LLW26] Enyan Li, Fukang Liu, and Gaoli Wang. Beyond linear subspace trails: Nonlinear subspaces for Gröbner basis attacks on Poseidon/Poseidon2 and Neptune. Cryptology ePrint Archive, Paper 2026/1792, 2026. URL: https://eprint.iacr.org/2026/1792.

[LM22] Charlotte Lefevre and Bart Mennink. Tight preimage resistance of the Sponge construction. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part IV*, volume 13510 of *LNCS*, pages 185–204. Springer, Cham, August 2022. doi:10.1007/978-3-031-15985-5_7.

[LTYW18] Yongqiang Li, Shizhu Tian, Yuyin Yu, and Mingsheng Wang. On the generalization of butterfly structure. *IACR Trans. Symm. Cryptol.*, 2018(1):160–179, 2018. doi:10.13154/tosc.v2018.i1.160-179.

[LW26] Enyan Li and Gaoli Wang. Revisiting linear subspace trails in Poseidon. Cryptology ePrint Archive, Report 2026/967, 2026. URL: https://eprint.iacr.org/2026/967.

[LWL23] Zhengnan Li, Baofeng Wu, and Dongdai Lin. Algebraic-Differential Attacks on a Family of Arithmetization-Oriented Symmetric Ciphers. *J. Syst. Sci. Complex.*, 36(6):2681–2702, Dec 2023. doi:10.1007/s11424-023-1511-7.

[MAPB23] Irati Manterola Ayala, Atharva Phanse, and Clémence Bouvier. STAP Zoo. https://stap-zoo.com/, 2023. Maintained by the STAP-Zoo Team and Zoo-keepers.

[Mat94] Mitsuru Matsui. Linear cryptanalysis method for DES cipher. In Tor Helleseth, editor, *EUROCRYPT'93*, volume 765 of *LNCS*, pages 386–397. Springer, Berlin, Heidelberg, May 1994. doi:10.1007/3-540-48285-7_33.

[MBKM19] Mary Maller, Sean Bowe, Markulf Kohlweiss, and Sarah Meiklejohn. Sonic: Zero-knowledge SNARKs from linear-size universal and updatable structured reference strings. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 2111–2128. ACM Press, November 2019. doi:10.1145/3319535.3339817.

[MG26] Simon-Philipp Merz and Àlex Rodríguez García. Skipping Class: Algebraic Attacks exploiting weak matrices and operation modes of Poseidon2(b). IACR Cryptology ePrint Archive, Paper 2026/306, 2026. URL: https://eprint.iacr.org/2026/306.

[MMMS23] Loïc Masure, Pierrick Méaux, Thorben Moos, and François-Xavier Standaert. Effective and efficient masking with low noise using small-mersenne-prime ciphers. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part IV*, volume 14007 of *LNCS*, pages 596–627. Springer, Cham, April 2023. doi:10.1007/978-3-031-30634-1_20.

[Moe73] R. T. Moenck. Fast computation of GCDs. In *Proceedings of the Fifth Annual ACM Symposium on Theory of Computing*, STOC '73, page 142–151, New York, NY, USA, 1973. Association for Computing Machinery. doi:10.1145/800125.804045.

[MSS24] Thorben Moos, Sayandeep Saha, and François-Xavier Standaert. Prime masking vs. faults - exponential security amplification against selected classes of attacks. *IACR TCHES*, 2024(4):690–736, 2024. doi:10.46586/tches.v2024.i4.690-736.

[NIS15a] NIST. Secure Hash Standard (SHS). National Institute of Standards and Technology, NIST FIPS PUB 180-4, U.S. Department of Commerce, August 2015. doi:10.6028/NIST.FIPS.180-4.

[NIS15b] NIST. SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions. National Institute of Standards and Technology, NIST FIPS PUB 202, U.S. Department of Commerce, August 2015. doi:10.6028/NIST.FIPS.202.

[NIS25] NIST. Stateless Hash-Based Digital Signature Standard. National Institute of Standards and Technology, NIST FIPS PUB 205, U.S. Department of Commerce, August 2025. doi:10.6028/NIST.FIPS.205.

[NO14] Yusuke Naito and Kazuo Ohta. Improved indifferentiable security analysis of PHOTON. In Michel Abdalla and Roberto De Prisco, editors, *SCN 14*, volume 8642 of *LNCS*, pages 340–357. Springer, Cham, September 2014. doi:10.1007/978-3-319-10879-7_20.

[NZW+23] Jianqiang Ni, Jianhui Zhang, Gaoli Wang, Rui Li, and Yanzhao Shen. Algebraic attacks against Grendel: An arithmetization-oriented primitive with the Legendre symbol. *Symmetry*, 15(8):1563, 2023. doi:10.3390/SYM15081563.

[OSC26] OSCAR – Open Source Computer Algebra Research system, Version 1.7.2, 2026. URL: https://www.oscar-system.org.

[Per24] Léo Perrin. Security analysis of XHASH8/12. Cryptology ePrint Archive, Report 2024/605, 2024. URL: https://eprint.iacr.org/2024/605.

[PGV94] Bart Preneel, René Govaerts, and Joos Vandewalle. Hash functions based on block ciphers: A synthetic approach. In Douglas R. Stinson, editor, *CRYPTO'93*, volume 773 of *LNCS*, pages 368–378. Springer, Berlin, Heidelberg, August 1994. doi:10.1007/3-540-48329-2_31.

[PHGR13] Bryan Parno, Jon Howell, Craig Gentry, and Mariana Raykova. Pinocchio: Nearly practical verifiable computation. In *2013 IEEE Symposium on Security and Privacy*, pages 238–252. IEEE Computer Society Press, May 2013. doi:10.1109/SP.2013.47.

[Pol22] Polygon Zero Team. Plonky2: A SNARK Implementation Based on PLONK and FRI. https://github.com/0xPolygonZero/plonky2, 2022.

[Pol24] Polygon Zero Team. Plonky3: A Toolkit for Polynomial IOPs (PIOPs). https://github.com/Plonky3/Plonky3, 2024.

[QCC+25] Kexin Qiao, Mengyu Chang, Junjie Cheng, Changhai Ou, An Wang, and Liehuang Zhu. Programming Equation Systems of Arithmetization-Oriented Primitives with Constraints. In *Information Security and Cryptology – Inscrypt 2025, Part II*, LNCS, pages 85–106. Springer, 2025. doi:10.1007/978-981-95-6203-9_5.

[QSH24] Wenxiao Qiao, Siwei Sun, and Lei Hu. New Algebraic Attacks on Grendel with the Strategy of Bypassing SPN Steps. *Chinese Journal of Electronics*, 33(3):635–644, 2024. doi:10.23919/cje.2023.00.127.

[RAS20] Arnab Roy, Elena Andreeva, and Jan Ferdinand Sauer. Interpolation cryptanalysis of unbalanced Feistel networks with low degree round functions. In Orr Dunkelman, Michael J. Jacobson, Jr., and Colin O'Flynn, editors, *SAC 2020*, volume 12804 of *LNCS*, pages 273–300. Springer, Cham, October 2020. doi:10.1007/978-3-030-81652-0_11.

[Rec23] Christian Rechberger. On the History of FHE, MPC, and ZK-Friendly Symmetric Cryptography. STAP (Symmetric Techniques for Advanced Protocols), 2023. URL: https://who.paris.inria.fr/Leo.Perrin/rescale/slides/Christian-STAP-2023.pdf.

[Rij24] Vincent Rijmen. Cryptanalytic audit of the XHash Sponge function and its components. Cryptology ePrint Archive, Report 2024/656, 2024. URL: https://eprint.iacr.org/2024/656.

[RS24] Arnab Roy and Matthias Johann Steiner. Generalized triangular dynamical system: An algebraic system for constructing cryptographic permutations over finite fields. In Maria Eichlseder and Sébastien Gambs, editors, *SAC 2024, Part II*, volume 15517 of *LNCS*, pages 139–165. Springer, Cham, August 2024. doi:10.1007/978-3-031-82841-6_6.

[RST23] Arnab Roy, Matthias Johann Steiner, and Stefano Trevisani. Arion: Arithmetization-Oriented Permutation and Hashing from Generalized Triangular Dynamical Systems, 2023. URL: https://arxiv.org/abs/2303.04639, arXiv:2303.04639.

[SAD20] Alan Szepieniec, Tomer Ashur, and Siemen Dhooghe. Rescue-Prime: a standard specification (SoK). Cryptology ePrint Archive, Report 2020/1143, 2020. URL: https://eprint.iacr.org/2020/1143.

[Sag26] The Sage Developers. SageMath, the Sage Mathematics Software System (Version 10.9), 2026. URL: https://www.sagemath.org.

[Sal23] Robin Salen. Two additional instantiations from the Tip5 hash function construction, 2023. URL: https://toposware.com/paper_tip5.pdf.

[Set20] Srinath Setty. Spartan: Efficient and general-purpose zkSNARKs without trusted setup. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part III*, volume 12172 of *LNCS*, pages 704–737. Springer, Cham, August 2020. doi:10.1007/978-3-030-56877-1_25.

[Set26] Srinath Setty. BinarySpartan: Spartan over binary fields. Cryptology ePrint Archive, Paper 2026/1656, 2026. URL: https://eprint.iacr.org/2026/1656.

[SLST23] Alan Szepieniec, Alexander Lemmens, Jan Ferdinand Sauer, and Bobbin Threadbare. The Tip5 hash function for recursive STARKs. Cryptology ePrint Archive, Report 2023/107, 2023. URL: https://eprint.iacr.org/2023/107.

[Sta19] StarkWare. STARK-Friendly Hash Challenge. https://web.archive.org/web/20230927184931/https://starkware.co/hash-challenge/, 2019. Archived version. Accessed 2026-05-27.

[Ste25a] Matthias Johann Steiner. Gröbner basis cryptanalysis of Ciminion and Hydra. *IACR Trans. Symm. Cryptol.*, 2025(1):240–275, 2025. doi:10.46586/tosc.v2025.i1.240-275.

[Ste25b] Matthias Johann Steiner. A note on the Walsh spectrum of power residue S-boxes. *CiC*, 2(3):13, 2025. doi:10.62056/a69qdk5vt.

[Ste25c] Matthias Johann Steiner. A note on the Walsh spectrum of the Flystel. *DCC*, 93(6):2245–2262, 2025. doi:10.1007/s10623-025-01589-w.

[Ste26] Matthias Johann Steiner. *Algebraic Aspects of Symmetric Cryptography*. PhD thesis, Alpen-Adria-Universität Klagenfurt, 2026. URL: https://resolver.obvsg.at/urn:nbn:at:at-ubk:1-61485.

[SV25] Antonio Sanso and Giuseppe Vitto. Attacking Poseidon via Graeffe-based root-finding over NTT-friendly fields. Cryptology ePrint Archive, Report 2025/937, 2025. URL: https://eprint.iacr.org/2025/937.

[SV26] Antonio Sanso and Giuseppe Vitto. Top Gun: Degree annihilation attacks on Poseidon. Cryptology ePrint Archive, Report 2026/1254, 2026. URL: https://eprint.iacr.org/2026/1254.

[Sze21] Alan Szepieniec. On the use of the Legendre symbol in symmetric cipher design. Cryptology ePrint Archive, Report 2021/984, 2021. URL: https://eprint.iacr.org/2021/984.

[TABR26] Stefano Trevisani, Elena Andreeva, Rishiraj Bhattacharyya, and Arnab Roy. Boosting efficiency and security in arithmetization-oriented hashing for zero-knowledge proof systems. Cryptology ePrint Archive, Report 2026/1271, 2026. URL: https://eprint.iacr.org/2026/1271.

[Thr] Bobbin Threadbare. Polygon Miden. https://polygon.technology/solutions/polygon-miden.

[UG26] Matilda Urani and Lorenzo Grassi. Corrigendum to "Invertible Quadratic Non-Linear Layers for MPC-/FHE-/ZK-Friendly Schemes over $\mathbb{F}_p^n$: Application to Poseidon". *IACR Trans. Symm. Cryptol.*, 2026(1):527–532, Mar. 2026. doi:10.46586/tosc.v2026.i1.527-532.

[vdHL21] Joris van der Hoeven and Grégoire Lecerf. Fast computation of generic bivariate resultants. *J. Complex.*, 62:101499, 2021. doi:10.1016/j.jco.2020.101499.

[Vil25] Gilles Villard. Bivariate polynomial reduction and elimination ideal over finite fields. *J. Symb. Comput.*, 127:102367, 2025. doi:10.1016/j.jsc.2024.102367.

[Vit26] Giuseppe Vitto. Slipway: Accessing finite subspace trails in Poseidon. Cryptology ePrint Archive, Report 2026/1579, 2026. URL: https://eprint.iacr.org/2026/1579.

[vzGG13] Joachim von zur Gathen and Jürgen Gerhard. *Modern Computer Algebra*. Cambridge University Press, 3 edition, 2013. doi:10.1017/CBO9781139856065.

[WCC+23] Howard Wu, Collin Chin, Raymond Chu, Pratyush Mishra, and Alessandro Chiesa. Varuna: An Optimized SNARK for R1CS. https://github.com/ProvableHQ/varuna-sage-impl, 2023. Aleo / Provable Inc.

[Wie86] D. Wiedemann. Solving sparse linear equations over finite fields. *IEEE Trans. Inf. Theory*, 32(1):54–62, 1986. doi:10.1109/TIT.1986.1057137.

[WOS+25] Anna P. Y. Woo, Alex Ozdemir, Chad Sharp, Thomas Pornin, and Paul Grubbs. Efficient proofs of possession for legacy signatures. In Marina Blanton, William Enck, and Cristina Nita-Rotaru, editors, *2025 IEEE Symposium on Security and Privacy*, pages 3291–3308. IEEE Computer Society Press, May 2025. doi:10.1109/SP61157.2025.00080.

[WW24] Bolin Wang and Wenling Wu. Security analysis of P-SPN schemes against invariant subspace attack with inactive S-boxes. *DCC*, 92(11):3753–3782, 2024. doi:10.1007/s10623-024-01465-z.

[WXXZ24] Virginia Vassilevska Williams, Yinzhan Xu, Zixuan Xu, and Renfei Zhou. New bounds for matrix multiplication: from alpha to omega. In David P. Woodruff, editor, *35th SODA*, pages 3792–3835. ACM-SIAM, January 2024. doi:10.1137/1.9781611977912.134.

[XZZ+19] Tiancheng Xie, Jiaheng Zhang, Yupeng Zhang, Charalampos Papamanthou, and Dawn Song. Libra: Succinct zero-knowledge proofs with optimal prover computation. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part III*, volume 11694 of *LNCS*, pages 733–764. Springer, Cham, August 2019. doi:10.1007/978-3-030-26954-8_24.

[YZY+24a] Hong-Sen Yang, Qun-Xiong Zheng, Jing Yang, Quan feng Liu, and Deng Tang. A new security evaluation method based on resultant for arithmetic-oriented algorithms. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part VII*, volume 15490 of *LNCS*, pages 457–489. Springer, Singapore, December 2024. doi:10.1007/978-981-96-0941-3_15.

[YZY+24b] Hong-Sen Yang, Qun-Xiong Zheng, Jing Yang, Quan feng Liu, and Deng Tang. A new security evaluation method based on resultant for arithmetic-oriented algorithms. Cryptology ePrint Archive, Report 2024/886, 2024. URL: https://eprint.iacr.org/2024/886.

[ZBK+22] Arantxa Zapico, Vitalik Buterin, Dmitry Khovratovich, Mary Maller, Anca Nitulescu, and Mark Simkin. Caulk: Lookup arguments in sublinear time. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *ACM CCS 2022*, pages 3121–3134. ACM Press, November 2022. doi:10.1145/3548606.3560646.

[ZCF24] Hadas Zeilberger, Binyi Chen, and Ben Fisch. BaseFold: Efficient field-agnostic polynomial commitment schemes from foldable codes. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part X*, volume 14929 of *LNCS*, pages 138–169. Springer, Cham, August 2024. doi:10.1007/978-3-031-68403-6_5.

[ZD25] Ziyu Zhao and Jintai Ding. Breaking Poseidon challenges with Graeffe transforms and complexity analysis by FFT lower bounds. Cryptology ePrint Archive, Report 2025/950, 2025. URL: https://eprint.iacr.org/2025/950.

[ZSVD25] Ziyu Zhao, Antonio Sanso, Giuseppe Vitto, and Jintai Ding. Graeffe-based attacks on Poseidon and NTT lower bounds. Cryptology ePrint Archive, Report 2025/1916, 2025. URL: https://eprint.iacr.org/2025/1916.

[ZSVD26] Ziyu Zhao, Antonio Sanso, Giuseppe Vitto, and Jintai Ding. Graeffe-based attacks on Poseidon and NTT lower bounds. In Nadia Heninger and Mike Rosulek, editors, *Advances in Cryptology - CRYPTO 2026, Part VI*, volume 16805 of *LNCS*, pages 471–498. Springer, 2026. doi:10.1007/978-3-032-35412-9_16.

Supplementary Material

Contents

1 Introduction **1**
1.1 Importance of ZK-Friendly Hash Functions . . . 2
1.2 Our Contribution . . . 3
1.2.1 Classification of Existing ZK-friendly Hash Functions . . . 4
1.2.2 Extensive Open-Source Implementation Suite . . . 4
1.2.3 Restoring Security . . . 5
1.2.4 GMiMCHash2: An Updated Variant of GMiMCHash . . . 6
1.2.5 Efficiency Comparison across Different ZK-Proof Systems . . . 6
1.3 Organization . . . 7

2 Overview of Zero-Knowledge Protocols **9**
2.1 Background . . . 9
2.2 Arithmetization in ZK-SNARKs . . . 10
2.2.1 Rank-1 Constraint System (R1CS) . . . 10
2.2.2 PLONK Arithmetization . . . 11
2.2.3 Algebraic Intermediate Representation (AIR) . . . 12
2.2.4 Lookup Arguments . . . 12
2.3 Practical Parameter Instantiations . . . 13
2.4 Traditional Cryptographic Hash Functions Are Not ZK-friendly . . . 13

3 Modes of Operation **14**
3.1 Sponge Hash Function . . . 14
3.1.1 "Original" Sponge Hash Function . . . 15
3.1.2 Sponge Variants . . . 15
3.2 Tree Hash and Compression Functions . . . 16
3.2.1 Tree-Hash Function . . . 16
3.2.2 Compression Functions . . . 18

4 Overview of Algebraic Cryptanalytic Techniques **18**
4.1 The Constrained-Input Constrained-Output Problem . . . 20
4.2 Polynomial System Solving Techniques . . . 21
4.2.1 Gröbner Bases . . . 21
4.2.2 Resultant . . . 23
4.2.3 Univariate Root-Finding . . . 25
4.3 Algebraic Models of Arithmetization-Oriented Primitives . . . 25
4.3.1 Skipping Rounds in Substitution-Permutation Networks . . . 26
4.3.2 Invariant Subspaces in Partial Substitution Layers . . . 28
4.3.3 FreeLunch Approach . . . 29
4.3.4 Change of Coordinates . . . 30
4.4 Cryptanalysis Challenges . . . 30

5 ZK-Friendly Hash Functions: Type-I (Low Degree) **31**
5.1 MiMC Designs . . . 31
5.1.1 GMiMC (ESORICS'19) . . . 32
5.1.2 GMiMCHash2 . . . 35
5.2 Hades-Like Designs . . . 36
5.2.1 Poseidon (USENIX'21), Poseidon2 (Africacrypt'23) . . . 37
5.2.2 Neptune (ToSC'22) . . . 40
5.3 Generalized Feistel Designs . . . 43
5.3.1 Moirai . . . 43
5.3.2 pSquare-hash . . . 44

6 ZK-Friendly Hash Functions: Type-II (CCZ-Equivalence) 45
6.1 *Marvellous* Designs . . . 46
6.1.1 *Rescue* (ToSC'20) . . . 47
6.1.2 XHash (DCC'26) . . . 50
6.2 `Anemoi` (CRYPTO'23) . . . 53
6.3 Griffin (CRYPTO'23) . . . 58
6.4 Arion (ToSC'26) . . . 61

7 ZK-Friendly Hash Functions: Type-III (Look-up Tables) 63
7.1 Reinforced Concrete (CCS'22) . . . 64
7.2 Tip5 Variants . . . 65
7.3 `Monolith` (ToSC'24) . . . 66
7.4 Skyscraper-v2 (TCHES'25) . . . 67

8 ZK-Friendly Hash Functions: Type-IV (Power Residues) 68
8.1 *Grendel* . . . 69
8.2 Polocolo (EUROCRYPT'25) . . . 70

9 Theoretical Proof System Performance 72
9.1 General Arithmetization Strategies . . . 72
9.1.1 Power Maps and Inverse Power Maps . . . 72
9.1.2 Composite Non-linear Layers . . . 73
9.1.3 The Split-and-Lookup Paradigm . . . 73
9.1.4 Power-Residue S-boxes . . . 74
9.2 R1CS . . . 75
9.2.1 R1CS Costs for the XHash Cubic Extension-Field Power Map . . . 76
9.2.2 Split-and-Lookup S-boxes . . . 76
9.2.3 Power-Residue S-boxes . . . 78
9.2.4 Type-I Primitives . . . 78
9.2.5 Type-II Primitives . . . 79
9.2.6 Type-III Primitives . . . 79
9.2.7 Type-IV Primitives . . . 79
9.3 AIR . . . 80
9.3.1 Power Maps and Inverse Power Maps . . . 80
9.3.2 Split-and-Lookup S-boxes . . . 81
9.3.3 Power-Residue S-boxes . . . 82
9.3.4 Type-I Primitives . . . 82
9.3.5 Type-II Primitives . . . 82
9.3.6 Type-III Primitives . . . 83
9.3.7 Type-IV Primitives . . . 83
9.4 PLONK . . . 84
9.4.1 Constraining Affine Transformations . . . 84
9.4.2 Constraining Non-linear Layers . . . 86
9.4.3 Type-I Primitives . . . 86
9.4.4 Type-II Primitives . . . 88
9.4.5 Type-III Primitives. . . . 89
9.4.6 Type-IV Primitives . . . 89

10 Performances 90
10.1 Plain Implementations . . . 90
10.2 Proof System Performance . . . 93
10.2.1 Plonky3 Benchmarks . . . 93
10.2.2 Gnark Benchmarks . . . 94

11 Conclusion and Outcomes **96**

References **98**

A Overview Modes of Operation **120**

B Overview Cryptanalysis Coverage **121**

C GMiMC Appendix **123**
C.1 Updated Python Script to Compute $\text{GMiMC}^{\pi}_{\text{erf}}$ . . . 123
C.2 Generalized Truncated Differential for $\text{GMiMC}^{\pi}_{\text{erf}}$ . . . 123
C.3 Efficient $\text{GMiMC}^{\pi}_{\text{erf}}$ Circuit . . . 124

D GMiMCHash2 Appendix **124**
D.1 Security Analysis of $\text{GMiMC}^{\pi}_{\text{erf}}2$ . . . 124
D.1.1 (Truncated) Differential Cryptanalysis . . . 124
D.1.2 Other Statistical Attacks . . . 126
D.1.3 Interpolation Attack . . . 127
D.1.4 Gröbner Basis Attacks . . . 128
D.2 Efficient $\text{GMiMC}^{\pi}_{\text{erf}}2$ Circuit . . . 133

E Poseidon Appendix **134**
E.1 Equivalent Constant Representation for Internal Poseidon Rounds . . . 134
E.2 Reducing the PLONK Constraints of Internal Poseidon Rounds . . . 134

A Overview Modes of Operation

Table 25: Modes of operations for several ZK-friendly primitives.

Primitive Name	Sponge Mode	Compression Mode
Type-I Primitives		
GMiMCHash [AGP+19a]	✓	
GMiMCHash2 (Section 5.1.2)	✓	✓†
Neptune [GOPS22]	✓	
Poseidon [GKR+21]	✓	
Poseidon2 [GKS23a]	✓	✓
pSquare-hash [GMBM+26]	✓	✓
Type-II Primitives		
`Anemoi` [BBC+23]	✓	✓*
Arion [CRST26]	✓	✓
Griffin [GHR+23]	✓	✓
Rescue [AAB+20] / *Rescue-Prime* [SAD20]	✓	
RPO [AKM+22]	✓	
XHash [ABK+26]	✓	
Type-III Primitives		
`Monolith` [GKL+24]	✓	✓
Reinforced Concrete [GKL+22]	✓	
Skyscraper [BGK+25a, BGK+25b]	✓	✓
Tip5 [SLST23]	✓	
Tip4/Tip4' [Sal23]	✓	✓*
Type-IV Primitives		
Grendel [Sze21]	✓	
Polocolo [HHL+25]	✓	

* Jive compression mode.
† Only for $\log_2(q) \approx 32, 64$.

B Overview Cryptanalysis Coverage

Table 26: Cryptanalysis coverage of Type-I primitives, grouped by attack technique. (No third-party cryptanalysis has been proposed for pSquare-hash yet.) A checkmark indicates that the paper analyzes the corresponding primitive.

Attack	GMiMCHash	Poseidon	Poseidon2	Neptune
Statistical Cryptanalysis				
Links in Statistical Analysis [CGG+23]	✓			
Mind the Middle Layer [KR21]		✓		
Algebraic-Differential Attacks [LWL23]	✓			
Invariant Subspaces				
GB exploiting Subspace Trails [GKR25]		✓	✓	✓
Invariant Subspaces Attacks [WW24, DW25, DW26, LW26, MG26]		✓		
Non-Linear Subspace Attack [LLW26]		✓	✓	✓
Degree prediction				
Interpolation Cryptanalysis [RAS20]	✓			
(Ultra-Metric) Integral Cryptanalysis in $\mathbb{F}_p$ [BV25a]	✓	✓	✓	✓
Integral Cryptanalysis in $\mathbb{F}_p$ [LCC+26]	✓			
Algebraic Cryptanalysis				
GB and Algebraic Analysis [Ste26]	✓			
GB with Initial Full Round Skipping [BBLP22]		✓		
GB at Round Level [ABM24]		✓	✓	
Algebraic Cheap-Lunch [BBB+25a]		✓	✓	✓
Graeffe Transform [ZSVD26]		✓		
EF Challenges for Poseidon [vdHL21, Vil25, BBB+26, BTA26, SV26]		✓		
Structural & Algebraic Cryptanalysis				
Out of Oddity [BCD+20]	✓	✓		

Table 27: Cryptanalysis coverage of Type-II primitives, grouped by attack technique. A checkmark indicates that the paper analyzes the corresponding primitive.

Attack	*Rescue*	*Rescue-Prime*	RPO	XHash	Anemoi	Griffin	Arion
General GB Cryptanalysis							
STARK-Friendly Hash Survey [BGL20]	✓						
Algebraic Attacks [BBLP22]		✓					
Six Worlds of GB Cryptanalysis [KLR24]					✓		
GB Cryptanalysis of Anemoi [CR25a]					✓		
Programming Equation Systems [QCC+25]					✓	✓	
Analysis of XHash8/12 [Per24, Rij24]				✓			
FreeLunch Approaches							
Algebraic FreeLunch [BBL+24a]				✓†	✓	✓	✓
Algebraic CheapLunch [BBB+25a]				✓†		✓	✓
Resultant Approaches							
Improved Resultant Attack [BBB+25b]		✓			✓	✓	✓
Resultants Meet Resultant [BBHN26]		✓			✓	✓	
Resultant-Based Evaluation [YZY+24a]		✓			✓		
Structural & General Analysis							
Subspace Propagation [BCP23]	✓						
Security of Rescue [BCL+20]	✓						
Linear Cryptanalysis							
Walsh Spectrum Study [BB24, Ste25c, BB26]					✓		

† Only XHash8 due to partial S-Box layer.

Table 28: Cryptanalysis coverage of Type-III primitives, grouped by attack technique. A checkmark indicates that the paper analyzes the corresponding primitive.

Attack Paper	RC	Tip5	Tip4/Tip4'	Monolith	Skyscraper
Algebraic and Statistical Cryptanalysis					
Differential Properties S&L and Polynomial System Solving [LKG+24]		✓	✓		
Linear Properties of S&L and Polynomial System Solving [BP25a]	✓	✓	✓	✓	
Rebound Attack [Bak25]					✓
Probabilistic Polynomial System Solving [BJGP25]				✓	✓

Table 29: Cryptanalysis coverage of Type-IV primitives, grouped by attack technique. A checkmark indicates that the paper analyzes the corresponding primitive.

Attack	*Grendel*	Polocolo
Algebraic Cryptanalysis		
Guessing Power-Residue [GKRS22]	✓	
Guessing Power-Residue and Bypassing Rounds [NZW+23, QSH24, BP25a]	✓	
Linear Cryptanalysis		
Walsh Spectrum Study [Ste25b]		✓

C GMiMC Appendix

C.1 Updated Python Script to Compute GMiMC$^{\pi}_{\text{erf}}$

```
from math import *
import sys

if len(sys.argv) != 4:
    print("Usage: <script> <n> <t> <d>")
    exit()

n = int(sys.argv[1])
t = int(sys.argv[2])
d = int(sys.argv[3])

def Lambda(t):
    return (2 * ceil(log(t, 2)))
# ERF
erf_guess_gcd = ceil(2 * log(2, d) * n - 4 * log(n, d)) + 3 * t - 3
erf_inter = ceil(2 * log(2, d) * n) + 3 * t - 2
erf_grobner = ceil(log(2, d) * n + 2 * log(t, d)) + 4 * t - 5
erf_higher_order = 1 + 3 * t + ceil(2 * log(t, d))
erf_trunc_diff = 1 + t + ceil((t**2 + t) * (float(n) / float(2 * (n - 1))))
erf_imp_diff = 3 * t - 1
erf_gen_attack = 4 * t - 2

erf_round_number = max(erf_guess_gcd, erf_inter,
                       erf_grobner,   erf_higher_order,
                       erf_trunc_diff, erf_imp_diff,
                       erf_gen_attack)

print("n:", n)
print("t:", t)
print("---- ERF ----")
print("Guess/GCD:", erf_guess_gcd)
print("Interpolation:", erf_inter)
print("Grobner:", erf_grobner)
print("Higher-Order Diff.:", erf_higher_order)
print("Trunc. Diff.:", erf_trunc_diff)
print("Imp. Diff.:", erf_imp_diff)
print("Generic:", erf_gen_attack)
print("-> Total number of rounds:", erf_round_number)
```

C.2 Generalized Truncated Differential for GMiMC$^{\pi}_{\text{erf}}$

We present the generalization of Beyne et al.'s truncated differential for the GMiMC$^{\pi}_{\text{erf}}$ round function, see Section 5.1.1. Let $k \leq t$, and let $\Delta_1, \Delta'_1, \ldots, \Delta_k, \Delta'_k \in \mathbb{F}_p$ be input differences with transitions $\Delta_1 \xrightarrow{x^\alpha} \Delta'_1, \Delta_2 + \Delta'_1 \xrightarrow{x^\alpha} \Delta'_2, \ldots, \Delta_k + \Delta'_1 + \ldots \Delta'_{k-1} \xrightarrow{x^\alpha} \Delta'_k$. In addition, let $\tilde{\Delta}_i = \Delta_i + \sum_{\substack{j=1 \\ j \neq i}}^{k} \Delta'_j$. Then we have the truncated differential

$$(0, \ldots, 0, \Delta_1, \ldots, \Delta_k)^\intercal \xrightarrow[\text{prob. } 1]{\mathsf{R}^{(t-k)}} (\Delta_1, \ldots, \Delta_k, 0, \ldots, 0)^\intercal \tag{81}$$

$$\xrightarrow{\mathsf{R}} (\Delta_2 + \Delta'_1, \ldots, \Delta_k + \Delta'_1, \Delta'_1, \ldots, \Delta'_1, \Delta_1)^\intercal \tag{82}$$

$$\xrightarrow{\mathsf{R}^{(k-1)}} \left(\sum_{i=1}^{k} \Delta'_i, \ldots, \sum_{i=1}^{k} \Delta'_i, \tilde{\Delta}_1, \ldots, \tilde{\Delta}_k \right)^\intercal . \tag{83}$$

On average we expect that $\sum_{i=1}^{k} \Delta'_i = 0$ with probability p^{-1}, in that case we have the following cyclic truncated differential trail

$$(0, \ldots, 0, \Delta_1, \ldots, \Delta_k)^\intercal \xrightarrow[\text{prob. } 1/p]{\mathsf{R}^{(t)}} \left(0, \ldots, 0, \tilde{\Delta}_1, \ldots, \tilde{\Delta}_k\right)^\intercal. \tag{84}$$

C.3 Efficient GMiMC$^{\pi}_{\text{erf}}$ Circuit

Algorithm 1 Efficient GMiMC$^{\pi}_{\text{erf}}$ circuit for $t > 4$.

Input: $\mathbf{x} \in \mathbb{F}_p^t$.
Output: GMiMC$^{\pi}_{\text{erf}}(\mathbf{x})$.

Initialize $x_{\text{acc}} = 0$, $\mathbf{x}_i = \mathbf{x}$ and $\mathbf{a} = \mathbf{0} \in \mathbb{F}_p^{t-1}$.
for $r \leftarrow 1$ to R **do**
  $x_\alpha = \left(x_{i,1} + c^{(r)}\right)^\alpha$.
  Rotate $\mathbf{a}$ one position to the left.
  $x_{\text{acc}} = x_{\text{acc}} - a_1$.
  $a_1 = x_\alpha$.
  $x_{\text{acc}} = x_{\text{acc}} + x_\alpha$.
  Rotate $\mathbf{x}_i$ one position to the left.
  $x_{i,1} = x_{i,1} + x_{\text{acc}}$.
for $j \leftarrow 2$ to $t - 1$ **do**
  Rotate $\mathbf{a}$ one position to the left.
  $x_{\text{acc}} = x_{\text{acc}} - a_1$.
  $x_{i,j} = x_{i,j} + x_{\text{acc}}$.
return $\mathbf{x}_i$.

D GMiMCHash2 Appendix

D.1 Security Analysis of GMiMC$^{\pi}_{\text{erf}}$2

In principle the cryptanalysis of GMiMCHash2 resembles the one of GMiMCHash. In Appendix D.1.1 we perform truncated differential cryptanalysis. Round numbers are estimated with the trail in Appendix C.2. In Appendix D.1.3 we analyze GMiMC$^{\pi}_{\text{erf}}$2 with respect to the interpolation attack. We use the tools of [BV25a] to estimate the degree over 32 bit fields during the saturation phase. In Appendix D.1.4 we perform Gröbner basis cryptanalysis of GMiMCHash2. For 32 and 64 bit fields we can compute DRL Gröbner bases via a change of coordinates. For 256 bit fields we develop a novel round skipping technique for expanding Feistel permutations.

D.1.1 (Truncated) Differential Cryptanalysis

Differential cryptanalysis introduced by Biham and Shamir [BS91] studies the statistical propagation of input differences to output differences through an interated permutation. Truncated differential cryptanalysis introduced by Knudsen [Knu95] generalizes this concept by allowing differentials where only parts of the differences are specified.

We observe first that the generalized truncated differential from Equation (84) for GMiMC$^{\pi}_{\text{erf}}$ also applies to GMiMC$^{\pi}_{\text{erf}}$2 round function. Moreover, if α does not induce a power permutation of $\mathbb{F}_p$, then there exist valid differentials $\Delta \xrightarrow[\text{prob. } (\alpha-1)/p]{x^\alpha} 0$. In this case, there also exist cyclic differential trails of length t for GMiMC$^{\pi}_{\text{erf}}$2 of the form

$$(0, \ldots, 0, \Delta)^\intercal \xrightarrow[\text{prob. } (\alpha-1)/p]{\mathsf{R}^{(t)}} (0, \ldots, 0, \Delta)^\intercal. \tag{85}$$

Thus, we conjecture that any valid (truncated) differential trail for $\text{GMiMC}^{\pi}_{\text{erf}}2$ over t rounds has at most probability p^{-1}.

We also note that there exists a probability 1 truncated differential over t rounds for the round function

$$(0,\ldots,0,\Delta)^{\intercal} \xrightarrow[\text{prob. } 1]{\mathsf{R}^{(t)}} (*,\ldots,*,\Delta)^{\intercal}, \tag{86}$$

where $*$ denotes a truncated state. Though, such a truncated differential cannot be used for a collision search or a distinguisher unless we impose conditions on the truncated parts.

Due to the clustering effect (i.e., the fact that several differential characteristics can be used together for setting up the attack) and due to the possibility to exploit a Meet-in-the-Middle approach for setting up the attack, we claim that the scheme is secure if every differential characteristic has probability smaller than $2^{-2.5\cdot\kappa}$ for a factor 2.5. Thus, we require that the number of rounds R must satisfy

$$p^{-\frac{R-t}{t}} \leq 2^{-2.5\cdot\kappa} \qquad \Rightarrow \qquad R \geq t\cdot\left(\left\lceil \frac{2.5\cdot\kappa}{\log_2(p)}\right\rceil + 1\right). \tag{87}$$

We note that we deliberately ignored the Input/Output matrix to simplify the analysis.

In Table 30 we summarize the minimum round numbers required for $\text{GMiMC}^{\pi}_{\text{erf}}2$ to resist differential attacks.

Table 30: Round numbers for $\text{GMiMC}^{\pi}_{\text{erf}}2$ to resist differential attacks.

$\log_2(p)\backslash t$	8	12	16	20	24	32	40
32	–	–	176	–	264	352	440
64	48	72	96	120	–	–	–

Let us also do a sanity check for the round numbers in Table 30 with respect to generalized structures, see Equation (49). Suppose we have a differential structure with l variables, i.e. it has size p^l, which cycles over t rounds with probability p^{-1}. Then the structure admits $\approx p^{2\cdot l}$ unique pairs, and we can search at most $\frac{2^{\kappa}}{p^l}$ many structures. Then, the expected number of collisions found by searching the structures is lower bounded by $\mathsf{EC} \geq 2^{\kappa}\cdot p^{l-\lfloor\frac{R}{t}\rfloor}$.[36] Note that this term is maximized for $l=\frac{c}{2}$, i.e. we search over a single structure with maximal data complexity $p^{\frac{c}{2}}\approx 2^{\kappa}$. Then, for $\kappa=128$ we have that $\mathsf{EC} \geq \begin{Bmatrix} 2^{-96}, & \log_2(p)=32, \\ 2^{-128}, & \log_2(p)=64 \end{Bmatrix}$. On the other hand, with a full data search we expect to find a collision with probability $\frac{1}{2}$ due to the birthday bound. Therefore, we conjecture that $\text{GMiMC}^{\pi}_{\text{erf}}2$ resists (truncated) differential attacks within security level 128 bits even when structures have been taken into account.

Algebraically Controlled Differential Attacks. Analogous to $\text{GMiMC}^{\pi}_{\text{erf}}$ we can apply Beyne et al.'s [BCD+20] algebraically controlled differential attack and its generalization by Li et al. [LWL23] to $\text{GMiMC}^{\pi}_{\text{erf}}2$'s round function, also see the cryptanalysis discussion in Section 5.1.1. For this purpose we ignore the Input/Output matrix.

With Beyne et al.'s approach we can find an input difference for up to $4\cdot t-2$ rounds with negligible bivariate polynomial system solving complexity.

[36] If x^{α} is not a permutation we can slightly improve the lower bound, since differences in our structure might satisfy either the truncated differential from Equation (84) or the differential trails like Equation (86) which activate 1 S-box over t rounds. (There are k such input differences.) Then, we have $p^l\cdot(p^l-1)$ pairs for the trail of Equation (84) and $l\cdot p^{l-1}\cdot p\cdot(p-1)$ pairs for the trail of Equation (86). So, the total number of pairs is $p^{2\cdot l}+l\cdot p^{l+1}-(l-1)\cdot p^l$. However, this effect is negligible since in our regime of interest the total number of pairs is $\leq 2\cdot p^{2\cdot l}$.

For the generalization of Li et al. [LWL23] the polynomial system solving complexity with k variables is bounded by

$$\mathcal{O}\left(\left(\binom{\frac{\alpha^{2\cdot k-2}-\alpha^{k-1}}{2}+1}{k-1}\right)^{\omega}+\left(\binom{\frac{\alpha^{2\cdot k}-\alpha^{k}}{2}+1}{k}\right)^{\omega}\right) \tag{88}$$

field operations, and up to $k\cdot(t-1)+t-c$ many rounds can be covered. In Table 31a we evaluated this complexity for $\omega=2$, and in Tables 31b and 31c we evaluated the maximal round numbers that can be attacked.

Table 31: Maximal attackable round numbers for $\text{GMiMC}^{\pi}_{\text{erf}}2$ with algebraically controlled differential attacks.

(a) Complexity of polynomial system solving (Equation (88)) with $\omega=2$.

α	2	3	4	5	8
k / Complexity (bits)	6 / 113	4 / 85	4 / 111	3 / 73	3 / 97

(b) Round numbers for $\log_2(p)\approx 64$ and $c=4$.

$t\backslash\alpha$	2	3	4	5	8
8	46	32	32	25	25
12	74	52	52	41	41
16	102	72	72	57	57
20	130	92	92	73	73

(c) Round numbers for $\log_2(p)\approx 32$ and $c=8$.

$t\backslash\alpha$	2	3	4	5	8
16	98	68	68	53	53
24	154	108	108	85	85
32	210	148	148	117	117
40	266	188	188	149	149

Algebraic-Differential Attacks. Analogous to $\text{GMiMC}^{\pi}_{\text{erf}}$ we can perform a hybrid algebraically controlled differential attack where we split $R=R_1+R_2$, we find input pairs with appropriate input/output differences for the first R_1 rounds via polynomial system solving, and propagete these pairs another R_2 rounds probabilistically, also see the cryptanalysis discussion in Section 5.1.1. For this purpose we ignore the Input/Output matrix.

Li et al. [LWL23] proposed for GMiMCHash and k variables to use $R_1=t-k-c+t\cdot(k-2)=t\cdot(k-1)-k-c$, to find p^2 pairs that satisfy Equation (84). The pairs are then propagated another $l\cdot t$ rounds with probability p^{-k}. So, with $R_2=2\cdot t+t-d-k$ we expect to find at least one collision for up to $R=t\cdot(k+2)-c-d-2\cdot k$ rounds. Li et al. estimated the system solving complexity with

$$\mathcal{O}\left(2\cdot p\cdot\left(\binom{\frac{\alpha^{2\cdot k-2}-\alpha^{k-1}}{2}+1}{k-1}\right)^{\omega}\right) \tag{89}$$

field operations. In Table 32a we evaluated this complexity for $\omega=2$, and in Tables 32b and 32c we evaluated the maximal round numbers that can be attacked.

D.1.2 Other Statistical Attacks

We conjecture that security against differential attacks also implies security against other statistical attack vectors like linear cryptanalysis [Mat94, BSV07], impossible differentials [BBS99, BBS05] or zero-correlation [BR14].

Table 32: Maximal attackable round numbers for $\text{GMiMC}^{\pi}_{\text{erf}}2$ with algebraic-differential attacks.

(a) Complexity of polynomial system solving (Equation (89) with $\omega = 2$.

	Complexity (bits)				
α	2	3	4	5	8
$k \,/\, \log_2(p) \approx 64$	5 / 111	4 / 110	4 / 125	3 / 95	3 / 106
$k \,/\, \log_2(p) \approx 32$	6 / 108	5 / 117	4 / 92	4 / 105	3 / 74

(b) Round numbers for $\log_2(p) \approx 64$ and $c, d = 4$.

$t \backslash \alpha$	2	3	4	5	8
8	38	32	32	26	26
12	66	56	56	46	46
16	94	80	80	66	66
20	122	104	104	86	86

(c) Round numbers for $\log_2(p) \approx 32$ and $c, d = 8$.

$t \backslash \alpha$	2	3	4	5	8
16	100	86	72	72	58
24	164	142	120	120	98
32	228	198	168	168	138
40	292	254	216	216	168

D.1.3 Interpolation Attack

When interpolating a polynomial function over $\mathbb{F}_p^t$ we consider it as an element of $R = \mathbb{F}_p[x_1, \ldots, x_t]/(x_1^p - x_1, \ldots, x_t^p - x_t)$, i.e., the degree in x_i can be at most $p-1$. Heuristically, the degree growth of an iterated function is split into two phases:

(1) The exponential phase: No reduction with respect to any $x_i^p - x_i$ has occurred yet.

(2) The saturation phase: At least one reduction with respect to some $x_i^p - x_i$ has occurred.

During the exponential phase, the degree growth of iterated polynomials in R behaves identical to iteration over $\mathbb{F}_p[x_1, \ldots, x_t]$. Though, this behavior is lost once a reduction occurs. We also recall that the number of monomials in $\mathbb{F}_p[x_1, \ldots, x_t]$ up to degree d is given by $\binom{t+d}{d}$.

For the interpolation attack on $\text{GMiMC}^{\pi}_{\text{erf}}2$ we do a case distinction depending on the field size.

$\log_2(p) \approx 256$. In this case we can always reduce GMiMCHash2 to a univariate function. We expect the function to be of degree $\approx \alpha^R$ and to be dense after R rounds. So, roughly $\log_\alpha(q) \geq \kappa$, where κ denotes the security level, should be efficient. The polynomial degree can be optimized via efficient guessing. Since this is closely related to the univariate Gröbner basis attack on GMiMCHash2, we postpone the discussion to Appendix D.1.4.

$\log_2(p) \approx 64$. Over a 64 bit prime field and 128 bit security level we have for the capacity/rate that at least $c, r \geq 4$ and $t \geq 8$. For the interpolation attack we have two choices: Either we interpolate the full function with r variables or we split the permutation in the middle and interpolate two functions with t variables each. For 64-bit fields and small α we also expect that security against the interpolation attack can already be achieved during the exponential phase. To visualize this intuition we list in Table 33 the quantity $\lceil 64 \cdot \log_\alpha(2) \rceil$ together with the minimal round numbers such that $\binom{r+\alpha^R}{\alpha^R} \geq 2^{128}$ and $\binom{t+\alpha^{\frac{R}{2}}}{\alpha^{\frac{R}{2}}} \geq 2^{128}$ for small α.

Table 33: Naive estimation of the minimal round numbers of $\text{GMiMC}^{\pi}_{\text{erf}}2$ over 64-bit fields and security level 128 bits.

		R			
α	$\lceil 64 \cdot \log_\alpha(2) \rceil$	$\binom{4+\alpha^R}{\alpha^R} \geq 2^{128}$	$\binom{8+\alpha^{\frac{R}{2}}}{\alpha^{\frac{R}{2}}} \geq 2^{128}$	$\binom{8+\alpha^R}{\alpha^R} \geq 2^{128}$	$\binom{12+\alpha^{\frac{R}{2}}}{\alpha^{\frac{R}{2}}} \geq 2^{128}$
2	64	34	36	18	28
3	41	21	24	12	18
4	32	17	18	9	14
5	28	15	16	8	12
8	16	12	12	6	10

However, it is important to keep in mind that the degree growth in $\text{GMiMC}^{\pi}_{\text{erf}}2$ is not uniform, see Figure 7. An input is only fully mixed after t rounds, and an adversary could exploit this fact together with linear relations between the polynomials to lower the degree. Therefore, we add $2 \cdot t$ rounds as margin, and we require over 64-bit fields that

$$\binom{r+\alpha^{R-2\cdot t}}{\alpha^{R-2\cdot t}} \geq 2^{128}, \qquad \text{and} \qquad \binom{t+\alpha^{\frac{R}{2}-2\cdot t}}{\alpha^{\frac{R}{2}-2\cdot t}} \geq 2^{128}. \tag{90}$$

$\log_2(p) \approx 32$. Over 32 bit fields and security level 128 bits we have capacity/rate $c, r \geq 8$ and $t \geq 16$. However, as one can easily verify security against the interpolation attack is not achieved during the exponential growth phase. We can estimate the degree growth during the saturation phase with tools recently developed by Beyne & Verbauwhede [BV25a]. In Figure 24 we plot the degree growth of the $\text{GMiMC}^{\pi}_{\text{erf}}2$ permutation over the Mersenne31 prime $p = 2^{31} - 1$. In Table 24c we list the minimum round numbers $\tilde{R}$ that reach approximately the maximum degree $t \cdot (p-1)$. Over 31-bit fields we then require for then require that the round numbers satisfy

$$R \geq 2 \cdot (\tilde{R} + t), \tag{91}$$

where we double $\tilde{R}$ due to the Meet-in-the-Middle approach and we add $2 \cdot t$ rounds to prevent linear combinations of the components. Moreover, we conjecture that the $\text{GMiMC}^{\pi}_{\text{erf}}2$ permutation is dense once the maximum degree is (almost) reached.

In Table 34 we summarize the minimum round numbers required for $\text{GMiMC}^{\pi}_{\text{erf}}2$ to resist interpolation attacks.

Table 34: Round numbers for $\text{GMiMC}^{\pi}_{\text{erf}}2$ to resist interpolation attacks.

(a) $\log_2(p) \approx 64$.

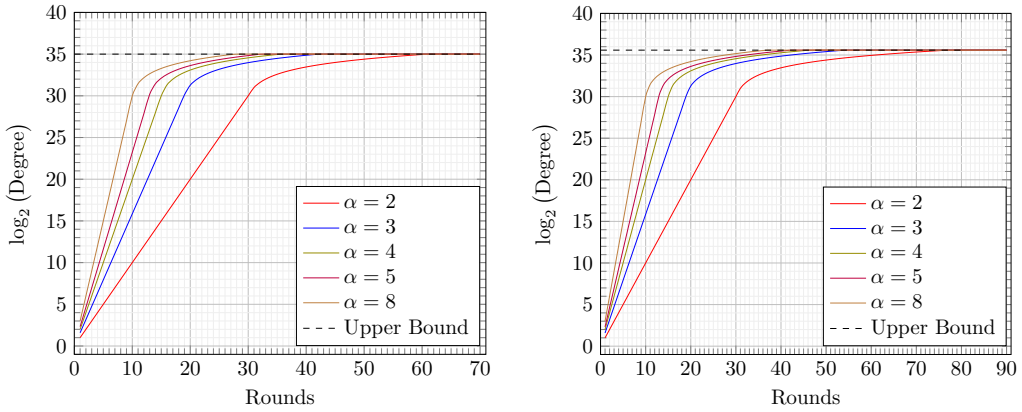
$t \backslash \alpha$	2	3	4	5	8
8	52	40	34	32	28
12	52	42	38	36	34

(b) $\log_2(p) \approx 32$.

$t \backslash \alpha$	2	3	4	5	8
16	164	122	108	100	90
24	210	168	144	136	124

D.1.4 Gröbner Basis Attacks

For Gröbner basis cryptanalysis of GMiMCHash2 we distinguish between capacity $c = 1$ and $c > 1$. In the former case, preimage and CICO problems can be reduced to a univariate or a bivariate MitM attack. The other case requires a Gröbner basis computation to solve the system.



(a) $t = 16$. **(b)** $t = 24$.

(c) Minimum round number for the $\log_2$-degree to exceed 34.999 ($t = 16$) and 35.584 ($t = 24$).

	R	
$\alpha \backslash t$	16	24
2	66	81
3	45	60
4	38	48
5	34	44
8	29	38

Figure 24: $\mathrm{GMiMC}^{\pi}_{\mathrm{erf}}$ and $\mathrm{GMiMC}^{\pi}_{\mathrm{erf}}2$ degree growth over $q = 2^{31} - 1$.

$\log_2(p) \approx 256$. For 256 bit fields we have capacity $c = 1$ for security level 128 bits. We analyze the forward modeling, which reduces to a univariate polynomial, as well as a bivariate MitM model.

Univariate Attack. We can reduce the sponge preimage and CICO-1 problems

$$\pi\begin{pmatrix}\mathbf{x}_{\mathrm{in}}\\ \mathtt{IV}\end{pmatrix} = \begin{pmatrix}h\\ \mathbf{x_{out}}\end{pmatrix}, \qquad \pi\begin{pmatrix}\mathbf{x}_{\mathrm{in}}\\ \mathtt{IV}_{\mathrm{in}}\end{pmatrix} = \begin{pmatrix}\mathbf{x}_{\mathrm{out}}\\ \mathtt{IV}_{\mathrm{out}}\end{pmatrix}, \tag{92}$$

where $\mathbf{x}_{\mathrm{in}}$ and $\mathbf{x}_{\mathrm{out}}$ are variable vectors of $t-1$ variables each and $\mathtt{IV}, \mathtt{IV}_{\mathrm{in}}, \mathtt{IV}_{\mathrm{out}}, h \in \mathbb{F}_p$, into univariate equations by guessing $t-2$ variables on the input or output. We also note that the last round in the CICO-1 problem can be linearized. In particular, by guessing the first $t-2$ entries of $\mathbf{x}_{\mathrm{in}}$ we linearize $t-2$ rounds. Thus, for the preimage problem we yield a univariate polynomial of degree $\alpha^{R-(t-2)}$ and for the CICO-1 problem we yield degree $\alpha^{R-(t-1)}$. For this naive variable guessing strategy and security level 2^{κ} we would require that at least $R \geq t + \kappa \cdot \log_{\alpha}(2)$.

However, with a refined round skip it is possible to linearize more rounds. We first discuss odd α. For simplicity we set $\mathtt{IV} = 0$. Let us expand the first round

$$\begin{pmatrix}x_{\mathrm{in},1}\\ \vdots\\ x_{\mathrm{in},t-1}\\ 0\end{pmatrix} \xrightarrow{\mathsf{R}} \begin{pmatrix}x_{\mathrm{in},2} + (x_{\mathrm{in},1} + c_1)^{\alpha}\\ \vdots\\ x_{\mathrm{in},t-1} + (x_{\mathrm{in},1} + c_1)^{\alpha}\\ (x_{\mathrm{in},1} + c_1)^{\alpha}\\ x_{\mathrm{in},1} + c_1\end{pmatrix} = \begin{pmatrix}\tilde{x}_{\mathrm{in},2}\\ \vdots\\ \tilde{x}_{\mathrm{in},t-1}\\ (x_{\mathrm{in},1} + c_1)^{\alpha}\\ x_{\mathrm{in},1} + c_1\end{pmatrix}, \tag{93}$$

where we performed the substitutions $\tilde{x}_{\mathrm{in},1} = x_{\mathrm{in},1}$ and $\tilde{x}_{\mathrm{in},2} = x_{\mathrm{in},2} + (x_{\mathrm{in},1} + c_1)^{\alpha}$ for

$2 \leq i \leq t-1$. We can perform analogous substitution for the next $t-3$ rounds, at this stage we arrive at

$$\begin{pmatrix} x_{\text{in},1} \\ \vdots \\ x_{\text{in},t-1} \\ 0 \end{pmatrix} \xrightarrow{\mathsf{R}^{(t-2)}} \begin{pmatrix} \tilde{x}_{\text{in},t-1} \\ \sum_{i=1}^{t-2} (\tilde{x}_{\text{in},i} + c_i)^\alpha \\ \tilde{x}_{\text{in},1} + c_1 + \sum_{i=2}^{t-2} (\tilde{x}_{\text{in},i} + c_i)^\alpha \\ \vdots \\ \tilde{x}_{\text{in},t-3} + c_{t-3} + (\tilde{x}_{\text{in},t-4} + c_{t-})^\alpha \\ \tilde{x}_{\text{in},t-2} + c_{t-2} \end{pmatrix} \tag{94}$$

$$\xrightarrow{\mathsf{R}} \begin{pmatrix} \sum_{i=1}^{t-1} (\tilde{x}_{\text{in},i} + c_i)^\alpha \\ \tilde{x}_{\text{in},1} + c_1 + \sum_{i=2}^{t-1} (\tilde{x}_{\text{in},i} + c_i)^\alpha \\ \vdots \\ \tilde{x}_{\text{in},t-3} + c_{t-3} + (\tilde{x}_{\text{in},t-2} + c_{t-2})^\alpha + (\tilde{x}_{\text{in},t-1} + c_{t-1})^\alpha \\ \tilde{x}_{\text{in},t-2} + c_{t-2} + (\tilde{x}_{\text{in},t-1} + c_{t-1})^\alpha \\ \tilde{x}_{\text{in},t-1} + c_{t-1} \end{pmatrix}. \tag{95}$$

Now we perform the following guesses

$$\tilde{x}_{\text{in},i} + c_i + \alpha_i = 0, \qquad 1 \leq i \leq t-3, \tag{96}$$

$$(\tilde{x}_{\text{in},t-2} + c_{t-2}) + (\tilde{x}_{\text{in},t-1} + c_{t-1}) = 0, \tag{97}$$

where $\alpha_i \in \mathbb{F}_q$. Then

$$\begin{pmatrix} x_{\text{in},1} \\ \vdots \\ x_{\text{in},t-1} \\ 0 \end{pmatrix} \xrightarrow{\mathsf{R}^{(t-1)}} \begin{pmatrix} \beta_1 \\ \beta_2 \\ \vdots \\ \beta_{t-2} \\ -\tilde{x}_{\text{in},t-1} + \beta_{t-1} + (\tilde{x}_{\text{in},t-1} + c_{t-1})^\alpha \\ \tilde{x}_{\text{in},t-1} + c_{t-1} \end{pmatrix} \tag{98}$$

$$\xrightarrow{\mathsf{R}^{(t-2)}} \begin{pmatrix} -\tilde{x}_{\text{in},t-1} + \gamma_1 + (\tilde{x}_{\text{in},t-1} + c_{t-1})^\alpha \\ \tilde{x}_{\text{in},t-1} + \gamma_2 \\ \gamma_3 \\ \vdots \\ \gamma_t \end{pmatrix}, \tag{99}$$

where $\beta_1, \ldots, \beta_{t-1}, \gamma_1, \ldots, \gamma_t \in \mathbb{F}_q$. Thus, after $2 \cdot t - 3$ rounds the components are polynomials of degrees $(\alpha, 1, 0, \ldots, 0)$ in $\tilde{x}_{\text{in},t-1}$. So, for the preimage problem we yield a univariate polynomial of degree $\alpha^{R-2\cdot t+4}$ and for the CICO-1 problem we yield a polynomial of degree $\alpha^{R-2\cdot t+3}$. We note that this guessing strategy can be applied to GMiMCHash (with minor modifications for the constants).

For even α we have to modify the guess in Equation (97) to

$$(\tilde{x}_{\text{in},t-2} + c_{t-2})^\alpha + (\tilde{x}_{\text{in},t-1} + c_{t-1})^\alpha = 0. \tag{100}$$

Then

$$\begin{pmatrix} x_{\text{in},1} \\ \vdots \\ x_{\text{in},t-1} \end{pmatrix} \xrightarrow{\mathsf{R}^{(2\cdot t-3)}} \begin{pmatrix} -\tilde{x}_{\text{in},t-2} + \gamma_1 + (\tilde{x}_{\text{in},t-1} + c_{t-1})^\alpha \\ \tilde{x}_{\text{in},t-1} + \gamma_2 \\ \gamma_3 \\ \vdots \\ \gamma_t \end{pmatrix}. \tag{101}$$

So, we yield two bivariate polynomials $f, g \in \mathbb{F}_q[\tilde{x}_{\text{in},t-2}, \tilde{x}_{\text{in},t-1}]$ with degrees

$$\begin{aligned} &\deg_{\tilde{x}_{\text{in},t-2}}(f) = \alpha, && \deg_{\tilde{x}_{\text{in},t-2}}(g) &&= \alpha, \\ &\deg_{\tilde{x}_{\text{in},t-1}}(f) = \alpha, && \deg_{\tilde{x}_{\text{in},t-1}}(g) &&= \alpha^{R-2\cdot t+3} \cdot \begin{cases} \alpha, & \text{pre}, \\ 1, & \text{CICO-1}. \end{cases} \end{aligned} \tag{102}$$

Their resultant polynomial has degree $\leq \left\{ \begin{matrix} \alpha^{R-2\cdot t+5}, & \text{pre}, \\ \alpha^{R-2\cdot t+4}, & \text{CICO-1} \end{matrix} \right\}$. In summary, for $\log_2(p) \approx 256$ and security level 2^κ we require that at least

$$R \geq 1.25 \cdot \left(2 \cdot t + \kappa \cdot \log_\alpha(2)\right), \tag{103}$$

where we add a factor 1.25 as security margin.

Bivariate Attack. Let $R_1, R_2 \in \mathbb{Z}_{\geq 1}$ be such that $R = R_1 + R_2$, we split the $\text{GMiMC}^{\pi}_{\text{erf}}2$ permutation into two parts $\pi = \pi_2 \circ \pi_1$, where π_1 represents the first R_1 rounds and π_2 the final R_2 rounds. Let the variables $\mathbf{x} = (x_1, \ldots, x_t)^\intercal$ denote the middle state after R_1, then we can rewrite the CICO-1 problem as follows

$$\begin{pmatrix} \mathbf{x}_{\text{in}} \\ \texttt{IV}_{\text{in}} \end{pmatrix} = \pi_1^{-1}(\mathbf{x}), \qquad \pi_2(\mathbf{x}) = \begin{pmatrix} \mathbf{x}_{\text{out}} \\ \texttt{IV}_{\text{out}} \end{pmatrix} \tag{104}$$

(An analogous system can be set up for the preimage problem.) Since we have just one constraint on the input and output respectively, we can guess $t-2$ variables in $\mathbf{x}$ and reduce to two bivariate polynomials. We denote these polynomials with $f_{\text{bw}}(x, y) = \texttt{IV}_{\text{in}}$ and $f_{\text{fw}}(x, y) = \texttt{IV}_{\text{out}}$. We can solve these polynomials by computing their resultant polynomial, and then factoring the univariate resultant polynomial. Let $d_* = \max\left\{\deg_*(f_{\text{bw}}), \deg_*(f_{\text{fw}})\right\}$, where $* \in \{x, y\}$, then their resultant can be computed in $\tilde{\mathcal{O}}\big(d_x \cdot d_y \cdot \log_2(p)\big)$ field operations [vdHL21, Vil25], and it has degree $\leq d_x \cdot d_y$. To minimize the degrees of f_{bw} and f_{fw} the partially guessed middle state should be of the form $\mathbf{x} = (\alpha_1, \ldots, \alpha_{k-1}, x, y, \alpha_{k+2}, \ldots, \alpha_t)^\intercal$, where $1 \leq k \leq t-2$ and $\alpha_i \in \mathbb{F}_p$. Such a guess linearizes the first $t-k-1$ rounds of π_1^{-1} and the first $k-1$ rounds of π_2. Working out the degrees of f_{bw} and f_{fw} in the CICO-1 problem is a straightforward generalization from the univariate case

$$\begin{aligned} &\deg_x(f_{\text{bw}}) = \alpha^{R_1-(t-k-1)-1}, && \deg_x(f_{\text{fw}}) = \alpha^{R_2-(k-1)-1}, \\ &\deg_y(f_{\text{bw}}) = \alpha^{R_1-(t-k-1)}, && \deg_y(f_{\text{fw}}) = \alpha^{R_2-(k-1)-2}. \end{aligned} \tag{105}$$

Thus, the complexity of computing the resultant becomes $\approx \tilde{\mathcal{O}}\big(\alpha^{R-(t-2)}\big)$. Comparing the complexity to our univariate degree estimations we conclude that the bivariate Meet-in-the-Middle approach is not expected to outperform the univariate attack.

We have implemented the bivariate Meet-in-the-Middle attack for GMiMCHash in `OSCAR` [OSC26]. In small scale experiments with $R_1 = \left\lceil \frac{t}{2} \right\rceil$ we observed that the resultant polynomial indeed has degree $\alpha^{R-(t-2)}$.

We also note that the substitution trick from the univariate attack cannot be applied to the bivariate Meet-in-the-Middle attack. If we would apply the substitution trick in the backward direction for the elements of $\mathbf{x}$ we would increase the degrees in forward direction, and vice versa if we would perform the substitution in the forward.

$\log_2(p) \approx 32, 64$. For Gröbner basis cryptanalysis of GMiMCHash2 we work with an iterated polynomial model, where we introduce intermediate state variables after every round. We illustrate the model for the sponge preimage problem. Let $\mathbf{x}_{\text{in}} = (x_{\text{in},1}, \ldots, x_{\text{in},r})^\intercal$,

$\mathbf{x}^{(i)} = \left(x_1^{(i)}, \ldots, x_t^{(i)}\right)^\intercal$ and $\mathbf{x}_{\text{out}} = (x_{\text{out},1}, \ldots, x_{\text{out},t-d})^\intercal$, where d denotes the digest size in field elements. Then, for preimage recovery of the hash value $\mathbf{h} \in \mathbb{F}_p^d$ we have to solve the following polynomial system

$$\begin{aligned} \mathsf{R}^{(1)}\left(\mathsf{M}_{\text{IO}}\begin{pmatrix}\mathbf{x}_{\text{in}}\\ \mathtt{IV}\end{pmatrix}\right) &= \mathbf{x}^{(1)}, \\ \mathsf{R}^{(i)}\left(\mathbf{x}^{(i-1)}\right) &= \mathbf{x}^{(i)}, \qquad 2 \le i \le R-1, \\ \mathsf{R}^{(i)}\left(\mathbf{x}^{(R-1)}\right) &= \mathsf{M}_{\text{IO}}^{-1}\begin{pmatrix}\mathbf{h}\\ \mathbf{x}_{\text{out}}\end{pmatrix}. \end{aligned} \tag{106}$$

Analogous polynomial systems can be set up for the sponge CICO and the compression preimage problems. Let us expand the polynomials in Equation (106)

$$\begin{pmatrix} x_2^{(i-1)} + \left(x_1^{(i-1)} + c_i\right)^\alpha \\ \vdots \\ x_t^{(i-1)} + \left(x_1^{(i-1)} + c_i\right)^\alpha \\ x_1^{(i-1)} + c_i \end{pmatrix} = \begin{pmatrix} x_1^{(i)} \\ \vdots \\ x_t^{(i)} \end{pmatrix} \tag{107}$$

$$\Rightarrow \begin{pmatrix} x_2^{(i-1)} + \left(x_1^{(i-1)} + c_i\right)^\alpha \\ -x_2^{(i-1} + x_3^{(i-1)} \\ \vdots \\ -x_2^{(i-1)} + x_t^{(i-1)} \\ x_1^{(i-1)} + c_i \end{pmatrix} = \begin{pmatrix} x_1^{(i)} \\ -x_1^{(i)} + x_2^{(i)} \\ \vdots \\ -x_1^{(i)} + x_{t-1}^{(i)} x_t^{(i)} \end{pmatrix}. \tag{108}$$

Analogous transformations can be performed for the first and the last round. So, the polynomials in every round can be transformed into a degree α polynomial and $t-1$ affine polynomials. In particular, after the transformation the GMiMCHash2 polynomials satisfy the structure assumption for the change of coordinates from Section 4.3.4. We have implemented the GMiMCHash2 polynomial models together with the change of coordinates in the computer algebra system `OSCAR` [OSC26]. In particular, success of the coordinate change can be verified with the rank of an associated linear system. In the following we will discuss the success of the change of coordinates.

Let us start with the sponges with $t \in \{8, 16\}$ and $c = r = d = \frac{t}{2}$. Recall that the Input/Output matrix is $\text{circ}\,(1, 0, \ldots, 0, 2, 0, \ldots, 0)$ with 2 on the $\frac{t}{2}$-th position. Here, the preimage and CICO polynomial systems consist of $t \cdot R$ polynomials in $t \cdot R$ variables. So, the polynomial systems are fully determined. For the Goldilocks prime and $t = 8$ the change of coordinates works for all round numbers $t \le R \le 170$. For the BabyBear, KoalaBear and Mersenne31 primes and $t = 16$ the change of coordinates works for all round numbers $t \le R \le 180$. Then, we have the $\mathbb{F}_p$-vector space dimension $D = \alpha^R$.

Next we analyze compression mode with $t \in \{8, 16\}$ and $d = \frac{t}{2}$. Again we use the Input/Output matrix $\text{circ}\,(1, 0, \ldots, 0, 2, 0, \ldots, 0)$. In compression mode we have $t \cdot R$ polynomials in $t \cdot R + \frac{t}{2}$ variables. So, the system is underdetermined and we can fix some of the variables. For example, we can linearize the first $\frac{t}{2}$ rounds by guessing the inputs of the power functions. For the Goldilocks prime and $t = 8$ we then observed that the change of coordinates works for all round numbers $t \le R \le 170$. For the BabyBear, KoalaBear and Mersenne31 primes and $t = 16$ the change of coordinates also works for all round numbers $t \le R \le 180$. Then, we have the $\mathbb{F}_p$-vector space dimension $D = \alpha^{R-\frac{t}{2}}$.

Finally, for the sponges with $t \in \{12, 24\}$ and $c = \frac{t}{3}$, $r = \frac{2 \cdot t}{3}$ and $d = \frac{t}{3}$ we have the Input/Output matrix $\text{circ}(1, 0, \ldots, 0, 2, 0, \ldots, 0, 2, 0, \ldots, 0)$, where the 2s are on the $\frac{t}{3}$-th and $\frac{t}{2}$-th positions. Here, the preimage and CICO polynomial systems consist of

$t \cdot R$ polynomials in $t \cdot R + \frac{t}{3}$ variables. So, we can again guess some variables. Again, we chose to linearize the first $\frac{t}{3}$ rounds. For the Goldilocks prime and $t = 8$ we then observed that the change of coordinates works for all round numbers $t \leq R \leq 170$. For the BabyBear primes and $t = 24$ the change of coordinates also works for all round numbers $t \leq R \leq 120$. (Due to the length of the computation we did not run the test up to the maximal round numbers in Table 4 and all fields.) Then, we have the $\mathbb{F}_p$-vector space dimension $D = \alpha^{R-\frac{t}{3}}$.

Summarizing our Gröbner basis observations we require for security level 2^κ that

$$R \geq 1.25 \cdot \left(\frac{t}{2} + \kappa \cdot \log_\alpha (2) \right), \tag{109}$$

where we add the summand $t/2$ and factor 1.25 as security margin.

In Table 35 we compute the minimal round numbers which satisfy Equations (103) and (109).

Table 35: Round numbers for GMiMCHash2 to resist Gröbner basis attacks within security level 128 bits.

(a) $\log_2(p) \approx 256$.

$t\backslash\alpha$	2	3	4	5	8
3	168	109	88	77	61
4	170	111	90	79	64
5	173	114	93	82	66
8	180	121	100	89	74

(b) $\log_2(p) \approx 64$.

$t\backslash\alpha$	2	3	4	5	8
8	165	106	85	74	59
12	168	109	88	77	61
16	170	111	90	79	64
20	173	114	93	82	66

(c) $\log_2(p) \approx 32$.

$t\backslash\alpha$	2	3	4	5	8
16	170	111	90	79	64
24	175	116	95	84	69
32	180	121	100	89	74
40	185	126	105	94	79

D.2 Efficient GMiMC$^{\pi}_{\text{erf}}$2 Circuit

Algorithm 2 Efficient GMiMC$^{\pi}_{\text{erf}}$2 circuit for $t > 3$.

Input: $\mathbf{x} \in \mathbb{F}_p^t$.
Output: GMiMC$^{\pi}_{\text{erf}}2(\mathbf{x})$.

Initialize $x_{\text{acc}} = 0$, $\mathbf{x}_i = \mathbf{x}$ and $\mathbf{a} = \mathbf{0} \in \mathbb{F}_p^{t-1}$.
$\mathbf{x}_i = \mathsf{M}_{\text{IO}} \mathbf{x}_i$.
$x_{i,1} = x_{i,1} + c^{(1)}$.
for $r \leftarrow 1$ to R **do**
    $x_\alpha = x_{i,1}^\alpha$.
    Rotate $\mathbf{a}$ one position to the left.
    $x_{\text{acc}} = x_{\text{acc}} - a_1$.
    $a_1 = x_\alpha$.
    $x_{\text{acc}} = x_{\text{acc}} + x_\alpha$.
    Rotate $\mathbf{x}_i$ one position to the left.
    $x_{i,1} = x_{i,1} + x_{\text{acc}} + c^{(r+1)}$, where $c^{(R+1)} = 0$.
for $j \leftarrow 2$ to $t - 1$ **do**
    Rotate $\mathbf{a}$ one position to the left.
    $x_{\text{acc}} = x_{\text{acc}} - a_1$.
    $x_{i,j} = x_{i,j} + x_{\text{acc}}$.
$\mathbf{x}_i = \mathsf{M}_{\text{IO}} \mathbf{x}_i$.
return $\mathbf{x}_i$.

E Poseidon Appendix

E.1 Equivalent Constant Representation for Internal Poseidon Rounds

Let us take a look at the first internal Poseidon round

$$\mathsf{R}_{\mathrm{I}}(\mathbf{x}) = \mathsf{M}_{\mathrm{I}} \begin{pmatrix} (x_t + c_{1,0})^\alpha \\ x_1 + c_{1,1} \\ \vdots \\ x_{t-1} + c_{1,t-1} \end{pmatrix} = \mathsf{M}_{\mathrm{I}} \begin{pmatrix} (x_t + c_{1,0})^\alpha \\ x_1 \\ \vdots \\ x_{t-1} \end{pmatrix} + \mathsf{M}_{\mathrm{I}} \begin{pmatrix} 0 \\ c_{1,1} \\ \vdots \\ c_{1,t-1} \end{pmatrix}. \tag{110}$$

As we can see from the equation, addition of $\tilde{\mathbf{c}}_1 = \mathsf{M}_{\mathrm{I}}(0, c_{1,1}, \ldots, c_{1,t-1})^\intercal$ commutes with application of the partial S-box layer. Thus, we can absorb $\tilde{\mathbf{c}}_1$ into the constant of the second round $\mathbf{c}_2$, and then iterate the commutative relation for $\tilde{\mathbf{c}}_2 = \mathsf{M}_{\mathrm{I}}(0, c_{2,1}, \ldots, c_{2,t-1})^\intercal$. Via inductive repetition we can then reduce the constant vector addition in every partial round to a single constant addition in the first component. For the final partial round we then absorb $\tilde{\mathbf{c}}_{R_{\mathrm{P}}}$ into the constants of the following full round.

E.2 Reducing the PLONK Constraints of Internal Poseidon Rounds

We follow the approach in [GKR+21] to fold most of the matrix costs in partial layers of Hades designs. Let the partial S-box layer be $\mathsf{S}_p(\mathbf{x}) = (x_0^\alpha, a_1, \ldots, x_{t-1})$. Let N be a transformation applying matrix $N = \mathrm{diag}(1, \tilde{M})$, where $N \in \mathbb{F}_p^{t \times t}$ and $\tilde{M}$ is any matrix in $\mathbb{F}_p^{t-1 \times t-1}$. N commutes with the partial S-box layer S_p s.t. $\mathsf{S}_p \circ \mathsf{N}(\mathbf{x}) = \mathsf{N} \circ \mathsf{S}_p(x)$ as they modify distinct branches. For simplicity, lets assume the first partial round starts at $i = 0$ and the last at $i = R$. For the partial rounds, we define an intermediate representation $\mathbf{v}_i = N_i \mathbf{x}_i$ where for correctness it is required that $N_R = I$ s.t. $\mathbf{v}_R = \mathbf{y}_R$. Per round, we derive the helper matrices N and G as

$$N_{i-1} = \begin{pmatrix} 1 & \mathbf{0}^T \\ \mathbf{0} & (N_i \mathsf{M}_{\mathrm{I}})_{1:,1:} \end{pmatrix}, \qquad G_i = N_i \mathsf{M}_{\mathrm{I}} N_{i-1}^{-1} = \begin{pmatrix} a_i & \mathbf{b}_i^\intercal \\ \mathbf{c}_i & I \end{pmatrix}. \tag{111}$$

In the forward direction, we compute the next round state as $\mathbf{v}_i = G_i \mathsf{S}_p(\mathbf{v}_{i-1}) + N_i \mathbf{c}_i$, where the evaluation costs $t-1$ gates to compute $v_{i,0}$ and $t-1$ gates to compute $\mathbf{v}_{i,1:}$ (the constant terms can be folded into the addition gates). While by construction $\mathbf{v}_R = \mathbf{y}_R$, we need to constrain the correct initial state $\mathbf{v}_0 = N_0 \mathbf{y}_0$, where N_0 is a $(t-1) \times (t-1)$ dense matrix. However, as the last round of the first block of full rounds ends with a matrix M_{E}, we can fold that basis shift into that matrix. For Poseidon M_{E} is already dense and there is no overhead in arithmetization, while for Neptune or Poseidon2 there is some small overhead.